

грокаем

проектирование реляционных баз данных

Цян Хао
Михаил Цикердекис



grokking

Relational Database Design

Qiang Hao, Michail Tsikerdekis


MANNING
SHELTER ISLAND

прокаем

проектирование реляционных баз данных

Цян Хао, Михаил Цикердекис



Санкт-Петербург • Москва • Минск

2026

ББК 32.988.02-018-07+32.973.2
УДК 004.738.5.056.53+004.65
X19

Хао Цян, Цикердекис Михаил

X19 Грокаем проектирование реляционных баз данных. — СПб.: Питер, 2026. — 272 с.: ил. — (Серия «Библиотека программиста»).

ISBN 978-5-4461-4405-1

Реляционные базы данных используются практически в каждой компании. И разбираться в том, как они работают, приходится и разработчикам, и аналитикам, создающим дашборды и отчеты, и специалистам, которым просто нужна актуальная информация. Это увлекательное руководство по миру баз данных и SQL написано в доступной и юмористической манере. Авторы, опытные преподаватели из Университета Торонто, превращают сложные концепции в простые и понятные объяснения с помощью ярких примеров, забавных иллюстраций и практических заданий.

Книга охватывает основы SQL, проектирование сущностей и связей, нормализацию, безопасность, оптимизацию и даже роль генеративного ИИ в дизайне БД. Идеальный выбор для тех, кто хочет освоить реляционные базы данных без скучных лекций, а с удовольствием и практическим применением.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ББК 32.988.02-018-07+32.973.2
УДК 004.738.5.056.53+004.65

Права на издание получены по соглашению с Manning Publications. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. В книге возможны упоминания организаций, деятельность которых запрещена на территории Российской Федерации, таких как Meta Platforms Inc., Facebook, Instagram и др. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими.

ISBN 978-1633437418 англ.

ISBN 978-5-4461-4405-1

Authorized translation of the English edition © 2025 Manning Publications.
This translation is published and sold by permission of Manning Publications,
the owner of all rights to publish and sell the same.

© Перевод на русский язык ООО «Прогресс книга», 2026
© Издание на русском языке, оформление ООО «Прогресс книга», 2026
© Серия «Библиотека программиста», 2026

*Нашим семьям и друзьям за их бесконечную поддержку
и содействие. Всем тем, кто верит в силу пытливости
и настойчивости, — пусть эта книга станет
вашим попутчиком на дороге к знаниям.*

Оглавление



От издательства	9
Предисловие	11
Благодарности	13
Об этой книге	15
Для кого эта книга	15
Структура книги	16
О коде в книге	17
Форум liveBook	17
Об авторах	18

Часть 1 НАЧАЛО РАБОТЫ

Глава 1. Знакомство с базами данных и SQL	20
Введение	21
Ваш первый SQL-запрос	27
Основы SQL-запросов	29
Управление таблицами и данными	37
Итоги	43

Глава 2. Связанные таблицы и углубление в SQL	44
Введение	45
Связанные таблицы	45
Снова об управлении таблицами и данными	54
Самостоятельное изучение SQL	61
Итоги	65
Глава 3. Общие сведения о проектировании баз данных	67
Введение	67
Цели проектирования баз данных	68
Общий обзор процесса проектирования	72
Основные этапы проектирования баз данных	77
Итоги	86

Часть 2

НАВЫКИ ПРОЕКТИРОВАНИЯ БАЗ ДАННЫХ

Глава 4. Сущности и свойства	88
Введение	89
Сущности и свойства	89
Ключи	94
Типы данных	101
Итоги	117
Глава 5. Связи	118
Введение	119
Диаграммы сущностей и связей	119
Соединение связанных сущностей	121
Кратность	124
Сильные и слабые сущности	143
Итоги	150
Глава 6. Нормализация и реализация	152
Введение	153
Нормализация	153

Реализация	171
Итоги	188
Глава 7. Безопасность и оптимизация	190
Введение	191
Безопасность	191
Стоимость хранения	202
Индексация	209
Денормализация	216
Итоги	225

Часть 3

ИСКУССТВЕННЫЙ ИНТЕЛЛЕКТ В ПРОЕКТИРОВАНИИ БАЗ ДАННЫХ

Глава 8. Проектирование баз данных в эпоху генеративного ИИ	228
Введение	229
Анализ требований	232
Сущности и свойства	233
Связи между сущностями	248
Реализация	261
Итоги	268



От издательства

Авторы знакомят читателей с базами данных, используя яркие и веселые образы, забавные названия сущностей и их свойств. Чтобы вам было так же интересно, как и англоязычным читателям, мы решили перевести на русский язык содержание таблиц этой БД. Перевод достаточно вольный, не буквальный. Обратите внимание, что в авторском репозитории книги на GitHub (см. далее раздел «О коде в книге») они, разумеется, остались на английском. Примеры SQL-кода с названиями полей и содержимым таблиц в книге тоже на английском, поскольку на уровне БД не принято использовать русские буквы (хотя технически это не запрещено). Мы надеемся, что это кажущееся противоречие не мешает знакомству с темой, поскольку материал в книге изложен в понятной и увлекательной манере, свойственной всей серии книг «Грокаем».

Мы выражаем огромную благодарность клубу рецензентов ИТ-литературы ReadIT Club за помощь в работе над русскоязычным изданием книги и вклад в повышение качества переводной литературы.

Ваши замечания, предложения, вопросы отправляйте по адресу comp@piter.com (издательство «Питер», компьютерная редакция).

Мы будем рады узнать ваше мнение!

На веб-сайте издательства www.piter.com вы найдете подробную информацию о наших книгах.

О научном редакторе русского издания

Евгений Войнов — тимлид Java-разработчиков в Яндекс Маркете. Ранее работал в компании КРОК: 6 лет руководил группой Java-разработчиков, 3 года работал техническим менеджером. Был ментором разработчиков и будущих руководителей, преподавал в учебных программах для студентов и школьников. Более 15 лет опыта разработки систем для государственного и коммерческого сектора.



Предисловие

В течение многих лет мы читаем лекции по системам управления базами данных. Одна из проблем, с которыми мы сталкиваемся постоянно, — значительные трудности при обучении студентов по традиционным учебникам. Несмотря на полноту изложения материала, они утомительны и тяжелы для освоения, что делает учебный процесс сложнее, чем он должен быть.

Наш опыт преподавания привел нас к идее создания учебных материалов совсем иного типа, с доступным и увлекательным изложением принципов проектирования баз данных. Такая возможность представилась, когда к нам обратился Даниэль Зингаро (Daniel Zingaro), редактор издательства Manning и наш коллега по Университету Торонто в Миссиссоге. Даниэль предложил написать учебник, в котором достаточно сложные понятия преподносились бы так, чтобы учащимся было не только понятно, но и интересно.

Так появилась книга «Грокаем проектирование реляционных баз данных», в которой мы уделили внимание ясности изложения технических проблем, практической применимости материала, его пошаговому, эффективному освоению. Она подойдет как новичкам, так и тем, кто желает освежить давно полученные знания.

Сегодня, когда искусственный интеллект и машинное обучение стремительно трансформируют целые отрасли, реляционные базы данных по-прежнему играют важную роль. Несмотря на появление новых технологий, в том числе NoSQL и методов обработки Big Data, реляционные базы данных все еще остаются самой распространенной технологией хранения и доступа, особенно в средах, требующих структурированных, надежных и масштабируемых решений. Более того, развитие искусственного интеллекта и машинного обучения только повысило спрос на четкие, хорошо организованные данные — такие, которые и содержатся в реляционных базах. Освоив их проектирование, вы не только приобретете полезные навыки, но и сможете полностью раскрыть потенциал искусственного интеллекта и машинного обучения, обеспечив целостность и доступность данных, лежащих в основе этих технологий.

Надеемся, что наш труд станет ценным источником информации для всех, кто пробует силы в этой важнейшей технологической области, поможет преодолеть разрыв между фундаментальной теорией баз данных и их практическим применением в быстро меняющемся современном мире.



Благодарности

Мы искренне благодарим всех перечисленных ниже за их неоценимый вклад в эту книгу:

- *Коди Фэм (Cody Pham)*. Коди нарисовал все иллюстрации к этой книге. Яркие и живые рисунки помогают читать текст и упрощают понимание самых сложных понятий.
- *Даниэль Зингаро (Dr. Daniel Zingaro)*. Даниэль Зингаро помог определиться с тематикой нашей книги, а также дал ценные советы, как лучше распределить материал по главам.
- *Элеша Хайд (Elesha Hyde)*. Элеша активно помогала нам на этапе редактирования. Трудно переоценить ее подробные отзывы и предложения.
- *Луис Дэвидсон (Louis Davidson)*. Луис дал множество полезных советов и рекомендаций, благодаря которым буквально каждая глава стала более четкой и логичной.
- *Джонатан Генник (Jonathan Gennick)*. Джонатан внес немало значимых предложений для доработки этой книги.
- *Команда издательства Manning*. Мы благодарны команде Manning за опыт и преданность делу, которые помогли воплотить эту книгу в жизнь.

- *Все рецензенты, которые не пожалели времени на содержательные отзывы о книге.* Дэвид Шпенн (David Spenn), Ганеш Фалак (Ganesh Falak), Ганеш Шваминатан (Ganesh Swaminathan), Гран Колли (Grant Colley), Хен Чжан (Heng Zhang), Джим Амрхейн (Jim Amrhein), Хосе Альберто Рейес Квеведо (Jose Alberto Reyes Quevedo), Маркус Гезель (Marcus Geselle), Максим Волгин (Maxim Volgin), Надир Доктор (Nadir Doctor), Оливер Кортен (Oliver Korten), Орландо Мендес Моралес (Orlando Mendez Morales), Патрик Реган (Patrick Regan), Петер А. Шотт (Peter A. Schott), Потито Колуччелли (Potito Colucelli), Радхакришна МВ (Radhakrishna MV), Саша Санкова (Sasha Sankova), Тревор Уильямс (Trevor Williams), Валери Пархам-Томсон (Valerie Parham-Thompson), Виктор Дуран (Victor Duran), Войта Тума (Vojta Tuma), Уильям Джамир Сильва (William Jamir Silva) и Илунь Чжан (Yilun Zhang). Ваши предложения помогли сделать книгу лучше.



Об этой книге

«Прокаем проектирование реляционных баз данных» — книга, которая научит вас искусству проектирования. Полная ярких иллюстраций и жизненных проектов, она ориентирована на действенное, практическое обучение. В отличие от многих учебников для новичков, где основное внимание уделяется синтаксису языка SQL и формальной теории баз данных, здесь оно смещено на фундаментальный подход к пониманию реляционных БД. Вы научитесь планировать их структуру с нуля, создавать эффективные, но при этом интуитивно понятные системы — базы данных, которые пройдут испытание временем и с которыми будет приятно работать долгое время после реализации.

Для кого эта книга

Книга написана в первую очередь для тех, кто только начинает заниматься базами данных, их проектированием или же хочет освежить в памяти понятия, изученные много лет назад. Все главы написаны доступным языком, содержат подробные практические рекомендации по всем ключевым вопросам, важным для создания эффективных реляционных БД.

Структура книги

Книга состоит из восьми глав, которые разделены на три части:

- *Часть 1 «Начало работы»* познакомит вас с основами. В главах 1 и 2 вы найдете основные сведения о языке SQL, знание которого необходимо для создания эффективных баз данных, а в главе 3 — описание самых важных принципов и процессов, определяющих успешность дизайна БД.
- *Часть 2 «Навыки проектирования баз данных»* состоит из четырех глав, которые посвящены основным навыкам, необходимым для проектирования БД:
 - *«Сущности и свойства»*. В этой главе вы научитесь анализировать предъявляемые заказчиками требования и на основе этого анализа выявлять и определять сущности, а также их свойства, закладывая фундамент для хорошо структурированной базы данных.
 - *«Связи»*. Из этой главы узнаете, как, исходя из принципов согласованности и удобства работы, устанавливать значимые связи между сущностями.
 - *«Нормализация и реализация»*. В этой главе познакомитесь с процессом нормализации, который позволяет минимизировать риски возникновения аномалий добавления, обновления и удаления. Кроме того, вы научитесь переводить готовый проект в код на языке SQL, пригодный для создания базы данных.
 - *«Безопасность и оптимизация»*. В этой главе найдете практические советы по обеспечению безопасности базы данных, а также ее оптимизации для повышения производительности и обеспечения целостности данных.
- *Часть 3 «Искусственный интеллект в проектировании баз данных»* расскажет о возможностях генеративных моделей ИИ, о том, как они ускоряют и упрощают процесс проектирования, делают его более эффективным. Вместе с ChatGPT вы снова пройдете по всем этапам создания базы данных с нуля.

О коде в книге

Исполняемые фрагменты кода можно загрузить из версии liveBook (электронной) по адресу <https://livebook.manning.com/book/grokking-relational-database-design>. Полный код примеров книги доступен для загрузки на сайте Manning по адресу www.manning.com, а также в репозитории на GitHub по адресу <https://github.com/Neo-Hao/grokking-relational-database-design>. Для каждой главы там создана отдельная папка, в которой находятся скрипты и пошаговые инструкции по их выполнению.

Форум liveBook

Приобретая книгу «Грожаем проектирование реляционных баз данных», вы также получаете бесплатный доступ к платформе для онлайн-чтения liveBook издательства Manning (на английском языке). Эксклюзивные возможности liveBook позволяют оставлять комментарии как к книге в целом, так и к отдельным ее разделам или абзацам. Можно легко делать заметки для себя, задавать технические вопросы и отвечать на них, а также получать помощь от авторов и других пользователей. Чтобы получить доступ к форуму, посетите страницу <https://livebook.manning.com/book/grokking-relational-database-design/discussion>. Информацию о форумах Manning и правилах поведения на них см. на <https://livebook.manning.com/discussion>.

В рамках своих обязательств перед читателями издательство Manning предоставляет ресурс для содержательного общения читателей и авторов. Эти обязательства не подразумевают конкретную степень участия автора, которое остается добровольным (и неоплачиваемым). Задавайте авторам хорошие вопросы, чтобы им было интересно участвовать в диалоге! Форум и архивы обсуждений доступны на сайте Manning, пока книга продолжает издаваться.



Об авторах



Цян Хао (Qiang Hao) — адъюнкт-профессор computer science в Университете Западного Вашингтона. Его работы в области компьютерного образования широко известны и часто цитируются в научном сообществе. Цян Хао — опытный преподаватель: в его активе лекции по многочисленным предметам, включая системы баз данных, структуры данных и разработку мобильных приложений. Благодаря инновационному подходу к обучению он пользуется признанием среди студентов и коллег.



Михаил Цикердекис (Michail Tsikerdekis) — адъюнкт-профессор computer science в Университете Западного Вашингтона. Его работы в области кибербезопасности опубликованы во многих ведущих журналах и на конференциях. На протяжении многих лет Михаил Цикердекис преподавал различные дисциплины, включая социологию, computer science, information science, системы и основы баз данных. Он является старшим членом IEEE и стипендиатом программы Фулбрайта, США (24/25).

Часть 1

Начало работы



Добро пожаловать в мир проектирования реляционных баз данных! В этой части мы познакомимся с основами языка SQL, знание которого необходимо для создания эффективных баз данных и позволит уверенно осваивать более сложные понятия. В этой части мы заложим прочный фундамент для всех структур, которые мы затем построим на нем. Она будет полезна как новичкам, так и тем, кто хочет освежить свои познания в области SQL.

В главе 1 мы начнем изучать язык SQL в объеме, необходимом для проектирования баз данных. Мы напишем первые SQL-запросы (своего рода «Привет, мир!»), и это позволит нам взаимодействовать с данными более осмысленно.

В главе 2 углубим познания в области SQL, узнаем о том, как устроены связи между таблицами. На этом наше знакомство с SQL завершится: мы будем знать все, что потребуется для работы.

В главе 3 мы посмотрим на проектирование баз данных с высоты птичьего полета, поговорим о том, из каких этапов состоит этот процесс и что наиболее важно для каждого из них.

Итак, вооружитесь планшетом или ноутбуком с приложением для работы с базами данных и сосредоточьтесь. Мы начинаем погружение в восхитительный мир проектирования баз данных.

1 | Знакомство с базами данных и SQL



В этой главе

- ✓ Фундамент для дальнейшей работы
- ✓ Основы реляционных баз данных
- ✓ Обзор проектирования баз данных
- ✓ Основные понятия языка SQL для написания первого SQL-запроса

Необходимые знания

В этой главе вам встретятся фрагменты кода. Если вы захотите их выполнить или изучить их варианты для разных систем управления базами данных (СУБД), откройте репозиторий GitHub к этой книге (<https://github.com/Neo-Hao/grokking-relational-database-design>). Скрипты из этой главы находятся в папке `chapter_01`, а инструкции по их запуску — в файле `README.md`.

Введение

Проектирование БД — критически важный этап программной разработки, которым, однако, часто пренебрегают. Практически каждое приложение должно где-то хранить данные и как-то управлять ими, но далеко не все могут похвастаться хорошо спроектированной базой данных. Подходя к этой задаче без понимания принципов эффективного построения баз данных, можно столкнуться с неожиданными проблемами, в том числе с беспорядочной организацией информации или задержками при выполнении запросов, которые требуют слишком много ресурсов. Все это может привести к неудобству для пользователей и багам.



Напротив, хорошо спроектированная база данных станет прочной основой для разработки эффективных приложений: грамотная организация и структура данных облегчают выборку и другие операции с ними, а значит, снижается количество ошибок и пользователям становится более комфортно работать.

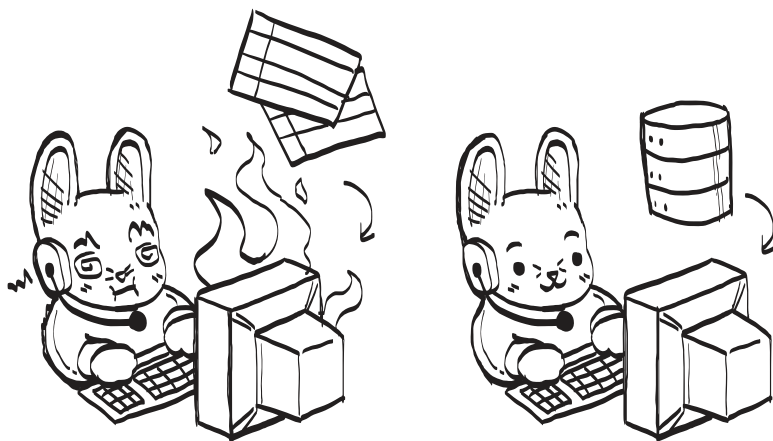
Каким бы ни был ваш уровень как программиста и разработчика приложений, важно уметь создавать эффективные базы данных и объяснять принципы их работы людям, возможно, далеким от технологий, не вводя их в ступор. Поможет вам в этом книга, которую вы держите в руках и где вы найдете многочисленные примеры, определения и пояснения, написанные простым языком. Даже при нулевых начальных

познаниях, перелистнув последнюю страницу, вы будете хорошо разбираться в том, как строятся реляционные базы данных.

А в этой главе мы познакомимся с ними, определим несколько ключевых понятий, которые будем использовать в дальнейшем, а также изучим основы языка *структурированных запросов* (Structured Query Language, SQL). SQL — язык программирования, который предназначен для работы с информацией, хранящейся в реляционных базах данных. Для их разработки важно уверенно владеть этим языком.

Реляционные базы данных

Давным-давно в одной небольшой компании сотрудники хранили данные о клиентах в Excel. И все работало: сведения были всегда под рукой, а обновлять их не составляло труда. Но время шло, продажи росли, все больше было клиентов и товаров, все больше данных. Таблицы стали огромными, так что теперь на поиск и обновление уходило гораздо больше времени. К тому же местами данные повторялись, местами противоречили друг другу.



И вот однажды в компанию позвонил недовольный клиент, с которого дважды взяли плату за товар. Открыв Excel, менеджер попытался решить проблему, но с ужасом понял, что данные о покупке повреждены и ничего с этим сделать уже нельзя. Вот только клиент такой был не один: вскоре история повторилась, а потом и еще много раз.

Чем чаще такое происходило, тем лучше в компании понимали, что электронные таблицы — не выход: необходима база данных, способная справиться с большим объемом информации, а также обеспечить целостность этой информации.

Если объемы данных невелики, а их структура проста, электронные таблицы вполне подойдут для работы. Действительно, при таких условиях не нужно заботиться о том, как обеспечить согласованность, целостность и безопасность информации, масштабируемость ее структуры, как облегчить ежедневный анализ данных. С увеличением объема и усложнением организации рано или поздно придется задуматься о базе данных.

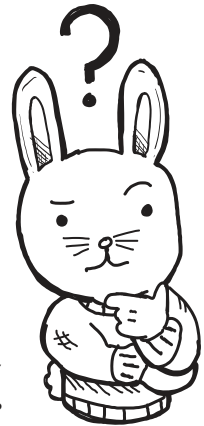
Реляционные базы данных уже давно являются стандартной технологией хранения и доступа к данным в тех случаях, когда вопросы масштабируемости, согласованности и целостности критически важны. Искусственный интеллект и машинное обучение, бурное развитие которых мы наблюдаем в последние годы, способствуют сохранению и даже росту популярности реляционных баз данных.

В этом разделе мы познакомимся с основными понятиями из области реляционных баз данных, в том числе таблицами, сущностями и СУБД.

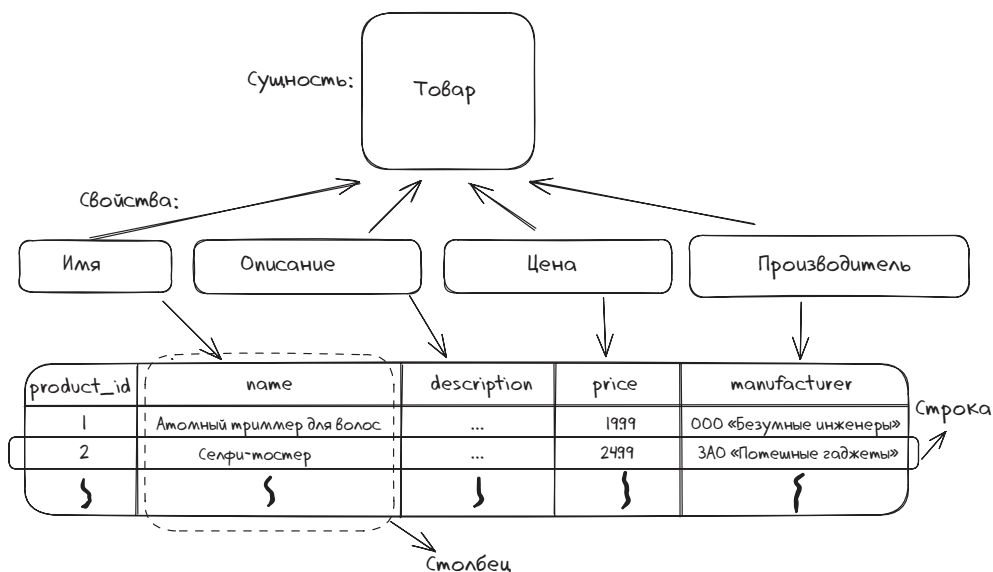
Таблицы, сущности и первичные ключи

Реляционная база данных — это набор таблиц, в которых хранится информация. Эти таблицы похожи на то, что мы видим в Excel, с которым вы наверняка знакомы, — они состоят из строк и столбцов. Таблица может описывать сущность или связь между сущностями: каждая строка таблицы содержит отдельную запись о сущности, а каждый столбец — одно из свойств сущности.

Что такое *сущность* (entity)? Это объект или понятие, которые описываются набором свойств. Допустим, мы обслуживаем интернет-магазин Sci-Fi Collective («Фантастическая лавка»), продающий научно-фантастические товары (к примеру, машину времени, которая вернет вас на пять минут назад, если вы вдруг забудете дома ключи). Каждый такой товар — это сущность, которую можно описать четырьмя свойствами: name (имя),



description (описание), price (цена) и manufacturer (производитель). Если представить список товаров в виде таблицы, каждое из свойств станет столбцом, а запись о каждом товаре — строкой.



Как можно заметить, в этой таблице не четыре, а пять столбцов. Первый из них — `product_id` — это *первичный ключ* (primary key), который нужен для идентификации строк («чтобы всех отыскать, воедино созвать»¹). В каждой таблице может быть только один такой столбец, а все значения в нем уникальны. Мы будем подробно обсуждать первичные ключи в главе 4.

Нередко в одной таблице Excel хранятся данные о нескольких сущностях. Вы спросите, можно ли сделать так в базе данных? К примеру, объединить информацию о товарах и клиентах «Фантастической лавки», как на рисунке ниже, где добавлено имя клиента (`customer_name`), ID клиента (`customer_id`), его электронный адрес (`customer_email`) и количество (`quantity`).

¹ Цитата из «Властелина колец» по переводу Н. Григорьевой и В. Грушевского — *Примеч. пер.*

product_id	name	price	manufacturer	customer_id	customer_name	customer_email	quantity
1	Атомный триммер ...	1999	ООО «Безумные инженеры»	a1	Боб	bob@gmail.com	5
2	Селфи-мостер	2499	ЗАО «Потешные гаджеты»	b2	Дэйв	dave@outlook.com	15
3	Кошачий ...	2999	АО «Абсурдные аксессуары»	a1	Боб	bob@gmail.com	2
...	...	...	...	...	...	...	...
9	Генератор ...	999	ООО «Всякая всячина»	j8	Ажон	john@123.net	1
10	Нейтрализатор	33.55	ООО «Всякая всячина»	p9	Кейм	katy@123.net	2

Эта таблица — типичный пример плохого дизайна. Помимо очевидной избыточности данных, такая структура чревата неожиданными проблемами. Допустим, какой-то клиент совершил лишь одну покупку и данные о нем есть только в одной строке таблицы. Тогда, удаляя купленный им товар, мы потеряем и сведения о клиенте. Эта проблема известна как *аномалия удаления* (delete anomaly). Еще одна возможная проблема — *аномалия добавления* (insertion anomaly) — будет возникать при добавлении новых строк с товарами. До тех пор, пока товар никто не купил, нам нечем заполнить столбцы, описывающие клиента, однако мы обязаны это сделать, ведь такова структура таблицы. Как результат, мы не сможем внести товар в базу.

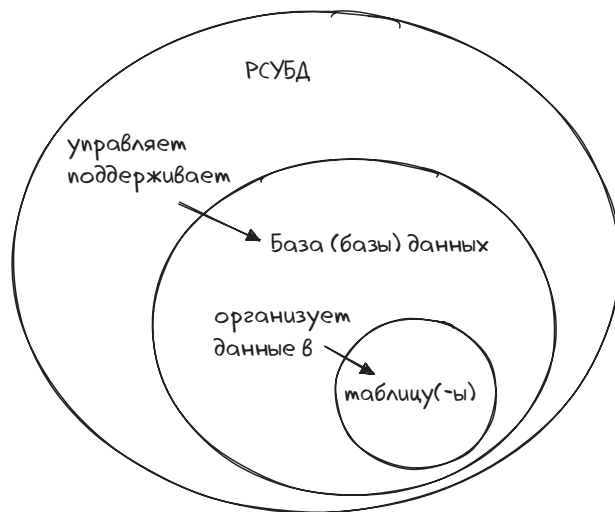
Итак, неудачные решения, принятые при проектировании, могут привести к проблемам, которые неизбежно снизят качество приложения. И чтобы их избежать, нужно освоить базовые принципы и лучшие практики проектирования баз данных.

Реляционные системы управления базами данных и SQL

Физически управление реляционными базами данных и работу с хранящейся в них информацией осуществляют *реляционные системы управления базами данных* (РСУБД). Впервые, еще в 70-х годах, такую систему создал Эдгар Кодд (Edgar Codd) из IBM.

РСУБД — это специальное ПО, обеспечивающее физическое хранение данных и управление ими. В его состав также входят средства для создания и обновления таблиц, а также для выборки данных. Вы, вероятно, слышали о таких РСУБД, как MySQL, MariaDB, PostgreSQL

и SQLite. Есть и другие. Одна из них непременно потребуется вам, чтобы работать с базой данных, которую вы создадите.



Одно из наиболее примечательных средств, которое поддерживают почти все РСУБД, — SQL, язык программирования, позволяющий создавать таблицы, получать и изменять хранящиеся в них данные. Несмотря на то что реализации SQL в разных РСУБД могут немного отличаться друг от друга, со временем сложился некий общий стандарт этого языка. Поэтому можно сказать, что РСУБД довольно последовательны в отношении SQL, а небольшие различия не имеют особого значения для нашей книги, поскольку речь в ней идет в основном о проектировании.

При разработке баз данных язык SQL можно вообще не использовать: в некоторых РСУБД имеются графические инструменты, которые автоматически генерируют SQL-скрипты, необходимые для создания задуманных баз данных и таблиц. Однако знание этого языка позволит вам лучше понять строение баз данных, особенно в том, что касается параметров структуры и дизайна, в том числе целостности данных, оптимизации и масштабирования. Кроме того, поскольку SQL — стандартизованный язык, который поддерживается большинством РСУБД, знакомство с ним позволит вам меньше зависеть от графических инструментов, а значит, расширит круг доступных вам РСУБД. Поэтому в первых двух главах книги мы и рассмотрим основы этого языка.

Ваш первый SQL-запрос

В этом разделе мы начнем изучать SQL и выполним первый запрос. Для этого мы воспользуемся примером из прошлого раздела — таблицей из базы данных магазина Sci-Fi Collective («Фантастическая лавка»). В этой базе много таблиц, но сейчас нас интересует одна: product (Товары), которая выглядит так:

product_id	name	description	price	manufacturer
1	Атомный триммер для волос	...	1999	ООО «Безумные инженеры»
2	Селфи-тостер	...	2499	ЗАО «Потешные гаджеты»
3	Кошачий наполнитель с ароматом кофе	...	2999	АО «Абсурдные аксессуары»
...	...	...	...	...
9	Генератор нулевой вероятности	...	999	ООО «Всякая всячина»
10	Нейтрализатор	...	33.55	ООО «Всякая всячина»

Прежде всего, необходимо создать саму базу данных и эту таблицу. Для этого можно использовать готовый SQL-скрипт, который находится в папке chapter_01 репозитория GitHub к этой книге (<https://github.com/Neo-Hao/grokking-relational-database-design>). Выполнить этот скрипт в выбранной вами СУБД¹ помогут инструкции из файла README.md, который находится в той же папке. Проще всего воспользоваться редактором SQLite Online:

1. Клонировать или загрузить репозиторий (<https://github.com/Neo-Hao/grokking-relational-database-design>).
2. Откройте редактор SQLite Online (<https://sqliteonline.com>).
3. Выберите нужную СУБД на левой панели, а затем нажмите Click to connect (Нажмите, чтобы подключиться).
4. Нажмите Import (Импортировать) и загрузите скрипт для выбранной СУБД (например, для MariaDB используйте файл mysql_db.sql).
5. Нажмите Ok.

¹ Здесь и далее под СУБД всегда будет пониматься РСУБД. — *Примеч. ред.*

Теперь мы готовы обратиться с запросом к таблице `product`. Введите следующий код (все три строки) в окне редактора SQLite Online и нажмите Run (Выполнить).

```
SELECT name
FROM product
WHERE price > 20;
```

Что делает этот запрос? Достаточно посмотреть на условие `price > 20` (цена больше 20). Как результат мы получим список товаров из таблицы `product`, стоимость которых превышает 20 (см. ниже). Мы знаем, что в таблице десять позиций, но пять из них (к примеру, атомный триммер для волос) не соответствуют этому условию.

name
Селфи-тостер
Кошачий наполнитель с ароматом кофе
Надувной чемодан
Световые мечи
Нейтрализатор

Как можно заметить, язык SQL во многом похож на обычный английский, что необычно. Все дело в том, что большинство языков программирования *императивны*: когда вы пишете код на Java или Python, вы указываете компьютеру, *что* нужно сделать и *как*. Особенность же SQL в том, что он *декларативен*: вы говорите лишь то, *что* следует делать, а всю остальную работу берет на себя СУБД. Мы, люди, намного чаще высказываем пожелания, чем предлагаем способы их осуществления, а потому английский и SQL действительно схожи друг с другом.

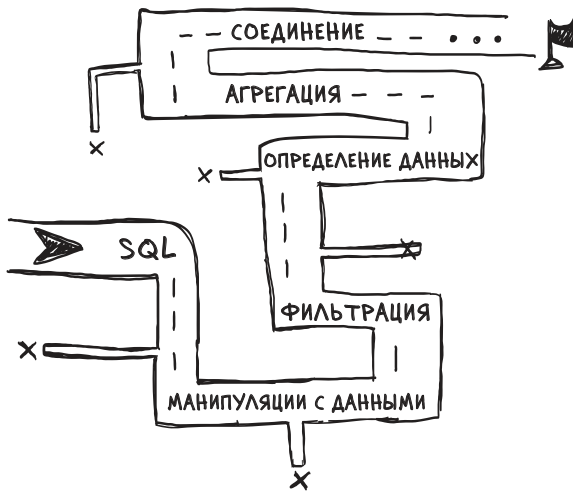
А если точнее, то SQL — это лаконичный английский. Впрочем, нам и не нужен огромный выбор слов и конструкций, чтобы сформулировать *запрос* (query). Для этого достаточно нескольких *выражений* (clause), также называемых *инструкциями* или *операторами* (statement), и небольшого набора правил. Для первого запроса нам потребуется три оператора:

- **SELECT** («выбрать») — указывает, какие столбцы нужно взять из таблицы. В данном случае нас интересуют названия товаров, то есть столбец `name` (имя), а потому мы написали `SELECT name`.
- **FROM** («из») — указывает одну или несколько таблиц, откуда требуется получить данные. Сейчас мы работаем только с одной таблицей: `product`, поэтому написали `FROM product`.
- **WHERE** («где») — задает параметры выбираемых данных. Поскольку нам нужны товары, которые стоят больше 20, следует написать `WHERE price > 20`.

В конце SQL-запроса необходимо поставить точку с запятой (;), которая говорит СУБД о том, что текст запроса завершен, а все, что следует дальше, к нему не относится.

Основы SQL-запросов

Авторы книги считают, что достаточно изучить лишь самые важные операторы, а все остальные — только по мере необходимости. Операторов в SQL много, однако не все они одинаково важны. Поэтому достаточно нескольких самых распространенных, чтобы усвоить основы и понять, куда двигаться дальше.



Поэтому вместо того чтобы обсуждать все операторы SQL по очереди, мы рассмотрим лишь самые нужные нам для работы, а в этом разделе займемся теми, что составляют запросы к одной таблице.

Фильтрация

Фильтрация (filtering) — одна из самых распространенных задач при выборке данных. Если из общего массива информации необходим лишь определенный набор, подходящий под заданные критерии, нужно отфильтровать данные при помощи условного оператора `WHERE`.

Еще составляя свой первый запрос в этой книге, мы говорили о том, что `WHERE` задает параметры фильтрации данных. К примеру, следующая инструкция позволяет получить из таблицы `product` названия и описания товаров, цена которых не больше 30:



```
SELECT name, description
FROM product
WHERE price < 30;
```

Когда нас интересует несколько столбцов, мы просто перечисляем их в операторе `SELECT`, используя запятую как разделитель.

А что, если нужно найти все товары конкретного производителя, к примеру ООО «Безумные инженеры» (Mad Inventors Inc)? Используем следующий запрос:

```
SELECT name
FROM product
WHERE manufacturer = 'Mad Inventors Inc';
```

Ответ на него будет таким¹:

name
Атомный триммер для волос
Мозговой зонд
Световые мечи

¹ Если вы будете работать с авторскими примерами, взятыми на GitHub, то получите результат на английском языке (см. раздел «От издательства» в начале книги). — *Примеч. ред.*

В этом запросе для проверки используется одиночный знак равенства (=). Также необходимо обратить внимание на одинарные кавычки (' '), при помощи которых выделяются строковые данные. Вы спросите: в SQL есть разные типы данных? Конечно. И все их можно разделить на шесть категорий:

- числовые данные (например, INT);
- строковые данные (например, TEXT);
- временные данные (например, DATE);
- строковые данные в формате Unicode (например, VARCHAR);
- двоичные данные (например, BINARY);
- прочие данные (например, XML).

Столбец `manufacturer` (Производитель) таблицы `product` относится к строковому типу. В отличие от него столбец `price` (Цена) является числовым.

Освоив фильтрацию числовых и строковых данных, поговорим о том, как объединять параметры в одном фильтре при помощи логических операторов. Чаще всего используются AND (И) и OR (ИЛИ), назначение которых несложно понять из перевода. К примеру, можно объединить два ранее упомянутых параметра при помощи оператора AND следующим образом:

```
SELECT *
FROM product
WHERE price < 30 AND manufacturer = 'Mad Inventors Inc.';
```

В результате получим:

product_id	name	description	price	manufacturer
1	Атомный триммер для волос	...	1999	ООО «Безумные инженеры»
6	Мозговой зонд	...	1999	ООО «Безумные инженеры»
7	Световые мечи	...	25.00	ООО «Безумные инженеры»

В отличие от прошлых запросов, в этом после `SELECT` мы указали не список столбцов из таблицы `product`, а звездочку (*), что означает, что нам нужны все столбцы. Благодаря фильтрации по двум параметрам мы получили данные о товарах, изготовленных ООО «Безумные инженеры» и стоящих не более 30.

Агрегация

Агрегация (aggregation) — еще одна важная задача, позволяющая выполнять вычисления с набором строк и получать единый результат. Благодаря агрегации можно узнать о тенденциях и шаблонах, которые сложно заметить, просматривая записи по отдельности. Чаще всего для вычислений используются следующие функции:

- `COUNT()` — подсчет количества строк;
- `SUM()` — вычисление суммы значений числового столбца;
- `AVG()` — вычисление среднего значения числового столбца;
- `MAX()` — определение максимального значения в столбце;
- `MIN()` — определение минимального значения в столбце.

Чтобы составить SQL-запрос с агрегатной функцией, следует поместить ее в инструкцию `SELECT`. Вот, например, как найти количество строк в таблице `product`:

```
SELECT COUNT(*) FROM product;
```

Результат будет таким:

COUNT(*)
10

Как вы, должно быть, заметили, имя столбца в ответе MariaDB повторяет написание агрегатной функции из запроса. В других СУБД этот столбец может называться `COUNT()` или иначе. Но если нам нужно другое, определенное имя, его можно указать при помощи оператора `AS`. К примеру, следующий запрос вычисляет среднюю стоимость товара и сохраняет ее в столбце `avg_price` (Средняя стоимость):

```
SELECT AVG(price) AS avg_price
FROM product
WHERE manufacturer = 'Mad Inventors Inc.';
```

В ответ мы получим столбец с именем `avg_price` и единственное значение: среднюю стоимость товаров из таблицы:

avg_price
21.66

В обоих примерах агрегатная функция применялась ко всем строкам таблицы. Однако вычисления можно выполнить и по нескольким наборам строк, сгруппировав их по одному или нескольким столбцам при помощи оператора `GROUP BY`, который часто применяется совместно с агрегатными функциями. После него всегда указывается один или несколько столбцов, которые разделяются запятыми. К примеру, можно посчитать количество товаров каждого из производителей:

```
SELECT COUNT(*) AS product_count, manufacturer
FROM product
GROUP BY manufacturer;
```

При этом мы получим следующий ответ, порядок строк в котором может быть иным:

product_count	manufacturer
2	АО «Абсурдные аксессуары»
1	ЗАО «Потешные гаджеты»
3	ООО «Безумные инженеры»
3	ООО «Всякая всячина»
1	ООО «Необычные товары»

Еще один пример: вычислим среднюю стоимость товаров по производителям:

```
SELECT AVG(price) AS avg_price, manufacturer
FROM product
GROUP BY manufacturer;
```

Результат этого запроса:

avg_price	manufacturer
22.5449...	АО «Абсурдные аксессуары»
2499	ЗАО «Потешные гаджеты»
21.659...	ООО «Безумные инженеры»
17.843...	ООО «Всякая всячина»
3999	ООО «Необычные товары»

Используя агрегатные функции и оператор GROUP BY, необходимо включить все столбцы из него в инструкцию SELECT. Иначе можно получить результат, не имеющий смысла. К примеру, в следующем запросе осуществляется группировка по столбцу manufacturer, который не указан в SELECT:

```
-- comment: результат будет сложно интерпретировать
SELECT COUNT(*) AS product_count
FROM product
GROUP BY manufacturer;
```

В ответ мы получим только один столбец с цифрами, смысл которых понять невозможно:

product_count
2
1
3
3
1

Еще один пример: вычислим среднюю стоимость товаров по производителям, не указав столбец manufacturer в SELECT:

```
-- comment: результат будет сложно интерпретировать
SELECT AVG(price) AS avg_price
FROM product
GROUP BY manufacturer;
```


Мы снова получим бессмысленный набор цифр:

avg_price
22.5449...
2499
21.659...
17.843...
3999

Не менее важно: в инструкции SELECT не должно быть столбцов, отсутствующих в GROUP BY (за исключением случаев, когда они используются как параметры агрегатной функции). Так, следующий запрос пытается подсчитать количество товаров по производителям, однако столбец name из условия SELECT не указан в условии GROUP BY и не передается агрегатной функции:

```
-- comment: этот запрос завершится ошибкой
-- comment: или результат будет бессмысленным
SELECT COUNT(*) AS product_count, manufacturer, name
FROM product
GROUP BY manufacturer;
```

В зависимости от СУБД такой запрос завершится ошибкой или бессмысленным результатом. К примеру, PostgreSQL сообщит об ошибке:

```
ERROR: column "product.name" must appear
in the GROUP BY clause or be used in
an aggregate function1
```

С другой стороны, SQLite отработает без ошибки, но результат не будет иметь смысла:

¹ ОШИБКА: столбец "product.name" необходимо включить в GROUP BY или использовать в агрегатной функции. — Примеч. ред.

product_number	manufacturer	name
2	АО «Абсурдные аксессуары»	Кошачий наполнитель с ароматом кофе
1	ЗАО «Потешные гаджеты»	Селфи-тостер
3	ООО «Безумные инженеры»	Атомный триммер для волос
3	ООО «Всякая всячина»	Полироль для рога единорога
1	ООО «Необычные товары»	Надувной чемодан



Как можно заметить, просматривая все десять строк таблицы product, атомный триммер для волос упоминается в ней лишь один раз, а не три. Но так как СУБД не понимает, что делать с указанным в запросе столбцом name, она выдает значение из первой его строки, которая попала в группу по производителю.

Еще один пример: попробуем посчитать среднюю стоимость товаров по производителям, добавив столбец product_id (Идентификатор товара) в SELECT, но не в GROUP BY:

```
-- comment: этот запрос завершится ошибкой
-- comment: или же результат будет бессмысленным
SELECT product_id, AVG(price) AS avg_price, manufacturer
FROM product
GROUP BY manufacturer;
```

И снова в зависимости от СУБД мы получим ошибку или бессмысленный результат. PostgreSQL, как и раньше, не выполняет запрос:

```
ERROR: column "product.product_id" must appear
in the GROUP BY clause or be used in an
aggregate function
```

SQLite же опять выдаст бессмысленный результат:

product_id	avg_price	manufacturer
3	22.5449...	АО «Абсурдные аксессуары»
2	24.99	ЗАО «Помешные гаджеты»
1	21.659...	ООО «Безумные инженеры»
5	17.843...	ООО «Всякая всячина»
4	39.99	ООО «Необычные товары»



Проверив строку с идентификатором 3 из таблицы product, можно заметить, что упомянутый в ней товар продается за 29.99, а не за 22.5449... Выходит, что агрегатная функция выполняет расчет неверно? Нет, просто, как и в прошлом примере, мы получили ответ, в котором в столбец product_id попало значение из первой строки соответствующей группы.

Таким образом, применяя агрегатные функции и оператор GROUP BY, не забывайте включить все упомянутые в нем столбцы в инструкцию SELECT. При этом в последнем не должно быть *неагрегатных свойств* (nonaggregate attributes), то есть таких, которые не используются в агрегатных функциях, если они не указаны в блоке GROUP BY.

Управление таблицами и данными

До сих пор мы писали запросы к таблице-примеру из репозитория. Однако время от времени приходится создавать и собственные таблицы, а также управлять как самими таблицами, так и данными в них.

Управление таблицами и данными — важный раздел языка SQL. Команды, предназначенные для решения этих задач, часто выделяют в отдельный язык *определения данных* (Data Definition Language, DDL),



а операторы, что мы рассматривали выше, — в *язык управления данными* (Data Manipulation Language, DML).

Знание основ DDL исключительно важно для проектирования баз данных. Поэтому в этом разделе мы познакомимся с тремя важнейшими операциями: созданием, изменением и удалением таблиц. Мы также поговорим о том, как заполнить таблицу данными.

Создание таблиц и добавление данных

Рассмотрим создание таблиц и добавление данных на примере таблицы `product` из базы данных «Фантастической лавки». Соответствующий скрипт (например, `mysql_db.sql` для MySQL и MariaDB) находится в репозитории к этой главе. Чтобы создать таблицу `product`, необходимо выполнить команду:

```
CREATE TABLE product (  
    product_id INT PRIMARY KEY,  
    name TEXT NOT NULL,  
    description TEXT NOT NULL,  
    price DECIMAL(5, 2) NOT NULL,  
    manufacturer TEXT NOT NULL  
);
```

Чтобы понять, как она работает, нужно ответить на два вопроса:

- Каков базовый синтаксис команды для создания таблиц?
- Что означают ключевые слова, которые мы используем в этом примере?

Особенность SQL: ошибки из-за дублирования данных

Если вы выполняли примеры из прошлых разделов и уже импортировали SQL-скрипт из репозитория, попытка выполнить команду `CREATE TABLE` приведет к ошибке.

Почему? Эта команда уже выполнялась при импорте скрипта (в разделе «Ваш первый SQL-запрос»), а значит, таблица `product` уже имеется в базе данных. Однако в одной базе данных не может быть двух таблиц с одинаковыми именами. То же самое можно сказать и о данных: в одной таблице не может быть двух одинаковых строк, точнее, двух строк с идентичным значением первичного ключа.

Поэтому, чтобы выполнить команду `CREATE TABLE`, а затем и команды для добавления данных, которые мы рассмотрим чуть позже, необходимо поступить так:

- При работе в редакторе SQLite Online — произвести сброс, обновив страницу в браузере.
- При работе в локальной или удаленной СУБД — удалить уже существующую таблицу. Как это сделать, читайте далее в этой главе.

Каков же базовый синтаксис команды для создания таблиц? Для этой цели используется команда `CREATE TABLE` со следующим синтаксисом:

```
CREATE TABLE имя_таблицы (  
    имя_столбца1 тип_данных [дополнительные_параметры],  
    имя_столбца2 тип_данных [дополнительные_параметры],  
    ...  
    имя_столбцаN тип_данных [дополнительные_параметры],  
    PRIMARY KEY (имя_столбцаХ)  
);
```

Ключевое слово `PRIMARY KEY` можно поместить непосредственно в описание нужного столбца, как это сделано в скрипте из примера. В отличие от дополнительных параметров, первичный ключ должен быть у каждой таблицы¹.

Что означают ключевые слова, используемые в этом примере? Прежде всего, мы указали, что таблица будет иметь имя `product`, а затем определили пять столбцов:

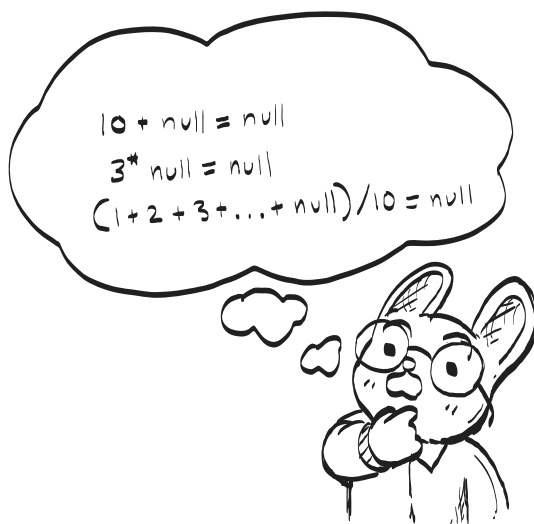
- `product_id` — числового типа (`INT`)
- `name` — строкового типа (`TEXT`)
- `description` — строкового типа (`TEXT`)
- `price` — числового типа (`DECIMAL`)
- `manufacturer` — строкового типа (`TEXT`)

Ключевые слова `INT` и `TEXT`, соответственно, определяют целочисленный и строковый типы данных. Некоторых пояснений, возможно, требует ключевое слово `DECIMAL`. Оно также определяет числовой тип, но принимает два параметра, первый из которых указывает общее ко-

¹ В некоторых СУБД, например в SQLite, вполне могут существовать таблицы и без первичных ключей. — *Примеч. пер.*

личество цифр, а второй — количество цифр после десятичной точки. В данном примере мы написали `DECIMAL(5, 2)`, то есть цена товара состоит из пяти цифр, две из которых отведены под дробную часть.

После определения типа данных в описании всех столбцов стоят ключевые слова `NOT NULL`. В языке SQL `NULL` представляет собой неизвестное значение. Как будто кто-то пытается вспомнить, как зовут собеседника, но не может. Любые вычисления с участием `NULL` дают в результате `NULL` (прибавив десять к неизвестному числу, мы получим другое, но столь же неизвестное число). Поэтому, во избежание неожиданных последствий, следует запретить хранение записей, свойства которых равняются `NULL`.



Наконец, ключевое слово `PRIMARY KEY` позволяет указать столбец, который будет первичным ключом таблицы. Имя столбца нужно указать в скобках за `PRIMARY KEY`.

Теперь у нас есть таблица `product`, готовая к загрузке данных. Для этого мы воспользуемся командой `INSERT INTO`, при помощи которой в таблицу можно добавить одну или несколько строк. Базовый синтаксис этой команды таков:

```
INSERT INTO имя_таблицы (столбец1, столбец2, ...)
VALUES (значение1, значение2, ...);
```

Пример добавления данных в таблицу `product` можно найти в том же скрипте. Мы можем добавить одну строку данных¹:

```
INSERT INTO product (product_id, name, description, price, manufacturer)
VALUES (
    1,
    'Atomic Nose Hair Trimmer',
    'Trim your nose hairs... of an atomic clock!',
    19.99,
    'Mad Inventors Inc.'
);
```

Или же сразу несколько строк²:

```
INSERT INTO product (product_id, name, description, price, manufacturer)
VALUES
(
    2,
    'Selfie Toaster',
    'Get your face on... with our selfie toaster',
    24.99,
    'Goofy Gadgets Corp.'
),
(
    3,
    'Cat-Poop Coffee',
    'The only coffee... the finest cat poop ...',
    29.99,
    'Absurd Accessories'
);
```

Изменение и удаление таблиц

Порой существующие таблицы приходится изменять и даже удалять — точно так же, как делать перестановку в квартире.

Есть много способов изменить таблицу: дать ей другое имя, добавить столбец или же выбрать для него другой тип данных. Для выполнения этих и многих других операций используется команда `ALTER TABLE`.

¹ Продукт: атомный триммер для волос, его описание: «Заряда этой машинки хватит на миллион лет» (вольный перевод), производитель: ООО «Безумные инженеры». — *Примеч. пер.*

² Второй продукт: селфи-тостер, его описание: «Позавтракайте тостом со своей фотографией», производитель: ЗАО «Потешные гаджеты». Третий: кошачий наполнитель с ароматом кофе, описание: «Ваша кошка оценит запах отборного кофе» (вольный перевод), производитель: АО «Абсурдные аксессуары». — *Примеч. пер.*

К примеру, чтобы добавить в таблицу `product` столбец для хранения артикулов, нужно написать следующий запрос:

```
ALTER TABLE product  
ADD serial_number INT;
```

После его выполнения в таблицу будет добавлен новый столбец `serial_number` (Артикул) целочисленного типа. Впрочем, быстро поняв, что не все артикулы записываются только цифрами, мы можем сделать этот столбец строковым:

```
-- comment: в SQLite изменение типа данных  
-- comment: столбца не поддерживается  
ALTER TABLE product  
ALTER COLUMN serial_number TEXT;
```

Способов изменить таблицу много, а вот удалить ее — только один. Поэтому синтаксис команды `DROP TABLE` очень прост: достаточно указать имя таблицы. К примеру, если таблица `product` больше не нужна, можно сделать такой запрос:

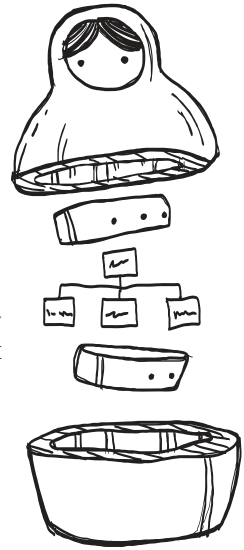
```
DROP TABLE product;
```

Но нужно действовать осторожно: эта команда безвозвратно удалит и саму таблицу, и все сохраненные в ней данные.

В этом разделе мы не будем рассматривать все возможные варианты команд для изменения и удаления таблиц. Тем, кто желает знать больше, стоит прочесть руководство по SQL к соответствующей СУБД. Однако мы сделали важный шаг на пути изучения и проектирования баз данных. Знаний, полученных в этой главе, вполне достаточно для освоения оставшегося материала книги, и, кто знает, возможно, дизайн, который вы создадите на их основе, окажется по-настоящему выдающимся.

Итоги

- Реляционная база данных — это набор таблиц, в которых хранится информация.
- В таблице содержатся данные о сущности или о связях нескольких сущностей в составе базы данных.
- Сущность — это объект или понятие, которые описываются набором свойств.
- СУБД — это специальное ПО, которое отвечает за физическое управление базами данных и работу с хранящейся в них информацией.
- Фильтрация данных осуществляется с помощью как минимум трех операторов SQL: `SELECT`, `FROM` и `WHERE`.
- Агрегатные функции часто используются совместно с оператором `GROUP BY`.
- Команды SQL, предназначенные для управления таблицами, обычно выделяют в отдельный язык — DDL. Чаще других используются три команды: `CREATE TABLE`, `ALTER TABLE` и `DROP TABLE`.
- При помощи команды `INSERT INTO...` в таблицу можно добавить одну или несколько строк.



2 | Связанные таблицы и углубление в SQL



В этой главе

- ✓ Знакомство со связанными таблицами и запросами к ним
- ✓ Обзор управления таблицами и данными
- ✓ Как изучить SQL самостоятельно

Необходимые знания

Чтобы усвоить материал из этой главы, необходимо понимать, что такое базы данных и таблицы, а также уметь писать простые SQL-запросы. При необходимости повторно прочтите главу 1.

В главе 2 вам встретятся фрагменты кода. Если вы захотите их выполнить или изучить их варианты для разных СУБД, откройте репозиторий GitHub к этой книге (<https://github.com/Neo-Hao/grokking-relational-database-design>). Скрипты из главы 2 находятся в папке `chapter_02`, а инструкции по их запуску — в файле `README.md`.

Проще всего воспользоваться инструментом SQLite Online:

1. Клонировать или загрузить репозиторий (<https://github.com/Neo-Hao/grokking-relational-database-design>).
2. Найдите скрипты в папке `chapter_02`.

3. Откройте веб-песочницу SQLite Online (<https://sqliteonline.com>). Выберите нужную СУБД на левой панели, а затем нажмите Click to connect.
4. Нажмите Import и загрузите скрипт для выбранной СУБД (например, для MariaDB используйте файл `mysql_db.sql`).
5. Нажмите Ok.

Введение

В прошлой главе мы изучили основы языка SQL и команды, при помощи которых можно создать одиночную таблицу и получить из нее данные. В этой главе продолжим знакомство с SQL и поговорим о том, как работать со связанными таблицами. Кроме того, вы узнаете, как самостоятельно углублять познания в этом языке.

Связанные таблицы

В этом разделе мы рассмотрим работу с двумя или большим количеством *связанных таблиц* (related tables), то есть таблиц, хранящихся в одной базе данных и объединенных одним или несколькими общими столбцами.

К примеру, клиенты Sci-Fi Collective («Фантастической лавки» — онлайн-магазина, известного нам по главе 1) могут регистрироваться на сайте, делать заказы, а также писать отзывы о купленных товарах. Для этого создана база данных из нескольких таблиц, две из которых — `product` и `review` — связаны друг с другом, как показано на рисунке далее. В первой из них хранятся сведения о товарах, а во второй — отзывы о них.

<u>product_id</u>	name	price	manufacturer
1	Атомный триммер ...	1999	ООО «Безумные инженеры»
2	Селфи-мостер	2499	ЗАО «Потешные гаджеты»
3	Кошачий наполнитель...	2999	АО «Абсурдные аксессуары»
...	...	...	...

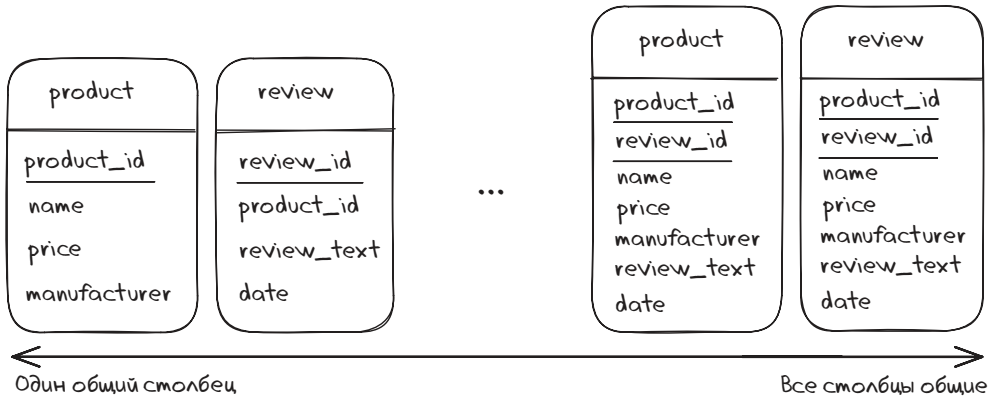
<u>review_id</u>	<u>product_id</u>	review_text	date
1	3	Классная штука...	2023-01-01 12:30:00
2	5	... лучшая покупка...	2023-01-02 13:45:00
3	2	...не рекомендую	2023-01-03 14:15:00
...	...	...	...

В обеих таблицах имеется столбец `product_id`, который и связывает их. В таблице `product` он выполняет функцию первичного ключа, а в таблице `review` — так называемого *внешнего ключа* (*foreign key*), который ссылается на первичный ключ другой таблицы, в данном случае `product`. Иными словами, значения в столбце `product_id` соединяют строки двух таблиц друг с другом. К примеру, строка с `product_id`, равным 3, в таблице `product` описывает кошачий наполнитель с ароматом кофе, а в строке с `review_id`, равным 1, таблицы `review` содержится положительный отзыв об этом товаре. Как понять, что клиент оставил отзыв именно о кошачьем наполнителе? По одинаковым значениям в столбце `product_id`.

Количество общих столбцов

Пытливый читатель, возможно, спросит: можно ли сделать так, чтобы таблицы `product` и `review` содержали несколько общих столбцов? Хороший вопрос с точки зрения проектирования. В более общем виде он звучит так: могут ли у связанных таблиц быть общие столбцы кроме первичного и внешнего ключей? Нет, не могут.

Для понимания рассмотрим пример с двумя связанными таблицами, которые используют в качестве первичных ключей отдельные столбцы. Теоретически у таких таблиц может быть как минимум один общий столбец, а как максимум — столько, сколько вообще в них имеется. Вот, например, таблицы `product` и `review`:



Если все столбцы у двух таблиц общие, то очевидно, что все данные дублируются, а значит, избыточны и не нужны. Удаляя или обновляя запись в одной из таблиц, мы столкнемся с аномалией удаления или обновления, как в главе 1, когда обсуждалось хранение двух сущностей в одной таблице. Итак, когда у двух таблиц все столбцы общие, это плохо.

А что, если общими будут не все столбцы, а только один или два (помимо ключей)? Все так же плохо. Во-первых, избыточность никуда не денется, хоть и не будет заметна так сильно, как в предыдущем случае. А во-вторых, сложности с обновлением продолжатся: изменяя данные в одной из таблиц, мы не должны забывать про обновление другой, иначе получим несоответствие.

Допустим, мы решили, в дополнение к `product_id`, сделать общим у таблиц `product` и `review` еще один столбец — `manufacturer`. Тогда, если кто-то из производителей поменяет название, нам придется обновлять его в обеих таблицах: иначе окажется, что один и тот же товар производится разными компаниями (см. рисунок ниже). Эта проблема известна как *аномалия обновления* (update anomaly). Итак, мы пришли к выводу, что общими у таблиц могут быть лишь столбцы, составляющие первичный и внешний ключи.

product_id	name	price	manufacturer
1	Атомный триммер ...	19.99	ООО «Безумные инженеры»
2	Селфи-мостер	24.99	ЗАО «Потешные гаджеты»
3	Кошачий наполнитель...	29.99	ООО «Безумные инженеры»
...	...	...	...

Замена на:

ООО «Безумные инженеры»

приводит к конфликту

review_id	product_id	manufacturer	review_text	date
1	3	ООО «Безумные инженеры»	Классная штука...	2023-...
2	5	ООО «Всякая всячина»	... лучшая покупка...	2023-...
3	2	ЗАО «Потешные гаджеты»	...не рекомендую	2023-...
...	...	...	...	...



Соединение данных из связанных таблиц

Бывает, что нужно соединить данные из связанных таблиц. Например, если владельцу «Фантастической лавки» интересно почитать отзывы о товарах, для этого нужно взять названия из таблицы `product` и совместить их с отзывами из таблицы `review`, обеспечив при этом соответствие данных друг другу. С этой задачей справится следующий SQL-запрос:

```
SELECT name, review_text
FROM product
JOIN review
ON product.product_id = review.product_id;
```

В результате получим:

name	review_text
Кошачий наполнитель...	Классная штука...
Полироль для рога единорога	...лучшая покупка...
Селфи-тостер	...не рекомендую
...	...

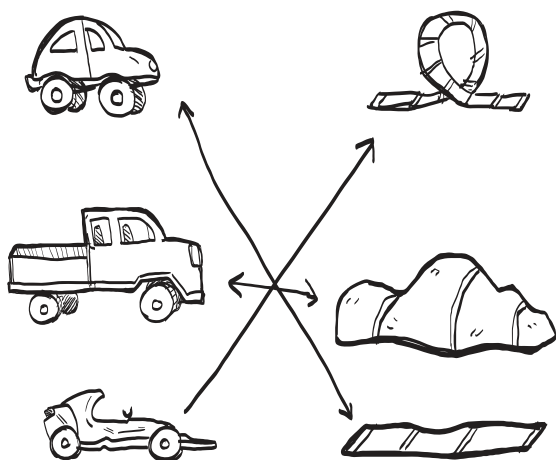
Этот запрос нуждается в некоторых пояснениях. Вот вопросы, которые могут возникнуть:

- Каков базовый синтаксис команд, получающих данные из связанных таблиц?
- Что именно делает оператор JOIN... ON...?
- Зачем нужна точка в product.product_id и review.product_id и как ею пользоваться?

Для начала ответим на первый вопрос. Синтаксис для получения данных из двух связанных таблиц следующий:

```
SELECT столбец1, столбец2, ...
FROM таблица1
JOIN таблица2
ON таблица1.столбец = таблица2.столбец;
```

Теперь ответим на второй. Представьте себе две коробки с детскими игрушками: в одной машинки, в другой — модельки дорог. Ребенку нужно их сопоставить. Для этого он высыпает все на пол, а затем начинает перебирать. Так же работает и оператор JOIN.



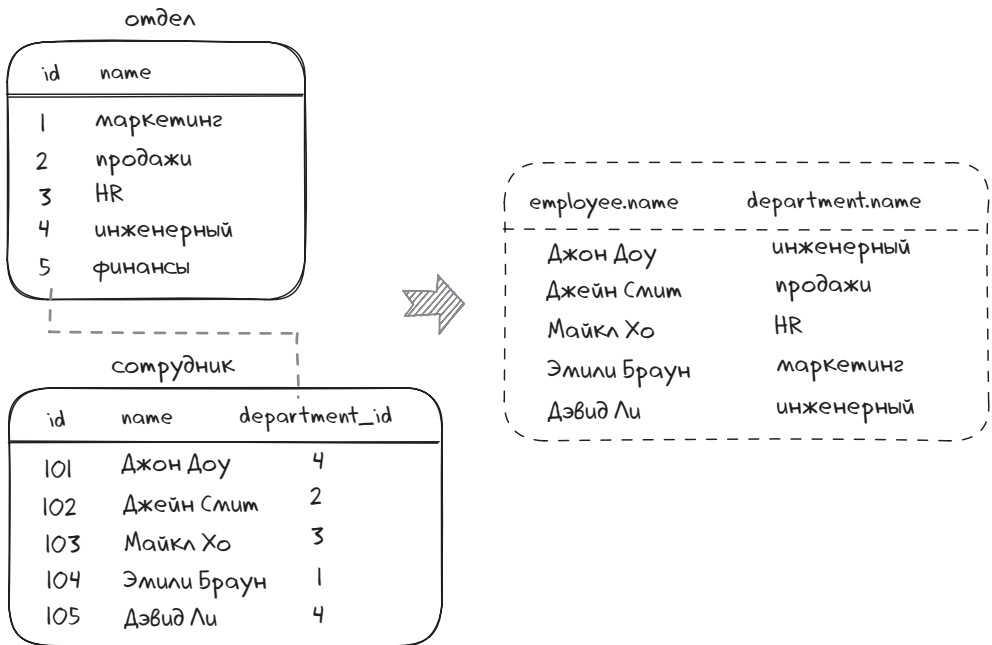
Если конкретнее, JOIN берет две таблицы (коробки) и высыпает все строки (игрушки) на пол, после чего ищет совпадающие значения в общем столбце (соответствия между машинками и дорогами). Если совпадение найдено, оператор заносит данные в новую таблицу (место, где ребенок будет играть).

Третий вопрос: зачем нужна точка в записи типа `product.product_id` и `review.product_id`? В языке SQL точка используется как разделитель частей имени. К примеру, запись `product.product_id` означает столбец `product_id` таблицы `product`, а запись `product.name` — столбец `name` той же таблицы. Точка особенно удобна, когда необходимо указать одноименные столбцы из разных таблиц, как в случае с `product.product_id` и `review.product_id`, не создавая путаницы.

Если в двух таблицах, которые нужно соединить, есть столбцы с одинаковыми именами, то чтобы не запутаться, их можно переименовать при помощи оператора SELECT.

Допустим, у нас есть две таблицы: `employee` (Сотрудник) и `department` (Отдел), которые необходимо соединить, чтобы узнать, в каком отделе числится тот или иной сотрудник.





В обеих таблицах есть столбец name, поэтому в блоке SELECT нужно использовать точки:

```
SELECT employee.name, department.name
FROM employee
JOIN department
ON employee.department_id = department.id;
```

В зависимости от используемой СУБД имена столбцов соединенной таблицы будут такими же, как на рисунке выше (с точками), или будут представлять собой два одинаковых имени (например, name). Поэтому, чтобы не запутаться, можно указать нужные имена при помощи ключевого слова AS:

```
SELECT employees.name AS employee_name,
       departments.name AS department_name
FROM employees
JOIN departments
ON employees.department_id = departments.id;
```

Типы оператора JOIN

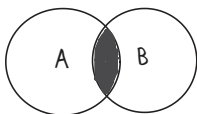
Познакомившись в общем с принципами работы оператора JOIN, поговорим о его разновидностях. Прежде всего, вспомним базовый синтаксис:

```
SELECT столбец1, столбец2, ...
FROM таблица1
JOIN таблица2
ON таблица1.столбец = таблица2.столбец;
```

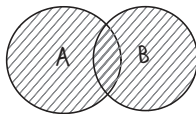
Не нарушая его, заменим JOIN на похожие команды. При этом изменится и результат выполнения запроса:

- INNER JOIN возвращает лишь соответствующие друг другу строки двух таблиц. Фактически команды INNER JOIN и JOIN идентичны.
- LEFT JOIN возвращает все строки таблицы 1 и соответствующие им строки таблицы 2, а если совпадений не находится — строки таблицы 1, заполненные значениями NULL.
- RIGHT JOIN возвращает все строки таблицы 2 и соответствующие им строки таблицы 1, а если совпадений не находится — строки таблицы 2, заполненные значениями NULL.
- FULL OUTER JOIN возвращает все строки из обеих таблиц, включая несовпадающие. Если запись в таблице 1 имеет соответствие в таблице 2, результат будет содержать одну строку с данными из обеих таблиц. А если строке из таблицы 1 не находится соответствия в таблице 2, результат будет содержать строки таблицы 1 со значениями NULL в различающихся столбцах. То же самое, если строке из таблицы 2 не находится соответствия в таблице 1.

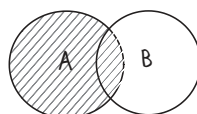
Возможные взаимосвязи между левой и правой таблицами, а также результаты запросов обобщены на следующем рисунке:



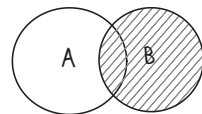
```
SELECT *
FROM A
INNER JOIN B
ON A.A_id = B.A_id
```



```
SELECT *
FROM A
FULL OUTER JOIN B
ON A.A_id = B.A_id
```



```
SELECT *
FROM A
LEFT JOIN B
ON A.A_id = B.A_id
```



```
SELECT *
FROM A
RIGHT JOIN B
ON A.A_id = B.A_id
```

Необходимо отметить, что блоки `LEFT JOIN`, `RIGHT JOIN` и `FULL OUTER JOIN` могут выдавать результаты, содержащие `NULL`, а значит, с ними нужно работать осторожно. Значение `NULL` приводит к ошибкам при вычислениях и сравнениях (к примеру, $10 + \text{NULL} = \text{NULL}$). Допустим, в таблице `product` есть несколько новых товаров без отзывов, а значит, и без записей в таблице `review`. Соединяя две таблицы при помощи `LEFT JOIN`, мы получим `NULL` в соответствующих столбцах результата. Проверим это при помощи запроса `LEFT JOIN`:

```
SELECT name, review_text
FROM product
LEFT JOIN review
ON product.product_id = review.product_id;
```

В ответ получим таблицу, в которой названия товаров совмещены с отзывами о них:

name	review_text
Атомный триммер для волос	Приличный товар...
Атомный триммер для волос	Не рекомендую...
...	...
Мозговой зонд	NULL
Ультразвуковая отвертка	NULL
...	...

Поскольку об атомном триммере для волос написано два отзыва, он упоминается дважды. Если же отзывов нет, как в случае, например, с мозговым зондом, в столбце `review_text` появляется `NULL`.

WHERE или JOIN

Любопытным читателям, возможно, придет на ум идея соединить таблицы при помощи оператора `WHERE`. Как мы узнали из главы 1, он позволяет фильтровать набор данных по указанному параметру. Поскольку в условии `FROM` можно перечислить несколько таблиц, можно сформулировать запрос, сопоставляющий `product` и `review`:

```
SELECT name, review_text
FROM product, review
WHERE product.product_id = review.product_id;
```

При этом не будет ошибок и мы получим тот же результат, что и выше, с оператором JOIN? Да. Однако есть три причины не использовать WHERE вместо JOIN:

- *Читаемость.* JOIN делает назначение запроса более четким и понятным.
- *Удобство обслуживания.* Блок JOIN менее подвержен ошибкам, его проще изменять и отлаживать.
- *Оптимизация.* Оператор WHERE выполняет перекрестное соединение таблиц. С точки зрения оптимизации для большинства СУБД эта операция намного сложнее, чем обычный оператор JOIN.

Перекрестное соединение: дикий танец каждого с каждым

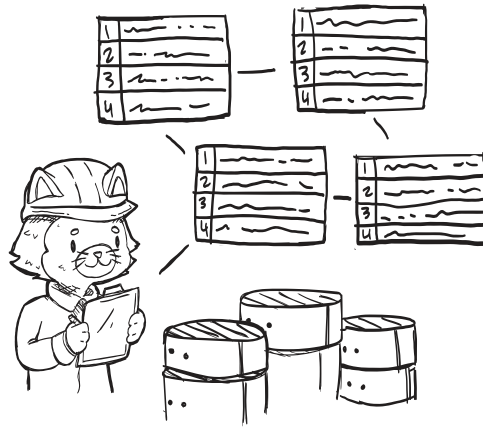
В SQL под *перекрестным соединением* (cross join) понимается операция, при которой каждая из строк одной таблицы соединяется с каждой из строк другой, то есть находятся все возможные сочетания строк из двух таблиц. И это очень большой объем данных.

Перекрестное соединение выполняется всякий раз, когда в блоке FROM указано больше одной таблицы. К примеру, FROM product, review означает, что с каждым из товаров будет сопоставлен каждый отзыв, независимо от того, соотносятся они друг с другом или нет.

Перекрестное соединение требует сканирования обеих таблиц по отдельности, что занимает очень много памяти.

Снова об управлении таблицами и данными

В главе 1 мы говорили об управлении одиночными таблицами и данными, которые в них хранятся. Теперь же речь пойдет о связанных таблицах: распространяя на них полученные ранее знания, мы создадим и заполним данными таблицы, которые уже использовались в этой главе.



Управление связанными таблицами

Рассмотрим, как создаются связанные таблицы, на примере скрипта, который прилагается к этой главе (необходимо выбрать ту его версию, что подходит для используемой вами СУБД, к примеру `mysql_db.sql` для MySQL и MariaDB). Скрипт создает две таблицы — `product` и `review` для базы данных «Фантастической лавки».

Таблица `product` создается так же, как в главе 1:

```
CREATE TABLE product (  
  product_id INT PRIMARY KEY,  
  name TEXT NOT NULL,  
  description TEXT NOT NULL,  
  price DECIMAL(5, 2) NOT NULL,  
  manufacturer TEXT NOT NULL  
);
```

Команда для таблицы `review` выглядит несколько иначе:

```
-- comment: команда для MySQL и MariaDB  
-- comment: команды для других СУБД см. в репозитории  
CREATE TABLE review (  
  review_id INT PRIMARY KEY,  
  product_id INT NOT NULL,  
  review_text TEXT NOT NULL,  
  datetime DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  CONSTRAINT fk_product_review  
    FOREIGN KEY (product_id)  
    REFERENCES product (product_id)  
);
```

Анализируя эту команду, необходимо задаться двумя вопросами:

- Каков базовый синтаксис команды для создания связанных таблиц?
- Каково назначение блока `CONSTRAINT... FOREIGN KEY... REFERENCES...`?

Ответим на первый вопрос. Как вам уже известно, столбец `product_id` является первичным ключом таблицы `product` и внешним в таблице `review`. В подобных парах таблица с первичным ключом называется *родительской* (parent table), а таблица с внешним — *дочерней* (child table).



На примере `product` мы видели, что при создании родительских таблиц используется тот же синтаксис, что и при создании обычных, одиночных таблиц.

Однако в случае дочерней таблицы необходимо указать внешний ключ, ссылающийся на первичный в родительской. Естественно, при этом нужно определить столбцы, типы данных и первичный ключ, как обычно. Базовый синтаксис в этом случае выглядит так:

```
CREATE TABLE имя_дочерней_таблицы (
    столбец1 тип_данных1 [NOT NULL],
    столбец2 тип_данных2 [NOT NULL],
    ...,
    имя_внешнего_ключа тип_данных,
    CONSTRAINT имя_ограничения
        FOREIGN KEY (имя_внешнего_ключа)
        REFERENCES имя_родительской_таблицы (имя_первичного_ключа)
);
```

Как вариант, можно сначала создать две независимые таблицы, а затем добавить в ту из них, что будет дочерней, ограничение по внешнему ключу:

```
-- comment: предполагается, что родительская и
-- comment: дочерняя таблицы уже существуют
ALTER TABLE имя_дочерней_таблицы
  ADD CONSTRAINT имя_ограничения
    FOREIGN KEY (имя_внешнего_ключа)
    REFERENCES имя_родительской_таблицы (имя_первичного_ключа);
```

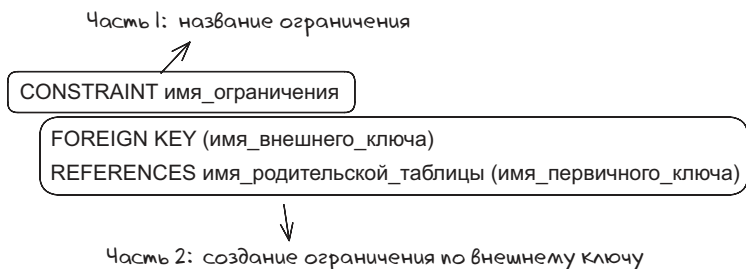
Второй вопрос: каково назначение блока `CONSTRAINT... FOREIGN KEY... REFERENCES...`? В двух словах, он налагает на таблицу ограничение по внешнему ключу, которое соединяет две связанные таблицы. Оно выполняет две функции:

- Гарантирует, что внешний ключ дочерней таблицы ссылается только на существующие значения первичного ключа родительской.
- Гарантирует, что при обновлении либо удалении строк из родительской таблицы согласованность данных в связанных таблицах не нарушается.



Эти две функции, а значит и блок `CONSTRAINT... FOREIGN KEY... REFERENCES...`, обеспечивают *ссылочную целостность* (referential integrity) данных.

Глядя на этот блок, можно сказать, что он состоит из двух частей, как показано ниже:



Во второй части блока (FOREIGN KEY... REFERENCES...) создается ограничение по внешнему ключу, которое обеспечивает ссылочную целостность данных. При этом в первой части блока (CONSTRAINT...) указывается имя этого ограничения. В примере с таблицей review оно называется `fk_product_review`. Вводя ограничение, мы не обязаны давать ему имя. Однако оно может пригодиться в дальнейшем: к примеру, если потребуется изменить или же удалить это ограничение, как это делается следующей командой:

```
-- comment: Команда ALTER TABLE DROP CONSTRAINT
-- comment: не поддерживается в SQLite
ALTER TABLE review
    DROP CONSTRAINT fk_product_review;
```

Имена должны быть у всех. Поэтому если не указать имя в команде, СУБД составит и присвоит его сама, по собственным стандартным правилам. Они не секретны, а значит, если нужно, такое имя можно узнать. Однако правила, а значит и имена, в каждой СУБД свои. Поэтому во избежание лишних хлопот лучше придумывать имена самостоятельно. Все-таки изменение и удаление ограничений — обычное дело при обновлении структуры базы данных.

Управление данными в связанных таблицах

При работе со связанными таблицами синтаксис команд для добавления, обновления и удаления данных ничем не отличается от того, что мы изучали в главе 1. Так, в скрипт из репозитория входят команды, которые загружают данные в обе таблицы: `product` и `review`¹:

¹ Отзыв в таблице review: «Классная штука, всячески рекомендую!». — *Примеч. пер.*


```
-- comment: добавление данных в таблицу product
INSERT INTO product (product_id, name, description, price, manufacturer)
VALUES
(
  1,
  'Atomic Nose Hair Trimmer',
  'Trim your nose hairs ... an atomic clock!',
  19.99,
  'Mad Inventors Inc.'
),
...;
-- comment: добавление данных в таблицу review
INSERT INTO review (review_id, product_id, review_text, datetime)
VALUES
(
  1,
  3,
  'Great product, would definitely recommend!',
  '2022-01-01 12:30:00'
),
...;
```

Единственное отличие, которое появляется при работе со связанными таблицами, — ограничение по внешнему ключу. Оно, как мы ранее говорили, стоит на страже ссылочной целостности данных в этих таблицах:

- Ограничение по внешнему ключу гарантирует, что внешний ключ дочерней таблицы ссылается только на существующие значения первичного ключа родительской.
- Ограничение по внешнему ключу гарантирует, что при обновлении либо удалении строк из родительской таблицы согласованность данных между таблицами не нарушается.

Новые данные, добавляемые в дочернюю таблицу, должны быть согласованы с теми, что уже есть в родительской. Иначе СУБД выдаст ошибку. Допустим, мы пытаемся добавить новый отзыв в таблицу review, но указанное значение product_id (например, 3000) в таблице product отсутствует:

```
INSERT INTO review (review_id, product_id, review_text, datetime)
VALUES (
  1,
  3000,
  'Great product!',
  '2023-05-01 12:30:00'
);
```

При выполнении этой команды СУБД выдаст примерно такое сообщение об ошибке:

```
ERROR 1452 (23000):
Cannot add or update a child row:
a foreign key constraint fails ...1
```

Аналогичная ошибка произойдет, если в результате изменения либо удаления записи из родительской таблицы в дочерней появятся «потерянные» строки. К примеру, попробуем удалить из таблицы `product` данные о товаре, на который в таблице `review` имеется отзыв. К счастью, сработает ограничение по внешнему ключу, а мы получим сообщение об ошибке:

product_id	name	price	manufacturer
1	Атомный триммер ...	19.99	ООО «Безумные инженеры»
2	Селфи-мостер	24.99	ЗАО «Потешные гаджеты»
3	Кошачий наполнитель...	29.99	АО «Абсурдные аксессуары»
...	...	...	...

Удаление строки
с ключом 3

ОШИБКА 1451 (23000):
невозможно удалить
или изменить строку
родительской таблицы:
нарушено ограничение
по внешнему ключу...

review_id	product_id	review_text	date
1	3	Классная штука...	2023-01-01 12: 30: 00
2	5	... лучшая покупка...	2023-01-02 13: 45: 00
3	2	...не рекомендую	2023-01-03 14: 15: 00
...	...	...	...



Наиболее радикальный способ удаления данных — стереть их вместе с родительской таблицей. Если такое произойдет, все строки дочерней

¹ ОШИБКА 1452 (23000): невозможно добавить или изменить строку дочерней таблицы: нарушено ограничение по внешнему ключу...

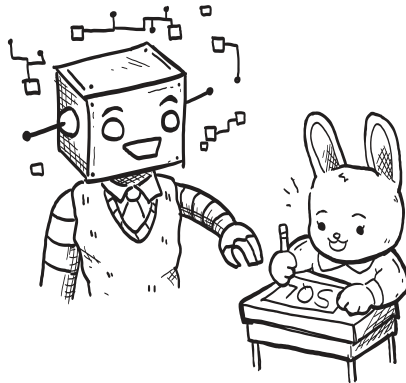
таблицы окажутся «потерянными». Поэтому ограничение снова действует: удаление не допускается, а СУБД выдает ошибку:

```
ERROR: cannot drop table ... because other objects depend  
on it  
DETAIL: constraint... on table...depends on table...1
```

Если вы все еще совершенствуете свой проект базы данных и вам необходимо изменить ее структуру, прочтите главу 1, в которой мы говорили о работе с одиночными таблицами, или же предыдущий раздел, где речь идет об ограничении по внешнему ключу. В крайнем случае придется удалить связанные таблицы вместе со всеми данными. Прежде чем так поступить, сначала сделайте резервную копию, после чего сначала удалите дочернюю таблицу, а затем родительскую. Этот порядок важен, поскольку при его нарушении сработает ограничение и вы получите сообщение об ошибке. Ограничение по внешнему ключу защищает ссылочную целостность данных.

Самостоятельное изучение SQL

В этом разделе поговорим о том, как изучать SQL самостоятельно. Освоение этого языка — долгий путь, однако если, начав читать эту книгу с начала, вы добрались до этой страницы, вы уже знаете все, что нужно для проектирования баз данных. Кроме того, полученные знания — это хорошая основа для дальнейшего совершенствования в этой области.



¹ ОШИБКА: невозможно удалить таблицу..., поскольку от нее зависят другие объекты
ПОДРОБНОСТИ: ограничение... на таблицу... зависит от таблицы...

Мы живем в эпоху бурного развития генеративных моделей искусственного интеллекта, таких как ChatGPT и Google Gemini. Сегодня освоить SQL самостоятельно стало значительно проще, чем еще несколько лет назад. При помощи ChatGPT можно быстро и эффективно получать новые знания, и в этом разделе вы найдете несколько тому примеров. Впрочем, авторы этой книги ни в коем случае не призывают полагаться в изучении SQL только на ChatGPT.

Естественно, многие методы традиционного обучения остались в силе. Для систематических занятий и освоения сложных концепций, таких как индексация и порядок выполнения запросов, все еще нужен качественный учебник. Ответы искусственного интеллекта здесь не помогут. Хороших книг по SQL немало. Вот, например, две из них:

- *Learn SQL in a Month of Lunches* Джеффа Яннуччи (Jeff Iannucci) (<https://www.manning.com/books/learn-sql-in-a-month-of-lunches>)
- *Learning SQL*, 3rd ed. Алана Болье (Alan Beaulieu)¹ (<https://mng.bz/Ge5V>)

С другой стороны, ChatGPT эффективен, когда вам нужен ответ на вопрос или же информация о конкретной команде. Главное преимущество генеративного ИИ в том, что он может предоставлять относительно надежную и своевременную помощь по общим вопросам. До его появления нам приходилось пользоваться поисковиками, то есть круг за кругом оттачивать запросы, после чего фильтровать результаты. Этот процесс, особенно фильтрация, требовал много времени. Благодаря ИИ все стало намного быстрее и проще.

Например, если нужно узнать, как работает агрегатная функция SUM(), можно спросить об этом ChatGPT:

Вопрос: Как использовать агрегатную функцию SUM() в SQL?

В ответ ИИ расскажет о базовом синтаксисе функции, а также приведет несколько примеров, причем объяснит материал намного понятнее, чем некоторые написанные людьми учебники.

¹ Болье А. «Изучаем SQL».

Вот как может ответить ChatGPT:

ChatGPT

В языке SQL функция `SUM()` используется для подсчета суммы значений, хранящихся в столбце таблицы. Вот базовый синтаксис этой функции:

```
SELECT SUM(имя_столбца) FROM имя_таблицы;
```

Пусть, например, у нас есть таблица `orders` (Заказы), в которой имеются столбцы `id` (Идентификатор), `customer_name` (Заказчик), `product_name` (Товар) и `price` (Стоимость). Необходимо определить общую стоимость всех заказов. Используем следующий запрос:

```
SELECT SUM(price) FROM orders;
```

В ответ мы получим одно число, которое будет равно сумме всех значений из столбца `price` таблицы `orders`.

Но если этот ответ для вас слишком сложен, можно спросить ChatGPT по-другому:

Вопрос: Как использовать агрегатную функцию `SUM()` в SQL? Поясни на примерах, как будто ты объясняешь это ребенку.

Вы сразу же заметите, что ChatGPT начнет объяснять, как учитель начальных классов:

ChatGPT

Конечно! Допустим, у нас есть таблица с числами:

id	value
1	2
2	4
3	6
4	8

В каждой строке в столбце `value` (Значение) имеется число. Пусть это будет количество конфет. Тогда значения в столбце `id` — это номера, обозначающие конфеты с разными вкусами.

Как узнать, сколько всего конфет у нас есть? Мы можем посчитать их, перебирая одну за другой, но это долго. Лучше использовать функцию `SUM()` языка SQL, которая сложит все числа в столбце `value`! Для этого нужно написать:

```
SELECT SUM(value) FROM my_table;
```

В ответ на этот запрос компьютер суммирует числа в столбце `value`. В этом примере получится, что у нас ровно 20 конфет!

Еще один пример: возьмем таблицу с ценами:

ID	Price
1	1.99
2	2.99
3	3.99
4	4.99

В каждой строке есть значение `price` — цена за игрушку или конфету. Нам нужно узнать, сколько денег понадобится, чтобы купить все товары из таблицы. Снова напишем запрос на языке SQL, используя функцию `SUM()`:

```
SELECT SUM(price) FROM my_table;
```

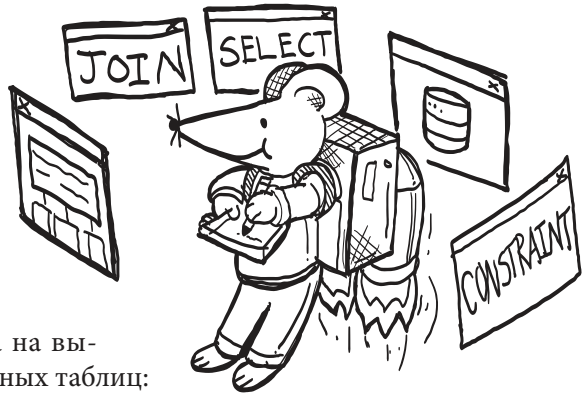
В ответ на него компьютер сложит все числа из столбца `price` и выдаст ответ: 13.96. Именно столько нам нужно денег на покупку!

Одно из самых значительных преимуществ использования генеративного ИИ при изучении SQL — возможность получать относительно надежную и своевременную помощь. Чтобы быстро получить информативный и хорошо структурированный ответ, не нужно быть мастером составления запросов. Кроме того, задавать вопросы ИИ обычно проще, чем пользоваться поисковиком.

Итоги

- Связанными называются таблицы, имеющие общий столбец.
- У таких таблиц не должно быть общих столбцов, кроме первичного и внешнего ключей.
- Базовый синтаксис запроса на выборку данных из двух связанных таблиц:

```
SELECT столбец1, столбец2, ...
FROM таблица1
JOIN таблица2
ON таблица1.столбец = таблица2.столбец;
```



- Для соединения чаще всего используются операторы INNER JOIN, LEFT JOIN, RIGHT JOIN и OUTER JOIN. Оператор INNER JOIN, или, что то же самое, просто JOIN, возвращает только те строки, в которых значения из обеих таблиц совпадают. Результат работы остальных трех операторов включает также строки, у которых нет такого соответствия.
- В общем случае для соединения двух таблиц следует использовать оператор JOIN, а не WHERE.
- В паре связанных таблиц та, у которой общий столбец является первичным ключом, называется родительской. Вторая таблица при этом называется дочерней.
- Родительская таблица создается так же, как и одиночная, не связанная с другими.
- Базовый синтаксис для создания дочерних таблиц следующий:

```
CREATE TABLE имя_дочерней_таблицы (
    столбец1 тип_данных1 [NOT NULL],
    столбец2 тип_данных2 [NOT NULL],
    ...,
    имя_внешнего_ключа тип_данных,
    CONSTRAINT имя_ограничения
        FOREIGN KEY (имя_внешнего_ключа)
        REFERENCES имя_родительской_таблицы (имя_первичного_ключа)
);
```

- Ограничение по внешнему ключу обеспечивает ссылочную целостность данных в связанных таблицах и действует при работе как с самими таблицами, так и с данными в них.
- В общем случае при создании связанных таблиц имеет смысл присвоить ограничению по внешнему ключу определенное имя.
- Генеративные модели искусственного интеллекта, подобные ChatGPT, помогают изучать SQL более эффективно.

Общие сведения о проектировании баз данных

3



В этой главе

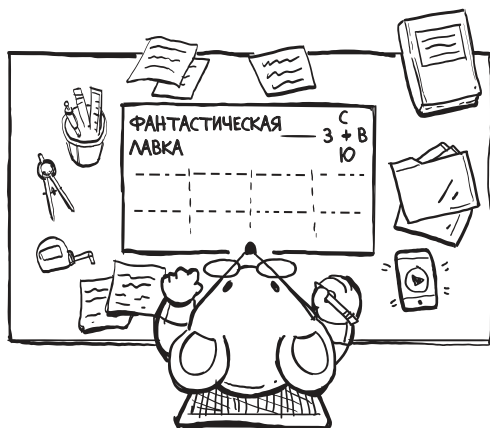
- ✓ Определение цели проектирования
- ✓ Знакомство с общим процессом проектирования
- ✓ Начальный анализ требований к базе данных

Необходимые знания

Эта глава содержит общий обзор принципов проектирования баз данных, то есть, в отличие от прошлых глав, в ней мы не будем рассматривать конкретные скрипты.

Введение

В главах 1 и 2 мы познакомились с основами реляционных баз данных и языка SQL. Начиная с этой главы, мы будем учиться проектированию и приступим к работе над базой данных «Фантастической лавки», которую создадим с нуля. Поэтому в этой главе поговорим о целях проектирования и его основных этапах. Затем займемся анализом требований к БД «Фантастической лавки».



Цели проектирования баз данных

Общая цель проектирования — создание хорошо структурированной и эффективной базы данных с учетом требований пользователей и организаций. Помимо этого, успешное проектирование преследует еще несколько целей:

- согласованность и целостность данных;
- простота применения и обслуживания;
- производительность и оптимизация;
- безопасность данных;
- масштабируемость и гибкость.

В этом разделе мы рассмотрим каждую из них и постараемся понять, к чему нужно стремиться.

Согласованность и целостность данных

Согласованность (consistency) и *целостность* (integrity) — это вопрос выбора подходящих типов данных, ограничений и отношений, которые гарантируют, что:

- между данными в разных таблицах не возникает противоречий;
- избыточность данных сведена к минимуму;
- возникновение аномалий исключено.

В главах 1 и 2 мы рассмотрели примеры плохо спроектированных баз данных, где недостатки структуры привели к избыточности и аномалиям трех типов. Чтобы такого не случилось, данные должны быть согласованы.

Аномалии добавления, обновления и удаления

Аномалия добавления проявляется, когда для добавления новых записей необходимы не связанные с ними данные.

Аномалия обновления проявляется, когда изменение данных приводит к их несогласованности.

Аномалия удаления проявляется, когда удаление данных приводит к непреднамеренной потере информации.

Простота применения и обслуживания

Хорошо спроектированная база данных интуитивно понятна и проста как в применении, так и в обслуживании для всех работающих с ней специалистов, включая разработчиков веб- и мобильных приложений, администраторов и аналитиков.

Как упростить обслуживание и применение базы данных? Есть множество способов. К примеру, соблюдать соглашения об именовании (см. следующий рисунок).

НЕПОСЛЕДОВАТЕЛЬНОЕ ИМЕНОВАНИЕ ИДЕНТИФИКАТОРОВ

PRODUCT 	REVIEW 	USER 	PAYMENT 
<u>Product_id</u> name price manufacturer	<u>reviewId</u> product_id review_text date	<u>User_identifier</u> username email password	<u>paymentKey</u> card/Number esc expiryDate



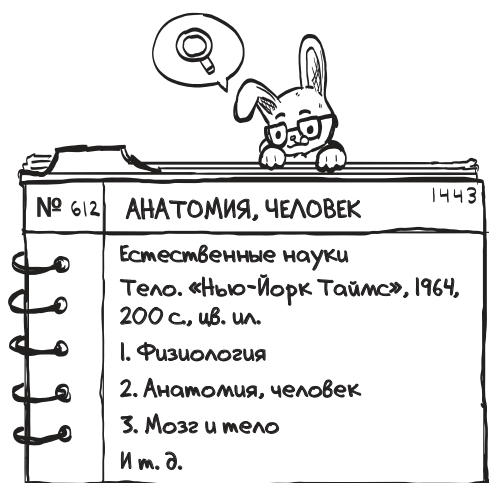
Это не потребует особых усилий от проектировщика, но при этом сэкономит немало времени другим специалистам. Так, разработчик, которому придется гадать, как называется первичный ключ нужной ему таблицы, перебирая `id`, `Id`, `identifier` и другие возможные варианты, вряд ли сочтет базу данных интуитивно понятной. О простоте обслуживания в подобных случаях и говорить не приходится.

Производительность и оптимизация

Хорошо спроектированная база данных оптимизирована с точки зрения производительности запросов и времени отклика. Как результат, снижаются расходы на эксплуатацию, повышается быстродействие работающих с базой приложений, а значит, и уровень комфорта пользователей.

Оптимизировать производительность можно разнообразными способами. Вспомните таблицу `review` из главы 2. Одни клиенты оставляют отзывы, другие — читают их на сайте «Фантастической лавки». При этом людей обычно интересуют самые свежие отзывы, а значит, таблицу часто придется сортировать по времени. Чтобы ускорить эту операцию, можно проиндексировать столбец `date`.

Что такое *индексация* (*indexing*)? Представьте себе, что данные из таблицы `review` хранятся в библиотеке в виде книг. Индексация — это процесс создания каталога, в котором все книги перечислены по алфавиту, причем для каждой указано, на какой полке она стоит. Благодаря этому можно быстро найти нужное издание.



Помните бумажные каталоги?

Безопасность данных

Хорошо спроектированная база данных надежно защищена: несанкционированный доступ, добавление, изменение и удаление данных в ней сильно затруднены. Если же вдруг такое произойдет, критически важные данные несложно будет восстановить.

Меры обеспечения безопасности данных разнообразны. Прежде всего не следует хранить информацию о платежных инструментах в виде простого текста. Это опасно: проникнув в базу данных, взломщик сразу получит номера кредитных карт всех покупателей. Поэтому нужно применять шифрование — универсальный способ, защищающий конфиденциальные данные даже в таких неблагоприятных сценариях.



Масштабируемость и гибкость

Хорошо спроектированная база данных должна быть готова к изменению требований и росту без снижения производительности.

Для этого можно использовать разные средства, к примеру, еще на этапе проектирования строить таблицы так, чтобы они были как мож-

но меньше по объему, а поиск в них выполнялся как можно быстрее. Кроме того, полезно использовать механизмы *кэширования* для данных, которые необходимы особенно часто, как, например, сведения из таблицы `product` базы данных «Фантастической лавки». Кэширование помещает такие данные в быстродействующую память (например, в оперативную), и это значительно повышает производительность и сокращает время отклика, особенно когда информации много. Среди популярных систем кэширования можно назвать Redis (<https://redis.io>) и Memcached (<https://www.memcached.org>).



Общий обзор процесса проектирования

В этом разделе мы рассмотрим процесс проектирования в целом и поговорим о его основных этапах. Существует несколько общепринятых подходов к проектированию. Например, при каскадном, или водопадном (waterfall), подходе работа осуществляется поэтапно, а переход на следующий шаг возможен только по завершении предыдущего. Agile-подход, напротив, достаточно гибок и допускает корректировки уже полученных результатов прямо по ходу работы. Однако независимо от того, какой из подходов выбран, в процессе проектирования можно выделить три основных этапа:

- сбор требований;
- анализ и составление проекта;
- реализация/интеграция и тестирование.



Сбор требований

Это сбор информации о будущей базе данных. Для этого существует несколько способов: беседы с теми, кто будет работать с новой базой данных, изучение уже существующих баз (если таковые имеются) и выявление прочих особенностей управления данными.

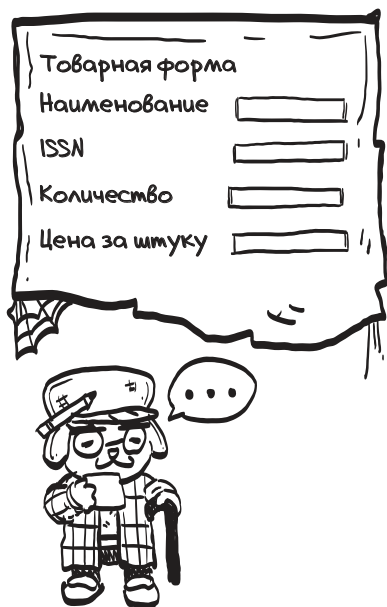
Чтобы поговорить со всеми, кто будет использовать базу данных или работать с ней, необходимо организовывать встречи и задавать правильные вопросы. При этом нашими собеседниками будут люди из разных групп: к примеру, в «Фантастической лавке» нам предстоит расспросить владельцев и менеджеров, будущих администраторов базы данных, а также разработчиков веб- и мобильных приложений. И каждый из них должен рассказать, какие данные ему нужны.



Если у заказчиков уже есть приложение и база данных, необходимо внимательно их изучить. Успех проекта во многом зависит от того, как хорошо мы поймем, чем старая версия не устраивает пользователей.

У «Фантастической лавки» база данных уже есть. Изучив ее и расспросив заинтересованных лиц, приходим к выводу: на данный момент приложение не отслеживает инвентарные номера товаров, находящихся на складе. Как следствие, иногда клиенты заказывают что-то, чего в наличии уже нет. Поэтому разработчики веб-приложения хотели бы добавить в него оповещения о том, что товар заканчивается.

Не менее важно добавить функциональность, связанную с управлением данными. Менеджер по закупкам берет информацию о товарных остатках из электронных таблиц и бумажных форм. Поэтому нужно внимательно изучить и то и другое, а также все действия, которые выполняет менеджер.

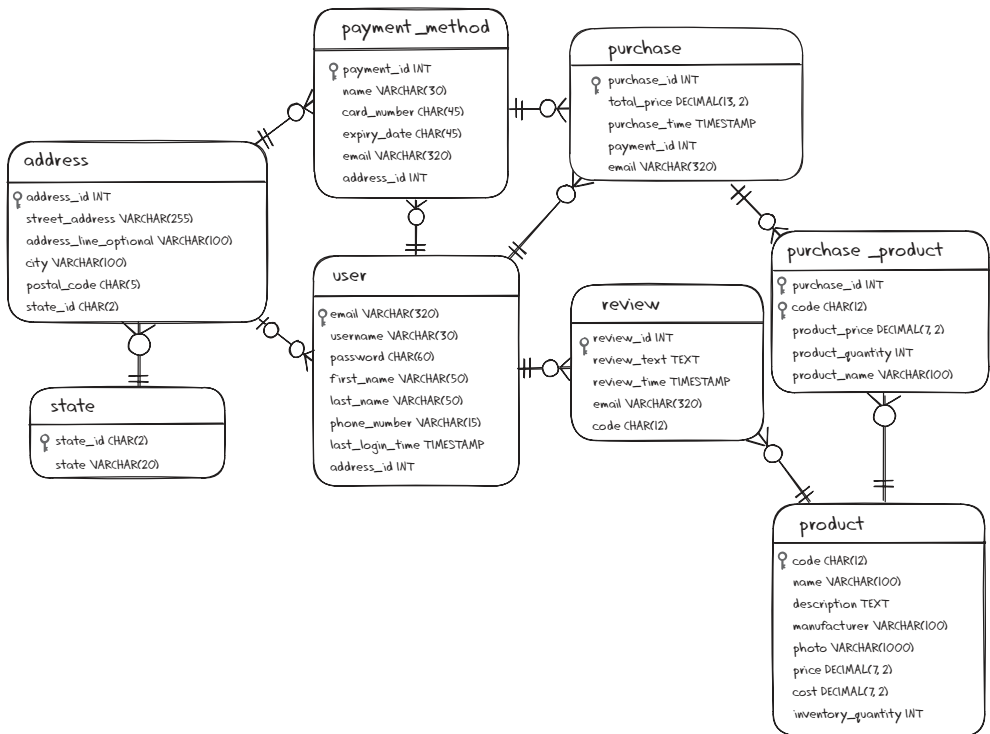


Анализ и составление проекта

На этапе анализа и составления проекта следует тщательно обдумать все требования, после чего составить четкий план того, как будет устроена и работать новая база данных. Итогом этой работы станет

подробный «чертеж» новой БД. Среди основных задач, которые решаются при этом, — моделирование данных и нормализация.

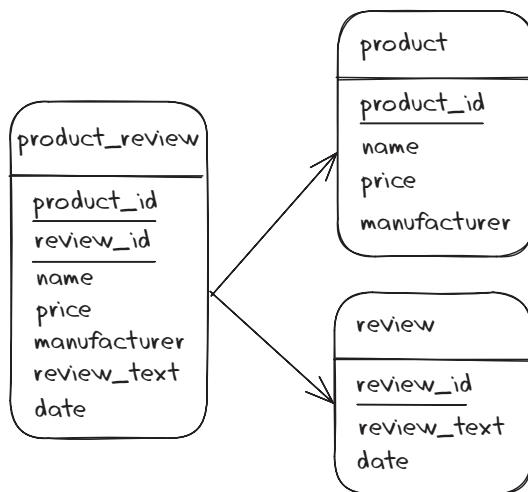
Цель *моделирования данных* (data modeling) — создание концептуального проекта, который объясняет, как будут связаны друг с другом и взаимодействовать разные части базы данных. Обычно для этой цели строятся *диаграммы сущностей и связей* (entity-relationship diagram), сокращенно — *ER-диаграммы*. К примеру, для «Фантастической лавки» ER-диаграмма может быть такой:



Многое из показанного на ней нам пока еще непонятно. Мы будем подробно обсуждать такие диаграммы и моделирование данных в главе 5.

ER-диаграммы — наш главный инструмент *нормализации* (normalization) — процесса, который позволяет свести к минимуму избыточность за счет разделения больших сущностей на более компактные, логичные блоки и выделения их в отдельные таблицы. К примеру, кто-то предложил хранить данные о товарах и отзывах в общей таблице

product_review (Товары и отзывы). Но как мы видели в главе 2, такой подход, как правило, приводит к избыточности и аномалиям. Поэтому мы разобьем эту таблицу на две: product и review.



О том, как проходит нормализация и какие таблицы нуждаются в ней, будет рассказано в главе 6.

Реализация/интеграция и тестирование

На этапе реализации/интеграции и тестирования мы создаем базу данных по «чертежам», разработанным при анализе и проектировании, а затем проверяем ее работу. Если используется водопадный подход, вся база данных строится за один прием, а если Agile, то работа выполняется постепенно, с одновременной интеграцией уже готовых таблиц. После этого база данных проходит тестирование, позволяющее убедиться, что все работает правильно и соответствует заданным требованиям и ожиданиям.

В ходе реализации создаются таблицы, которые состоят из столбцов нужного типа, устанавливаются связи между таблицами, применяются ограничения и правила, сформулированные при проектировании. На этом же этапе строятся индексы для повышения производительности. В главах 1 и 2 мы уже изучили, как создаются одиночные и связанные таблицы, и эти знания будут очень полезны при выполнении перечисленных задач.

После реализации готовую базу данных необходимо протестировать. Обычно при этом проверяются как минимум три качества:

- *Функциональность.* Следует убедиться, что база данных правильно выполняет поставленные задачи, в том числе создает, обновляет и удаляет записи.
- *Производительность.* Необходимо определить, как база данных будет справляться с большими объемами информации, а также интенсивной нагрузкой.
- *Безопасность.* Важно проверить достаточность принятых мер защиты конфиденциальных данных, включая данные банковских карт и пароли.

Естественно, что на этом этапе необходимо найти и устранить допущенные ошибки.

Более подробно обо всем этом мы будем говорить в главах 6 и 7.

Основные этапы проектирования баз данных

Итак, мы рассмотрели общий процесс проектирования и можем приступить к обсуждению его отдельных этапов. Мы будем продолжать работу над базой данных «Фантастической лавки», благодаря чему получим ценный опыт и знания, а также коснемся нескольких важных вопросов, которые можно упустить, если не отработать новые навыки на практике.

А в этом разделе мы разберем первый этап проектирования — сбор требований. Поскольку действия, которые нужно на нем выполнять, относятся не столько к науке, сколько к искусству, справиться с этой работой гораздо легче, если прислушиваться к советам экспертов и опытных специалистов.

Цели базы данных

Собирая сведения о будущей базе данных, необходимо ответить на крайне важный вопрос: каковы ее цели? Любая база создается для выполнения определенных задач, к примеру проведения финансовых операций или хранения информации о компании. Какими бы ни были

эти задачи, понятно, что база данных будет нужна для принятия важных решений.

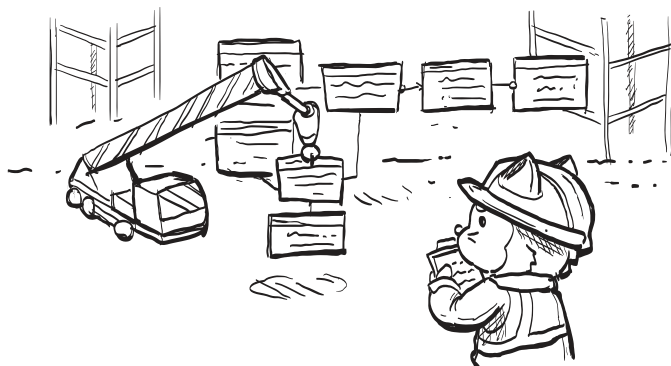
Порой на то, чтобы полностью осознать поставленные цели, уходит немало времени: больше, чем изначально планировалось. И нужно готовиться к такому сценарию. Чтобы понять, каковы цели «Фантастической лавки», следует пообщаться с ее владельцами, менеджерами и сотрудниками, а также разработчиками приложений. По результатам этой работы мы сделаем следующий вывод:

База данных нужна «Фантастической лавке», чтобы хранить информацию о товарах, включая отзывы покупателей и складские запасы, о клиентах, включая их финансовые данные и приобретенные товары, а также о совершенных сделках.

Существующая база данных и другие вопросы управления информацией

Иногда есть возможность изучить уже существующую базу данных. И хотя ее анализ может многое сообщить о необходимых вам свойствах и особенностях, скорее всего, в ее структуре есть недостатки. Поэтому не нужно впадать в искушение и строить новый проект на основе старого. Не забывайте: заказчики обратились к нам за новой базой, а не за обновлением того, что у них уже есть. Кроме того, часто бывает, что адаптация имеющейся базы данных к изменившимся условиям требует не меньше времени, чем проектирование новой с нуля.

Как определить насущные потребности? Разговаривая с людьми. Как разобраться в том, чем текущее состояние не соответствует этим потребностям? Все так же: разговаривая с людьми и задавая вопросы.



Расспрашивая людей об их работе, мы можем узнать много нового о том, какими хитроумными способами они применяют текстовые процессоры, электронные таблицы и бумажные бланки для сбора и обработки информации. Но если эти задачи планируется возложить на новую базу данных, нам предстоит сделать как минимум две вещи:

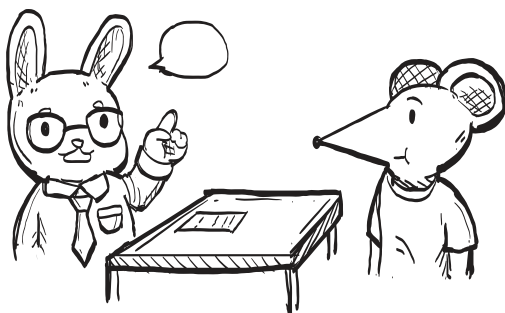
- Узнать у профильных специалистов, как они работают с данными. Необходимо выяснить все этапы этой работы, а поскольку описания обычно недостаточно, — разобрать несколько реальных примеров.
- По возможности попросить предоставить образцы данных. На следующем этапе проектирования они помогут проверить свойства и выбранные типы данных.

Рассмотрим пример. Менеджер по закупкам «Фантастической лавки» хранит все сведения о складских запасах в электронной таблице, на смену которой должна прийти новая база данных. В ходе общения с ним и его коллегами по отделу нам нужно понять, как происходит весь процесс закупки товаров: оформление заказов, ввод данных о новых товарах, обновление и удаление записей о тех, что уже поступили на склад. Эти этапы могут быть достаточно сложными, поэтому по возможности нужно просить собеседников наглядно показывать, что и как происходит. Не менее важно получить у них копию реальных данных (в идеале, конечно же, обезличенную). Такие данные помогут выявить проблемы, упущенные из виду в ходе бесед и демонстраций, в частности уточнить набор свойств и типы данных.

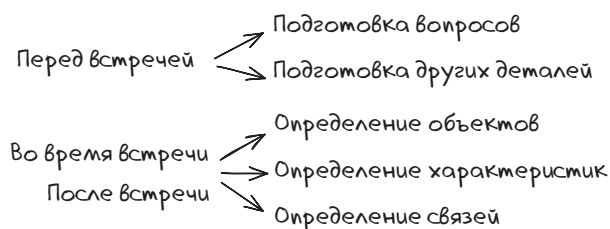
	A	B	C	D	F	G
1	Наименование товара	Производитель	Описание	Закупочная стоимость	Количество	Розничная цена
2	Нейроулучшатель	CyberDyne Corp	Мозговой имплант...	\$499.99	10	\$799.99
3	Квантовый генератор	HyperTech Labs	Революционный...	\$999.99	2	\$2999.99
4	Плазмобластер	NovaArmory	Портативный...	\$299.99	15	\$599.99
5	Наногель	BioSolutions Inc	Передовой наногель...	\$79.99	50	\$149.99
...	...	...	...	...	...	...

Встречи с людьми

Встречи — самая важная часть этапа сбора требований. Во время и после них мы получаем знания об объектах, их характеристиках и связях друг с другом — о том, что будет основой для проектирования на следующем этапе.



Объекты — это люди, места, предметы, события и многое другое, *характеристики* — свойства таких объектов. А что такое связи между объектами, должно быть ясно из самого слова.



Подготовка к встрече

Перед разговором со специалистами необходимо тщательно продумать его план и все детали, с ним связанные. Какие вопросы следует задать? Все зависит и от проекта, и от того, кем будут наши собеседники. Для простоты можно разбить их на несколько групп. Обычно (хотя все, конечно, зависит от обстоятельств) их три: стейкхолдеры, эксперты в предметных областях и ИТ-специалисты.



«Фантастическая лавка» — обычная компания. У нее два владельца и один менеджер, два минотавра, которые занимаются вводом данных и обслуживанием клиентов, а также три эльфа — разработчика приложений. Ниже приведены примерные списки вопросов, которые следует задать представителям этих групп:

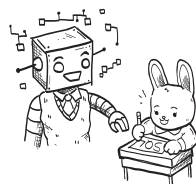
- *Стейкхолдеры.* Какова основная цель базы данных, какие специальные задачи планируется решать с ее помощью? Какие основные функции, включая генерацию отчетов и анализ информации, должна поддерживать база данных?
- *Эксперты в предметных областях.* Как сейчас осуществляется управление данными? С какими трудностями и ограничениями вы сталкиваетесь? Можете ли вы показать, как вы работаете с данными? Какие данные наиболее важны для вашей работы?
- *ИТ-специалисты.* Какие основные задачи и действия вы выполняете при организации хранения и извлечения данных? Какие отчеты обычно приходится генерировать на основе данных? Какие сведения содержатся в этих отчетах? Производите ли вы какие-то специфические расчеты на основе данных?

Попробуйте самостоятельно подготовить и другие вопросы. Из чего при этом следует исходить? Хороший вопрос поможет собрать важные сведения о данных, которые будут храниться в базе, и ограничениях, которые должны быть на них наложены. Впрочем, если не поджимает время, лучше задать неудачный вопрос, чем не спросить ничего. Всем новичкам в этом деле рекомендуем обратиться к ChatGPT: пусть он подготовит для вас примеры вопросов.

Промпт: Я проектирую базу данных и должен встретиться с руководством компании-заказчика. Какие вопросы ему обычно задают в таких случаях?

Помимо вопросов стоит подумать и о других деталях. Вот, например, несколько самых важных из них:

- Как записывать информацию в ходе встречи? В блокнот или на диктофон? Если второе, нужно ли просить разрешение на запись?
- Где будет проходить встреча?
- Стоит ли приглашать на встречу людей из нескольких групп? Какие группы лучше не совмещать? Какое максимальное количество участников допускается на встрече?



Попробуйте задать себе и другие подобные вопросы, учитывая особенности проекта и будущих собеседников. Главное — подготовить все необходимое, чтобы встречи прошли продуктивно и позволили получить все нужные сведения.

Определение объектов, характеристик и связей

Во время и после встреч нам предстоит определить объекты, их характеристики и взаимосвязи. Вы спросите, зачем? На следующем этапе проектирования мы будем сопоставлять эти объекты и характеристики с сущностями, их свойствами и взаимосвязями между сущностями.

Чтобы определить объекты и характеристики во время встречи или анализа собранной информации, нужно искать в полученных ответах имена существительные. Но как отличить объекты от характеристик? Обычно если два слова можно связать в предложение со словами *есть* или *имеет*, то первое из них будет объектом, а второе — его характеристикой.

Возьмем, к примеру, слова «пользователь» и «пароль». Мы можем сказать: у пользователя есть пароль. Стало быть, пользователь — это объект, а пароль — его характеристика.

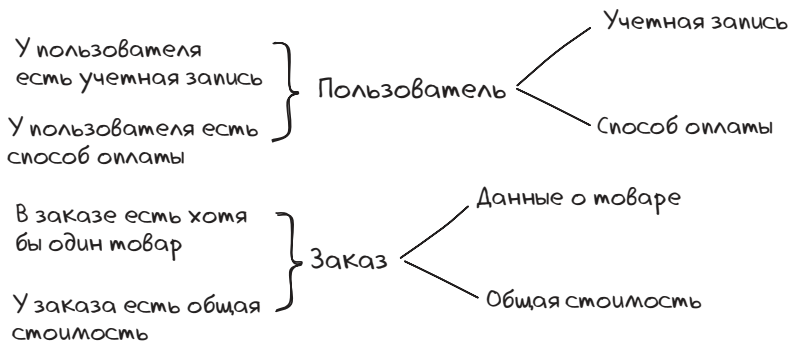
Похожие умозаключения можно делать еще во время встречи. К примеру, мы можем задать ИТ-специалисту из «Фантастической лавки» такой вопрос:

В: Какие действия вы в основном выполняете при организации хранения и извлечения данных?

Наш собеседник ответит примерно так (все объекты подчеркнуты):

О: Как разработчик я отвечаю за создание и поддержку системы оплаты заказов в интернет-магазине. Когда пользователь делает заказ, эта система должна найти в базе данных учетную запись пользователя и указанный им способ оплаты, сформировать заказ со всеми данными о выбранных товарах, вычислить общую стоимость, списать средства выбранным способом оплаты и сгенерировать счет.

Очень хороший ответ: наш собеседник сообщил немало полезной информации. Выделив все существительные и выстроив их в связки со словом «есть», мы можем определить такие объекты и характеристики:



Как можно заметить, из этого ответа нам стали известны четыре объекта: пользователь, заказ, счет и товар. У пользователя и заказа по две характеристики: учетная запись и способ оплаты у первого, данные о товаре и общая стоимость у второго. Счет и товар характеристик не имеют.

Чтобы составить полный список характеристик объекта, чаще всего нужны дополнительные вопросы. К примеру, рассмотрим такой диалог:

В: Какие данные содержит оформленный заказ?

О: Да все как в других магазинах. Цены и количество товаров, которые заказал покупатель. А что еще надо? Заказ — это несложно.

В: А общая стоимость? Она указывается в заказе?

О: Пожалуй, да. Мы вычисляем общую стоимость исходя из цен и количества купленных товаров.

В: А дата и время поступления заказа? Эти данные нужны?

О: Да, разумеется. Это очень важные сведения.

Ответы «как в других магазинах» и «пожалуй, да» довольно расплывчаты и часто требуют дополнительных или уточняющих вопросов. К счастью, на них собеседник ответил вполне однозначно. С учетом полученных сведений мы можем обновить характеристики заказа:



Составив предварительный список объектов и характеристик и завершая разговор о них, не забываем попросить у собеседника образцы данных. К примеру, так:

В: Можете ли вы привести примеры данных о товарах, заказах, счетах и пользователях? Любые будут полезны.

Настало время перейти к взаимосвязям между объектами. Необходимо рассмотреть все возможные пары. К примеру, спросим у собеседника о том, как связаны пользователи и заказы:

В: Как связаны пользователи и заказы? Может ли пользователь сделать несколько заказов? Могут ли несколько пользователей совместно сделать один заказ?

О: Пользователь, разумеется, может оформить сколько угодно заказов. А вот совместных заказов наша система не допускает.

В: Как связаны заказы и товары? Можно ли сделать заказ на несколько товаров? Может ли товар входить в несколько заказов?

О: В заказе может быть несколько товаров. И наоборот: на товар может поступить несколько заказов.

Не обязательно анализировать ответы прямо во время разговора. Главное, все тщательно записать.

Итак, мы рассмотрели проведение бесед с людьми. Опросив все группы представителей «Фантастической лавки», мы составили полный список объектов и их характеристик:

пользователь	товар	способ оплаты
* имя пользователя	* артикул	* название
* электронная почта	* наименование	* номер кредитной карты
* пароль	* описание	* срок действия
* имя	* производитель	* адрес для отправки счета
* фамилия	* фотография	
* телефон	* розничная цена	заказ
* адрес	* закупочная стоимость	* общая стоимость
отзыв	* количество на складе	* цена товара
* текст отзыва		* количество
* дата и время		* дата и время

Кроме того, мы выяснили, что объекты связаны друг с другом следующими правилами:

- Пользователь может сделать несколько заказов. Заказ может быть сделан только одним пользователем.
- Пользователь может оставить отзывы на несколько товаров (при условии, что он их купил). Отзывы на один товар могут оставить несколько пользователей.
- Пользователь может выбирать несколько способов оплаты. Способ оплаты может применяться только одним пользователем.
- В заказ может входить несколько товаров. Товар может входить в несколько заказов.

Имея на руках эти данные, можем перейти на следующий этап проектирования базы данных «Фантастической лавки».

Итоги

- Общая цель проектирования — создание хорошо структурированной и эффективной базы данных.
- Основными признаками эффективной базы данных являются: согласованность и целостность данных, простота применения и обслуживания, производительность и оптимизация, безопасность данных, масштабируемость и гибкость.
- При любом подходе к проектированию этот процесс можно разбить на три основных этапа: сбор требований, анализ и составление проекта, реализация/интеграция и тестирование.
- На этапе сбора требований осуществляется сбор информации о будущей базе данных. Для этого можно беседовать с людьми, которые будут ее использовать, изучать уже существующие базы данных, а также другие важные вопросы, связанные с управлением информацией.
- На этапе анализа и составления проекта создается четкий план, описывающий структуру и функциональность базы данных с учетом установленных требований.
- На этапе реализации/интеграции и тестирования на основе разработанного плана создается сама база данных, после чего выполняется ее проверка на соответствие требованиям.
- Встречи с людьми — важнейшая часть этапа сбора требований. Необходимо тщательно готовиться к встречам, планировать, какие вопросы следует задать. Во время встречи или после нее необходимо составить список объектов, а также их характеристик и взаимосвязей для будущей базы данных.



Часть 2

Навыки проектирования баз данных



Итак, мы приступаем к самой сути проектирования баз данных! В этой части мы выйдем за рамки основ и погрузимся в искусство и науку создания надежных, эффективных и масштабируемых баз данных. Именно эти главы научат вас смотреть на данные не только с позиций удобства их хранения, но и с учетом производительности и надежности работы приложений.

В главе 4 речь пойдет о сущностях и свойствах — кирпичиках, из которых строится база данных. Мы будем говорить о том, как сопоставлять с ними реальные объекты и понятия, а также о том, как правильно подобрать типы данных.

В главе 5 займемся связями между сущностями и узнаем, как на основе набора взаимосвязанных объектов реального мира создать логичную, хорошо структурированную базу данных.

В главе 6 обсудим нормализацию и реализацию баз данных: научимся корректировать и улучшать черновой вариант проекта, а также переводить заверченный проект на язык SQL, тем самым устраняя разрыв между теорией и практикой.

В главе 7 затронем проблемы безопасности и оптимизации: вооружимся лучшими стратегиями защиты конфиденциальных данных, а также повысим производительность базы путем индексации и денормализации.

К концу этой части вы получите навыки проектирования, достаточные для решения реальных задач. Неважно, насколько большую систему вы проектируете: вы сможете уверенно и точно работать и с небольшими проектами, и со сложными приложениями уровня предприятия.

4 | Сущности и свойства



В этой главе

- ✓ Этап составления проекта и анализа
- ✓ Ключи и основные типы данных
- ✓ Проектирование первых сущностей, определение их свойств и первичных ключей, выбор типов данных

Необходимые знания

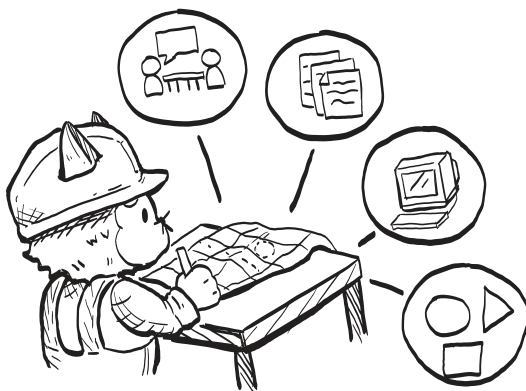
Проект рассматриваемой в этой главе базы данных (точнее, пока лишь описания сущностей и свойств), реализованный при помощи часто применяемых на практике инструментов dbdiagram.io и MySQL Workbench, можно найти в репозитории GitHub (<https://github.com/Neo-Hao/grokking-relational-database-design>). Чтобы загрузить проект в выбранный инструмент, перейдите в папку chapter_04 и следуйте инструкциям из файла README.md.

Используемые типы данных поддерживаются большинством СУБД, в том числе MySQL, MariaDB и PostgreSQL. Пользователям других систем, например SQL Server или SQLite, возможно, придется немного скорректировать проект: заменить некоторые из типов данных на их эквиваленты. Все нужные для этого сведения приведены в том же файле с инструкциями.

Введение

В прошлой главе мы рассмотрели процесс проектирования базы данных в целом, а также его первый этап — сбор требований — и подготовились к работе над базой данных «Фантастической лавки».

Эта глава посвящена следующему этапу — составлению проекта и анализу, который начинается с моделирования данных и построения ER-диаграмм. Продолжая работу над нашим примером, мы будем подробно говорить о сущностях, свойствах, ключах и типах данных.



Сущности и свойства

В этом разделе мы займемся преобразованием объектов и характеристик, сведения о которых мы собрали в ходе встреч с представителями «Фантастической лавки», в набор сущностей и свойств. Вот эти объекты:

пользователь

- * имя пользователя
- * электронная почта
- * пароль
- * имя
- * фамилия
- * телефон
- * адрес
- отзыв
- * текст отзыва
- * дата и время

товар

- * артикул
- * наименование
- * описание
- * производитель
- * фотография
- * розничная цена
- * закупочная стоимость
- * количество на складе

способ оплаты

- * название
- * номер кредитной карты
- * срок действия
- * адрес для отправки счета

заказ

- * общая стоимость
- * цена товара
- * количество
- * дата и время

Разговор о сущностях и свойствах мы начали еще в главе 1. Настало время вновь вспомнить о них. *Сущность* — это объект или понятие, которые описываются набором свойств. Кажется бы, объекты и характеристики — это почти готовые сущности и свойства, и преобразование почти не требует изменений: *объект* — это сущность, его *характеристики* — ее свойства. Однако следует обратить внимание на то, какие имена использовать.

Прежде всего, нужно определиться с числом: единственное или множественное. В этой книге мы будем использовать единственное число.

Единственное или множественное: добавлять ли s¹

Первую СУБД создал Эдгар Кодд (Edgar Codd) из IBM. Придумывая имена для сущностей в своих базах данных, он брал существительные в единственном числе (например, *employee* — работник). Другие разработчики последовали примеру Кодда.

Эта традиция пришла в базы данных из объектно-ориентированного программирования (ООП), где класс, описывающий сущность, содержит все объекты этого класса. И то же самое можно сказать о таблице, которая содержит записи об объектах одной и той же сущности.

С другой стороны, во множественном числе имена таблиц выглядят более естественно, ведь если таблица содержит записи о нескольких работниках, уместнее назвать ее *employees* — работники. Этот подход также допустим, однако важно применять его последовательно. Иначе разработчики, путаясь в формах слов, будут совершать ошибки при написании запросов.

Далее необходимо определить соглашения об именовании и в дальнейшем строго их придерживаться. Это уменьшит количество опечаток и других ошибок при составлении запросов и обслуживании базы данных.

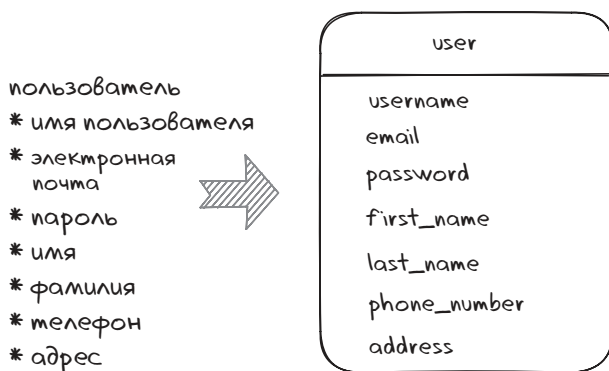
¹ В английском языке множественное число в большинстве случаев образуется добавлением буквы s к форме единственного числа. — *Примеч. пер.*

Наибольшее распространение получили следующие соглашения:

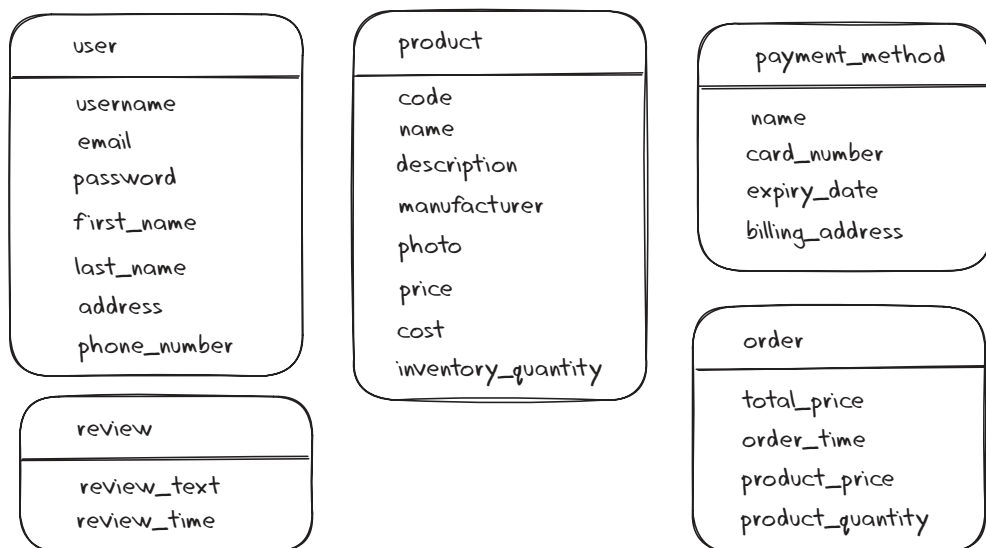
- змеиный регистр (`first_name`)
- верблюжий регистр (`firstName`)
- паскалевский регистр (`FirstName`)
- верхний регистр (`FIRST_NAME`)
- венгерская нотация, при которой в начале имени сокращенно указывается тип данных (`strFirstName`)
- семантическая нотация, при которой в имени указывается назначение переменной (`customerName`)

Особой популярностью пользуются змеиный и верблюжий регистры, паскалевский немного уступает им. Выбор обычно определяется личными предпочтениями разработчика или же требованиями к программному обеспечению. В этой книге мы будем использовать змеиный регистр.

Приняв решение по этим двум вопросам, несложно сопоставить объекты и характеристики с будущими сущностями и свойствами. К примеру, объект `user` (Пользователь) можно соотнести с одноименной сущностью и ее свойствами следующим образом:



Точно так же преобразуются и другие объекты, сведения о которых будут храниться в базе данных «Фантастической лавки»:

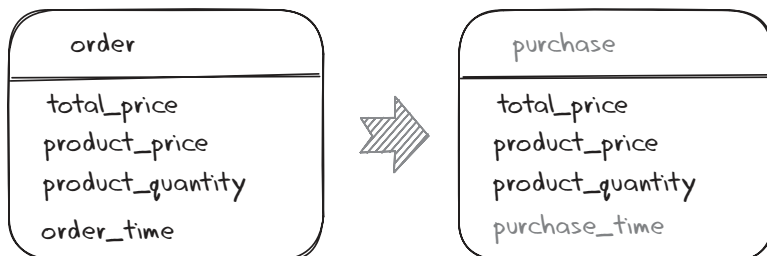


При выборе имени необходимо принять во внимание два важных требования:

- *Длина имен не должна превышать допустимую.* Во многих СУБД действует ограничение на длину имен. К примеру, в MariaDB в имени столбца должно быть не больше 64 символов. Поэтому чрезмерно длинные имена придется сократить.
- *В качестве имен нельзя использовать ключевые слова SQL.* Такие имена обязательно приведут к проблемам, прежде всего к синтаксическим ошибкам в запросах. К примеру, если вы назовете таблицу словом SELECT, СУБД не сможет отличить ее имя от соответствующего ключевого слова. Поэтому все имена, совпадающие с ключевыми словами, следует заменить.

Список ключевых слов у каждой СУБД свой. Поэтому в случае сомнений нужно обратиться к документации, к примеру <https://mng.bz/zZ6r> для MySQL. Однако есть ключевые слова, например SELECT, ORDER, INSERT, GROUP и JOIN, которые зарезервированы во всех СУБД. Как можно заметить, имя одной из наших сущностей — order (Заказ) —

имеется в этом списке. Поэтому мы выберем другое слово: purchase (Покупка)¹:



Зарезервированные ключевые слова SQL

Зарезервированные ключевые слова SQL имеют конкретные значения и функции: они нужны, чтобы определять данные и работать с ними. У каждой СУБД собственный список ключевых слов, но некоторые, самые распространенные из них, для всех систем одинаковы.

Возможно, самый простой способ узнать эти ключевые слова — задать вопрос генеративному ИИ, к примеру ChatGPT:

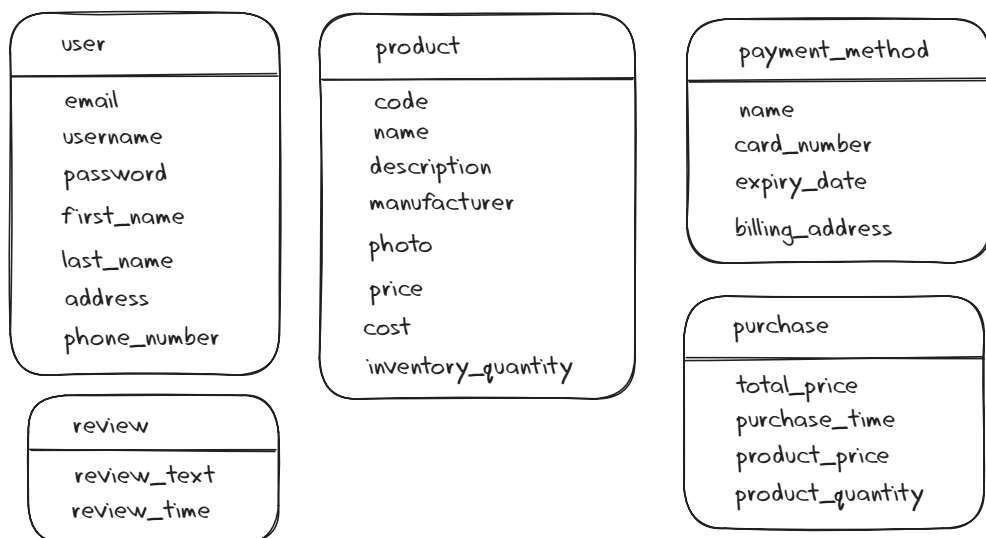
- Какие ключевые слова зарезервированы в языке SQL?

Если же нужен список ключевых слов конкретной СУБД, нужно добавить в запрос ее название. Например, если вас интересует MySQL, можно спросить так:

- Какие ключевые слова зарезервированы в MySQL?
- Какие зарезервированные ключевые слова указаны в документации по MySQL?

По завершении этих подготовительных действий мы получаем следующий набор сущностей для базы данных «Фантастической лавки»:

¹ Авторы здесь не упоминают о том, что user — также зарезервированное слово SQL. Об этом будет сказано в главе 7. — *Примеч. пер.*



Ключи

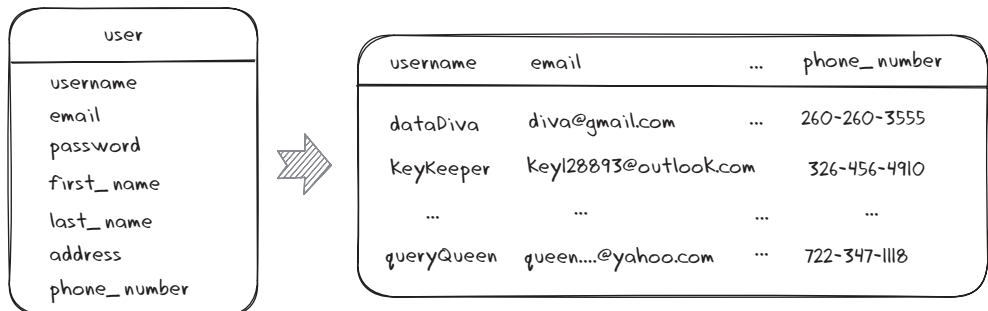
Теперь, когда вместо объектов и характеристик у нас появились сущности и свойства, можно определить их первичные ключи, с которыми мы познакомились еще в главе 1. *Первичный ключ* — это одно или несколько свойств, которые идентифицируют записи в таблице. Каждое значение первичного ключа уникально. Таблица может иметь только один первичный ключ.

Для чего сущности (набор которых в конечном счете представляет собой таблицу базы данных) нужен первичный ключ? Он однозначно идентифицирует каждую запись, что исключает дублирование данных, а также устанавливает связь с другими таблицами. Поэтому выбор первичных ключей — один из важнейших вопросов при проектировании базы данных.

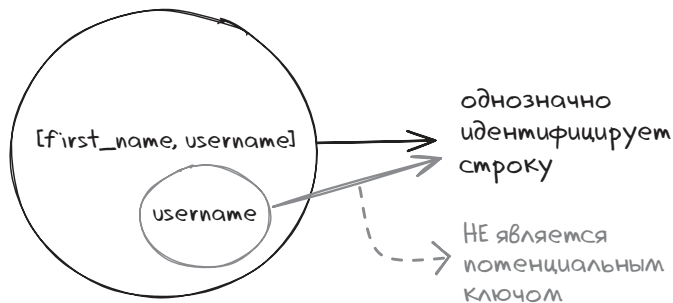
Принцип действия очень прост: нужно взять лучший из *потенциальных ключей* (candidate key), а если их нет, ввести для сущности новое свойство, которое и станет первичным ключом. В этом разделе мы обсудим потенциальные ключи, а также рассмотрим этот принцип на двух примерах.

Начнем с сущности `user`, которая имеет семь свойств. Сопоставляя примеры данных, полученные на этапе сбора требований, легко заметить свойства, которые, во избежание дублирования данных, должны быть уникальными:

- *Имя пользователя* (`username`). В базе не должно быть пользователей с одинаковыми именами. Нельзя зарегистрировать пользователя под именем, которое уже есть в базе данных.
- *Электронная почта* (`email`). Адреса электронной почты используются для подтверждения и восстановления учетных записей, а значит, должны быть уникальными. Каждый пользователь должен иметь свой собственный адрес.
- *Телефон* (`phone_number`). Номера телефонов также нужны для подтверждения и восстановления. Поэтому одинаковых телефонов в таблице быть не должно.



Начнем с главного: что такое потенциальный ключ. У сущности `user` имеется три свойства, которые можно рассматривать в этом качестве. *Потенциальный ключ* — это минимальное сочетание свойств, которое однозначно идентифицирует строку таблицы. При этом под словом «минимальный» имеется в виду, что для идентификации нельзя использовать лишь какую-то часть потенциального ключа. К примеру, можно идентифицировать строку по сочетанию свойств `username` и `first_name`, однако для этого нам хватит и одного `username`. Поэтому такое сочетание нельзя считать потенциальным ключом. А вот отдельно взятое свойство `username` — можно.



Существует ли систематический подход к выбору потенциальных ключей? Да. Необходимо перебрать все свойства и выделить среди них те, что однозначно идентифицируют записи. Если таких не находится, следует перебрать сочетания свойств.

Итак, мы определили, что у таблицы `user` есть три потенциальных ключа: `username`, `email` и `phone_number`. И чтобы сделать один из них первичным, нужно сравнить их с качественной точки зрения:

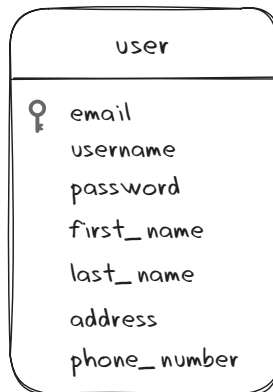
	(username)	(email)	(phone_number)
Уникальный	✓	✓	✓
Непустой (не NULL)	✓	✓	✗
Стабильный	✗	✓	✓
Простой	✓	✓	✓
Короткий	✓	✗	✓
Понятный	✓	✓	✓
Исключает избыточность	✓	✓	✓

Этот список критериев, естественно, неполон, но может использоваться в общем случае. Смысл каждого из критериев очевиден. Так, под *стабильностью* мы понимаем, что значение свойства, скорее всего, со временем не изменится, под *простотой* — что оно легко для понимания и применения. В столбцах `username` и `email` практически всегда хранятся *уникальные* и *непустые* (отличные от NULL) значения. А вот в столбце `phone_number` могут быть NULL, ведь даже если у клиента есть телефон, он может не согласиться его указать, но это не должно

помешать ему покупать товары, то есть зарегистрироваться на сайте магазина.

Все остальные критерии немного субъективны. К примеру, пользователь с большей вероятностью изменит свое имя на сайте, чем адрес электронной почты, но адреса обычно длиннее имен, что негативно скажется на скорости работы. Однако в случае с «Фантастической лавкой» выбор в пользу email очевиден: мы знаем, что изменение имен разрешено, а адресов — нет. Поскольку следует всеми силами избегать изменения первичных ключей, стабильность свойства email для нас важнее небольшого проигрыша в производительности.

Как видно из следующего рисунка, первичные ключи обозначаются особым символом слева от имени свойства. Кроме того, для обозначения первичного ключа можно использовать подчеркивание.



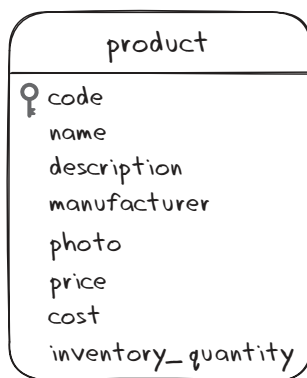
Другой пример. Среди столбцов таблицы product и их возможных сочетаний есть два потенциальных ключа:

- *Артикул.* В реальном мире большинство товаров имеют уникальные артикулы, по которым их можно идентифицировать. Уникальность артикула исключает избыточность данных.
- *Наименование и производитель товара.* Их сочетание однозначно идентифицирует товар и также исключает избыточность в таблице product.

Сравним потенциальные ключи:

	(code)	(name, manufacturer)
Уникальный	✓	✓
Непустой (не NULL)	✓	✓
Стабильный	✓	✓
Простой	✓	✗
Короткий	✓	✗
Понятный	✗	✓
Исключает избыточность	✓	✓

Естественно, что выбор в пользу артикула очевиден: использовать первичный ключ, состоящий из одного столбца, проще, чем состоящий из двух. К тому же артикул явно короче. Несмотря на то что понятие артикула знакомо не всем, оно несложно: любой товар во всех магазинах мира, включая интернет, имеет уникальный 12-значный номер по системе UPC (Universal Product Code — уникальный код изделия). Поэтому первичным ключом таблицы `product` мы сделаем столбец `code`.



В этих примерах мы выбрали первичный ключ из списка потенциальных. А что, если таких ключей в таблице нет либо с ними неудобно работать? Именно так обстоит дело с таблицей `review`, в которой есть два столбца, `review_text` и `review_time`. Значения в них не уникальны. Мы, разумеется, можем сказать, что их сочетание уникально в боль-

шинстве случаев, но все же не можем исключать вероятность того, что два одинаковых отзыва будут оставлены одновременно. В этой таблице нет потенциальных ключей:

	(review_txt)	(review_time)	(review_txt, review_time)
Уникальный	✗	✗	✗
Непустой (не NULL)	✓	✓	✓
Стабильный	✗	✓	✗
Простой	✗	✗	✗
Короткий	✗	✗	✗
Понятный	✓	✓	✓
Исключает избыточность	✗	✗	✗

В подобных ситуациях всегда есть выход: нужно создать новый столбец и сделать его первичным ключом. Такие ключи называются *суррогатными* (surrogate key). Обычно они имеют целочисленные значения, которые легко загружаются в память компьютера, что сильно ускоряет поиск. К тому же целые числа удобны с точки зрения масштабирования и стандартизации. Поэтому мы добавим в таблицу review столбец review_id. Он будет целочисленным, а его значения автоматически будут увеличиваться на единицу при добавлении новых записей.

Очевидно, что суррогатный ключ однозначно идентифицирует строки таблицы. Но может ли он предотвратить избыточность? Сам по себе нет. В таблице review могут быть две строки с одинаковыми данными, как на рисунке ниже:

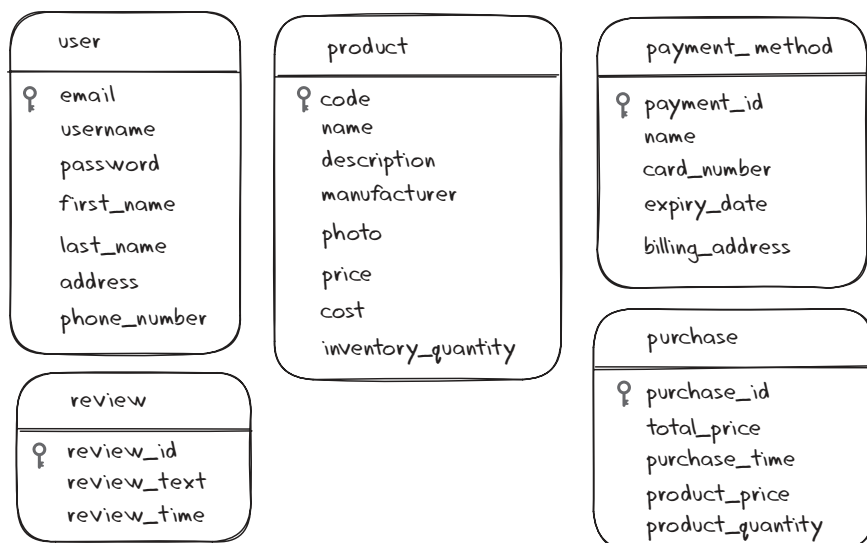
review_id	review_text	review_time
...	...	...
7	"Классная штука"	"9/14/2024, 9:15:15 AM"
8	"Классная штука"	"9/14/2024, 9:15:15 AM"

Не будет ли у нас проблем с избыточностью? В таблицах user и product важно ее исключить, поскольку двух записей, например, об одном и том же клиенте, быть не должно. Какую из них будет нужно обновить, если изменится имя пользователя? А вот в таблице review

картина немного иная. Отзывы — это просто текст на экране. Если мы однозначно идентифицируем отзыв клиента, мы сможем успешно его обновить или удалить. Поэтому избыточность в этой таблице не будет большой проблемой.

То же самое можно сказать и о таблицах `payment_method` и `purchase`: однозначная идентификация записей в них важнее необходимости исключить избыточность. В таблице `purchase` хранятся данные о покупках клиентов. Эти сведения не подлежат изменению, то есть с избыточностью можно смириться. Что же касается способов оплаты, то там избыточность даже вполне допустима, поскольку несколько клиентов «Фантастической лавки» могут оплачивать покупки одним и тем же способом (например, жена использует кредитку мужа). Поэтому в эти таблицы можно добавить суррогатные ключи с автоматическим увеличением значений: `payment_id` для `payment_method` и `purchase_id` для `purchase`.

Итак, теперь у каждой из наших сущностей имеется первичный ключ:

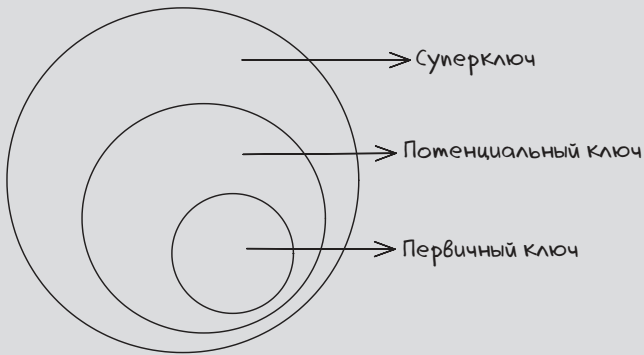


Потенциальные, первичные и суперключи

Необходимо упомянуть еще один тип ключей, о котором мы не успели поговорить. *Суперключ* (superkey) — это один или несколько столбцов, которые однозначно идентифицируют строку таблицы. Увы, в состав суперключа могут входить столбцы, которые для этой задачи не требуются.

Теперь мы можем посмотреть на потенциальные ключи под несколько иным углом: потенциальный ключ — это суперключ с минимальным количеством столбцов, то есть такой, в котором лишние столбцы отсутствуют. Первичный ключ выбирается из числа потенциальных.

Взаимосвязь между тремя типами ключей можно изобразить так:



Типы данных

Итак, мы выбрали или создали первичные ключи для всех сущностей. Теперь нужно определиться с типами данных для их свойств. Без этого реализация базы данных невозможна.

Мы уже говорили о типах данных. В главах 1 и 2 мы познакомились с тем, как они используются в SQL-запросах, а в главе 3 обсудили первый этап проектирования — сбор требований, — во время которого рекомендуется получить образцы будущих данных. И в этом разделе мы используем эти образцы, чтобы определить, какой из типов лучше всего подойдет для каждого из свойств. По ходу работы мы также продолжим изучение типов данных.

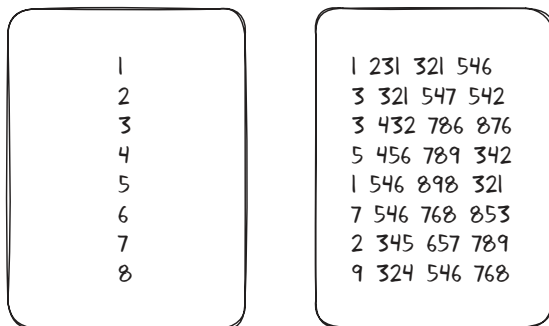
Строковые типы данных: сила слова

Строковые типы данных — одни из самых часто используемых. Поговорим о том, какие они бывают.

CHAR, VARCHAR и TEXT

Чаще всего применяются строковые типы CHAR, VARCHAR и TEXT. Главное различие между ними — требования к дисковому пространству, что связано с особенностями физической реализации. Иными словами, для хранения одной и той же строки может потребоваться меньше или больше места.

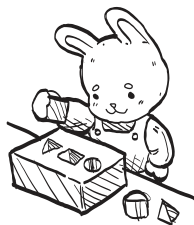
Вы спросите: разве это так важно? Сейчас нам доступны весьма объемные жесткие диски. Конечно, для небольших баз данных это неприципиально. Однако с ростом объема необходимость в вычислительных мощностях и объемах памяти возрастает. Рассмотрим такой пример:



Попробуйте найти цифру 8 на левом рисунке, а затем число 1 546 898 321 на правом. Вторая задача потребует больше времени, чем первая, не только у вас, но и у любого компьютера. Неудачный выбор типа данных может привести к излишнему увеличению их объема, что затруднит поиск (работу команды SELECT) и другие операции.

С учетом сказанного выше можно сформулировать следующие правила для выбора строковых типов данных:

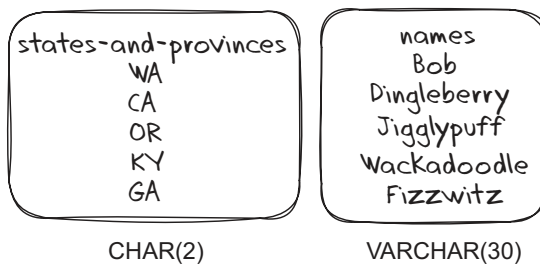
- Для строк одинаковой длины используйте CHAR.
- Для строк относительно небольшой, но разной длины используйте VARCHAR. При этом под словом



«относительно» понимается, что оценить их максимальную длину (скажем, 500 символов) несложно.

- Для текстовых фрагментов от 1000 символов используйте TEXT, особенно если максимальный объем оценить непросто.

Поясним эти правила на нескольких примерах (см. рисунок ниже). Для хранения двухбуквенных сокращенных названий штатов США и Канады (к примеру, «WA» означает Вашингтон), которые удобно использовать при вводе адресов, прекрасно подойдет тип CHAR. А вот для имен пользователей, которые могут быть короткими или длинными, но вряд ли выходят за рамки 30 символов, лучше выбрать VARCHAR.



Как можно заметить на этом рисунке, в скобках после названий CHAR и VARCHAR указаны цифры. Этот момент требует пояснения. Дело в том, что, используя эти типы, необходимо указывать длину строк, которые будут храниться в соответствующих столбцах. В данном случае CHAR(2) означает, что она составляет ровно два символа, а VARCHAR(30) — что она не превышает тридцати. В некоторых СУБД, например в MySQL, попытка сохранить более длинную строку приводит к ошибке или же обрезанию до максимальной длины.

Тип TEXT обычно служит для хранения текстов большого объема: описаний товаров, руководств и даже целых книг.

Строки и производительность базы данных

В некоторых СУБД данные типа TEXT хранятся отдельно от остальных, в специально предназначенных для этого файлах, и это приводит к задержкам при поиске и выборке информации. Используйте этот тип осторожно и только по необходимости (к примеру, для хранения описаний).

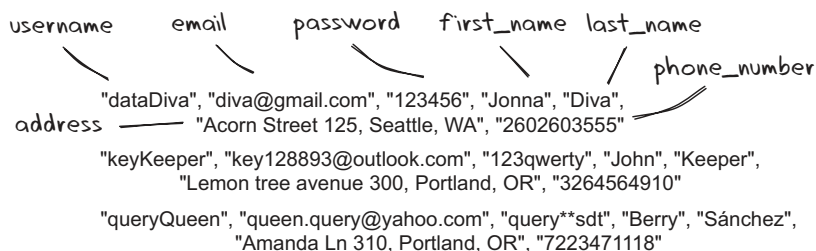
Кроме того, для индексации данных типа TEXT необходимы *полнотекстовые индексы* (full-text index), которые предназначены для поиска в текстах большого объема и поддерживаются не всеми СУБД. К примеру, на движке InnoDB в MySQL такой поддержки не было до версии 5.6. В отсутствие полнотекстового индекса поиск осуществляется линейно, а значит, медленно. Поэтому если строки не слишком длинны или же ожидается частый поиск по ним, лучше использовать тип VARCHAR.

Определение строкового типа данных на практическом примере

Итак, мы знаем все, что необходимо, чтобы определить, какие свойства сущностей из базы данных «Фантастической лавки» будут иметь строковый тип, а также выбрать для них наиболее подходящий из трех. Для каждой сущности эту работу мы выполним в три этапа:

1. Анализ списка требований и примеров данных, полученных ранее.
2. Определение свойств, которые должны быть строковыми.
3. Выбор строкового типа данных (CHAR, VARCHAR или TEXT).

Начнем с сущности user. Приглядимся к примерам данных на следующем рисунке:



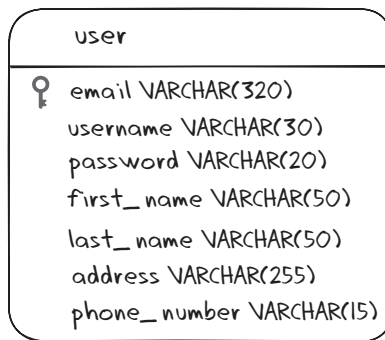
Как можно заметить, все семь свойств этой сущности имеют текстовые значения, а значит, должны быть строкового типа.

Осталось выбрать конкретные типы. Глядя на примеры, мы видим строки переменной длины, но оценить их максимальные размеры несложно. Например, номер телефона может состоять не более чем из 15 символов (с учетом форматирования). Иные международным стандартом не предусмотрены. Поэтому для свойства `phone_number` мы выберем тип `VARCHAR(15)`.

Если вам трудно оценить максимальный размер строки для какого-то свойства, спросите совета у экспертов в предметной области или у разработчиков. Хорошим советчиком может быть и искусственный интеллект. К примеру, вы можете задать вопрос ChatGPT:

Какова максимальная длина распространенных имен?

Определив возможную длину всех остальных свойств сущности `user`, добавим указание типа данных к имени свойства:



Вы можете возразить: хранить пароли в виде обычных строк типа `VARCHAR` небезопасно. Действительно, так и есть! К вопросу о конфиденциальных данных мы подойдем в главе 7. На данный момент `VARCHAR` нам вполне подойдет.

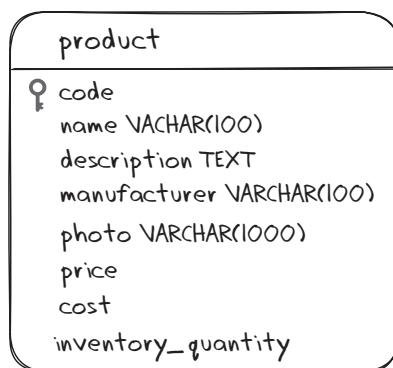
Перейдем к сущности `product` и соответствующим примерам данных:

name	description	manufacturer	photo
"Atomic Nose Hair Trimmer",	"Trim your nose ...",	"Mad Inventors Inc.", ...,	"https://hosting.photobucket.com/images/n627/....jpg"
"Selfie Toaster",	"Get your face on ...",	"Goofy Gadgets Corp", ...,	"https://hosting.photobucket.com/images/n631/....jpg"
"Cat-Poop Coffee",	"The only coffee made",	"Absurd Accessories", ...,	"https://hosting.photobucket.com/images/n627/....jpg"

Мы видим четыре свойства строкового типа, причем для `name` и `manufacturer` идеальным выбором будет `VARCHAR`. Длина URL фотографии довольно велика, но все же не запредельна, что позволяет нам отнести к типу `VARCHAR` и это свойство, установив максимальную длину,

скажем, в тысячу символов. А вот про свойство `description` можно сказать лишь, что его длина не ограничена ничем. Поэтому для перестраховки мы выберем для него тип `TEXT`.

Обновленная диаграмма сущности `product` выглядит так:



Проблемы преобразований: кодировка

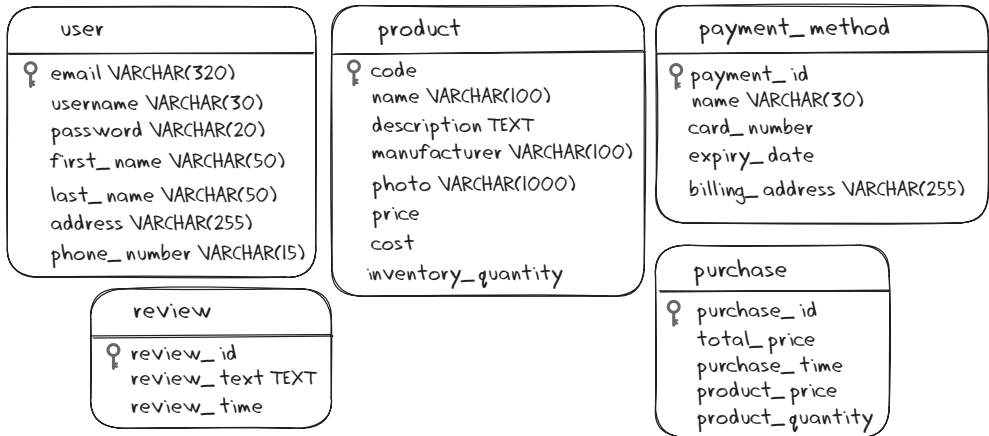
Проектируя любую базу данных и работая над ее реализацией, требуется принять одно очень важное решение, а именно выбрать кодировку для строковых данных. И лучше определиться с этим как можно раньше.

Кодировка (character set) — это набор правил отображения и хранения символов, который СУБД использует при работе с буквами, цифрами и знаками из разных языков и шрифтов.

Зачем нам думать о кодировках? Представьте жителей разных стран мира, которые говорят на разных языках и пишут разными буквами. В английском языке латинский алфавит, в русском — кириллица, в китайском — тысячи символов, которые не похожи ни на то ни на другое. От кодировки зависит, какие языки будут поддерживаться. К примеру, в `Latin1` входят лишь буквы из языков на основе латиницы, а в `UTF-8` — множество символов из других языков. Проектируя базу данных, следует позаботиться о тех, кто будет с ней работать. К тому же правильный выбор кодировки обеспечит точность и совместимость данных.

Обычно кодировка указывается при создании базы данных. В дальнейшем изменить принятое решение может быть невозможно (некоторые СУБД не позволяют менять кодировку) либо довольно сложно и с риском повреждения или потери данных. Вот почему кодировку следует выбрать как можно раньше.

Попробуйте на время отложить книгу, чтобы самостоятельно просмотреть все остальные сущности, определить их строковые свойства и выбрать для них типы данных. Но если у вас нет времени, взгляните на рисунок ниже. На нем показаны результаты этой работы.



Целочисленные типы: числа — путь к успеху

Прежде чем перейти к определению целочисленных свойств сущностей, поговорим о самых распространенных типах данных из этой категории.

TINYINT, SMALLINT, MEDIUMINT, INT и BIGINT

Чаще всего применяются целочисленные типы TINYINT, SMALLINT, INT и BIGINT¹. Они отличаются тем, сколько байтов отводится для хранения чисел (1, 2, 4 и 8 соответственно), а также их минимальными и максимальными значениями, которые различны для знаковых и беззнаковых данных.

Как следует из названия, беззнаковые числа не имеют при себе знака + или -, то есть бывают только положительными. В отличие от них знаковые числа могут быть как положительными, так и отрицательными. Диапазоны чисел разного типа с учетом наличия знака приведены ниже.

¹ Tiny — крошечный, small — маленький, big — большой. — *Примеч. пер.*

TINYINT	{	знаковые : -128 ... 127
		беззнаковые : 0 ... 255
SMALLINT	{	знаковые : -32 768 ... 32 767
		беззнаковые : 0 ... 65 535
INT	{	знаковые : -2 147 483 648 ... 2 147 483 647 (~2 млрд)
		беззнаковые : 0 ... 4 294 967 295 (~4 млрд)
BIGINT	{	знаковые : $-2^{63} \dots 2^{63} - 1$
		беззнаковые : $0 \dots 2^{64}$

Итак, TINYINT позволяет хранить беззнаковые значения до 255 и знаковые до 127, а INT — беззнаковые значения до 4 294 967 295, что немало. Тем, кто работает с еще большими числами, на помощь придет тип BIGINT.

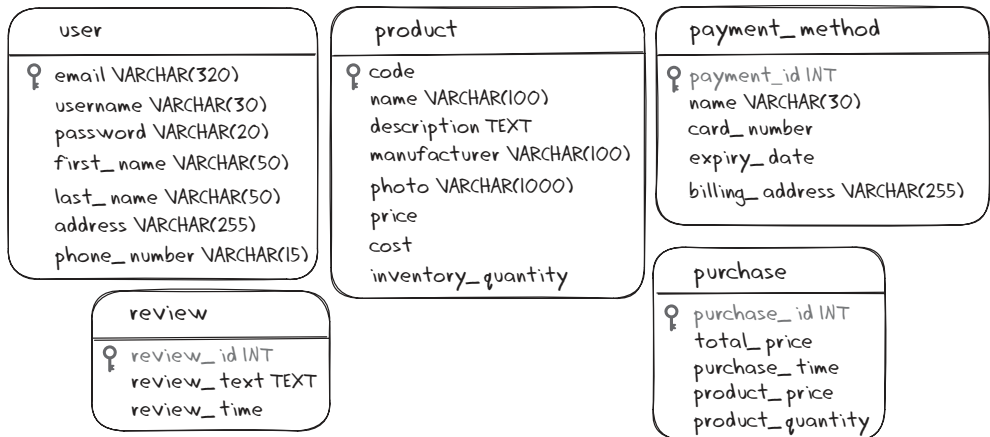
Целочисленные типы данных на практическом примере

Итак, мы знаем все, что необходимо, чтобы определить, какие свойства сущностей из базы данных «Фантастической лавки» будут иметь целочисленный тип, а также выбрать для них наиболее подходящий из пяти.

Будем действовать так же, как раньше, когда работали со строковыми типами: сначала рассмотрим примеры данных, изучим требования, затем определим, какие свойства будут целочисленными, а после этого выберем для них конкретные типы.

Прежде всего обратим внимание на идентификаторы. Целочисленные идентификаторы стабильны, уникальны, просты и эффективны. Следуя этой логике, для свойства `payment_id`, к примеру, необходимо выбрать один из пяти целочисленных типов. Понятно, что TINYINT не подойдет: его диапазон слишком мал. Возможно, в каких-то случаях можно было бы взять SMALLINT, но только если речь не идет о магазине с клиентами со всей Галактики. Конечно, всегда можно выбрать BIGINT: его диапазона хватит на всё. Однако мы остановимся на INT: он обеспечивает достаточное максимальное значение и добавок большую производительность БД (чем меньше разрядность, тем быстрее выполняется команда SELECT и другие). Несмотря на то что на

ER-диаграммах знаковость типов данных обычно не показывается, решение о знаках тоже нужно принять. Поскольку у первичных ключей нет причин быть отрицательными, все они будут беззнаковыми.



Обычно определить, что свойство будет целочисленным, несложно. Однако в нашей базе данных есть два исключения: сущности `payment_method` (способ оплаты) и `product` (товар). Начнем с примеров данных для первой из них:

```

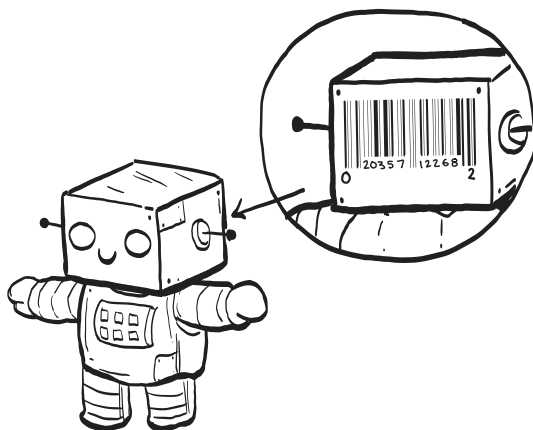
name    card_number    expiry_date
  |      |             |
"master card", 5260405254103227, 0227, ...
"boa visa", 3169596072241198, 1225, ...
"amex", 1677288846463190, 1129, ...
    
```

Значения свойств `card_number` (номер карты) и `expiry_date` (срок действия) состоят из цифр и могут быть целочисленными. Тем не менее для них лучше использовать строки, пожертвовав, таким образом, эффективностью хранения в пользу наглядности, практичности и отсутствия искажений. Поскольку математические операции ни с номерами карт, ни со сроками их действия не предусмотрены, хранить их именно в виде чисел не требуется. Более того, если все-таки сохранить их как числа, могут возникнуть нежелательные проблемы. Все дело в том, что и номера и сроки могут начинаться с нулей, которые должны отображаться на экране, но при выводе чисел обычно отбрасываются.

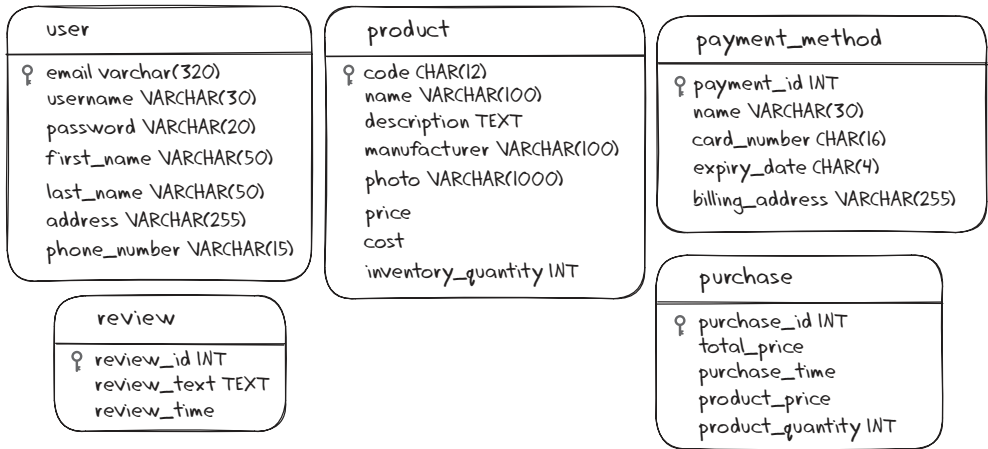
Поэтому лучше выбрать для свойства `card_number` тип `CHAR(16)`, а для свойства `expiry_date` — тип `CHAR(4)`, поскольку все номера карт состоят из шестнадцати цифр, а сроки действия — из четырех.

Вы можете снова указать на проблемы безопасности: хранить эти данные в виде простого текста нельзя. Все верно. Но к этому вопросу мы вернемся в главе 7, где будем обсуждать защиту информации. Сейчас, когда мы говорим о типах данных, простой текст нас вполне устраивает.

Теперь поговорим о сущности `product` и ее свойстве `code`. Образованные по системе UPC артикулы товаров представляют собой уникальные 12-значные цифры. Поэтому, по той же причине, что и номера карт (начальные нули), имеет смысл хранить артикулы в виде строк, а не чисел и выбрать для этого свойства тип `CHAR(12)`.



Аналогичным образом определим все остальные целочисленные свойства и выберем для них подходящие типы данных. Самое главное — определить рациональные максимальные значения, при которых итоговая база данных будет как можно меньше и эффективнее. Вот что должно получиться в результате:



Дробные типы данных: все дело в точке

Как вы, возможно, заметили, в прошлом разделе мы пропустили свойство `price` (цена) сущности `product`. Но если оно не целочисленное, то какое же? Ответ — дробное. Однако, прежде чем перейти к выбору таких типов данных, познакомимся с ними поближе.

Числа с плавающей и фиксированной точкой

Дробные типы данных подразделяются на два вида: с *плавающей* (floating-point) и *фиксированной* (fixed-point) *точкой*¹. Для хранения чисел с плавающей точкой используются типы `FLOAT` и `DOUBLE`, которые отличаются количеством разрядов: 32 или 64 соответственно. Иными словами, числа типа `DOUBLE` могут быть больше чисел типа `FLOAT`. Оба они применяются в случаях, когда значения не требуется выражать с точностью до определенного количества знаков после десятичного разделителя, к примеру при работе с датчиками, в статистическом анализе и моделировании физических явлений. В противном случае `FLOAT` и `DOUBLE` не подходят, поскольку их применение может привести к ошибкам округления.

Представим себе базу данных, в которой хранится информация об обмене валют (считать чужие деньги — одно удовольствие). Если перевести 1 000 001 доллар в евро по курсу 0.83, используя 32-разрядные

¹ В русскоязычной нотации — «число с фиксированной (или плавающей) запятой». — *Примеч. ред.*

числа с плавающей точкой, получим 830 000.8125, то есть на 1.75 евроцента меньше, чем должно быть. Казалось бы, ничего страшного, но только не для того, кто обменивает такие большие суммы часто.

$$\begin{array}{lcl}
 \$1\,000\,001 & \xrightarrow{\$1 \times 0.83\text{€}} & \begin{array}{l} \text{Фактически: } 830000.83 \\ \text{FLOAT (32 разряда): } 830000.8125 \\ \text{Погрешность: } -0.0175 \end{array}
 \end{array}$$

Если, как в примере выше, точность вычислений очень важна, лучше использовать дробные числа с фиксированной точкой. Для них характерны две величины: общее количество цифр и количество цифр после десятичного разделителя. К примеру, число 12.3456 состоит из шести цифр, четыре из которых стоят после разделителя.

$$\begin{array}{c}
 \text{Цифр после разделителя: } 4 \\
 \begin{array}{c} \text{12.3456} \end{array} \\
 \text{Всего цифр: } 6
 \end{array}$$

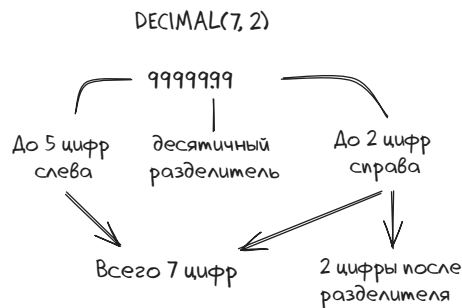
Для хранения таких дробных чисел предназначены типы данных DECIMAL и NUMERIC. В большинстве СУБД (например, в MySQL и PostgreSQL) между ними фактически нет различий, и можно смело выбрать любой. В других СУБД различия могут быть. Поэтому если вы не знаете тонкостей выбранной системы, прочтите документацию либо спросите у ChatGPT.

Дробные типы данных на практическом примере

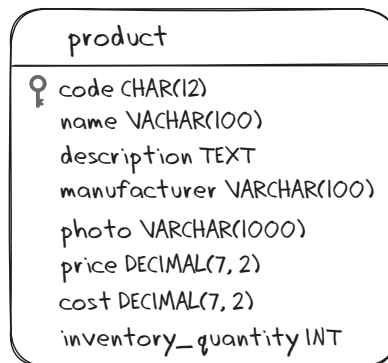
Закончив знакомство с дробными типами данных, вернемся к «Фантастической лавке» и будем работать так же, как ранее со строковыми и целочисленными типами. Посмотрим на примеры данных для сущности product:

name	description	manufacturer	price	cost
"Atomic Nose Hair Trimmer",	"Trim your nose ...",	"Mad Inventors Inc.",	19.99,	9.99, ...
"Selfie Toaster",	"Get your face on ...",	"Goofy Gadgets Corp",	24.99,	15.02, ...
"Cat-Poop Coffee",	"The only coffee made",	"Absurd Accessories",	29.99,	21.85, ...

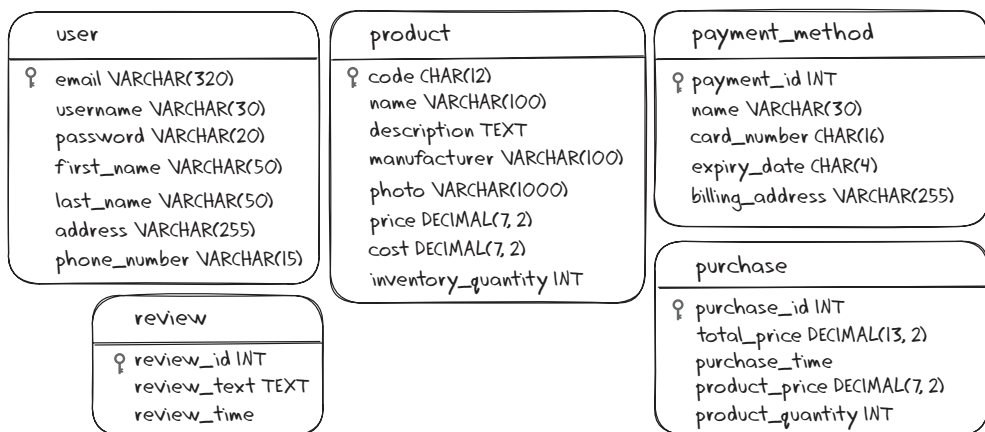
Обратите внимание на значения свойств `price` и `cost`. Поскольку это денежные суммы, они должны выражаться с точностью до определенного числа знаков после десятичного разделителя, в нашем случае — двух (центы), хотя в других монетарных системах это количество может быть иным. Итак, нам нужен тип `DECIMAL`. Поскольку стоимость товаров в «Фантастической лавке» не превышает тысячи долларов, для записи любых значений нам будет достаточно семи цифр:



В итоге свойства `price` и `cost` будут иметь тип `DECIMAL(7, 2)`, где 7 — общее количество цифр, а 2 — количество цифр после десятичного разделителя. Обновим диаграмму сущности `product`:



Аналогичные действия нужно выполнить и с другими сущностями, в частности со свойствами `product_price` и `total_price` сущности `purchase`. Поскольку сумма заказа может быть довольно внушительной, имеет смысл увеличить общее количество цифр свойства `total_price` и взять, например, `DECIMAL(13, 2)`. Обновленная схема базы данных будет выглядеть так:



Временные типы данных: счастливые часы не наблюдают

У нас осталось всего два свойства без определенных типов: `review_time` (время отзыва) сущности `review` и `purchase_time` (время заказа) сущности `purchase`. Для них нам понадобятся временные типы данных — последняя категория, с которой мы познакомимся в этой главе. Поговорим о том, какие типы к ней относятся.

DATE, TIME, DATETIME и TIMESTAMP

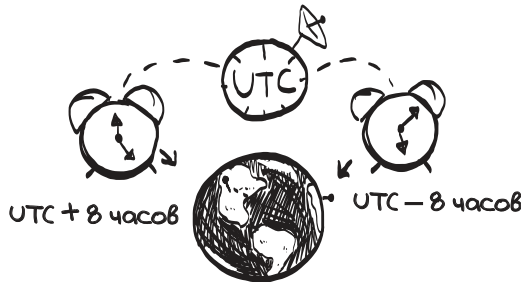
Большинство СУБД поддерживают временные типы данных, среди которых можно назвать `DATE`, `TIME`, `DATETIME` и `TIMESTAMP`. Несмотря на различия в реализации, у них есть общие черты, что позволяет установить такие правила применения:

- Используйте `DATE`, когда необходимо хранить информацию о дате без указания точного времени. К примеру, чей-то день рождения 15 мая 1980 года можно записать как `1980-05-15`.
- Используйте `TIME`, если необходимо хранить информацию о времени без указания даты. К примеру, если обед подают каждый день в половине третьего, мы можем записать это время как `14:30:00`.
- Используйте `DATETIME`, если нужна информация как о дате, так и о времени, когда произошли или произойдут какие-то события. При этом данные указываются с учетом часового пояса.
- Используйте `TIMESTAMP`, чтобы точно зафиксировать время какого-то события, к примеру записи в лог, добавления или обновления

строк таблицы. При сохранении данных в этом формате СУБД обычно автоматически учитывают часовой пояс.

Проектируя базу данных для приложения сугубо местного назначения (к примеру, электронного меню для ресторана), можно не думать о часовых поясах и связанных с ними проблемах. Если же с базой будут работать люди из разных стран, время необходимо указывать с учетом часового пояса. Поэтому, независимо от особенностей конкретной СУБД, следует использовать *всемирное координированное время* (UTC): именно оно обеспечивает согласованность данных и устраняет проблемы часовых поясов, летнего времени и путешествующих пользователей.

UTC — это огромные часы, со временем на которых согласны все люди. Оно едино для всех независимо от места пребывания. Жителю Сиэтла не нужно каждый раз спрашивать у друзей из Пекина, который у них час. Достаточно знать, что пекинское время опережает UTC на восемь часов. Сегодня все веб- и мобильные приложения определяют местное время при помощи UTC.



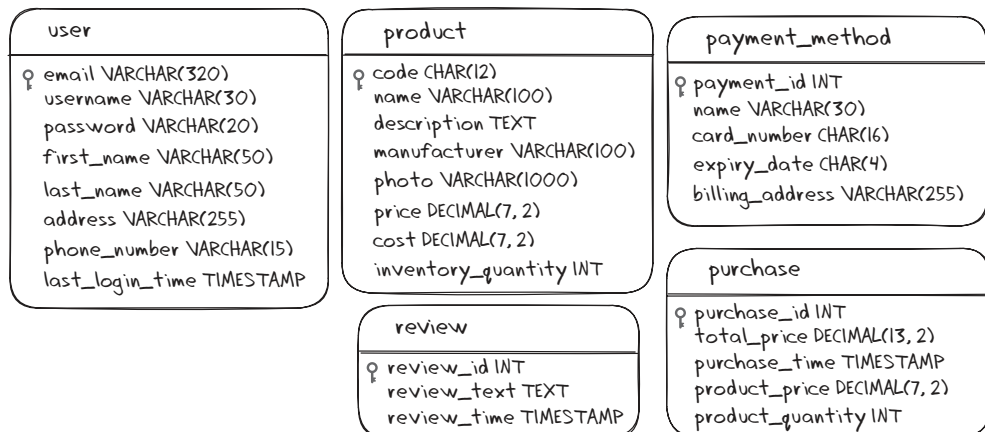
Временные типы данных на практическом примере

Теперь мы можем определить типы данных для оставшихся свойств из проекта: свойства `review_time` сущности `review` и свойства `purchase_time` сущности `purchase`. Мы можем действовать, как и раньше: начать с анализа примеров данных. Однако уже из имен, в которых имеется слово `time` — время, можно понять, что они относятся к временному типу. Это упростит нашу задачу.

Как мы установили во время сбора требований, и `review_time`, и `purchase_time` нужны для хранения информации о дате и времени: когда был сделан заказ и когда был оставлен отзыв. При этом в обоих случаях данные не должны зависеть от часового пояса. Для этих целей лучше всего подойдет тип `TIMESTAMP`.

Кроме того, чтобы отслеживать посещения магазина клиентами, имеет смысл добавить сущности `user` еще одно свойство: `last_login_time` (время последнего входа в систему), которое также будет типа `TIMESTAMP`.

С учетом всех обновлений ER-диаграмма базы данных «Фантастической лавки» будет выглядеть так:



Итак, мы выбрали типы данных для свойств всех имеющихся сущностей. Взгляните на этот шедевр еще раз! Мы отлично поработали.

Несоответствия типов данных в разных СУБД

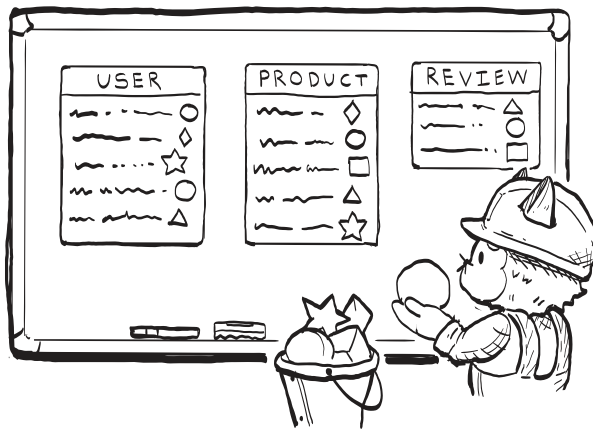
Нравится нам или нет, но между СУБД существуют несоответствия в отношении типов данных. Изученные нами типы поддерживаются большинством СУБД, но с рядом исключений и особенностей. Сущности и свойства, которые мы спроектировали в этой главе, будут отлично работать в MySQL, MariaDB и PostgreSQL, но чтобы перенести их в другую СУБД, потребуются корректировки, как незначительные, так и существенные. Необходимо знать, что:

- В SQLite вместо `TIMESTAMP` обычно используется `TEXT`.
- В Oracle вместо `INT` и `DECIMAL` используется `NUMBER`, а вместо `TEXT` — `VARCHAR2`.
- В SQL Server вместо `TEXT` используется `NVARCHAR(MAX)`, а вместо `TIMESTAMP` — `DATETIME2` или `DATETIMEOFFSET`.

Информация об этом приведена в репозитории GitHub к нашей книге.

Итоги

- При проектировании сущности необходимо определить ее свойства и первичный ключ, а также выбрать типы данных для каждого из свойств.
- Первичный ключ должен однозначно определять каждую строку в таблице. В некоторых случаях он может потребоваться, чтобы предотвратить избыточность. Если в таблице нет потенциальных ключей либо с ними неудобно работать, необходимо добавить новое целочисленное свойство, которое и будет первичным ключом.



- В базах данных чаще всего используются строковые, целочисленные, дробные и временные типы данных. Выбор зависит от задач, для которых будет использоваться этот тип, а также от конкретных требований сценария.
- Выбор типа данных необходимо начинать с анализа примеров данных, а также других сведений, полученных на этапе сбора требований. Определив общий тип данных (к примеру, строковый) и учитывая другие свойства (максимальная длина, особенности хранения), можно выбрать конкретный тип данных (например, TEXT).
- Правильный выбор типов данных позволяет хранить информацию эффективно, с учетом возможных запросов на выборку и обновление.

5 | Связи



В этой главе

- ✓ Установление связей между сущностями
- ✓ Определение кратности связей
- ✓ Выбор между сильными и слабыми сущностями

Необходимые знания

Проект рассматриваемой в этой главе базы данных, реализованный при помощи часто применяемых на практике инструментов dbdiagram.io и MySQL Workbench, можно найти в репозитории GitHub (<https://github.com/Neo-Hao/grokking-relational-database-design>). Чтобы загрузить проект в выбранный инструмент, перейдите в папку chapter_05 и следуйте инструкциям из файла README.md.

Упоминаемые в этой главе типы данных поддерживаются большинством СУБД, в том числе MySQL, MariaDB и PostgreSQL. Пользователям других систем, например SQL Server или SQLite, возможно, придется немного скорректировать проект: заменить некоторые из типов данных на их эквиваленты из соответствующей СУБД. Все нужные для этого сведения приведены в том же файле с инструкциями.

Введение

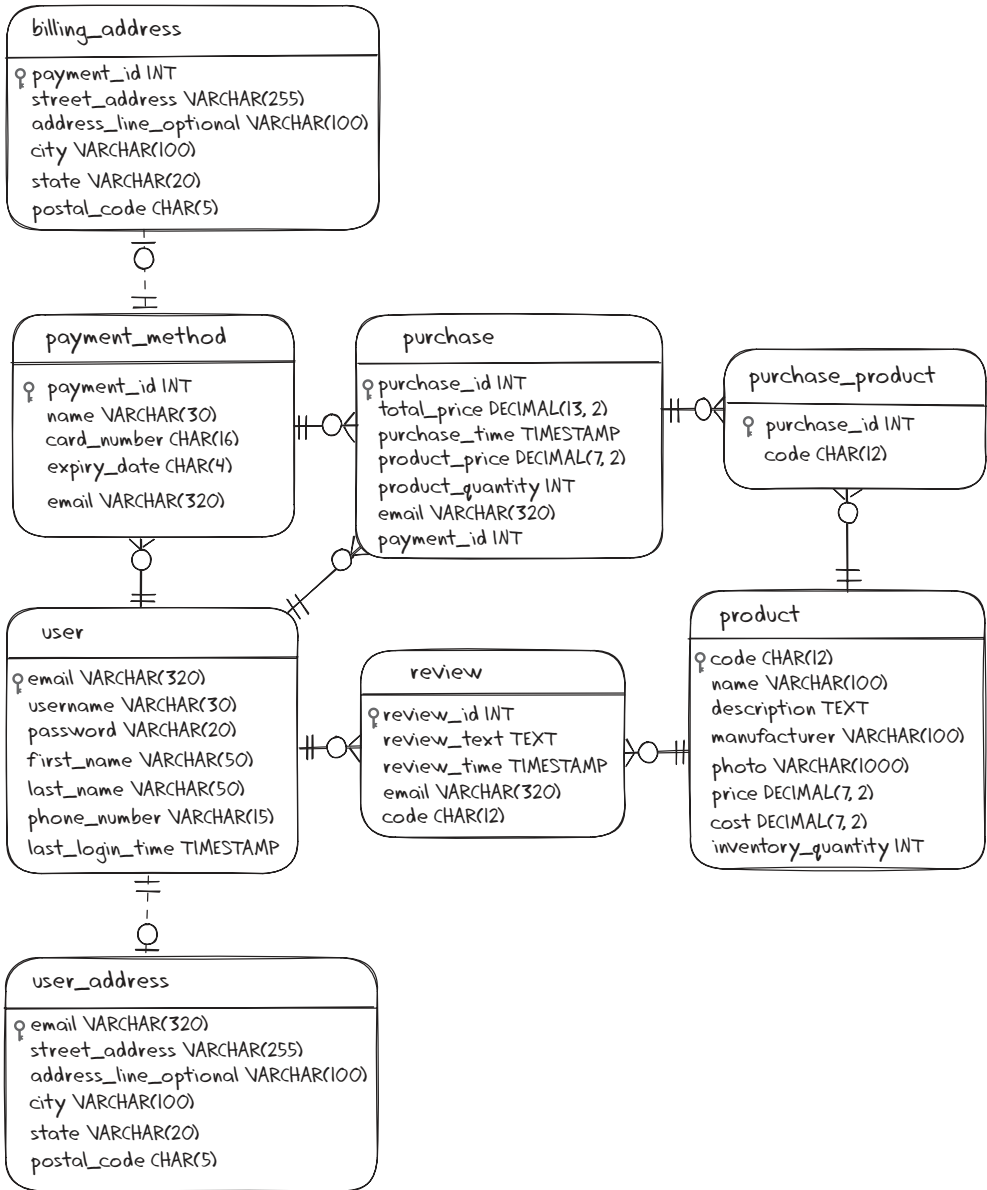
В этой главе мы продолжим работу над ER-диаграммой базы данных «Фантастической лавки». Теперь нам потребуется установить связи между сущностями, которые мы определили в прошлой главе. По ходу работы мы познакомимся с рядом важных понятий, в том числе кратностью и зависимостью.



Диаграммы сущностей и связей

ER-диаграмма — графическое представление сущностей и связей между ними — не просто визуальный инструмент, но и документ, который описывает базу данных и правила ее работы. За годы применения такие диаграммы стали универсальным языком, на котором общаются проектировщики.

ER-диаграмма обычно состоит из прямоугольников, которые соответствуют сущностям, и линий, которые показывают связи между ними, то есть структуру базы данных (или же *схему данных* — *data schema*), но не сами данные. Таблица с миллионами записей отображается на диаграмме как единая сущность. Вот, например, как выглядит ER-диаграмма, которую мы нарисуем к концу главы:

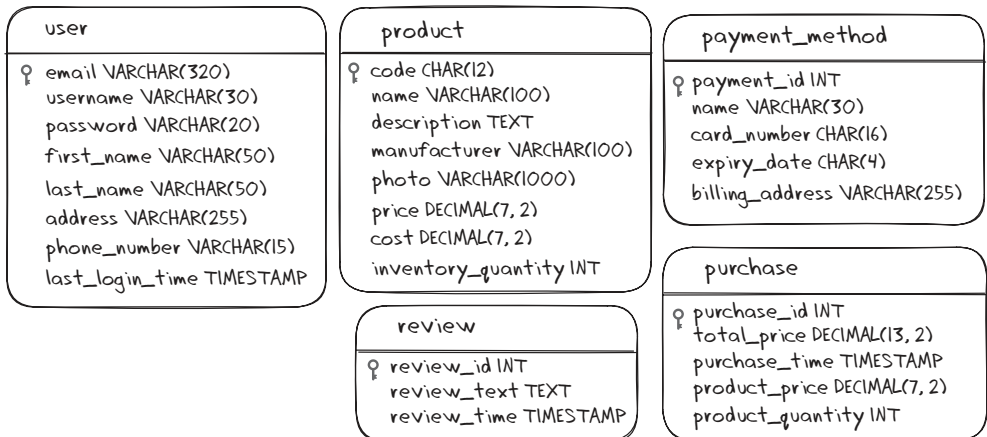


На ней используется особая система обозначений, известная как «птичьи лапки» (Crow's Foot). Чуть позже мы кратко поговорим и о других системах.

Для полного описания сложной базы данных может потребоваться несколько ER-диаграмм, однако в нашем случае вполне достаточно одной. Работу над ней мы начнем не с нуля: у нас уже есть набор сущностей, которые мы определили в главе 4, но они пока не связаны друг с другом. В следующих разделах мы рассмотрим различные типы связей, после чего определим, какие из них будут нужны в базе данных «Фантастической лавки».

Соединение связанных сущностей

Сначала еще раз посмотрим на то, что уже сделано. В главе 4 мы составили список сущностей:

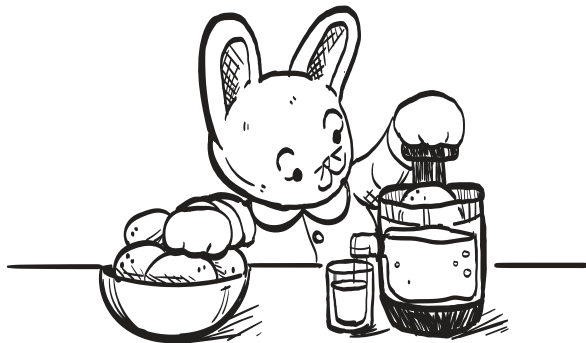


Важные сведения о связях между ними нам удалось получить чуть раньше, в главе 3, — на этапе сбора требований. Тогда мы выяснили следующее:

- Пользователь может сделать несколько заказов. Заказ может быть сделан только одним пользователем.
- Пользователь может оставить отзывы на несколько товаров (при условии, что он их купил). Отзывы на товар могут оставить несколько пользователей.
- Пользователь может выбрать несколько способов оплаты. Способ оплаты может применяться только одним пользователем.

- В заказ может входить несколько товаров. Товар может входить в несколько заказов.

На ER-диаграммах все связанные друг с другом сущности соединяются линиями. Чтобы нарисовать их, нам нужно прежде всего понять, в чем именно состоит связь между сущностями, то есть объединить составленный нами список и перечисленные выше правила.

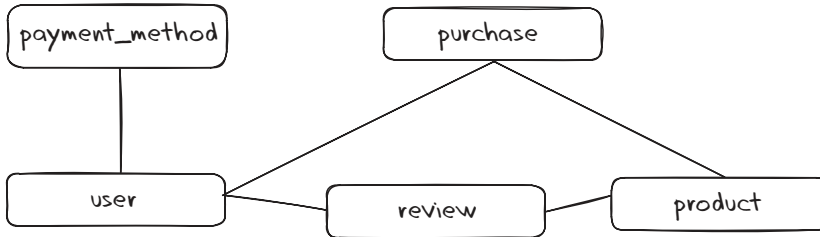


Работая над списком сущностей и свойств, мы получили результат, несколько противоречащий собранной информации. К примеру, мы ввели сущность `review`, хоть в диалоге с работником «Фантастической лавки» и не было слова «отзыв». Необходимо осмыслить подобные противоречия и убедиться, что «лишние» сущности действительно необходимы. Для этого адаптируем собранные сведения: составим список простых предложений, состоящих только из подлежащего, сказуемого и дополнения:

- Пользователь делает заказы.
- Пользователь пишет отзывы.
- На товар оставляют отзывы.
- Пользователь применяет способы оплаты.
- Товары входят в заказы.

Покажем эти предложения на диаграмме. Все существительные в них — это сущности, а если какие-то две из них удалось объединить при помощи глагола, мы можем считать, что они с большой вероятностью связаны друг с другом. Конечно, такой подход не исключает неточностей и ложных толкований, поэтому, возможно, придется сделать несколько заходов, действовать методом проб и ошибок. В итоге

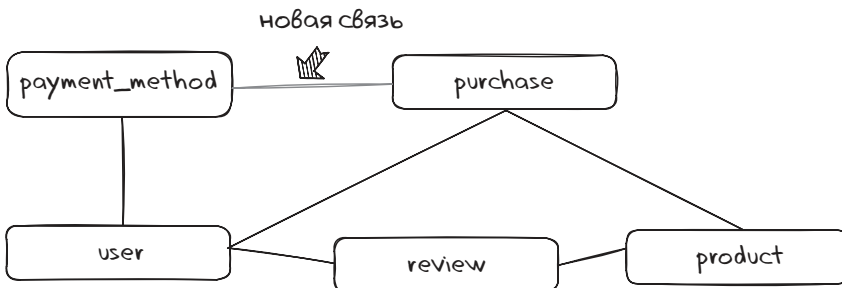
мы получим следующую диаграмму (для упрощения свойства на ней не показаны, поскольку сейчас они не имеют значения):



Нужно проверить все связи на этом черновике на соответствие собранным ранее сведениям и примерам данных. Также имеет смысл показать результаты работы заказчикам и рассказать о принятых решениях: возможно, мы где-то ошиблись или же не учли что-то критически важное. К примеру, разработчик из «Фантастической лавки» может уточнить, что онлайн-заказ невозможно сделать, не указав способ оплаты. Поэтому на основании новой информации нужно ответить на следующий вопрос:

Связаны ли друг с другом сущности `payment_method` и `purchase`?

Без информации о способе оплаты оформить заказ не удастся, поскольку магазин не сможет взять с покупателя деньги. Значит, с каждым заказом должен быть сопоставлен определенный способ оплаты, и это говорит о том, что связь между `payment_method` и `purchase` имеется. Отметим это на диаграмме:



Теперь, прежде чем продолжить работу над диаграммой, поговорим о том, какими свойствами обладают связи между сущностями.

Кратность

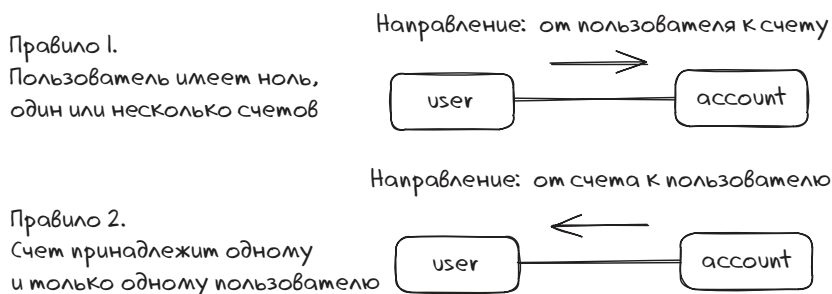
Кратность (cardinality) — это важное свойство связи между двумя сущностями. Оно показывает, сколько экземпляров одной из них можно соотнести с одним экземпляром другой. В зависимости от кратности, связи подразделяются на несколько типов, которые отображаются на ER-диаграммах по-разному. Поэтому чтобы завершить создание нашей ER-диаграммы, нам предстоит вернуться к собранным ранее сведениям, чтобы определить кратность каждой из обнаруженных нами связей.

Направление и обозначения

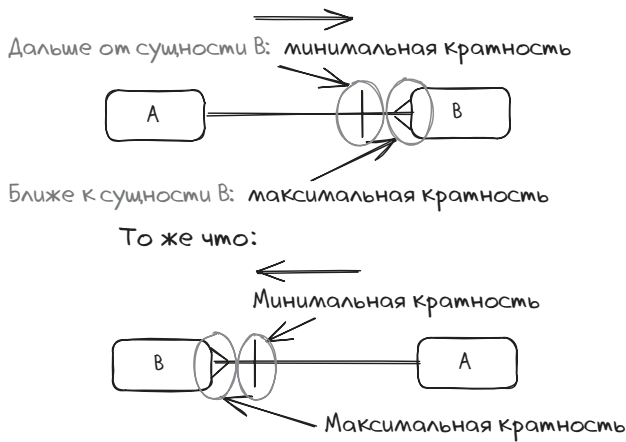
Связь между двумя сущностями может быть направлена в одну или другую сторону. Например, в базе данных банка имеются две сущности: user (пользователь) и account (счет), которые связаны друг с другом следующими правилами:

- *Правило 1.* Пользователь имеет ноль, один или несколько счетов.
- *Правило 2.* Счет принадлежит одному и только одному пользователю.

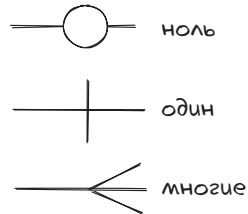
Эти правила показывают два направления, в которых действует связь между сущностями: в обоих случаях — от субъекта к объекту:



Для связи, действующей в направлении от сущности А к сущности В, кратность равна количеству экземпляров В, с которыми можно соотнести один экземпляр А. На диаграмме она обозначается двумя символами на линии связи, которые всегда используются парами. Символ, расположенный дальше от сущности В, показывает *минимальную кратность*, а находящийся ближе к этой сущности — *максимальную*.

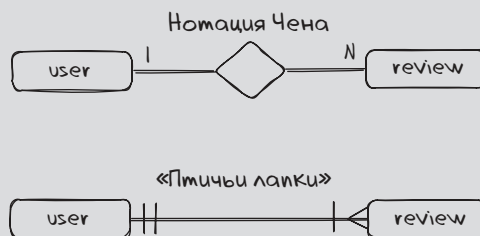


Как видно из этих рисунков, чтобы показать кратность связи в направлении от А к В, мы ставим на линии между ними два символа, располагая их рядом с В. Вам необходимо знать следующие символы:



Обозначение кратности: сложные связи, математические символы

Есть несколько вариантов обозначения кратности. Чаще всего используются два: нотация Чена (Chen), важная с исторической точки зрения, и «птичьи лапки», более простые и популярные среди профессионалов. В этой книге мы будем использовать «птичьи лапки».



Как показать кратность на диаграмме базы данных из прошлого примера о сущностях user и account? Все, что нам нужно для решения

этой задачи, изложено в двух правилах в начале этого раздела. Еще раз приведем их:

- *Правило 1.* Пользователь имеет ноль, один или несколько счетов.
- *Правило 2.* Счет принадлежит одному и только одному пользователю.

Поскольку пользователь может иметь ноль, один или несколько счетов, минимальная кратность в направлении пользователь — счет будет равна нулю, а максимальная — «многие». В обратном направлении оба значения кратности равны единице, так как у пользователя может быть один и только один счет.

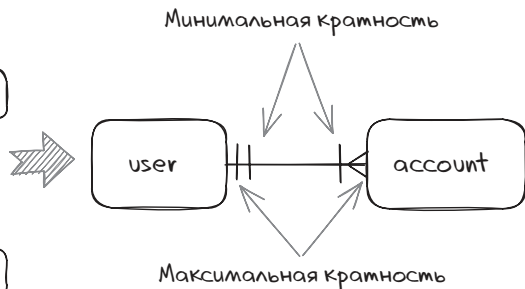
Правило 1.

Пользователь имеет ноль, один или несколько счетов



Правило 2.

Счет принадлежит одному и только одному пользователю

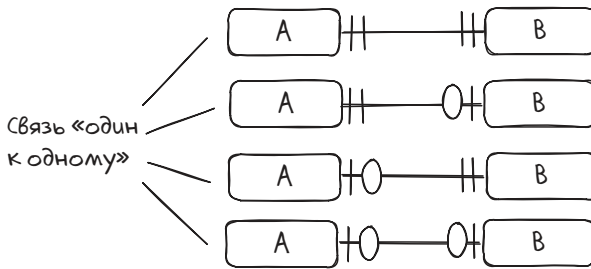


Как можно заметить, мы объединяем два направления связи, обозначая их одной линией. Символы, которые расположены рядом с сущностью `account`, показывают кратность в направлении к ней, а те, что находятся рядом с сущностью `user`, — в обратном направлении.

Определившись с тем, что такое кратность и как она обозначается, поговорим о трех типах связей и нанесем их на черновик нашей диаграммы.

Один к одному: идеальное совпадение

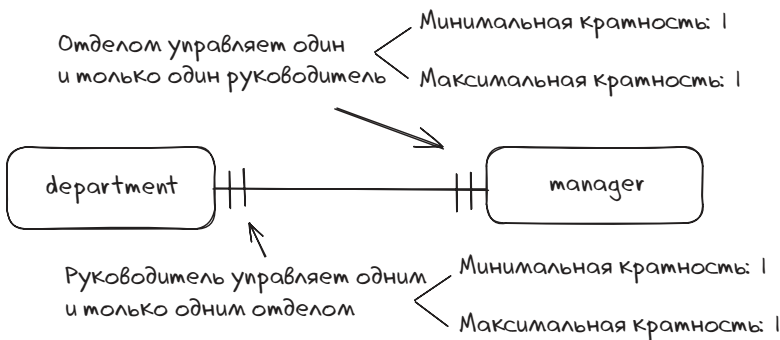
Связь относится к типу «один к одному» (one-to-one), если каждый экземпляр первой сущности соотносится максимум с одним экземпляром второй. Название отражает максимальную кратность: для обоих направлений она равна единице. При этом минимальная кратность равна единице или нулю.



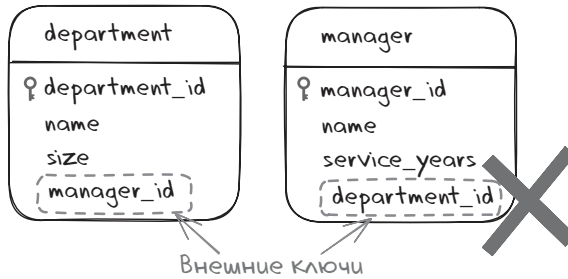
Отметим, что в случаях, когда минимальная кратность связи с обеих сторон равна единице, одно из значений обычно обнуляется. Так поступают для удобства реализации. Возьмем, к примеру, большую компанию, в которой есть несколько отделов (department) и в каждом из них по руководителю (manager). При этом действуют такие правила:

- Отделом управляет один и только один руководитель.
- Руководитель управляет одним и только одним отделом.

Изобразим эти правила на ER-диаграмме:



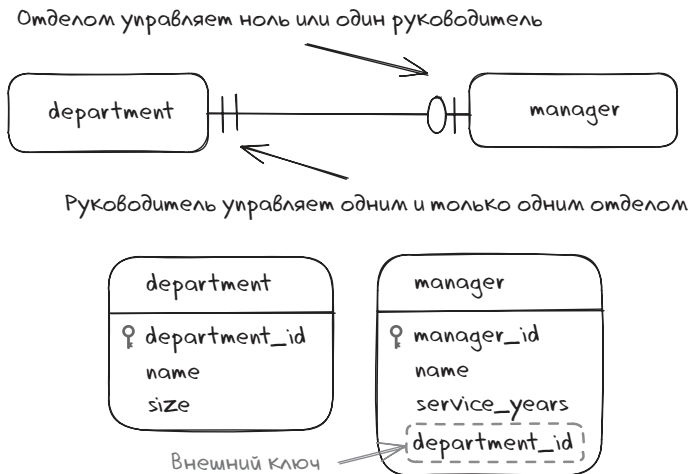
Теоретически все нарисовано правильно, но на практике это работать не будет. Чтобы связать две сущности, необходимы внешние ключи, которые придется добавить к свойствам обеих сущностей, поскольку каждый отдел связан с руководителем, а каждый руководитель — с отделом:



Но при такой реализации мы столкнемся с проблемами. Поскольку при создании обе таблицы будут пусты, нам будет нужно наполнить их информацией об отделах и руководителях. Если начать с таблицы department, сработает ограничение по внешнему ключу, поскольку данные о руководителях еще отсутствуют. Но то же самое произойдет, если сначала заняться таблицей manager, ведь данные об отделах еще не загружены. Проблема можно решить, обнулив одну из минимальных кратностей. Для этого нужно немного изменить правило в отношении отделов:

- Отделом управляет ноль или один руководитель.
- Руководитель управляет одним и только одним отделом.

И, соответственно, реализацию таблицы:



Можно сделать вывод: при связи «один к одному» минимальная кратность показывает, в какой таблице находится внешний ключ. Действи-

тельно, если таблицы А и В связаны, а минимальная кратность связи равна нулю, не каждая запись в таблице А может иметь соответствие в таблице В. Значит, внешний ключ можно определить в таблице В, что и решает проблему с загрузкой данных. В последнем примере таблица А — department, а таблица В — manager.

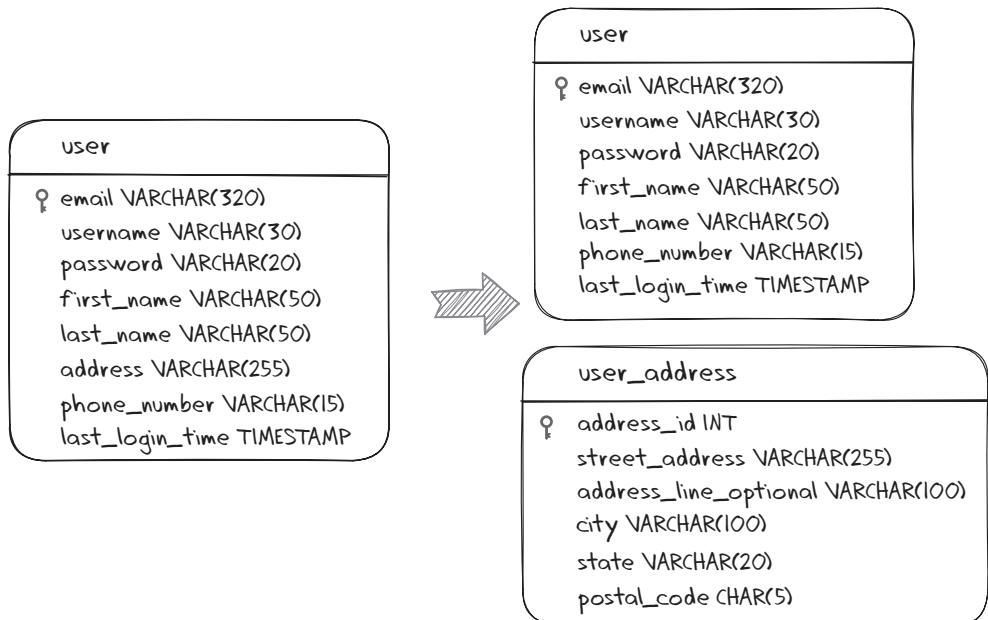
Вернемся теперь к базе данных «Фантастической лавки».

Связи «один к одному» на практическом примере

Анализируя первоначальные сведения, полученные на этапе сбора требований, можно сказать, что в этой базе данных нет сущностей со связями «один к одному». Однако в дальнейшем мы узнали новую информацию, из-за которой придется пересмотреть сущность user. Все дело в адресах, которые имеют примерно такой вид:

20 Baldwin Rd, Shelter Island, New York, 11964

Для таргетированного маркетингового анализа аналитику данных нужно иметь возможность фильтрации клиентов по городам, штатам или почтовым индексам. Выбранный нами тип данных VARCHAR не слишком подходит для этой задачи. Поэтому следует выделить адрес в отдельную сущность со своим набором свойств:

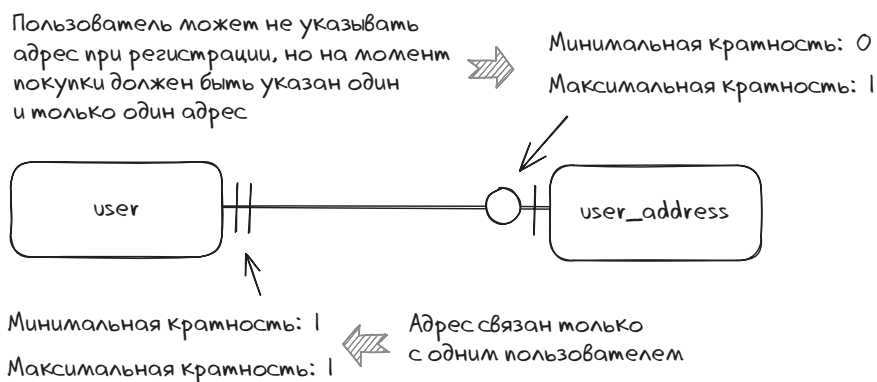


Как видно из этого рисунка, большинство свойств новой сущности (`user_address` — адрес пользователя) — строковые. Необходимо выбрать для них тип `CHAR` или `VARCHAR` с указанием максимальной длины. Для упрощения предположим, что все клиенты «Фантастической лавки» живут в США. Тогда, к примеру, свойство `state` (штат) будет иметь тип `VARCHAR(20)`: вполне достаточно для Северной Каролины — штата с самым длинным названием (16 букв). Для перестраховки, на случай возможного появления новых штатов, добавим несколько символов про запас. Таким же образом можно присвоить тип `CHAR(5)` свойству `postal_code` (почтовый индекс): будем использовать формат с пятью цифрами.

Как же новая сущность `user_address` связана с сущностью `user`? Вот информация от разработчика интернет-магазина:

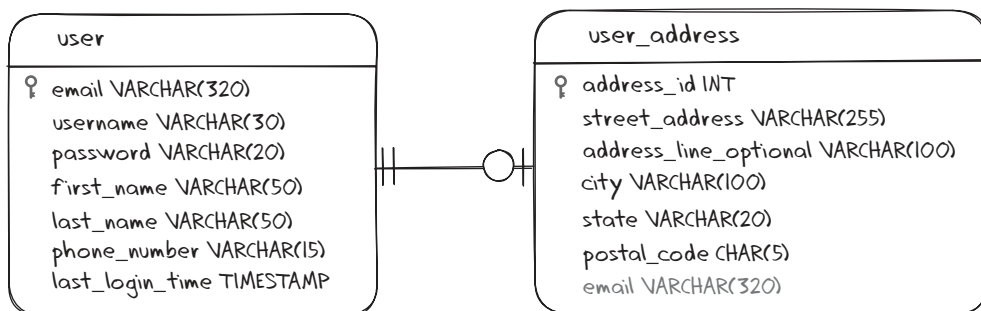
- Пользователь может не указывать адрес при регистрации, но на момент покупки должен быть указан один и только один адрес.
- Адрес связан только с одним пользователем.

Отсюда несложно определить минимальную и максимальную кратность связи в обоих направлениях (от `user` к `user_address` и от `user_address` к `user`), после чего нанести на линию соответствующие символы:



Связь «один к одному»? Да. Как можно заметить, на линию нанесены те же символы, что мы уже разбирали в примере с отделами и руководителями.

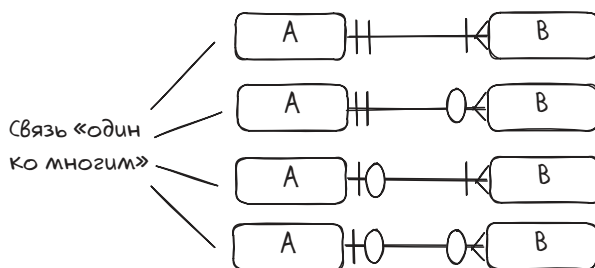
Чтобы связать две сущности: `user` и `user_address`, — потребуется внешний ключ. Как мы уже говорили, определить, куда его добавлять, можно по минимальной кратности. В данном случае она равна нулю в направлении от `user` к `user_address`, а значит, внешний ключ будет у последней:



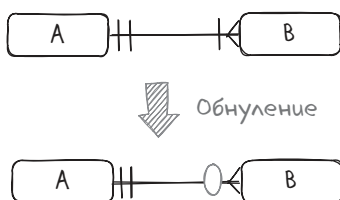
В заключение стоит добавить, что связи «один к одному» используются в базах данных реже других, поскольку нечасто встречаются и в реальных условиях.

Один ко многим: любовный треугольник

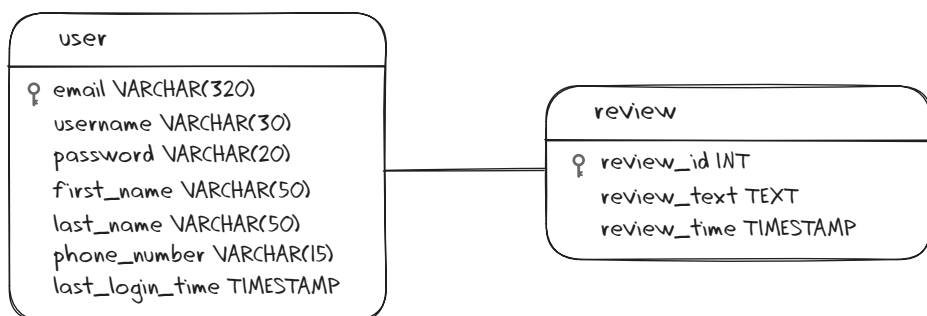
Напротив, связь «один ко многим» (one-to-many) — самая распространенная. Как и в прошлом случае, это название отражает максимальную кратность: в одном направлении она равна единице, а в другом — «многие».



Если в обоих направлениях минимальная кратность равна единице, при добавлении данных в таблицу будет срабатывать ограничение по внешнему ключу. Поэтому, по аналогии со связью «один к одному», минимальную кратность приходится обнулять, причем на стороне «многие»:



Поговорим о связях этого типа подробнее: рассмотрим, к примеру, сущности `user` и `review` из базы данных «Фантастической лавки»:



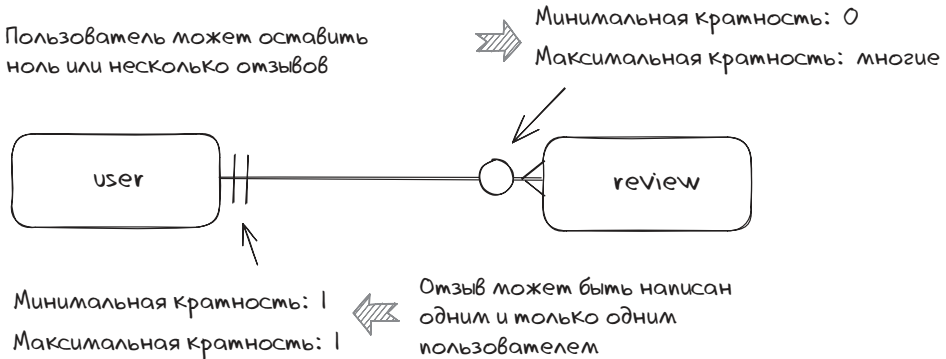
Как мы уже узнали на этапе сбора требований и в ходе последующих уточнений информации:

- пользователь может оставить ноль или несколько отзывов;
- отзыв может быть написан одним и только одним пользователем.

Покажем это на рисунке:



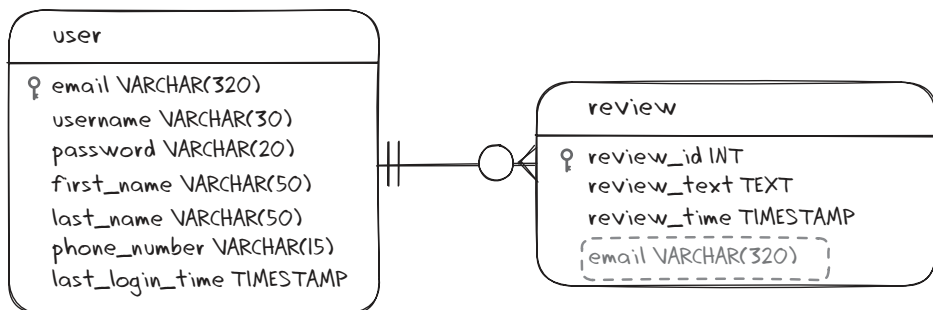
Исходя из этого можно определить минимальную и максимальную кратность в обоих направлениях (от user к review и от review к user), а затем нанести нужные символы на диаграмму:



По кратности связи мы определяем, у какой из сущностей будет внешний ключ. Связи «один ко многим» — не исключение: в данном случае ключ будет на стороне «многие», то есть у сущности, на которую указывает «птичья лапка».

Почему так? На стороне «многие» есть множество записей, которые соответствуют только одной на стороне единицы. И если внешний ключ будет со стороны последней, придется дублировать эту запись множество раз, ровно столько, сколько есть записей на стороне «многие». Но это невозможно.

В нашем случае на стороне «многие» находится сущность review, а ее внешний ключ ссылается на первичный ключ сущности user. Добавим обозначение связи на диаграмму:



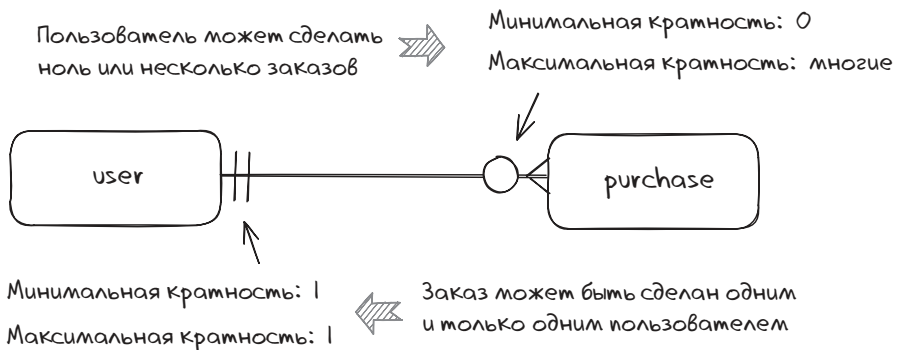
Связи «один ко многим» на практическом примере

Теперь, по аналогии со связью между user и review, нанесем на диаграмму связи «один ко многим» и между другими сущностями нашей базы данных. Вернемся к собранной ранее информации:

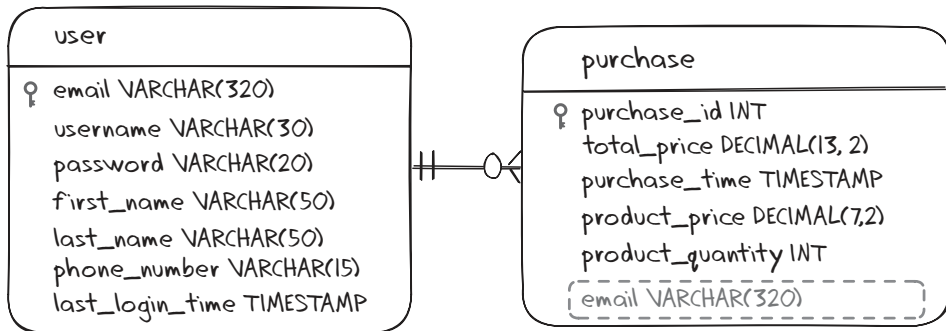
- Пользователь может сделать несколько заказов. Заказ может быть сделан только одним пользователем.
- Пользователь может выбрать несколько способов оплаты. Способ оплаты может применяться только одним пользователем.
- [Новое правило]. Способ оплаты может использоваться для нескольких заказов. Один заказ может быть оплачен только одним способом оплаты.
- В заказ может входить несколько товаров. Товар может входить в несколько заказов.

Легко заметить, что связь между product и purchase не соответствует типу «один ко многим», поскольку ни в одном из направлений максимальная кратность не равна единице.

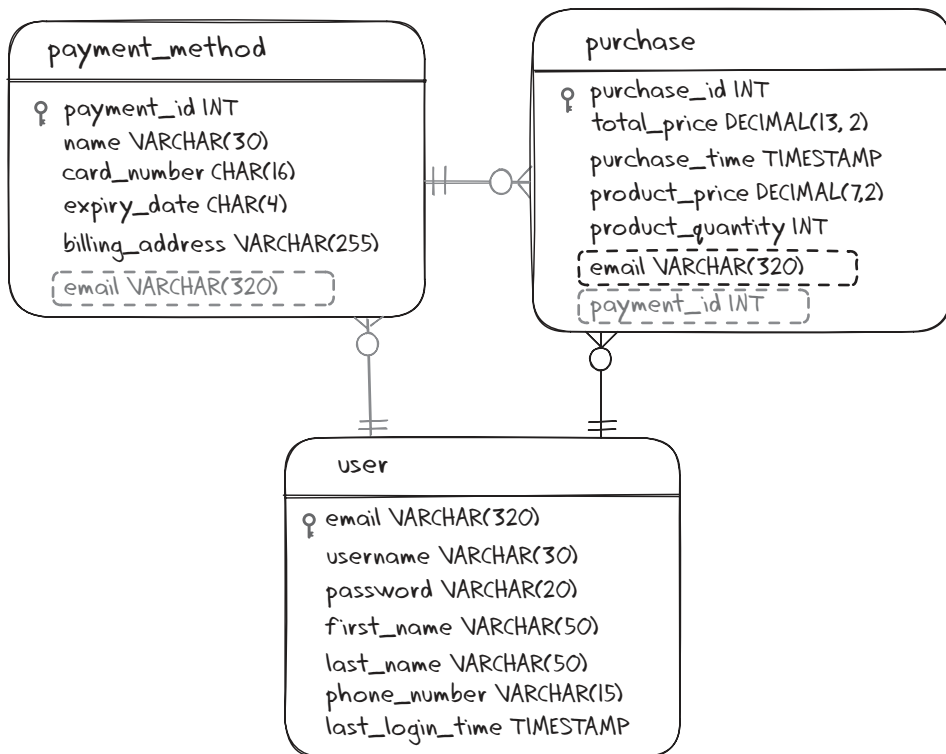
С другой стороны, связь между user и purchase подходит под это определение: максимальная кратность в одном направлении (от purchase к user) равна единице, а в противоположном (от user к purchase) — «многие». Определить минимальную кратность немного сложнее. Что означает фраза «пользователь может сделать несколько заказов»? Мы можем спросить об этом работников «Фантастической лавки» или же вспомнить о собственном опыте: иногда, просмотрев каталог, мы закрываем сайт магазина, так ничего и не выбрав. Пользователь может не сделать ни одного заказа, а значит, минимальная кратность равна нулю. Обновим сущности user и purchase на диаграмме:



Осталось добавить на сторону «многие» внешний ключ:



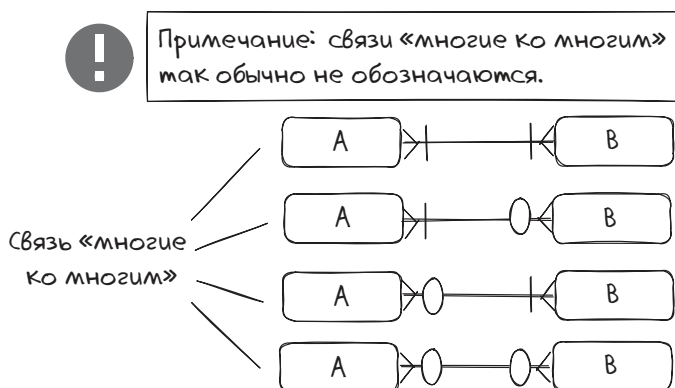
Аналогичным образом определим связи между **user** и **payment_method**, а также **purchase** и **payment_method**. Обновленная диаграмма будет выглядеть так:



Как видно из этого рисунка, у сущности `payment_method` две связи с сущностью `user`: прямая и через `purchase`. Подобные связи чреваты проблемами, которые мы обсудим в главе 6.

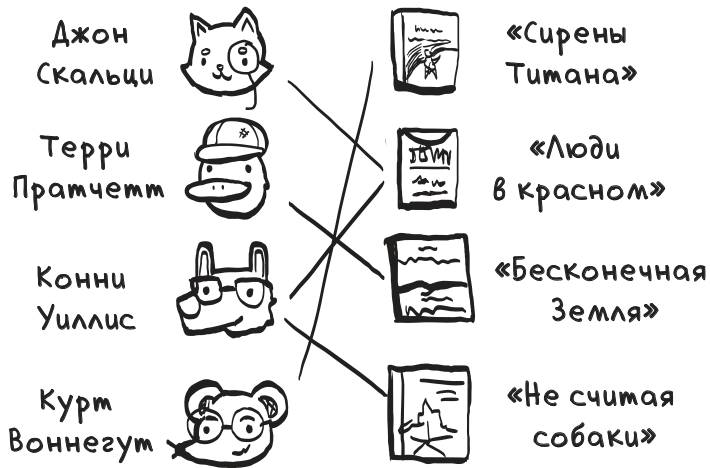
Многие ко многим: чем больше, тем веселее

Осталось рассмотреть связи последнего типа — «многие ко многим» (many-to-many). Как и в двух прошлых случаях, название отражает максимальную кратность связи: в обоих направлениях она равна «многие»:

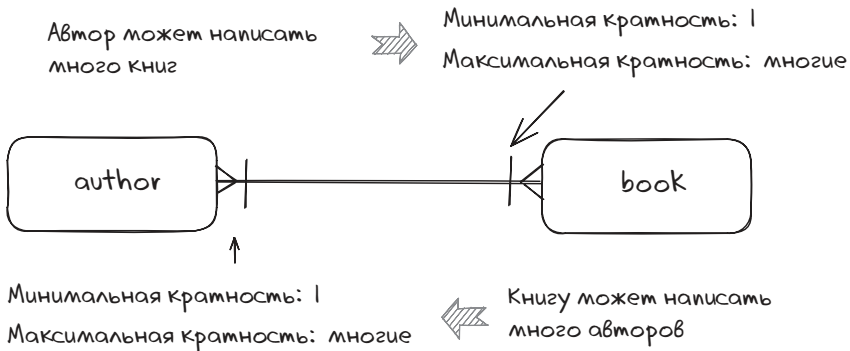


На диаграммах связи этого типа обозначаются по-другому, но этот рисунок поможет понять общий смысл. Как правило, связь «многие ко многим» реализуется при помощи *промежуточной таблицы* (junction table), с которой каждая из связанных сущностей имеет связь «один ко многим». Посмотрим, как это делается, на следующем примере.

Допустим, нам нужно спроектировать базу данных для книжного магазина. Сведения об авторах и книгах будут храниться в двух таблицах. Но как всем известно, один автор может написать несколько книг, а одну книгу — несколько авторов.

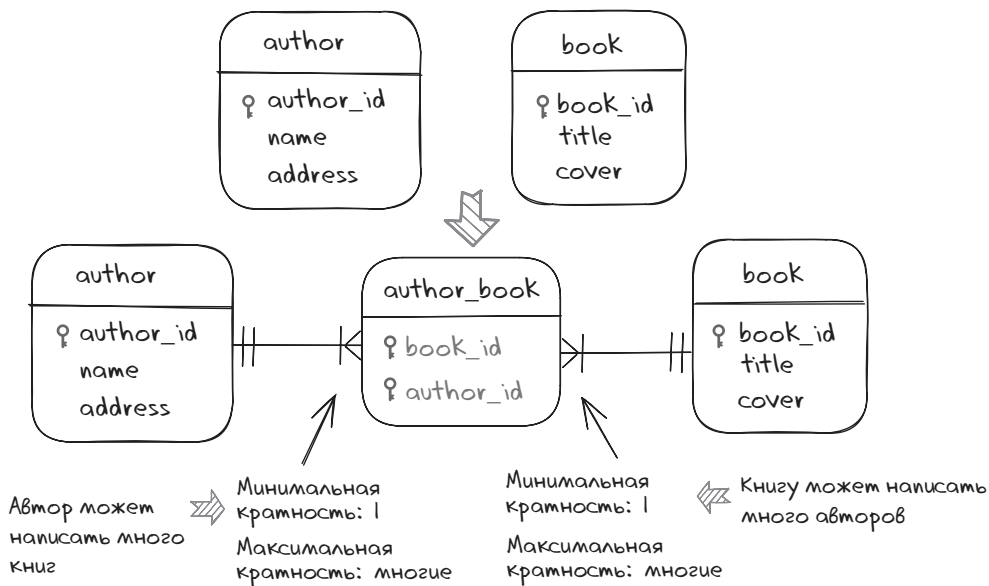


О том, что перед нами связь «многие ко многим», легко судить по максимальной кратности в обоих направлениях:



Реализация этой связи потребует дополнительного шага, а именно создания промежуточной таблицы, состоящей лишь из первичных ключей двух сущностей — author (автор) и book (книга) — и сопряженной с каждой из них связью «один ко многим». Эти две связи заменят

собой исходную связь «многие ко многим». Итоговая диаграмма будет выглядеть так:



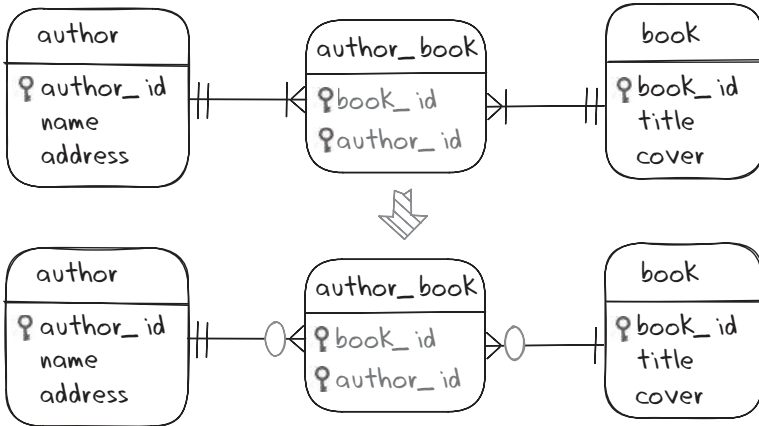
Новая, промежуточная таблица `author_book` содержит только первичные ключи сущностей `author` и `book` — свойства `author_id` и `book_id` соответственно. Вместе два этих свойства образуют так называемый *составной первичный ключ* (composite primary key). Сущности `author` и `book` теперь связаны косвенно, через промежуточную таблицу, а «птичьи лапки», как можно заметить, направлены только в ее сторону.

Составные первичные ключи: уникальные сочетания

В реляционной базе данных составной первичный ключ состоит из двух или большего количества свойств (столбцов), которые совместно однозначно идентифицируют каждую строку таблицы. В отличие от первичного ключа, основанного на одном столбце, составной первичный ключ формирует уникальный идентификатор каждой записи на основании нескольких столбцов.

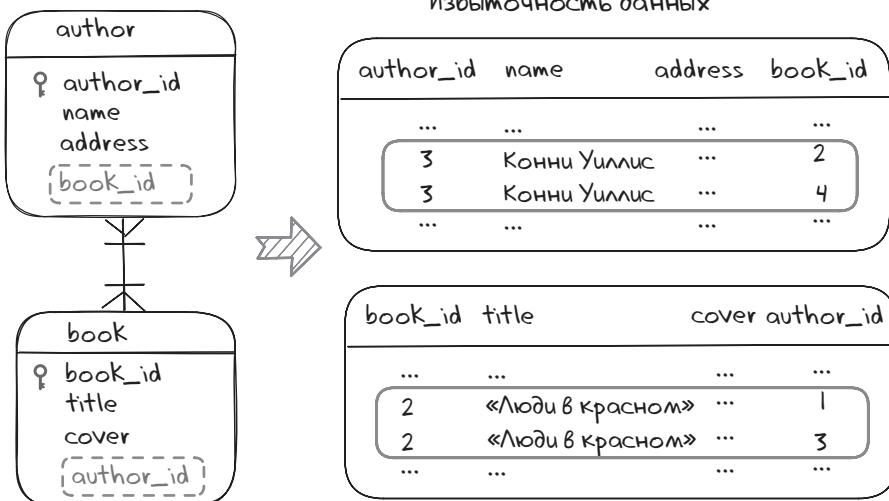
Помимо распространенного подхода к реализации связи «многие ко многим», такие ключи необходимо использовать, когда при помощи одного столбца нельзя гарантировать уникальность всех строк таблицы, а при помощи нескольких — можно.

Поскольку теперь мы имеем дело с двумя связями «один ко многим», поступим с ними так же, как раньше: обнулим минимальную кратность со стороны «многие», чтобы не допустить возможных проблем на этапе реализации:



Вы спросите: почему нельзя просто добавить по внешнему ключу к каждой из сущностей со связью «многие ко многим»? Все дело в том, что это затруднит изменение данных и усложнит запросы, но прежде всего приведет к избыточности. Вот, например, что мы увидим в таблицах **author** и **book**:

Плохое решение



Если автор написал несколько книг, данные о нем придется внести в таблицу `author` несколько раз, ведь каждое произведение имеет собственный `book_id`, на который должен ссылаться внешний ключ. То же самое происходит и в таблице `book`, если у книги несколько авторов. Помимо избыточности, мы неизбежно столкнемся с проблемами при загрузке данных в таблицы. Поэтому такая реализация связи «многие ко многим» невозможна.

Упражнения с кратностью: тренировка гибкости при обнулении

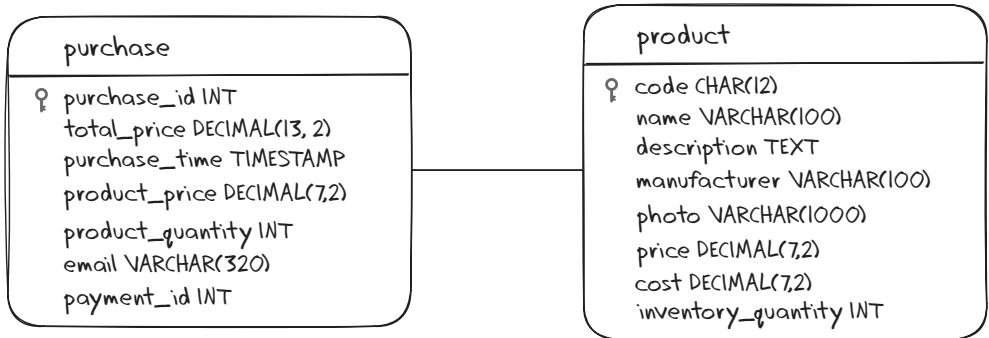
По умолчанию минимальная кратность часто равна единице. Но чтобы добавить в таблицу данные, в некоторых случаях ее приходится обнулять:

- При связи «один к одному» необходимо обнулить минимальную кратность с любой из сторон (обычно для той сущности, которая создается позже другой). Это единственная возможность внести записи в эту таблицу, пока соответствующие им записи в другой таблице отсутствуют.
- При связи «один ко многим» следует обнулить минимальную кратность со стороны «многие». В этом случае записи в таблице на стороне единицы смогут существовать без соответствующих записей в таблице на стороне «многие»: к примеру, покупатель может зарегистрироваться, не указывая адрес.
- При связи «многие ко многим» минимальную кратность часто обнуляют с обеих сторон, если обе сущности могут существовать независимо друг от друга. Если порядок их существования определяется строгими бизнес-правилами, можно оставить одну из двух минимальных кратностей равной единице.

Из этих сугубо практических правил могут быть исключения: к примеру, обнуление бывает запрещено правилами компании либо требованиями заказчика. Однако можно следовать этим правилам для выявления связей, минимальная кратность которых требует дальнейшего анализа.

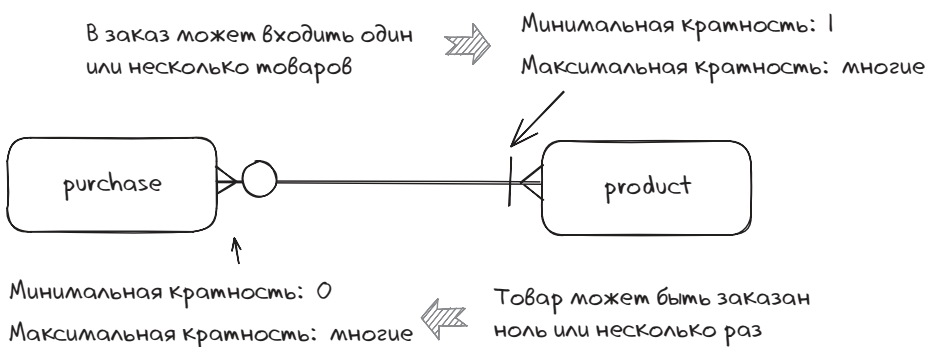
Связи «многие ко многим» на практическом примере

На диаграмме базы данных «Фантастической лавки» осталось всего две сущности — `purchase` и `product`, — над связью между которыми нам предстоит поработать.



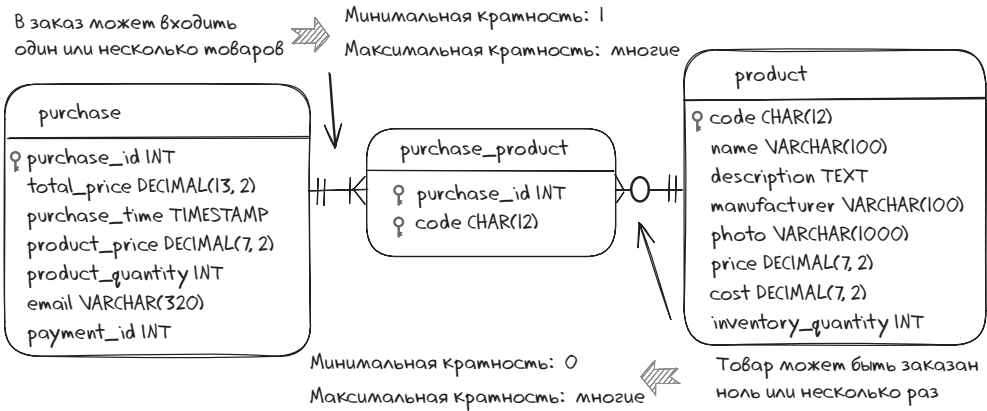
Как нам стало известно из первоначальных бесед со стейкхолдерами, в заказ может входить несколько товаров, а товар — в несколько заказов. В ходе уточнения требований мы выяснили, что заказ должен состоять хотя бы из одного товара, тогда как товар может быть не куплен ни разу.

Из этого можно сделать вывод, что перед нами связь «многие ко многим», а также определить минимальную и максимальную кратность:

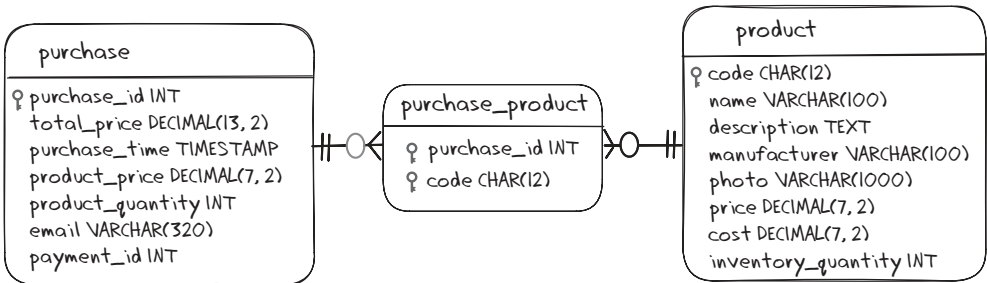


Для выполнения этих требований создадим промежуточную таблицу с двумя столбцами — первичными ключами сущностей `purchase`

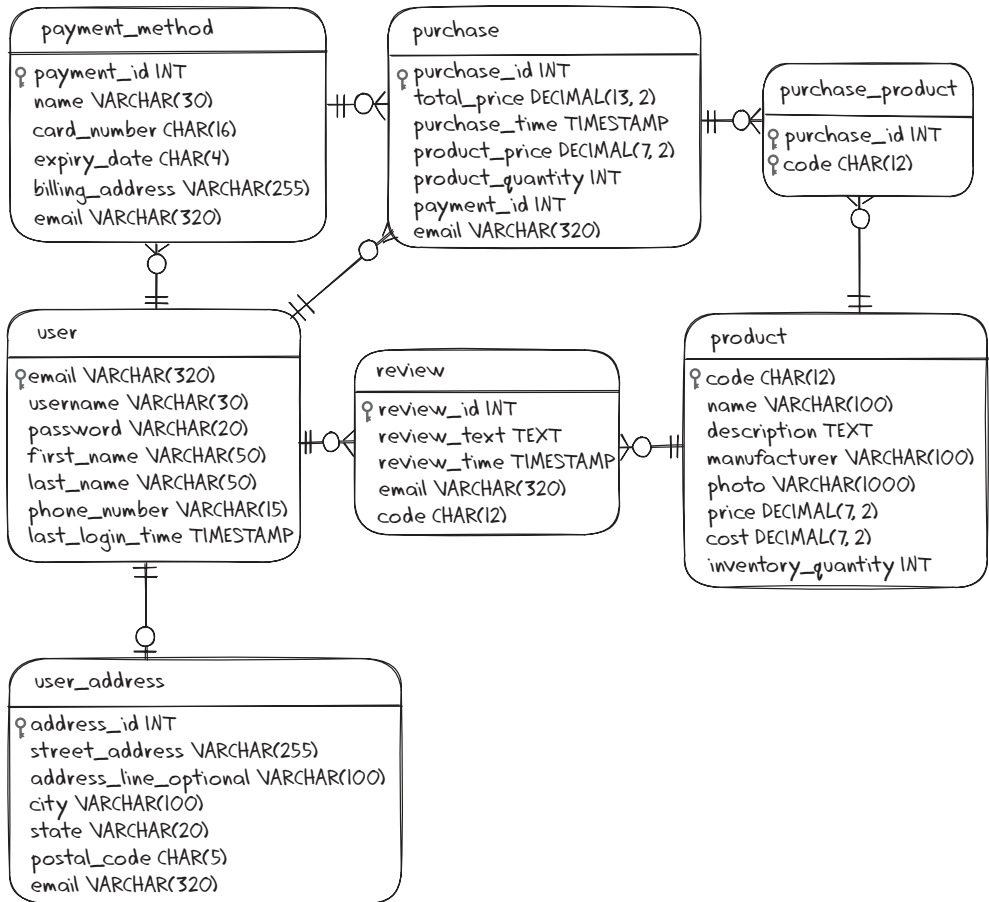
и product, благодаря чему преобразуем связь «многие ко многим» в две связи «один ко многим»:



Как видно из этого рисунка, минимальная кратность связи между **purchase** и **purchase_product** с обеих сторон равна единице. Во избежание проблем при загрузке данных необходимо обнулить кратность со стороны таблицы **purchase_product**, как на следующем рисунке.



После выполнения всех этих действий черновая ER-диаграмма будет выглядеть так:

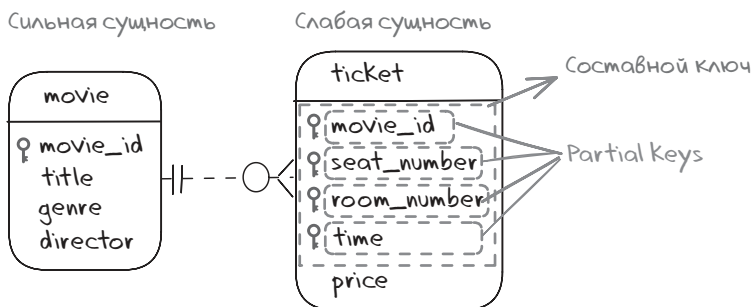


Сильные и слабые сущности

Сила и слабость сущностей — еще одно свойство связи между ними, которое помогает при разработке ER-диаграмм. Поговорив о нем в этом разделе, мы постараемся применить полученные знания на практике, продолжив работу над базой данных «Фантастической лавки».

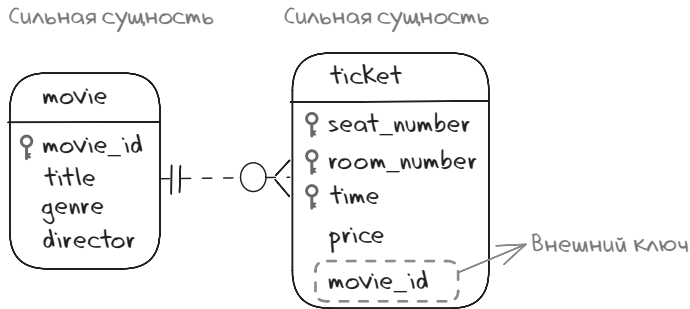
Называя сущности *сильными* (strong) или *слабыми* (weak), мы характеризуем их *зависимость* друг от друга. Если одна из двух связанных сущностей — сильная, а другая — слабая, первая может существовать сама по себе, тогда как вторая не может существовать без первой. Иными словами, свойства слабой сущности не позволяют однозначно определять ее экземпляры без ссылок на экземпляры сильной.

Рассмотрим пример такой пары сущностей. Допустим, мы проектируем базу данных для кинотеатра, в которой в том числе хранится информация о фильмах (movie) и билетах (ticket). Мы можем сказать, что сущность movie является сильной, поскольку данные о фильмах не нуждаются в сведениях о билетах на них, а сущность ticket — слабая: без информации из таблицы movie нельзя сказать, что за фильм мы будем смотреть. Изобразим это на диаграмме:



Обычно первичный ключ слабой сущности является составным, то есть сочетает в себе ее собственные свойства с первичным ключом связанной сильной сущности. Как видно из рисунка выше, у сущности ticket имеется составной ключ, в состав которого входит movie_id (идентификатор фильма) — первичный ключ сущности movie, а также другие свойства, которые помогают однозначно определить каждый билет на этот фильм. Такие свойства называются *частичными ключами* (partial key).

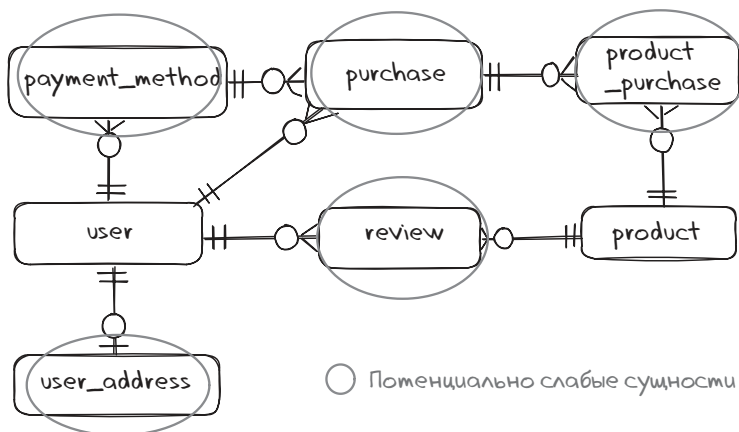
Нужно отметить, что слабую сущность всегда можно сделать сильной. Если нам нужно работать с билетами независимо от фильмов (к примеру, повторно продавать билеты, сданные зрителями), имеет смысл трансформировать сущность ticket в сильную:



Но если мы можем сделать сущность как сильной, так и слабой, какой вариант выбрать? Ответ зависит от множества факторов, к примеру потребностей бизнеса, требований к отображению данных, производительности и удобства обслуживания. Если возможны оба варианта, остановиться следует на том, который упростит ER-диаграмму и понизит вероятность появления ошибок. Как мы увидим далее, в нашем проекте такое решение придется принять не один раз.

Сильные и слабые сущности на практическом примере

В базе данных «Фантастической лавки» есть несколько сущностей, которые можно назвать слабыми, например сущность review (отзыв), которая зависит от user и product:



Решая, будет ли сущность сильной или же слабой, необходимо подумать о том, какой из вариантов упростит проект или повысит точность данных. Какие преимущества даст нам слабая сущность review? Никаких. При этом мы будем вынуждены работать с составным ключом, что увеличит объем базы данных и снизит ее производительность, поскольку индексы составных ключей занимают много места, а поиск по ним требует больше времени. Поэтому сущность review должна быть сильной. Все то же самое справедливо и для сущностей payment_method и purchase.

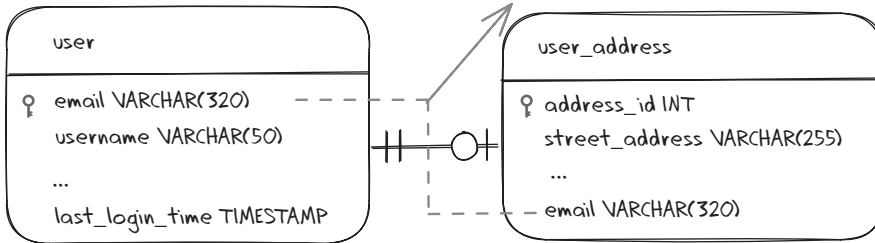


А как быть с сущностью purchase_product? С ней ничего решать не придется: промежуточные таблицы, которые связывают две сущности (в данном случае purchase и product), слабы по определению. Действительно, ее первичный ключ является составным и представляет собой сочетание первичных ключей двух сильных сущностей — purchase и product.

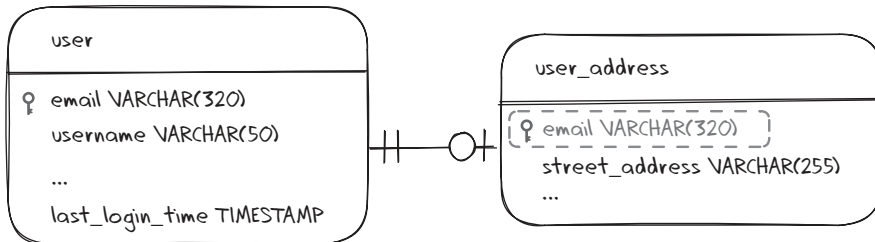
О том, что делать с сущностью user_address, придется подумать более тщательно. На данный момент она является сильной сущностью с первичным ключом address_id и внешним ключом email, который обеспе-

чивает связь с сущностью user. Если мы трансформируем user_address в слабую сущность, ее первичным ключом станет email, а внешний ключ исчезнет. Почему так? Связь между user и user_address относится к типу «один к одному», поэтому свойства email вполне достаточно для идентификации. Составной первичный ключ здесь не требуется. Такое преобразование имеет смысл, поскольку, избавившись от ограничения по внешнему ключу, мы упростим проект.

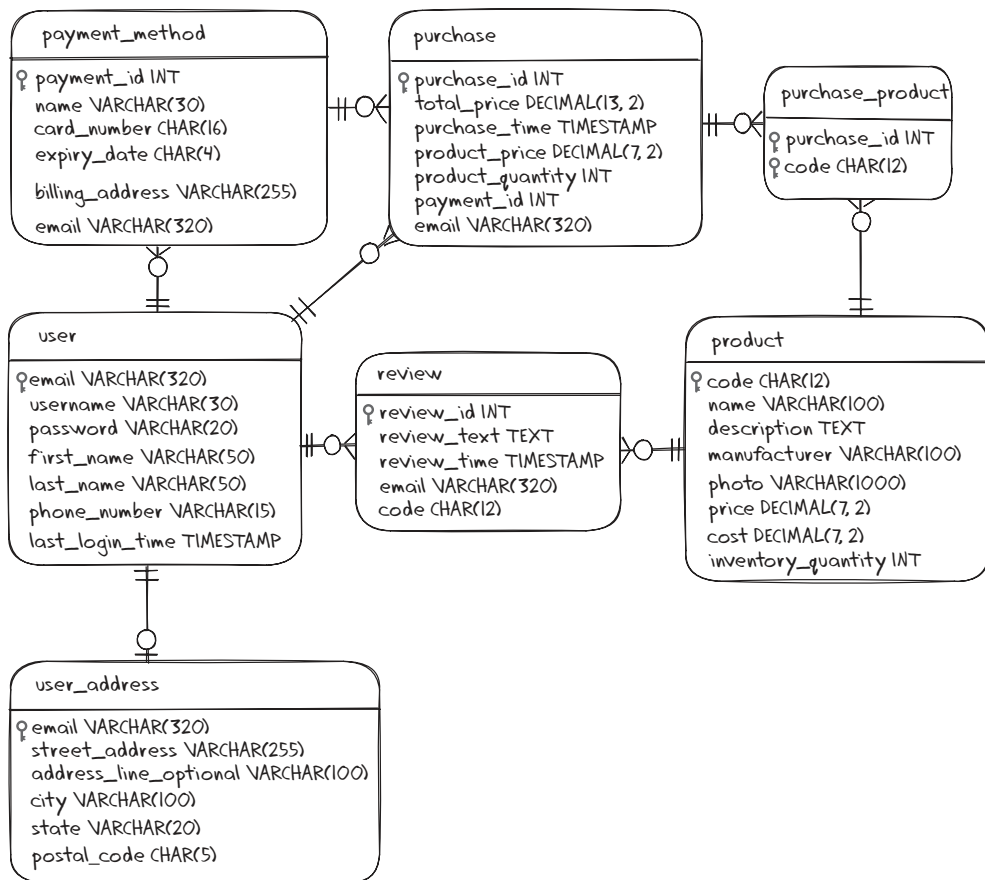
Две сильные сущности



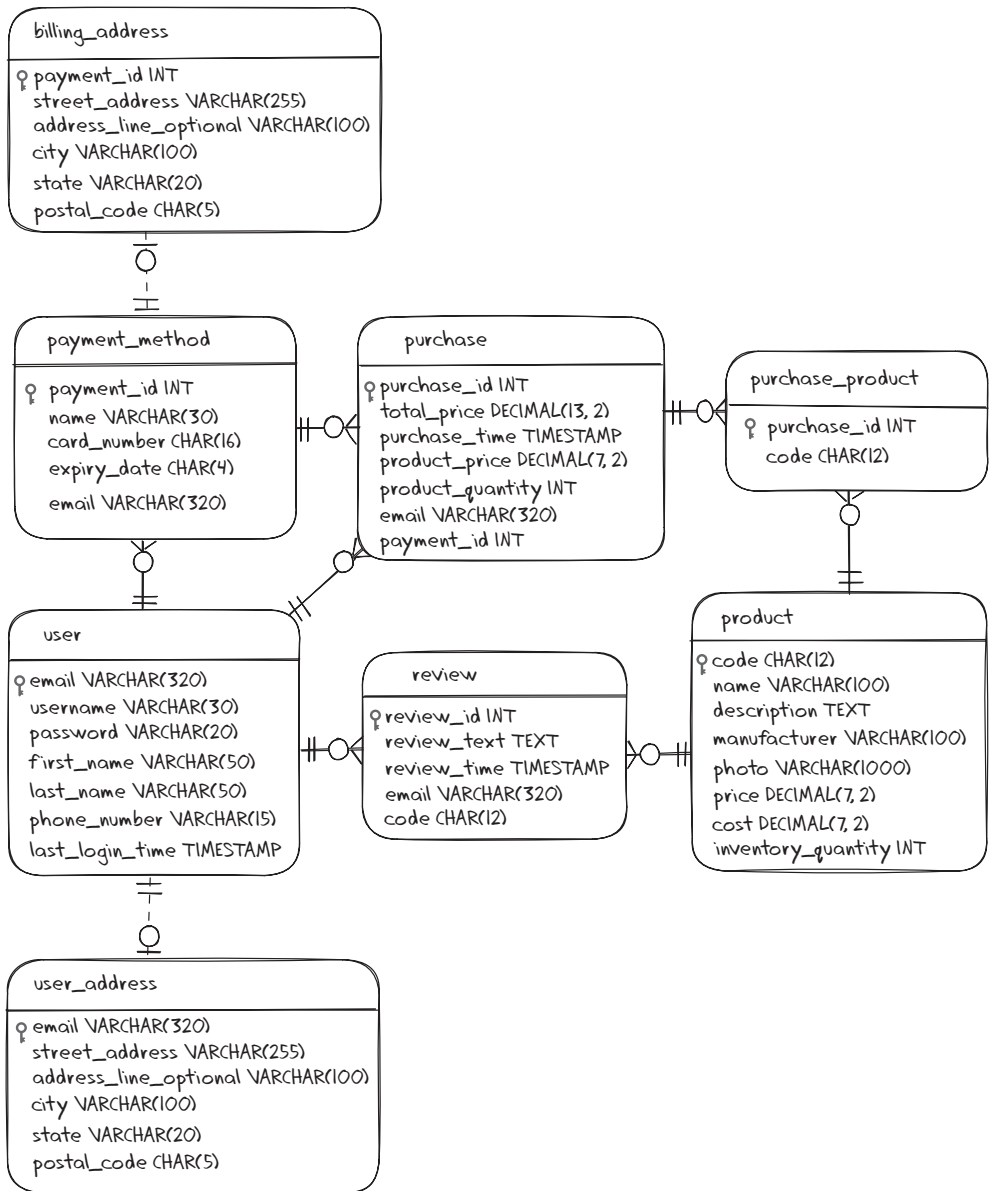
Сильная и слабая сущности



После преобразования `user_address` в слабую сущность ER-диаграмма проекта будет выглядеть так:



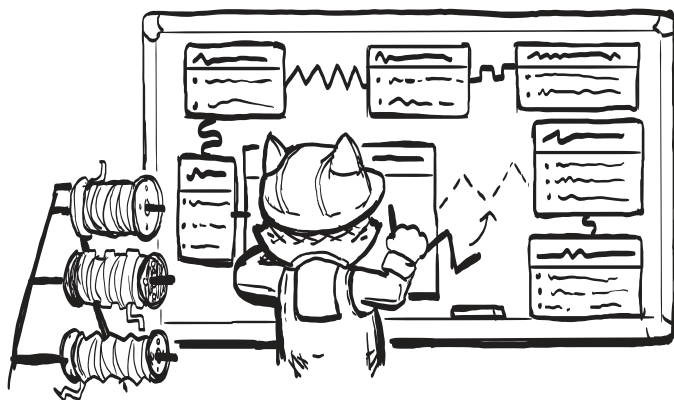
Если внимательно посмотреть на сущность `payment_method`, можно заметить аналогию между свойством `billing_address` — адресом для выставления счетов — и свойством `address` сущности `user`, которое мы заменили отдельной таблицей `user_address`. В обоих случаях мы имеем некую сущность (`user` и `payment_method`), одним из свойств которой является адрес (место жительства или место, куда нужно присылать счета). Поэтому будет логично выделить свойство `billing_address` в отдельную таблицу, подобную `user_address`. Это будет слабая сущность, связанная с сильной сущностью `payment_method`. В результате ER-диаграмма проекта изменится следующим образом:



Итак, мы завершили моделирование данных, создали и доработали ER-диаграмму проекта. Отличная работа!

Итоги

- Связи соединяют сущности друг с другом. Сведения, необходимые для определения связей, получают на этапе сбора требований, а также в ходе их дальнейшего анализа.
- Чтобы отобразить связь на ER-диаграмме, следует выполнить три действия: установить наличие связи, определить ее кратность, после чего решить, какую из сущностей можно и целесообразно трансформировать в слабую.



- Кратность — важная характеристика связи между двумя сущностями. В зависимости от нее связи делятся на три типа: «один к одному», «один ко многим» и «многие ко многим». Кратность определяет, как связь отображается на диаграмме, а также в какую из сущностей нужно добавить внешний ключ.
- У связи «один к одному» (обозначение — две линии) максимальная кратность с обеих сторон равна единице. Внешний ключ может быть у любой из связанных сущностей.
- У связи «один ко многим» максимальная кратность с одной стороны равна единице, с другой — «многие». Внешний ключ должен находиться со стороны «многие», она обозначается «птичьей лапкой».
- У связи «многие ко многим» максимальная кратность с обеих сторон равна «многие». Реализация связи такого типа требует создания промежуточной таблицы, с которой обе сущности соединяются по типу «один ко многим». Промежуточная таблица содержит ссылки на первичные ключи обеих сущностей.

- Еще одна характеристика связи заключается в том, что сущности, которые она соединяет, могут быть сильными или слабыми. Трансформировать сильную сущность в слабую, если это возможно, имеет смысл только при условии, что это упрощает ER-диаграмму и делает базу данных более эффективной.
- Первичный ключ слабой сущности, как правило, является составным и включает в себя первичный ключ сильной сущности. Графически сила и слабость сущностей не обозначаются.

6 | Нормализация и реализация



В этой главе

- ✓ Нормализация проекта базы данных
- ✓ Реализация проекта базы данных
- ✓ Использование ограничений и каскады

Необходимые знания

Проект рассматриваемой в этой главе базы данных, реализованный при помощи часто применяемых на практике инструментов dbdiagram.io и MySQL Workbench, можно найти в репозитории GitHub (<https://github.com/Neo-Hao/grokking-relational-database-design>). Чтобы загрузить проект в выбранный инструмент, перейдите в папку `chapter_06` и следуйте инструкциям из файла `README.md`.

В той же папке находятся SQL-скрипты для реализации почти готового проекта базы данных в разных СУБД, включая MySQL, MariaDB, PostgreSQL, SQLite, SQL Server и Oracle.

Введение

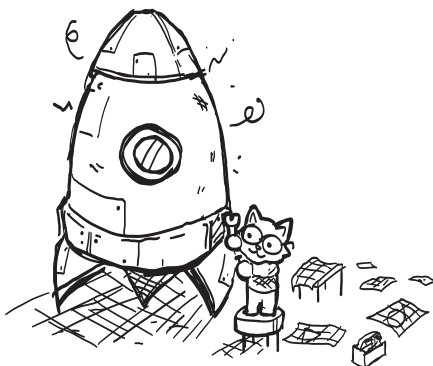
В этой главе мы займемся нормализацией проекта и реализацией базы данных «Фантастической лавки». При этом мы изучим несколько важных понятий, в том числе поговорим о функциональной зависимости, нормализации и ограничениях.



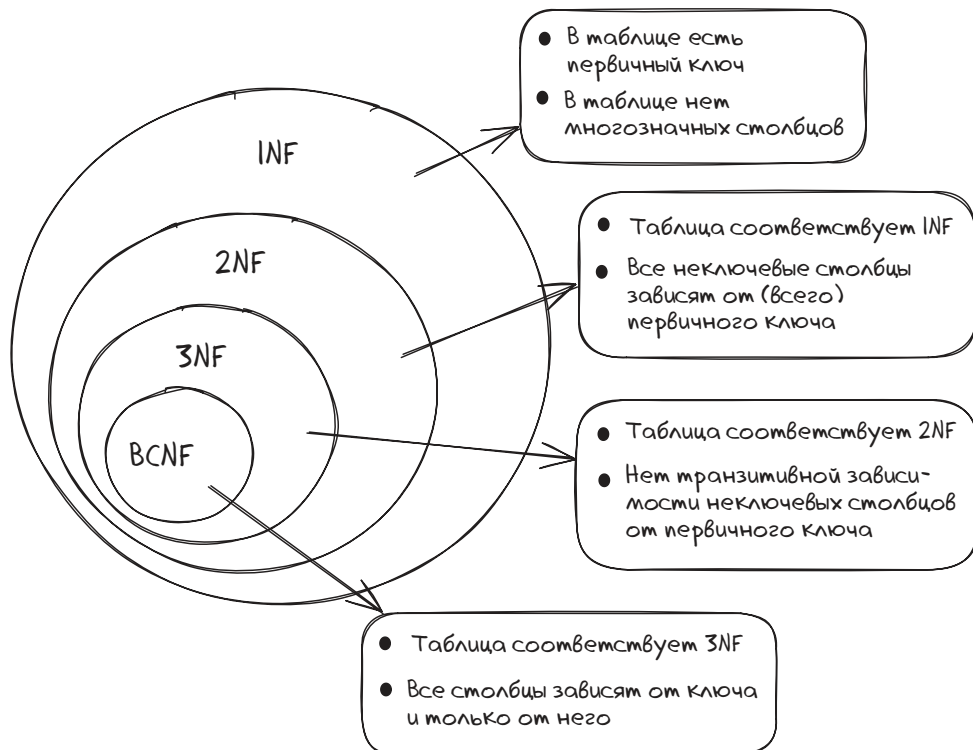
Нормализация

Прежде чем перевести проект базы данных на язык SQL и написать скрипт для создания соответствующих таблиц, необходимо выполнить нормализацию. Этот этап проектирования очень важен.

Нормализация (normalization) — это процесс оптимизации базы данных, в ходе которого избыточность и зависимости сводятся к минимуму, а целостность и согласованность приводятся к максимуму. Иными словами, нам предстоит разделить базу данных на менее объемные, но более удобные в работе таблицы, каждая из которых будет описывать только одну сущность или понятие. Основная цель нормализации — укрепление целостности данных. В предыдущих главах мы уже многое сделали в этом плане, сейчас же закончим начатое.



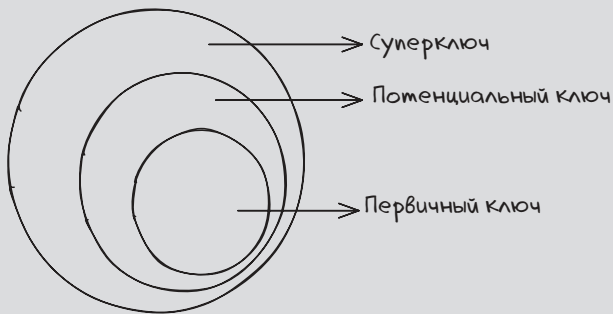
В процессе нормализации проект последовательно принимает несколько *нормальных форм* (normal form), в том числе первую (1NF), вторую (2NF), третью (3NF), а также нормальную форму Бойса — Кодда (BCNF). Все эти формы образуют иерархию, то есть требования к любой из них включают в себя требования ко всем предыдущим. Последней в этом ряду является BCNF.



Суперключи, потенциальные и первичные ключи

Помните, что это такое? Мы говорили о разных ключах еще в главе 4.

Суперключ — это сочетание одного или нескольких столбцов, которое однозначно идентифицирует строку таблицы, но может включать в себя столбцы, которые для этого не требуются. *Потенциальный ключ* — это минимальный суперключ: он не содержит ненужных столбцов. Из всех имеющихся в таблице потенциальных ключей один становится первичным. Взаимосвязь между этими видами ключей можно изобразить так:



На практике базу данных можно считать полностью нормализованной, если все таблицы в ней приведены к форме BCNF (а значит, и к формам 1NF-3NF), то есть соответствуют следующим условиям:

- в таблице есть первичный ключ;
- в таблице нет многозначных столбцов;
- все столбцы зависят от ключа и только от него;
- транзитивные зависимости отсутствуют.

Если хотя бы одно из условий не выполняется, мы, вероятно, столкнемся с такими проблемами, как аномалии обновления и удаления. Помните эксперимент с хранением данных о товарах и клиентах в одной таблице (см. главу 1)?

product_id	name	price	manufacturer	customer_id	customer_name	customer_email	quantity
1	Атомный триммер ...	1999	ООО «Безумные инженеры»	a1	Боб	bob@gmail.com	5
2	Селфи-мостер	2499	ЗАО «Потешные гаджеты»	b2	Дэйв	dave@outlook.com	15
3	Кошачий ...	2999	АО «Абсурдные аксессуары»	a1	Боб	bob@gmail.com	2
...	...	...	...	...	...	...	...
9	Генератор ...	999	ООО «Всякая всячина»	j8	Джон	john@123.net	1
10	Нейтрализатор	33.55	ООО «Всякая всячина»	p9	Кейм	katy@123.net	2

В следующих разделах мы подробно поговорим о каждом из этих условий, после чего приступим к нормализации базы данных «Фантастической лавки».

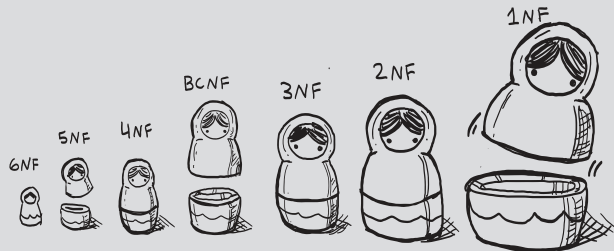
Нормальные формы: безумные матрешки

Вы спросите: BCNF — это последняя нормальная форма или же есть и другие? Конечно же есть. В теоретических работах по нормализации рассматривают четвертую, пятую и даже шестую нормальную форму (4NF, 5NF и 6NF соответственно).

Каждая из них строится на основе предшествующих и ориентирована на все более узкие и менее распространенные задачи. К примеру, 4NF устраняет многозначные зависимости, а 5NF — избыточность, вызванную совместными зависимостями, которые не затрагиваются формой 4NF. Форму 6NF, по большому счету, можно считать сугубо теоретической.

Но все они выходят за рамки этой книги. Желаящим познакомиться с ними стоит прочесть учебник по базам данных (к примеру, A. Silberschatz, H. F. Korth, S. Sudarshan: *Database System Concepts*, 7th ed. (McGraw-Hill Education, 2019)) или же обратиться к ChatGPT со следующим вопросом:

Расскажи мне о формах 4NF, 5NF и 6NF простым языком и с примерами.



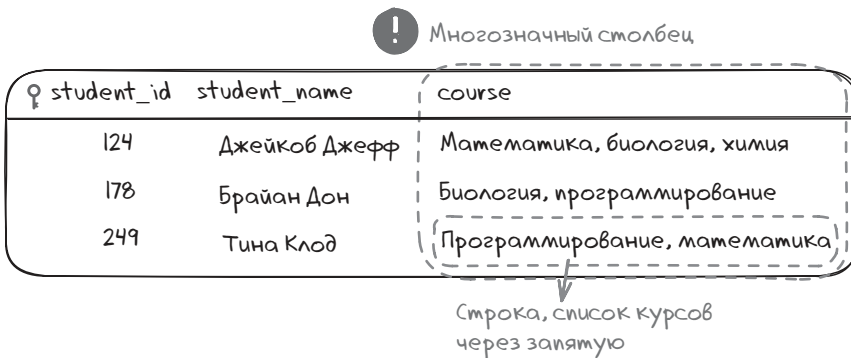
Наличие первичного ключа

В любой таблице, приведенной к форме BCNF, имеется первичный ключ. Взгляните на диаграмму, которую мы построили в прошлой главе. Вы видите таблицы без первичных ключей? Отлично. Стало быть, наш проект соответствует первому требованию формы BCNF.

А если бы это было не так? Тогда бы пришлось вернуться на шаг назад и заняться определением первичных ключей.

Отсутствие многозначных столбцов

Форма BCNF исключает наличие многозначных столбцов. Что это такое? Представьте себе таблицу `course_registration` (регистрация на курсы) с данными о студентах и курсах, на которые они зарегистрировались. Значения в столбце `course` (курс) представляют собой списки курсов, что и делает его многозначным.



student_id	student_name	course
124	Джейкоб Джефф	Математика, биология, химия
178	Брайан Дон	Биология, программирование
249	Тина Клод	Программирование, математика

Многозначные столбцы — это проблема: они усложняют запросы, содержат избыточные данные (как, например, в этой таблице), легко приводят к появлению неточностей и аномалий.

Кому решать, что столбец многозначен?

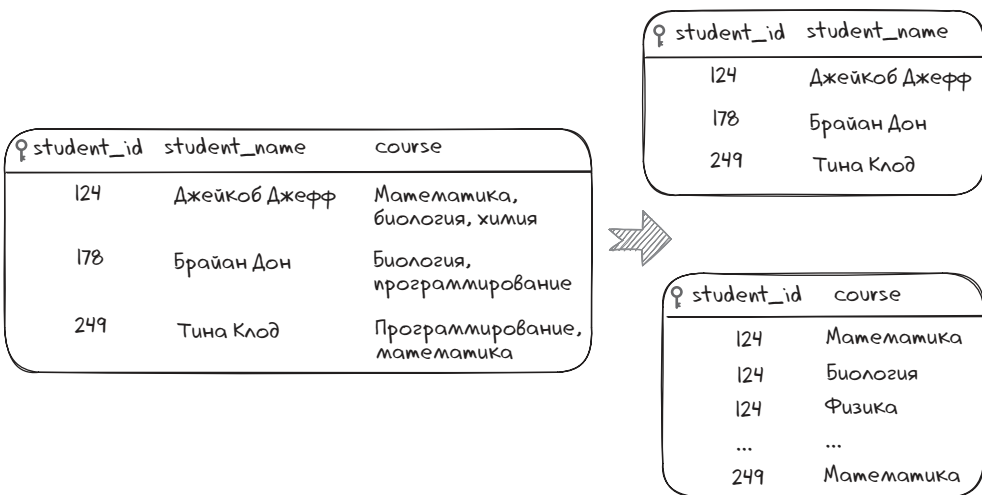
Таблица реляционной базы данных — это отображение сущности, сведения о которой нам нужно хранить, столбец же — отображение одного из свойств этой сущности. Каждый столбец имеет имя и тип данных. Но вот вопрос: а кто решает, что будет сущностью, а что — ее свойствами? Ответ — пользователи приложения, для которого мы

проектируем базу данных. Они же и формируют требования, которые позволяют нам судить о многозначности.

К примеру, у нас есть таблица с данными о клиентах магазина, а в ней — столбец с номерами телефонов. В каждом из них можно выделить код города и номер абонента, и в этом смысле сказать, что такой столбец многозначен. Однако в контексте магазина в подобном делении нет никакого смысла, поскольку телефон воспринимается неделимым свойством человека. По той же причине не стоит хранить отдельно друг от друга имена пользователей и доменов, составляющие адреса электронной почты.

Таким образом, решение о многозначности столбца зависит от целей базы данных, которые определяются требованиями пользователей.

Чтобы решить эту задачу, обычно следует изменить многозначный столбец так, чтобы в каждой строке в нем было только одно значение. Чаще всего в этом случае добавляется новая таблица, что устраняет избыточность и другие проблемы. К примеру, мы можем потребовать, чтобы в столбце `course` было не больше одного курса на строку. Но мы не можем использовать одно и то же значение `student_id` в нескольких строках, поскольку это первичный ключ. Поэтому мы разобьем таблицу `course_registration` на две: в одной из них будут данные о студентах, в другой — о курсах:



Остались ли в этом примере многозначные столбцы? Нет. Если создать таблицы согласно диаграмме, ни в одном из столбце не будет более одного значения. В противном случае работу придется продолжить аналогичным образом.

Все столбцы зависят от ключа и только от него

Все столбцы таблицы, приведенной к форме BCNF, должны функционально зависеть от ключа. Чтобы понять, что это означает, сначала нужно поговорить о функциональной зависимости.

Представьте себе функцию на любом языке программирования. Возьмем, к примеру, функцию `power(x)`, которая принимает один параметр `x` и возвращает его квадрат. При равных значениях `x` результат всегда будет одним и тем же. Иными словами, выходное значение функционально зависит от входного.

Теперь посмотрим, как это работает в базах данных. Если в какой-то таблице любому значению из столбца `A` однозначно соответствует значение из столбца `B`, можно сказать, что столбец `B` функционально зависит от столбца `A`. В следующей таблице, в которой хранятся данные о сотрудниках, столбец `employee_name` (имя сотрудника) функционально зависит от столбца `employee_id` (идентификатор сотрудника), ведь, зная идентификатор, мы без труда узнаем имя. Эту зависимость можно выразить так:

```
employee_id -> employee_name
```

employee_id	employee_name
101	Джон
102	Эмили
103	Майк
104	Сара

Функциональная зависимость имеет направление. На рисунке `employee_name` функционально определяется по `employee_id`, но не наоборот: по имени нельзя узнать идентификатор работника. Еще бы, ведь два совершенно разных человека могут быть тезками.

Теперь вернемся к требованиям формы BCNF:

Все столбцы таблицы должны быть функционально зависимы от ключа и только от него.

Вы спросите, почему просто от ключа, а не только от первичного? Это тонкий момент, так как имеются пограничные случаи. Впрочем, на практике чаще всего нам действительно не о чем беспокоиться: можно считать, что:

Все столбцы таблицы должны быть функционально зависимы от первичного ключа и только от него.

Глубокое понимание BCNF

Понимание BCNF и всех остальных нормальных форм помогает при нормализации (пока не сведет с ума). Пограничные случаи, которые помогут углубить это понимание, выходят за рамки этой книги, но все-таки о них стоит упомянуть:

1. В таблице R три столбца: a, b и c, первичный ключ — (a, b). Если c функционально зависит от a, R не находится в форме 2NF, не говоря уже о 3NF и BCNF.
2. В таблице R пять столбцов: a, b, c, d и e, первичный ключ — (a, b, c). Столбцы d и e функционально зависят от (a, b, c). Если c функционально зависит от d, R находится в форме 3NF, а не BCNF.
3. В таблице R пять столбцов: a, b, c, d и e, первичный ключ — (a, b). Имеется также потенциальный ключ (c, d). Если e функционально зависит только от (c, d), R все равно находится в форме BCNF.

Впрочем, не стоит ломать над этим голову: случаи типа 2 и 3 — крайняя редкость.

Иными словами, если один или несколько столбцов не находятся в функциональной зависимости от первичного ключа, необходима нормализация. Но как ее осуществить? Следует разделить нормализуемую таблицу на несколько таблиц, в простейшем случае — на две. В каждой из них должны остаться только столбцы, зависимые от первичного ключа. При необходимости можно добавить в таблицы внешние ключи.

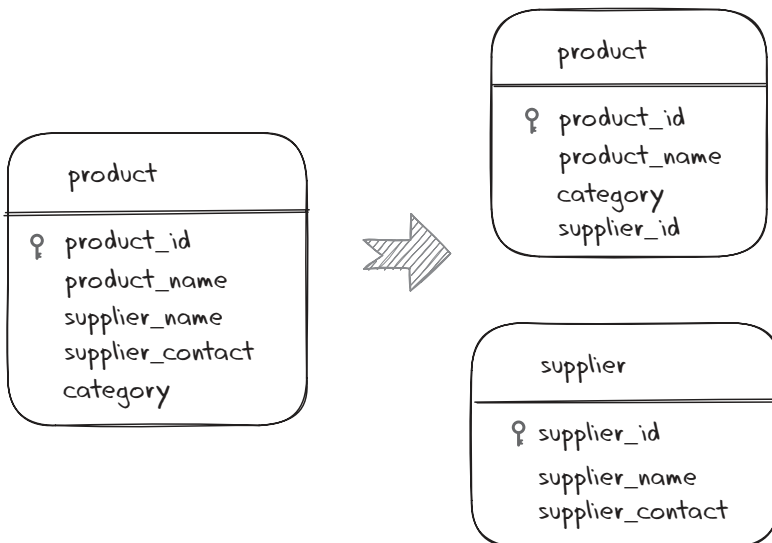
В качестве примера рассмотрим таблицу product, спроектированную новичком:

♀ product_id	product_name	supplier_name	supplier_contact	category
1224	Звездная пыль	Brewlux	555-0123	Приправы
3378	Машина времени	TimeCo	555-0456	Оборудование
2549	Волшебная палочка	Magico	555-0789	Оборудование

С первого взгляда логично: когда телефон поставщика находится рядом с его названием — это довольно удобно. Однако столбец `supplier_contact` (телефон поставщика) функционально зависит лишь от столбца `supplier_name` (название поставщика), то есть в этой таблице две зависимости:

```
product_id -> product_name, supplier_name, category
supplier_name -> supplier_contact
```

Чтобы исправить ситуацию, необходимо убрать столбцы, которые не зависят напрямую от первичного ключа. Куда же их поместить? В новую таблицу. Столбец `supplier_contact` функционально зависит от `supplier_name`. Если мы выделим их в таблицу с данными о поставщиках, мы сможем полностью выполнить требование к функциональной зависимости. У каждого поставщика появится уникальный идентификатор, который станет первичным ключом новой и внешним ключом нормализованной таблицы:



Нормализация на практическом примере

Анализируя базу данных «Фантастической лавки» на предмет функциональной зависимости столбцов, можно заметить проблемы в таблице purchase:

purchase	
♀	purchase_id INT
	total_price DECIMAL(13, 2)
	purchase_time TIMESTAMP
	product_price DECIMAL(7, 2)
	product_quantity INT
	email VARCHAR(320)
	payment_id INT

Без сомнения, многие из столбцов функционально зависят от первичного ключа, в том числе total_price, purchase_time, payment_id и email. Если посмотреть на чек, можно заметить, что все, что должно попасть на бумагу, включая total_price и purchase_time, можно узнать по значению purchase_id:

Фантастическая лавка		
Чек за покупку		
2023-01-01 00:00:00		
# 311205		
	Количество	Цена
Атомный триммер для волос	1	5
Селфи-тостер	2	3
Кошачий наполнитель с ароматом кофе	3	2
Надувной чемодан	3	2
	Итого:	12



Вот, например, как может выглядеть чек. Каждый заказ состоит из нескольких товаров по разной цене (`product_price`) и в разных количествах (`product_quantity`). Цена и количество функционально зависят от сочетания артикула товара (`code`) и номера заказа (`purchase_id`). Однако артикул в этой таблице отсутствует, а значит, у нас опять две зависимости в одной таблице:

`product_id` → `total_price`, `purchase_time`, `payment_id`, `email`
`purchase_id`, `code` → `product_quantity`, `product_price`

Итак, таблица `purchase` нарушает требования BCNF. Стоит начать вводить в нее данные, как проблема сразу станет очевидной. К примеру, добавить сведения с нашего чека попросту невозможно:

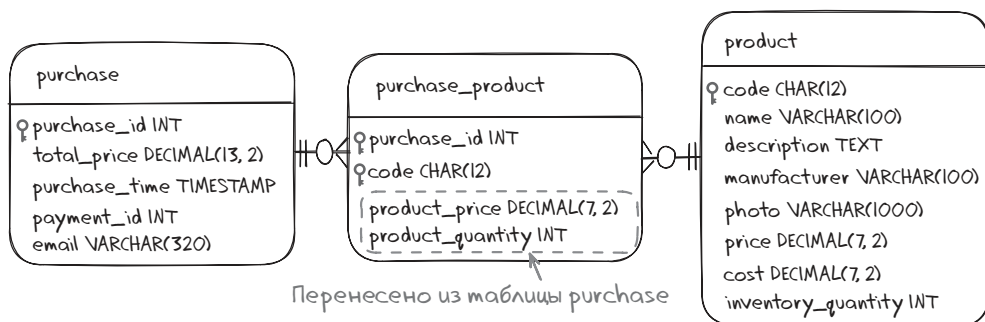
<code>purchase_id</code>	<code>total_price</code>	<code>purchase_time</code>	<code>product_id</code>	<code>product_price</code>	...
1335	12	01 01 2023 00:00:00	1	5	
1335	12	01 01 2023 00:00:00	2	3	...
1335	12	01 01 2023 00:00:00	3	2	
1335	12	01 01 2023 00:00:00	4	2	
...	...	...	...	...	

Дублировать значения первичного ключа нельзя. А если бы было и можно, мы бы столкнулись с избыточностью в других столбцах: `total_price` и `purchase_time`.

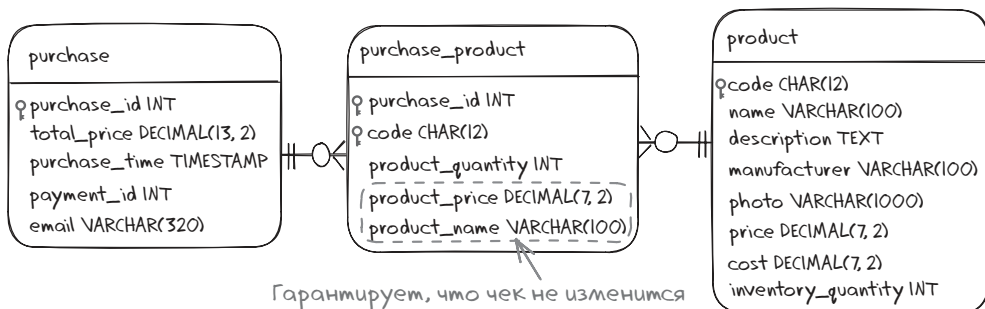
Поэтому нужно перенести столбцы, функционально независимые от первичного ключа, в другую таблицу. В нашем случае получается, что `product_quantity` и `product_price` зависят от `purchase_id` и `code`, причем не поодиночке, а совместно:

- Сам по себе `purchase_id` не определяет `product_quantity` или `product_price`. Чтобы определить количество и цену заказанного товара, необходимо знать его артикул.
- Сам по себе `code` не определяет `product_quantity` или `product_price`. Чтобы определить количество и цену товара в конкретном заказе, необходимо знать его номер.

К счастью, у нас уже есть промежуточная таблица `purchase_product`, которая соединяет заказы и товары, а в качестве первичного ключа использует сочетание `purchase_id` и `code`. Переместив `product_quantity` и `product_price` в эту таблицу, мы одним выстрелом убьем двух зайцев: приведем и `purchase` и `purchase_product` в соответствие с требованиями к функциональной зависимости:

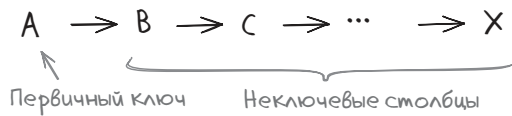


Вы спросите, а почему не удалить `product_price` из таблицы `purchase_product`, ведь цену товара мы можем узнать из таблицы `product`. Лучше так не делать. Со временем цена может измениться из-за инфляции или рыночной конкуренции. Поэтому, не сохраняя текущую стоимость товара в момент заказа, мы потеряем возможность просматривать чеки, выданные несколько месяцев назад. По той же причине стоит добавить в таблицу `purchase_product` столбец с названием товара: даже если оно изменится, это никак не скажется на старых чеках.

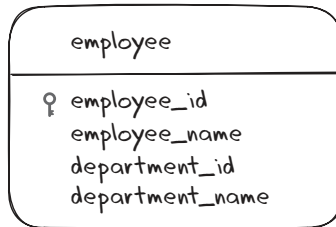


Отсутствие транзитивных зависимостей

В нормальной форме 3NF в таблице не должно быть *транзитивных зависимостей* (transitive dependency), которые появляются, когда что-то зависит от чего-то, зависящего от чего-то еще. В контексте базы данных транзитивная зависимость означает, что неключевой столбец функционально зависит от другого неключевого столбца, который функционально зависит от ключа таблицы. При этом промежуточных столбцов в такой цепочке может быть сколько угодно.



В таблице с данными о сотрудниках есть следующие столбцы и функциональные зависимости:



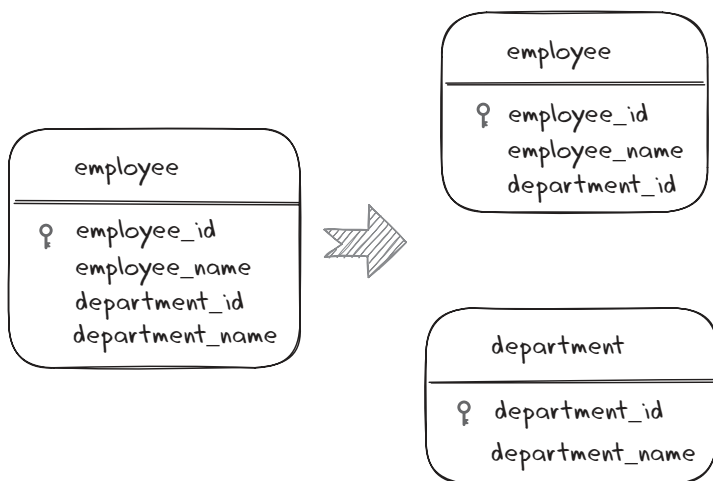
$employee_id \rightarrow department_id \rightarrow department_name$

Первичный ключ таблицы employee — столбец employee_id. В этой таблице department_name зависит от department_id, а тот, в свою очередь, — от employee_id. Такая цепочка функциональных зависимостей и называется транзитивной.

Транзитивная зависимость — это проблема, которая влечет за собой другие, от избыточности до аномалий всех типов. Чтобы изменить название отдела (`department_name`) в таблице `employee`, потребуется обновить все строки, где он упоминается:

<code>employee_id</code> ...	<code>department_id</code>	<code>department_name</code>	
125	30	Финансы	Избыточность
137	30	Финансы	
189	39	Маркетинг	
260	30	Финансы	Необходимо обновить все строки
310	42	ИТ	

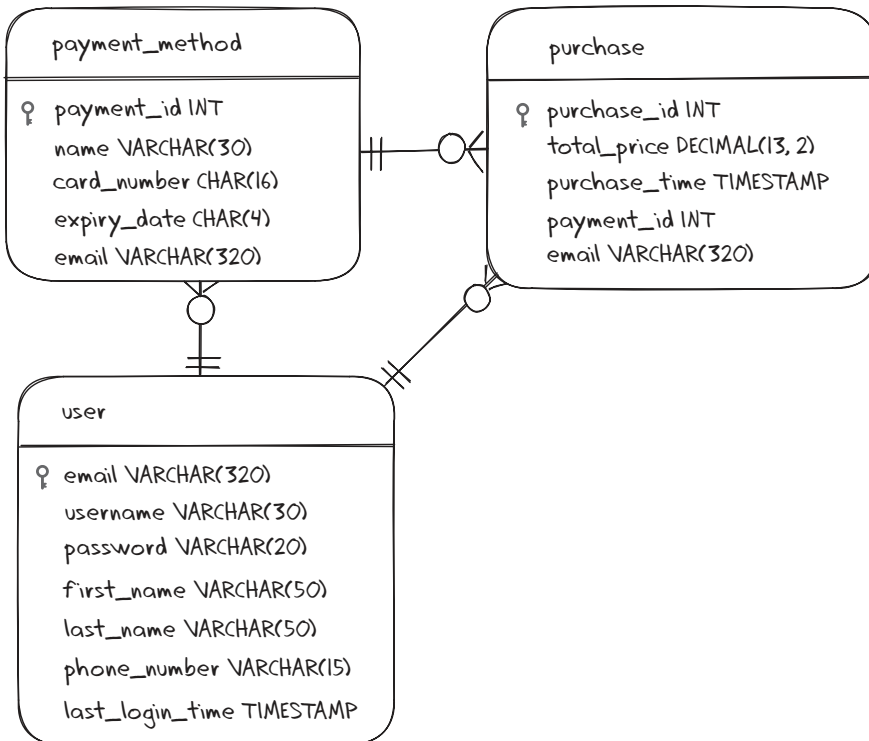
Как избавиться от транзитивной зависимости? Разделить таблицу на несколько таблиц. При этом все столбцы новых таблиц должны зависеть от первичных ключей и только от них. При необходимости следует добавить подходящие внешние ключи. Действуя по этому правилу, таблицу `employee` можно трансформировать так:



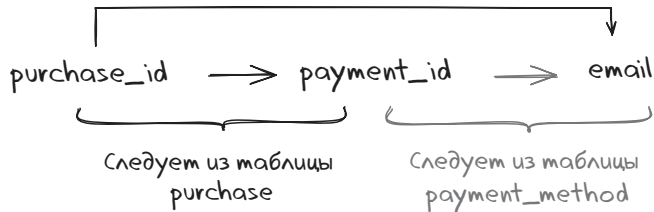
В новой таблице `department` оказался неключевой столбец, ранее бывший частью цепочки, а теперь функционально зависящий от `department_id` — нового первичного ключа. В таблице `employee` остались только столбцы, которые функционально зависят от первичного ключа (`employee_id`) и только от него, и среди них — столбец `department_id`, ставший теперь внешним ключом. Внешний ключ всегда зависит от первичного, поскольку их сочетание в любой строке уникально.

Нормализация на практическом примере: кольцо из трех таблиц

Заметить транзитивную зависимость бывает непросто. Для этого нужно глубоко понимать данные, тщательно изучить собранные требования и примеры данных. В таблице `purchase` из нашего примера имеется два внешних ключа: `email` и `payment_id`, которые связывают ее с таблицами `payment_method` и `user`:

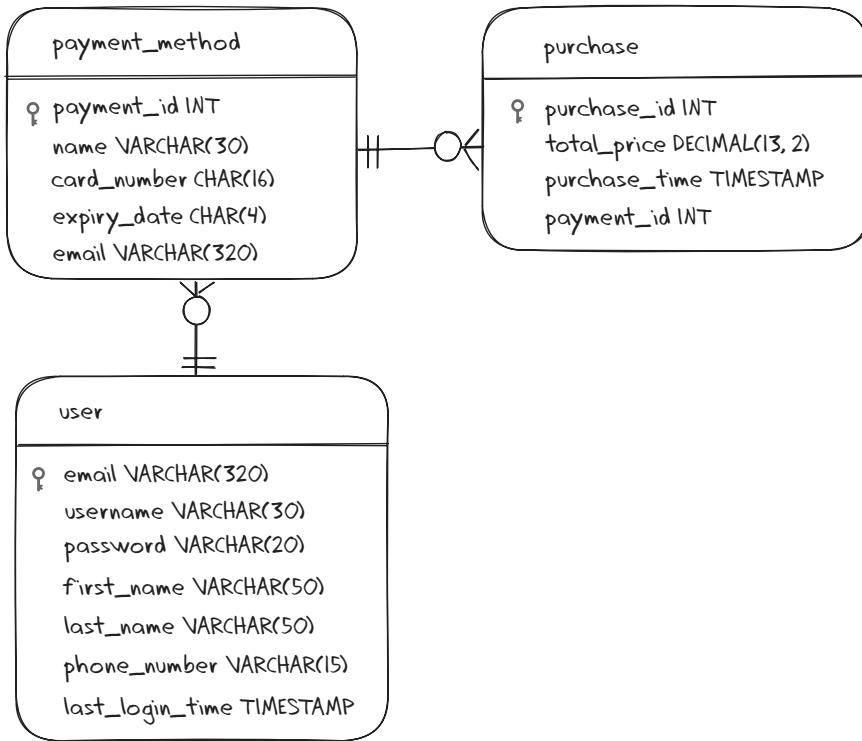


В самой таблице `purchase` все столбцы, включая внешние ключи, бесспорно, зависят от первичного ключа. Но если выйти за ее пределы, то можно заметить нечто неожиданное: два внешних ключа этой таблицы — `payment_id` и `email` — зависимы друг от друга. Действительно, в таблице `payment_method` столбец `payment_id` является первичным ключом, а `email` — зависимым от него внешним, связывающим таблицу `payment_method` с `user`. Иными словами, в таблице `purchase` имеет место транзитивная зависимость:



Теоретически для соединения трех таблиц достаточно двух связей. Если же связей три и они образуют кольцо на ER-диаграмме, где-то наверняка есть транзитивная зависимость.

Как устранить транзитивную зависимость? Все тем же способом: в таблице должны остаться только столбцы, которые зависят от первичного ключа и только от него. Удалим столбец `email` из таблицы `purchase`, чтобы устранить прямое соединение между ней и `user`. При этом таблицы будут по-прежнему связаны через `payment_method`, как видно на следующем рисунке.

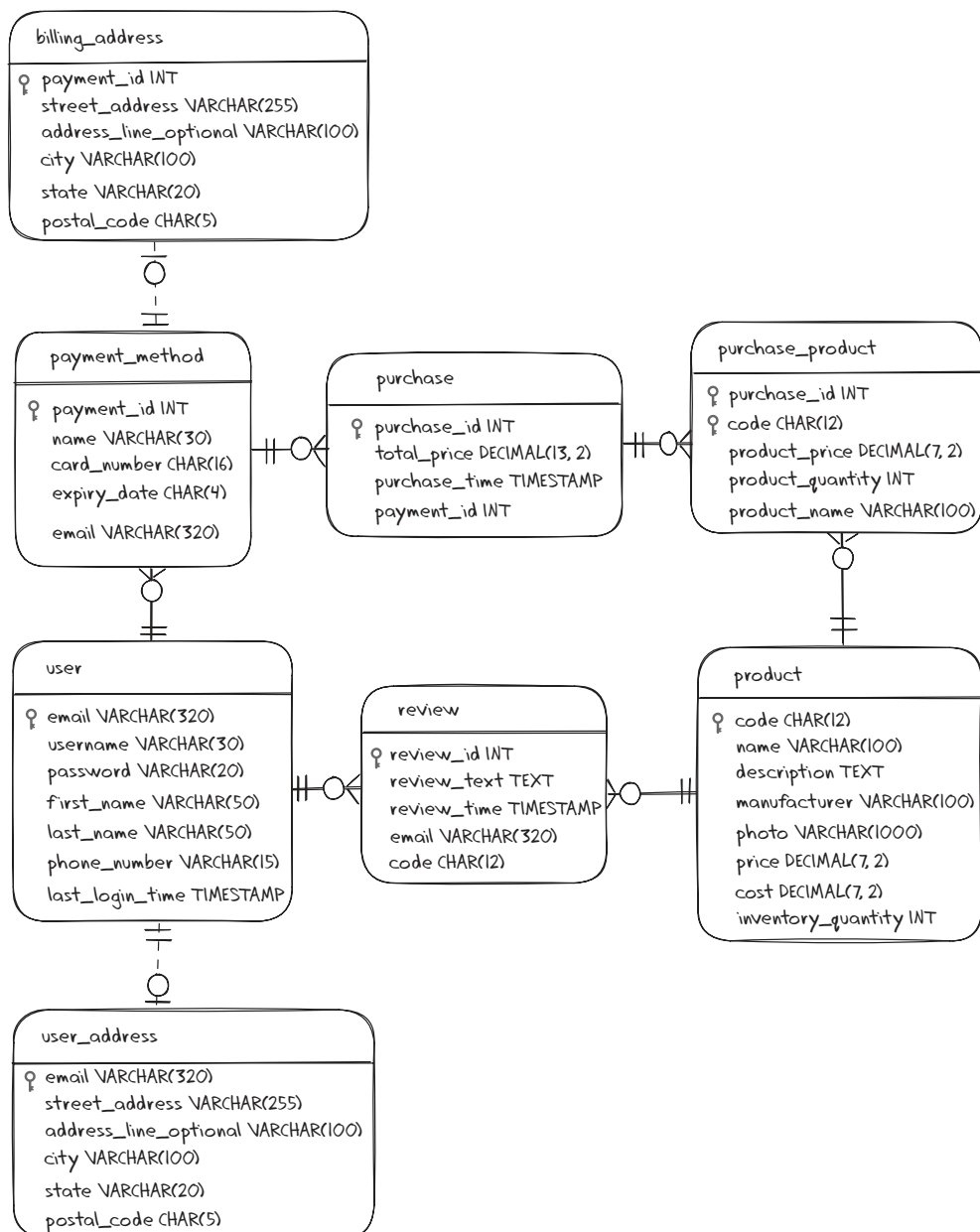


Если поставить нормализацию во главу угла, удаление связи между таблицами `user` и `purchase` будет отличным решением. Однако если задуматься об иных проблемах, к примеру о производительности и накладных расходах, оно может оказаться совсем не идеальным. Поэтому в следующей главе мы вновь поговорим об этих таблицах.

Просмотрев все остальные таблицы, мы не увидим других транзитивных зависимостей. Иными словами, мы привели базу данных к нормальной форме 3NF:

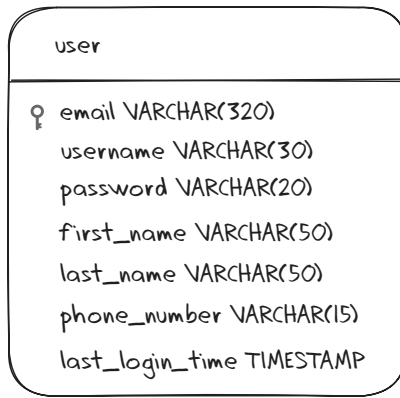
- у всех таблиц есть первичные ключи;
- ни в одной из таблиц нет многозначных столбцов;
- во всех таблицах все столбцы зависят от ключей и только от них;
- транзитивные зависимости отсутствуют (все неключевые столбцы зависят от ключей напрямую).

Обновленная ER-диаграмма будет выглядеть так:



Реализация

Завершив нормализацию базы данных, перейдем к следующему этапу — реализации. Знаний, полученных еще в главах 1 и 2, вполне достаточно, чтобы без особых усилий перевести диаграмму проекта на SQL и создать все нужные таблицы. К примеру, таблица `user` на диаграмме выглядит так:



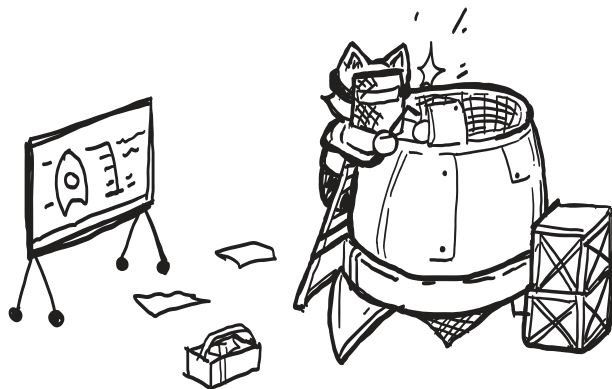
Чтобы ее создать, потребуется следующий код на языке SQL:

```
-- comment: команда для MySQL и MariaDB
-- comment: команду для других СУБД можно найти в репозитории
CREATE TABLE user (
    email VARCHAR(320) PRIMARY KEY,
    username VARCHAR(30),
    password VARCHAR(20),
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    phone_number VARCHAR(15),
    last_login_time TIMESTAMP
);
```

На этом все? Не совсем. Успешная реализация проекта потребует близкого знакомства с ограничениями. Умение работать с ними позволит принимать решения по вопросам, лежащим за пределами ER-диаграмм.

В главе 2 мы уже применяли ограничения, но так и не дали им формального определения. В языке SQL *ограничения* (constraints) — это правила, которые применяются к столбцам таблицы для обеспечения точности и достоверности данных. Ограничения гарантируют целост-

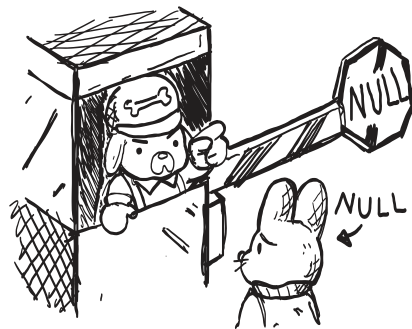
ность данных и могут вводиться как при создании таблицы, так и на более поздних этапах работы. Переживаете, что разработчики приложения будут работать с базой не так, как вы планировали (слоны в посудной лавке)? Введите ограничения, которые им придется соблюдать.



В следующих разделах нас ждет подробный разговор об ограничениях. Конечно же, не обо всех, а лишь о самых распространенных. Мы также обсудим, какие еще решения нужно принять в ходе реализации. При этом мы не будем рассматривать весь SQL-скрипт для создания базы данных: желающие могут найти его в репозитории GitHub (<https://mng.bz/4ao5>).

NOT NULL: всегда есть что сказать

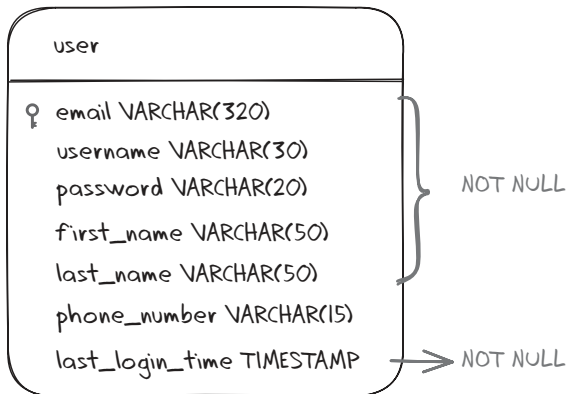
Ограничение NOT NULL гарантирует, что в столбце не будет пустых значений. Если оно введено, при загрузке данных такой столбец нельзя пропустить. Как мы уже знаем из главы 1, NULL — это неизвестное значение, из-за которого при выполнении скриптов часто возникают ошибки. Поэтому NOT NULL — весьма удобный способ предотвратить проблемы с хранением и анализом данных.



Когда же нужно вводить ограничение NOT NULL? Почти всегда. К примеру, без него не обойтись, если в столбце хранятся важные для бизнеса данные (имена пользователей, адреса электронной почты, пароли

и т. п.) или же сведения, наличие которых обязательно по закону или стандартам (к примеру, даты рождения в системах с ограничением по возрасту). Как правило, NOT NULL вводится для внешних ключей связанных таблиц. Чуть позже мы будем рассматривать такие случаи.

Понадобится ли NOT NULL в базе данных «Фантастической лавки»? Конечно да. В нашем проекте немало столбцов с данными, критически важными для бизнеса. К примеру, в таблице user таких столбцов шесть: username, email, password, first_name, last_name и last_login_time.



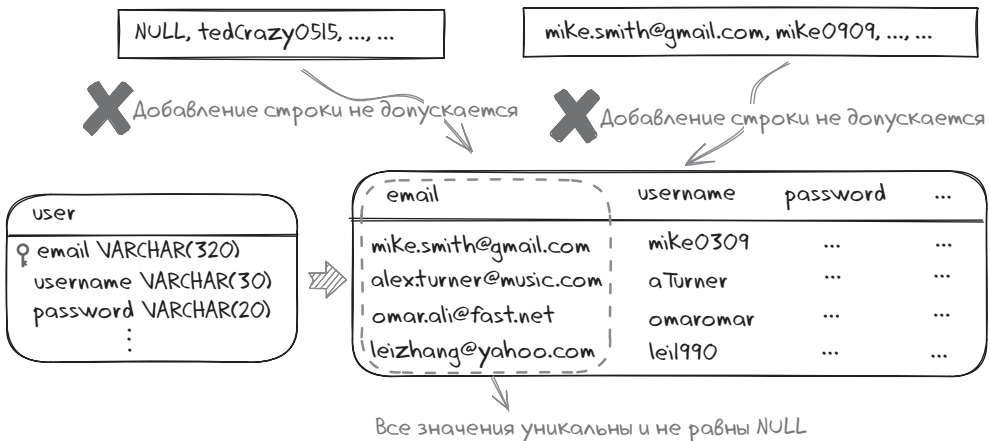
Чтобы ввести ограничение NOT NULL, необходимо добавить эти слова в определение столбца:

```
-- comment: команда для MySQL, MariaDB и SQLite
-- comment: команду для других СУБД можно найти в репозитории
CREATE TABLE user (
  email VARCHAR(320) PRIMARY KEY,
  username VARCHAR(30) NOT NULL,
  password VARCHAR(20) NOT NULL,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  phone_number VARCHAR(15),
  last_login_time TIMESTAMP NOT NULL
);
```

Обратите внимание: определение столбца email не изменилось. Все дело в том, что это первичный ключ, а значит, действует другое ограничение — PRIMARY KEY, которое автоматически включает в себя и NOT NULL.

PRIMARY KEY: единственный и неповторимый

Как мы уже знаем, в хорошем проекте в каждой таблице имеется особый столбец — первичный ключ. На языке SQL такие столбцы получают ограничение PRIMARY KEY, которое гарантирует отсутствие повторяющихся строк, а точнее, строк с одинаковым значением первичного ключа. Кроме того, не допускается наличие строк с первичным ключом, равным NULL.



Синтаксис ограничения очень прост: в определение столбца необходимо добавить слова PRIMARY KEY:

```
-- comment: команда для MySQL, MariaDB и PostgreSQL
-- comment: команду для других СУБД можно найти в репозитории
CREATE TABLE user (
    email VARCHAR(320) PRIMARY KEY,
    username VARCHAR(30) NOT NULL,
    password VARCHAR(20) NOT NULL,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    phone_number VARCHAR(15),
    last_login_time TIMESTAMP NOT NULL
);
```

Если проект базы данных еще не завершен и есть вероятность, что первичный ключ изменится, можно задать имя ограничения явным образом при помощи блока CONSTRAINT, который дополняет определение столбца, как в следующем фрагменте:

```
-- comment: команда для MySQL, MariaDB и PostgreSQL
-- comment: команду для других СУБД можно найти в репозитории
CREATE TABLE user (
  email VARCHAR(320),
  username VARCHAR(30) NOT NULL,
  password VARCHAR(20) NOT NULL,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  phone_number VARCHAR(15),
  last_login_time TIMESTAMP NOT NULL,
  CONSTRAINT pk_user PRIMARY KEY (email)
);
```

Именованное ограничение: лучшие практики

В языке SQL давать имена ограничениям не обязательно, но, в силу следующих причин очень желательно:

- Имена ограничений четко указывают их назначение, а это делает схему базы данных более понятной для всех, кому придется в ней разбираться.
- Если ограничение будет нарушено, его имя появится в сообщении об ошибке. Такие сообщения более информативны и позволяют быстрее устранить проблему.
- При отсутствии явно указанного имени СУБД сформирует его автоматически. Однако такие имена обычно не слишком осмысленны, а их формат зависит от СУБД, что может вызвать путаницу, особенно если понадобится перенести базу данных в другую систему.

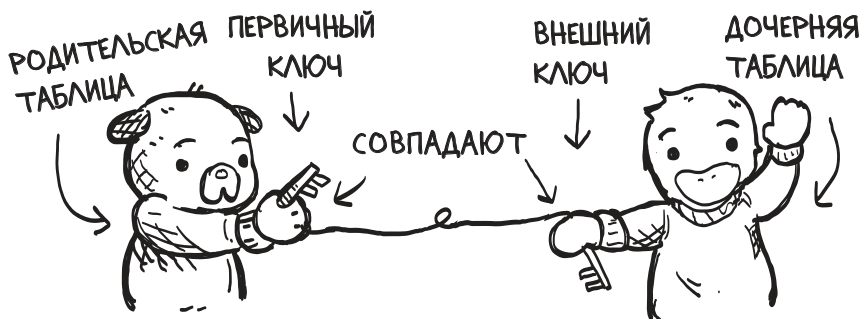
Некоторые ограничения, к примеру NOT NULL и DEFAULT, как правило, не имеют имен, поскольку вводятся при определении столбцов: такой подход облегчает и разработку, и чтение SQL-скриптов. Что же касается других ограничений, прежде всего PRIMARY KEY и FOREIGN KEY, то если есть хоть малейшая вероятность изменения или же переноса базы в другую СУБД, лучше всего заранее дать им явные имена.

А как быть с составными первичными ключами? Ключ из нескольких столбцов можно определить в отдельном блоке SQL, как показано ниже. В базе данных «Фантастической лавки» как раз имеется такая таблица — `purchase_product`, первичный ключ которой состоит из двух столбцов: `purchase_id` и `code`.

```
-- comment: команда для MySQL, MariaDB и PostgreSQL
-- comment: команду для других СУБД можно найти в репозитории
CREATE TABLE purchase_product (
  purchase_id INT NOT NULL,
  code CHAR(12) NOT NULL,
  product_price DECIMAL(7,2) NOT NULL,
  product_quantity INT NOT NULL,
  product_name VARCHAR(100) NOT NULL,
  CONSTRAINT pk_purchase_product
    PRIMARY KEY (purchase_id, code)
);
```

FOREIGN KEY: дружба навек

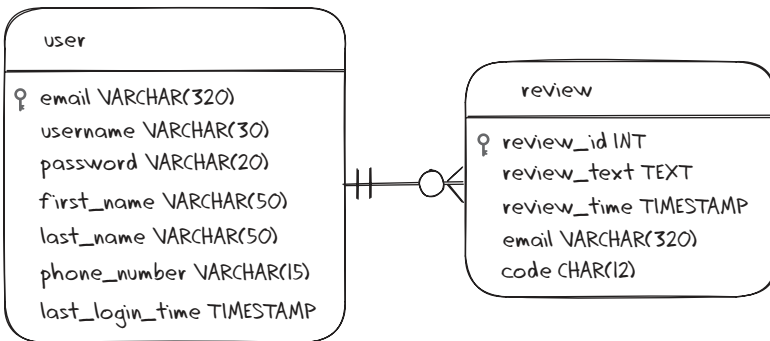
Как мы уже знаем, внешний ключ, который обычно состоит из одного столбца, поддерживает связь между двумя таблицами. На языке SQL ему соответствует ограничение FOREIGN KEY.



Ограничение по внешнему ключу обеспечивает ссылочную целостность. Что это такое? Чтобы связать друг с другом две таблицы, в одну из них нужно добавить внешний ключ — ссылку на первичный ключ второй. При этом первая таблица станет дочерней, а вторая — родительской. Попытка добавить в дочернюю таблицу запись со ссылкой на строку, которой нет в родительской, закончится сообщением об ошибке. Мы говорили об этом в главе 2.

Между таблицами `user` и `review` из базы данных «Фантастической лавки» имеется связь «один ко многим», которую поддерживает внешний ключ `email` таблицы `review`. При правильной реализации ограничения FOREIGN KEY СУБД заблокирует добавление отзыва, если пользователь (точнее, его электронная почта) отсутствует в таблице `user`.

Примечание: в столбце email таблицы user нет адреса alex99@gmail.com.



```
INSERT INTO review VALUES
```

```
(111, 'Классная шмuka!', '2024-01-02T06:25:16.670Z', 'alex99@gmail.com');
```

```
ERROR 1452 (23000): Cannot add or update a child row:
a foreign key constraint fails (`scifi_collective`.`review`,
CONSTRAINT `fk_user_review` FOREIGN KEY (`email`)
REFERENCES `user` (`email`))
```



С точки зрения реализации внешний ключ очень похож на первичный. К примеру, в таблице review будет два внешних ключа: для таблиц user и product. Можно определить и назвать их сразу при создании таблицы:

```
-- comment: команда для MySQL и MariaDB
-- comment: команду для других СУБД можно найти в репозитории
CREATE TABLE review (
  review_id INT PRIMARY KEY,
  review_text TEXT NOT NULL,
  review_time TIMESTAMP NOT NULL,
  email VARCHAR(320) NOT NULL,
  code CHAR(12) NOT NULL,
  CONSTRAINT fk_user_review
    FOREIGN KEY (email) REFERENCES user(email),
  CONSTRAINT fk_product_review
    FOREIGN KEY (code) REFERENCES product(code)
);
```

Или же можно добавить ограничения в уже существующую таблицу:

```
-- comment: команда для MySQL, MariaDB и PostgreSQL
-- comment: SQLite не поддерживает команду ALTER TABLE ADD CONSTRAINT
-- этап 1: создание таблицы review
CREATE TABLE review (
```

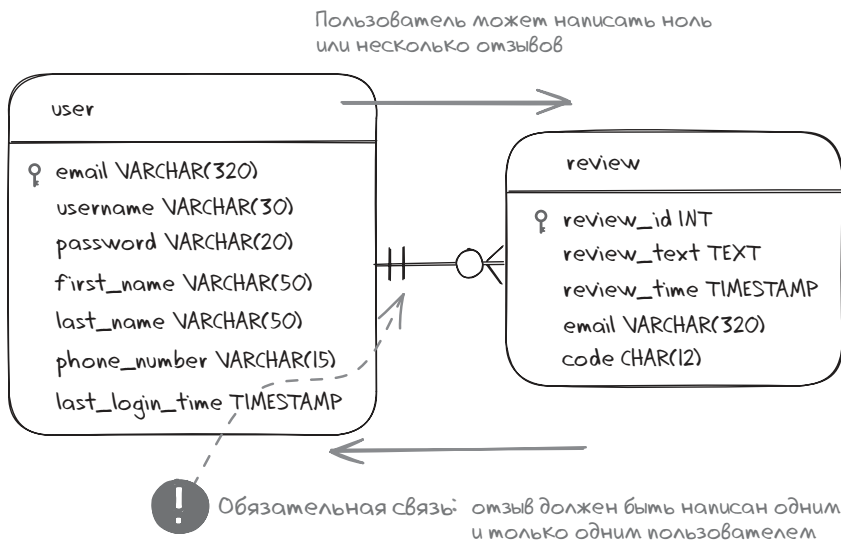
```

review_id INT PRIMARY KEY,
review_text TEXT NOT NULL,
review_time TIMESTAMP NOT NULL,
email VARCHAR(320) NOT NULL,
code CHAR(12) NOT NULL
);
-- этап 2: добавление ограничений по внешнему ключу
ALTER TABLE review
  ADD CONSTRAINT fk_user_review
    FOREIGN KEY (email) REFERENCES user(email),
  ADD CONSTRAINT fk_product_review
    FOREIGN KEY (code) REFERENCES product(code)

```

Так же как и в случае с первичным ключом, блок CONSTRAINT позволяет дать ограничению имя, а блоки FOREIGN KEY и REFERENCES создают внешний ключ, который обеспечит ссылочную целостность данных в двух таблицах.

Нужно отметить, что если связь между таблицами обязательна (то есть ее минимальная кратность в каком-то из направлений равна единице), на внешний ключ также полезно наложить ограничение NOT NULL. К примеру, такова связь между таблицами review и user, поскольку минимальная кратность в направлении последней равна единице (у отзыва должен быть хотя бы один автор). Действительно, добавить в таблицу review запись, которая не ссылается ни на кого из пользователей из таблицы user, не получится. Поскольку NULL — это заведомо нерабочая ссылка, разрешать такие значения внешнего ключа не имеет смысла.



Ссылочные действия

Ранее мы говорили о том, что ограничение по внешнему ключу гарантирует ссылочную целостность при добавлении данных. Если при вставке строки в дочернюю таблицу выясняется, что в родительской нет записи, на которую ссылается внешний ключ, такая операция блокируется: возникает ошибка.

Но ссылочную целостность нужно обеспечивать не только при добавлении данных. Что делать при удалении записи из родительской таблицы? Как быть, когда в ней изменяется значение первичного ключа? Что в этих случаях должно произойти со строками дочерней таблицы?

Таблица product (родительская)

code	name	price	manufacturer
20357	Атомный триммер...	1999	Mad Inventors Inc.
20468	Селфи-мостер	2499	Goofy Gadgets Corp.
30967	Кошачий наполнитель...	2999	Absurd Accessories
...	...	...	...

Удалить

Таблица review (дочерняя)

review_id	review_text	date	code
1	Классная штука...	2023-...	20357
2	...лучшая покупка...	2023-...	20468
3	...не рекомендую	2023-...	20357
...	...	...	...

Что делать с этими строками?

1. Прекратить удаление (по умолчанию)
2. Каскадный эффект
3. Установить в NULL

Такие проблемы можно решить с помощью специальных действий по умолчанию, которые будут выполняться при удалении и обновлении строк родительской таблицы, если на них ссылаются строки дочерней. Чтобы использовать такие действия, нужно либо оставить все как есть, либо изменить блок CONSTRAINT ограничений внешнего ключа, добавив в него несколько новых команд, к примеру:

```
ON DELETE RESTRICT
ON UPDATE RESTRICT
```

Оба словосочетания определяют, что делать с записями дочерней таблицы: `ON DELETE` — при удалении записей из родительской таблицы, `ON UPDATE` — при изменении значений первичного ключа в родительской таблице. Такие действия называются *ссылочными* (referential actions) и выполняются автоматически. `RESTRICT` (или, что то же самое, `NO ACTION`) — действие по умолчанию, которое заключается в том, что удаление либо изменение строк родительской таблицы полностью блокируется.

Но что, если нам действительно нужно удалить запись или изменить ее первичный ключ? Тогда, чтобы обеспечить целостность и согласованность данных, необходимо использовать другое ссылочное действие: `CASCADE`. Оно бывает двух видов:

- *Каскадное удаление.* Выполняется при удалении строки из родительской таблицы. СУБД автоматически находит в дочерней таблице все записи со ссылкой на эту строку и удаляет их (если это возможно). Благодаря этому удаление не приведет к появлению «потерянных» строк с нерабочими ссылками.
- *Каскадное обновление.* Выполняется при изменении первичного ключа в родительской таблице. СУБД автоматически находит в дочерней таблице все записи со ссылкой на эту строку и обновляет значение внешнего ключа. Благодаря этому ссылки на записи в родительской таблице продолжают работать без ручного вмешательства.

Каскадное удаление и обновление просты в реализации: достаточно изменить блок `CONSTRAINT`, добавив в него новые команды:

```
ON DELETE CASCADE
ON UPDATE CASCADE
```

После этого удаление записи либо изменение первичного ключа в родительской таблице будет сопровождаться каскадным эффектом: удалением или обновлением строк в дочерней.

Таблица product (родительская)

code	name	price	manufacturer
20357	Атомный триммер...	1999	Mad Inventors Inc.
20468	Селфи-мостер	2499	Goofy Gadgets Corp.
30967	Кошачий наполнитель...	2999	Absurd Accessories
...	...	...	...

Таблица review (дочерняя)

review_id	review_text	date	code
1	Классная штука...	2023-...	20357
2	...лучшая покупка...	2023-...	20468
3	...не рекомендую	2023-...	20357
...	...	...	...

Каскадный эффект

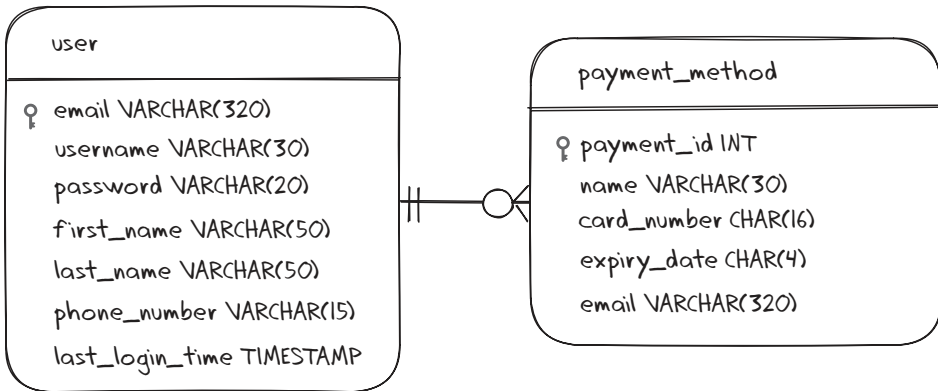
Удаление

В более редких случаях может потребоваться удалить запись из родительской таблицы, но сохранить все записи в дочерней. К примеру, в базе данных имеется две связанные таблицы: device (устройство) и employee (сотрудник). Когда сотрудник увольняется из компании, запись о нем необходимо удалить из таблицы employee. Однако устройства, которыми он пользовался, все еще на балансе компании, а потому из таблицы device удалять ничего не требуется. В подобных случаях можно использовать ссылочное действие SET NULL:

```
ON DELETE SET NULL
ON UPDATE SET NULL
```

При удалении строки из родительской таблицы СУБД автоматически находит в дочерней все записи со ссылкой на эту строку и обновляет в них внешний ключ: заменяет старое значение на NULL. Это довольно удобно, когда связь между двумя таблицами необязательна или же нужно сохранить информацию как, например, архивную. Однако появление NULL чревато проблемами в виде «потерянных» строк и усложнения запросов, ожидающих ненулевые внешние ключи.

Понадобятся ли ссылочные действия в базе данных «Фантастической лавки»? Да. К примеру, таблица `user` с первичным ключом `email` является родительской по отношению к нескольким другим, в том числе `payment_method`.



Если бы клиенты «Фантастической лавки» имели возможность сменить электронную почту или же полностью удалить учетную запись, нам бы пришлось обеспечивать целостность данных при удалении строк из таблицы `user` и обновлении ее первичного ключа `email`. Для этого можно добавить действие `CASCADE`, к примеру, в таблицу `payment_method`:

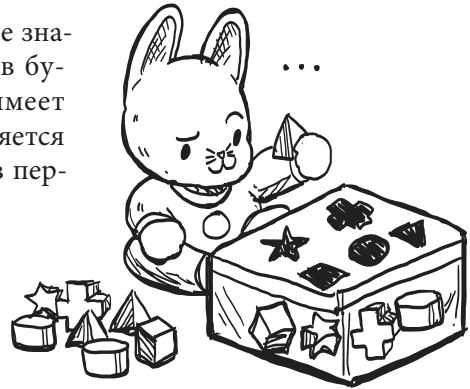
```
-- comment: команда для MySQL, MariaDB и PostgreSQL
-- comment: команду для других СУБД можно найти в репозитории
CREATE TABLE payment_method (
    payment_id INT PRIMARY KEY,
    name VARCHAR(30) NOT NULL,
    card_number CHAR(16) NOT NULL,
    expiry_date CHAR(4) NOT NULL,
    email VARCHAR(320) NOT NULL,
    CONSTRAINT fk_payment_method_user
        FOREIGN KEY (email) REFERENCES user (email)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

UNIQUE: простите, я занят

Ограничение UNIQUE гарантирует, что все значения столбца или сочетания столбцов будут уникальными. Использовать его имеет смысл, когда такое требование предъявляется к столбцу или столбцам, не входящим в первичный ключ.

Представим, что в таблице есть столбец с номерами социального страхования (SSN) — уникальными девятизначными идентификаторами, которые правительство США использует для контроля за гражданами: их доходами, налогами или, допустим, правами на льготы. Этот столбец не является частью ключа. Однако поскольку каждый номер представляет конкретного человека, у нескольких человек одинакового SSN быть не может. В подобных случаях стоит ввести ограничение UNIQUE.

У нас в базе данных такие столбцы тоже есть. Это несложно понять, если вернуться к анализу требований. Таблица user выглядит так:



user

```

└─ email VARCHAR(320)
   username VARCHAR(30)
   password VARCHAR(20)
   first_name VARCHAR(50)
   last_name VARCHAR(50)
   phone_number VARCHAR(15)
   last_login_time TIMESTAMP

```

Мы знаем, что значения в столбцах username и phone_number должны быть уникальными. Однако поскольку они не входят в первичный ключ, добиться этого можно при помощи ограничений UNIQUE. Добавим их, не забывая об именах:

```

-- comment: команда для MySQL, MariaDB и PostgreSQL
-- comment: команду для других СУБД можно найти в репозитории
CREATE TABLE user (

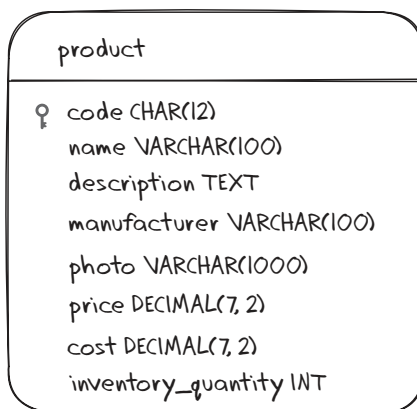
```

```

email VARCHAR(320) PRIMARY KEY,
username VARCHAR(30) NOT NULL,
password VARCHAR(20) NOT NULL,
first_name VARCHAR(50) NOT NULL,
last_name VARCHAR(50) NOT NULL,
phone_number VARCHAR(15),
last_login_name TIMESTAMP NOT NULL,
CONSTRAINT unq_username UNIQUE(username),
CONSTRAINT unq_phone_number UNIQUE(phone_number)
);

```

Бывают случаи, когда ограничение UNIQUE должно распространяться на сочетание столбцов. Вот, например, таблица product:



Если, согласно требованиям, необходимо обеспечить уникальность комбинаций двух свойств сущности, нужно ввести ограничение UNIQUE на соответствующую пару столбцов:

```

-- comment: команда для MySQL, MariaDB и PostgreSQL
-- comment: команду для других СУБД можно найти в репозитории
CREATE TABLE product (
    code CHAR(12) PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    description TEXT NOT NULL,
    manufacturer VARCHAR(100) NOT NULL,
    photo VARCHAR(1000) NOT NULL,
    price DECIMAL(7,2) NOT NULL,
    cost DECIMAL(7,2) NOT NULL,
    inventory_quantity INT,
    CONSTRAINT unq_name_manufacturer
        UNIQUE(name, manufacturer)
);

```

DEFAULT: заполнить пустоты

При помощи ограничения DEFAULT можно указать значение по умолчанию, которое будет использоваться, если при добавлении новой записи ее значение не указано. Это ограничение гарантирует, что у столбца всегда будет значение: принятое по умолчанию либо заданное пользователем.

Обычно ограничение DEFAULT применяется в следующих случаях:

- Имеется столбец для хранения меток времени, к примеру, добавления или изменения строки.
- Имеется столбец численного типа, необходимый для агрегации, например расчета количества или стоимости.
- Имеется столбец с исключительно справочной информацией, а потому вместо конкретных значений допустимо использовать «заглушки».

В базе данных «Фантастической лавки» такие столбцы, безусловно, есть. К примеру, в столбце `purchase_time` таблицы `purchase` хранятся метки времени:

purchase	
♀	<code>purchase_id INT</code> <code>total_price DECIMAL(13, 2)</code> <code>purchase_time TIMESTAMP</code> <code>payment_id INT</code>

Когда поступает заказ, в таблицу `purchase` добавляется новая строка, в которой должно быть проставлено время его оформления. В этом вопросе мы можем полностью положиться на разработчиков приложения или же облегчить их работу, введя ограничение DEFAULT. При этом значением по умолчанию как раз и будет время добавления новой строки.

Синтаксис ограничения DEFAULT очень прост: необходимо добавить в определение столбца ключевое слово DEFAULT, после которого указать значение по умолчанию:

```
CREATE TABLE purchase (
  purchase_id INT PRIMARY KEY,
  total_price DECIMAL(13,2) NOT NULL,
  purchase_time TIMESTAMP NOT NULL
    DEFAULT CURRENT_TIMESTAMP,
  payment_id INT NOT NULL,
  CONSTRAINT fk_payment_method_purchase
    FOREIGN KEY (payment_id)
    REFERENCES payment_method(payment_id)
);
```

В этом фрагменте кода важно отметить две вещи:

- В определение столбца `purchase_time` добавлены два ограничения: `NOT NULL` и `DEFAULT`. Но так ли нужно первое из них при наличии второго, ведь мы указали отличное от `NULL` значение по умолчанию? Все дело в том, что `DEFAULT` само по себе не препятствует вводу значения `NULL`: к примеру, `NULL` может быть в явном виде задано пользователем. Поэтому двойное ограничение бывает полезно, особенно в случаях, когда `NULL` может вызвать проблемы в работе приложения.
- В качестве значения по умолчанию указана функция `CURRENT_TIMESTAMP`, которая возвращает текущую дату и время. Она поддерживается большинством СУБД, включая MySQL, MariaDB, SQLite и PostgreSQL.

Базы данных с часами: отношение разных СУБД к UTC

В главе 4 мы говорили о том, что для хранения дат и времени лучше всего использовать всемирное координированное время (UTC), поскольку такой подход обеспечивает согласованность данных, а также предотвращает проблемы, связанные с переходом на летнее время и разностью между часовыми поясами.

А как работает функция `CURRENT_TIMESTAMP`? Использует ли она UTC автоматически? Ответ зависит от СУБД. К примеру, в MySQL UTC по умолчанию используется для всех временных значений, и результат функции `CURRENT_TIMESTAMP` не исключение. А вот в PostgreSQL есть метки времени двух типов:

- `TIMESTAMP WITHOUT TIME ZONE` (или просто `TIMESTAMP`) — для местного времени;
- `TIMESTAMP WITH TIME ZONE` — для UTC.

Поэтому чтобы получить значение в UTC, необходимо определить столбец `purchase_time` как `TIMESTAMP WITH TIME ZONE`. В этом случае

CURRENT_TIMESTAMP будет автоматически возвращать результат с учетом часового пояса:

```
-- PostgreSQL
CREATE TABLE purchase (
    purchase_id INT PRIMARY KEY,
    total_price DECIMAL(13,2) NOT NULL,
    purchase_time TIMESTAMP WITH TIME ZONE
        NOT NULL DEFAULT CURRENT_TIMESTAMP,
    payment_id INT NOT NULL,
    CONSTRAINT fk_payment_method_purchase
        FOREIGN KEY (payment_id)
        REFERENCES payment_method(payment_id)
);
```

Этот вопрос — один из тех, что стоит дважды проверить по документации к выбранной СУБД или же уточнить у ChatGPT, например, спросив его:

- Как в SQLite определить столбец для текущего времени, в котором значения хранились бы в UTC?
- Как в SQL Server определить столбец для текущего времени, в котором значения хранились бы в UTC?

CHECK: следить за порядком

Ограничение CHECK — это правило, по которому выполняется проверка данных, предназначенных для хранения в столбце. Оно обеспечивает целостность данных, ограничивая их ввод.

К примеру, такое ограничение полезно, когда значение должно находиться в пределах определенного диапазона или же соответствовать определенному формату. Допустим, если в столбце хранится возраст пользователя, разумно использовать CHECK, чтобы отсеивать все значения меньше 0 и больше 120:

```
-- comment: команда для MySQL, MariaDB и PostgreSQL
-- comment: команду для других СУБД можно найти в репозитории
CREATE TABLE person (
    person_id INT NOT NULL,
    last_name VARCHAR(255) NOT NULL,
    first_name VARCHAR(255),
    age INT,
    CONSTRAINT age_check CHECK (age >= 0 AND age <= 120)
);
```

В этом примере `age >= 0 AND age <= 120` — условие, которое проверяется ограничением CHECK.

Есть ли такие столбцы в базе данных «Фантастической лавки»? Да, и они идеально подходят для этого ограничения. В таблицах `user_address` и `billing_address` имеются столбцы `state`, в которых хранятся названия штатов США, а их всего пятьдесят. Мы можем поручить проверку фронтенд- или бэкенд-разработчикам, а можем справиться сами, используя CHECK. Поскольку список из пятидесяти слов довольно велик, удобнее ввести ограничение после создания таблицы:

```
-- comment: команда для MySQL, MariaDB, PostgreSQL и Oracle
-- comment: команду для других СУБД можно найти в репозитории
CREATE TABLE user_address (
    email VARCHAR(320) PRIMARY KEY,
    street_address VARCHAR(255),
    address_line_optional VARCHAR(100),
    city CHAR(100) NOT NULL,
    state VARCHAR(20) NOT NULL,
    postal_code CHAR(5) NOT NULL
);
ALTER TABLE user_address
ADD CONSTRAINT chk_state
CHECK (
    state IN (
        'Алабама', 'Аляска', 'Аризона',
        'Арканзас', 'Калифорния', ...,
        'Западная Вирджиния', 'Висконсин', 'Вайоминг'
    )
);
```

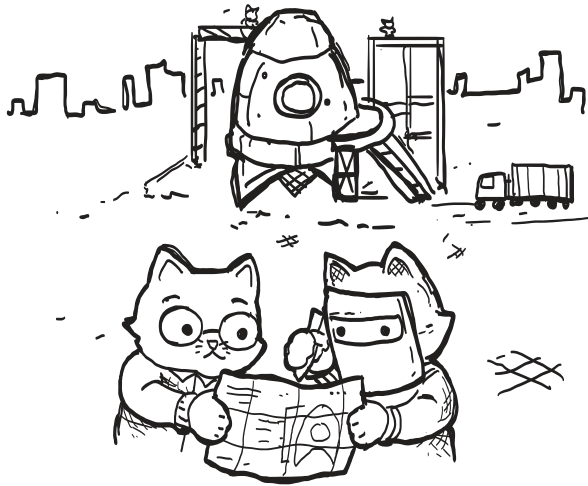
В списке ('Алабама', 'Аляска', ... 'Вайоминг') перечислены все допустимые значения столбца `state`. Любые иные варианты будут отвергнуты ограничением.

Итоги

- Нормализация — это процесс разделения больших таблиц на более мелкие и удобные в работе, соответствующие отдельным сущностям или понятиям.
- Обычно база данных считается полностью нормализованной, если все таблицы приведены к форме BCNF. Такая таблица соответствует следующим требованиям:
 - имеет первичный ключ;
 - не имеет многозначных столбцов;

- все ее столбцы зависят от ключа и только от него;
- переходные зависимости отсутствуют (все неключевые столбцы зависят от ключа напрямую).

Можно сказать, что нормализация сводится к проверке таблиц на соответствие требованиям BCNF, выявление и устранение отклонений от них.



- Ограничения NOT NULL и PRIMARY KEY/FOREIGN KEY имеют критически важное значение для проекта любой базы данных.
- На случай, если потребуется изменить структуру базы данных или перенести ее в другую СУБД, имеет смысл присвоить всем ограничениям имена.
- Ограничение NOT NULL гарантирует, что в столбце не будет значений NULL.
- Ограничение UNIQUE удобно, когда значения в выбранном столбце, отличном от первичного ключа, должны быть уникальными.
- Ограничение DEFAULT полезно при работе с метками времени и значениями-заглушками.
- Ограничение CHECK позволяет проверить формат и диапазон данных и может использоваться для реализации сложной бизнес-логики, с которой не справляются другие ограничения.

7 | Безопасность и оптимизация



В этой главе

- ✓ Как оценить безопасность базы данных и усилить ее
- ✓ Как повысить эффективность хранения в базе данных в дополнение к нормализации
- ✓ Индексация и ее применение, чтобы ускорить выполнение запросов
- ✓ Денормализация — еще один способ повысить производительность

Необходимые знания

Проект рассматриваемой в этой главе базы данных (наконец-то готовый), реализованный при помощи часто применяемых на практике инструментов `dbdiagram.io` и `MySQL Workbench`, можно найти в репозитории `GitHub` (<https://github.com/Neo-Hao/grokking-relational-database-design>). Чтобы загрузить проект в выбранный инструмент, перейдите в папку `chapter_07` и следуйте инструкциям из файла `README.md`.

В той же папке находятся SQL-скрипты для реализации полностью завершенного проекта базы данных в разных СУБД, включая `MySQL`, `MariaDB`, `PostgreSQL`, `SQLite`, `SQL Server` и `Oracle`.

Введение

В этой главе мы будем говорить о безопасности баз данных, а также об их оптимизации, о том, как оценить целостность и конфиденциальность информации, которая содержится в БД, и повысить эффективность ее хранения сверх уже достигнутого при нормализации. Мы также узнаем о том, что такое денормализация и как индексация позволяет ускорить выполнение запросов.

Безопасность

Безопасность — критически важный аспект проектирования баз данных. Чтобы предотвратить утечки информации, необходимо оценить проект и убедиться, что он соответствует хотя бы нескольким основным принципам безопасности. И в этом разделе мы обсудим два из них: целостность и конфиденциальность, а также научимся применять их на практике.



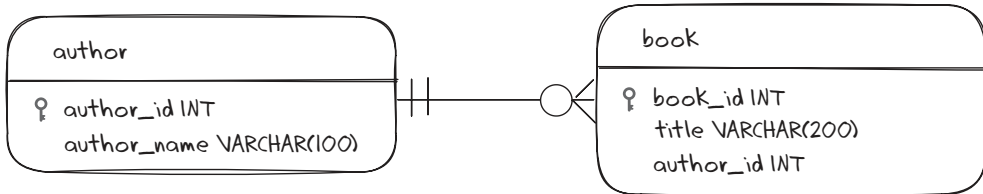
Целостность

Под *целостностью* (integrity) понимается возможность сохранения точности, полноты и достоверности данных и систем. Проектируя базу данных, необходимо убедиться, что ее таблицы, столбцы и связи спланированы и реализованы правильно, то есть соответствуют принципам проектирования и реализации, которые мы обсуждали в главах 4, 5 и 6, в том числе:

- для каждого столбца подобран оптимальный тип данных;
- у каждой таблицы есть первичный ключ;
- связи между таблицами обеспечиваются внешними ключами;
- введены ограничения на основе анализа требований.

База данных обычно проектируется в несколько этапов, поскольку небольшие ошибки и упущения практически неизбежны. К примеру, довольно легко забыть об ограничении по внешнему ключу.

Допустим, нам нужно создать базу данных для веб-приложения издательства. Чтобы хранить информацию об авторах и книгах, добавим в нее две связанные таблицы, `author` и `book`:



Поскольку на ER-диаграммах не отображаются внешние ключи и другие ограничения, при разработке SQL-скриптов легко их упустить:

```

CREATE TABLE author (
    author_id INT PRIMARY KEY,
    author_name VARCHAR(100) NOT NULL
);
-- comment: в таблице book отсутствует
-- comment: ограничение по внешнему ключу
CREATE TABLE book (
    book_id INT PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
    author_id INT NOT NULL
);
  
```

А без ограничения по внешнему ключу в дочерней таблице неминуемо появятся «потерянные» строки со ссылками на записи, которых уже нет в родительской. И дело не только в том, что такой базой данных сложнее управлять: в ней появляются лазейки для злоумышленников, что создает угрозу безопасности. Поэтому в SQL-скрипт необходимо добавить ограничение по внешнему ключу для таблицы `book`.

```

CREATE TABLE book (
    book_id INT PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
    author_id INT NOT NULL,
    CONSTRAINT FK_author_id
        FOREIGN KEY (author_id)
        REFERENCES author(author_id)
);
  
```

Конфиденциальность

Конфиденциальность (confidentiality) — это защита данных от несанкционированного доступа и разглашения. Необходимо, чтобы доступ

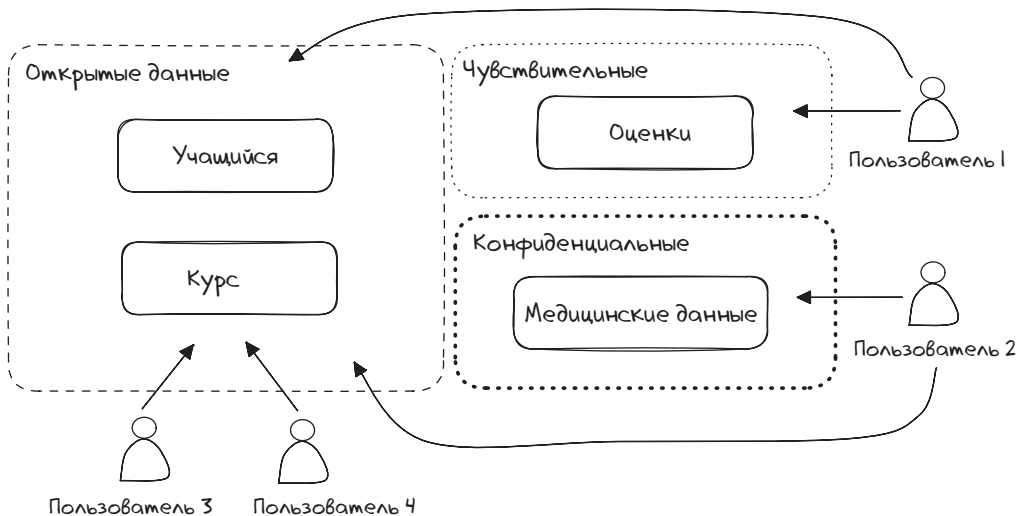
к данным имели лишь те процессы и люди, которым это разрешено. Существует два способа это обеспечить: *управление доступом* и *шифрование*.

Управление доступом

Управление доступом (access control) к базе данных похоже на проверку у входа в ночной клуб: охранник смотрит на посетителей и решает, кому из них можно войти и куда. Мы же будем определять, кто может просматривать, изменять и удалять данные. При этом важно наделить пользователей ровно тем уровнем прав, который им действительно необходим.

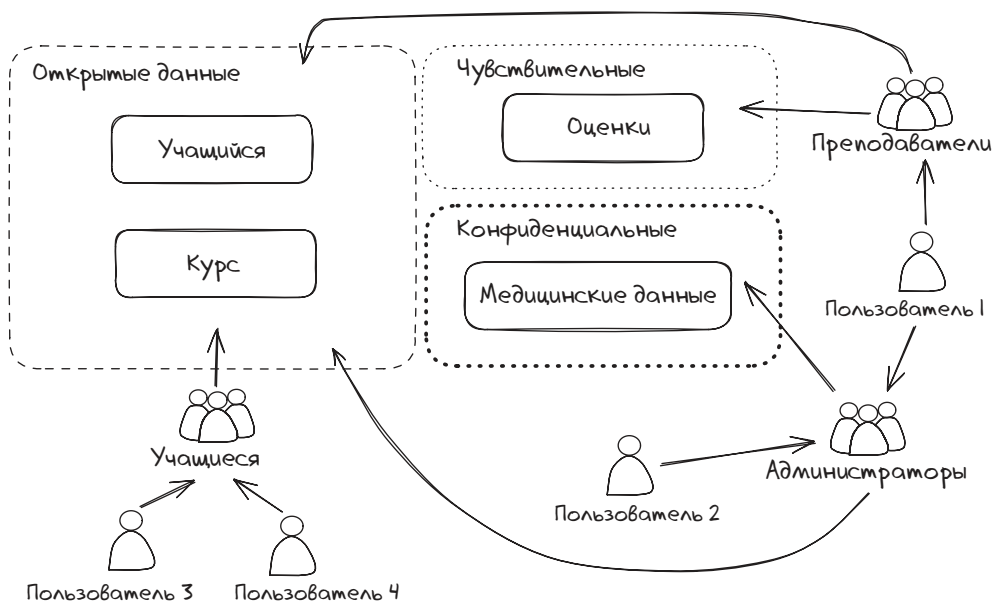
Существует два основных подхода к управлению доступом: *на основе полномочий* (mandatory access control, MAC) и *на основе ролей* (role-based access control, RBAC). Первый подход напоминает строгую школу с жесткими правилами: правами наделяет главный администратор, каждая таблица (а иногда и строка) получает уровень секретности (например, «чувствительные» (sensitive) или «конфиденциальные» (confidential)), а каждый пользователь — уровень доступа. И если уровень доступа не соответствует уровню секретности, чтение данных блокируется. И никаких исключений.

Представьте себе базу данных с информацией об учащихся школы. Таблицы в ней будут иметь такие уровни секретности:



Таблицы, в которых хранятся чувствительные и конфиденциальные данные, могут просматривать лишь те, кто имеет соответствующий уровень доступа. При этом уровни секретности настраиваются вручную.

Управление доступом на основе ролей — подход гораздо более гибкий и удобный. Вместо того чтобы предоставлять права отдельно взятым пользователям, мы создаем группы с тем или иным уровнем доступа и добавляем в них пользователей. Такой подход напоминает клуб и его членов: кому-то из них можно пройти в vip-зал, другим — в бар, ну а третьих не пропустят дальше общей зоны. В примере со школой в базе данных появится несколько ролей:



Используем подход на основе ролей для управления доступом к базе данных «Фантастической лавки». Для упрощения возьмем лишь четыре таблицы: `product`, `purchase`, `review` и `purchase_review`. Допустим, что на этапе сбора информации мы выяснили, что доступ к ним должны иметь пользователи трех групп:

- **USER (пользователь).** Просматривает список товаров, делает заказы и пишет отзывы о покупках.

- ANALYST (аналитик). Анализирует данные, просматривая записи из всех таблиц.
- ADMIN (администратор). Просматривает, изменяет и удаляет записи из всех таблиц.

С учетом этих требований можно создать роли, предоставить им нужные права, а затем распределить среди пользователей. Все это можно сделать как с помощью графического интерфейса СУБД, так и посредством команд SQL. Начнем с роли USER:

```
-- comment: команды для MySQL и MariaDB
-- comment: команды для других СУБД можно узнать у ChatGPT
-- step 1: создание роли
CREATE ROLE standard_user;
-- step 2: предоставление роли прав
GRANT SELECT ON database_name.product TO standard_user;
GRANT INSERT ON database_name.purchase TO standard_user;
GRANT INSERT ON database_name.purchase_product TO standard_user;
GRANT INSERT ON database_name.review TO standard_user;
-- step 3: определение роли для нового пользователя
CREATE USER 'morpheus'@'%' IDENTIFIED BY 'password';
GRANT standard_user TO 'morpheus'@'%';
```

Этот фрагмент кода нуждается в некоторых пояснениях:

- Роль будет называться `standard_user` (стандартный пользователь), а не просто `user`, поскольку это слово зарезервировано в языке SQL большинства СУБД.
- Блок `SELECT` (например, `SELECT ON database_name.product`) соответствует просмотру таблицы, а блок `INSERT` (например, `INSERT ON database_name.purchase`) — добавлению в нее данных. Роль `standard_user` позволяет просматривать список товаров, делать покупки и писать отзывы. Поэтому нам потребуется команда `GRANT SELECT ON... TO...` (просмотр) и команда `GRANT INSERT ON... TO...` (добавление данных) для нескольких таблиц.
- Все новые пользователи, созданные при помощи команды `CREATE`, автоматически назначаются на роль `standard_user`.
- В MySQL и MariaDB в имени пользователя обязательно указывается хост — место, откуда он будет подключаться к базе данных: `'username'@'host'`. При этом запись `'username'@'%'` означает, что пользователь может подключаться откуда угодно.

Для роли ANALYST нужно ввести такие команды:

```
-- comment: команды для MySQL и MariaDB
-- comment: команды для других СУБД можно узнать у ChatGPT
-- step 1: создание роли
CREATE ROLE analyst;
-- step 2: предоставление роли прав
GRANT SELECT ON database_name.product TO analyst;
GRANT SELECT ON database_name.purchase TO analyst;
GRANT SELECT ON database_name.purchase_product TO analyst;
GRANT SELECT ON database_name.review TO analyst;
-- step 3: определение роли для нового пользователя
CREATE USER 'smith'@'%' IDENTIFIED BY 'password';
GRANT analyst TO 'smith'@'%';
-- step 4: включение роли по умолчанию (MySQL 8.0+)
ALTER USER 'smith'@'%' DEFAULT ROLE analyst;
```

Поскольку аналитикам нужно только читать данные из таблиц, в этом фрагменте мы видим четыре команды GRANT SELECT.

И наконец, определим роль ADMIN:

```
-- comment: команды для MySQL и MariaDB
-- comment: команды для других СУБД можно узнать у ChatGPT
-- step 1: создание роли
CREATE ROLE admin;
-- step 2: предоставление роли прав
GRANT SELECT, INSERT, UPDATE, DELETE ON database_name.product TO admin;
GRANT SELECT, INSERT, UPDATE, DELETE ON database_name.purchase TO admin;
GRANT SELECT, INSERT, UPDATE, DELETE ON database_name.purchase_product TO admin;
GRANT SELECT, INSERT, UPDATE, DELETE ON database_name.review TO admin;
-- step 3: определение роли для нового пользователя
CREATE USER 'david'@'%' IDENTIFIED BY 'password';
GRANT admin TO 'david'@'%';
-- step 4: включение роли по умолчанию (MySQL 8.0+)
ALTER USER 'david'@'%' DEFAULT ROLE admin;
```

Управление доступом — важная задача, которую выполняют не только проектировщики баз данных, но и администраторы разного уровня. Ей можно посвятить не только отдельную главу, но и целую книгу. Здесь же мы рассмотрели лишь самое главное: то, что касается нашей темы — проектирования.

Шифрование

В базах данных чаще всего применяется шифрование двух типов: одностороннее и симметричное.

При *одностороннем шифровании* (one-way encryption), которое обычно называют *хешированием* (hashing), входные данные превращаются в случайную на вид строку фиксированной длины. Расшифровать ее практически невозможно, особенно если исходная строка достаточно длинна. Иными словами, имея на руках хешированное значение, полученное односторонним шифрованием, нельзя восстановить то, что было зашифровано.

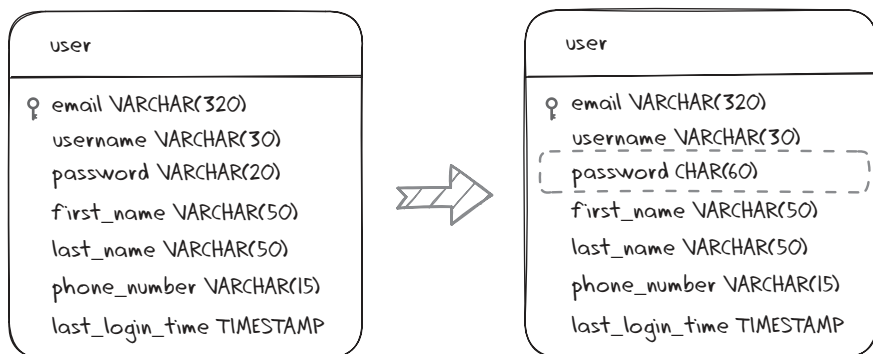
Одностороннее шифрование часто используется при хранении паролей. Благодаря этому, проникнув в базу данных, злоумышленник не сможет узнать пароль и применить его где-то еще (ведь у людей нередко один пароль на все случаи жизни). Так шифрование помогает уменьшить возможный ущерб от атаки. Наиболее распространенные алгоритмы одностороннего шифрования — bcrypt, PBKDF2 и Sha512.

В проекте базы данных «Фантастической лавки» есть несколько столбцов, которые стоит зашифровать. В столбце password таблицы user хранятся пароли пользователей, на данный момент — в виде обычного текста:

user	
♀	email VARCHAR(320)
	username VARCHAR(30)
	password VARCHAR(20)
	first_name VARCHAR(50)
	last_name VARCHAR(50)
	phone_number VARCHAR(15)
	last_login_time TIMESTAMP

Однако одностороннее шифрование потребует небольших изменений в проекте. К примеру, если мы будем применять алгоритм bcrypt,

каждый пароль превратится в 40 байт двоичных данных. Поскольку двоичные данные сложно выводить на экран или принтер, чаще всего их преобразуют в строку из ASCII-символов. Если использовать алгоритм Base64, длина такой строки составит 60 байт. Поэтому изначально выбранный нами тип `VARCHAR(20)` будет слишком мал. Изменим его на `CHAR(60)`:



Если проект базы данных уже реализован, замену типа можно произвести при помощи следующей команды:

```
-- comment: команда для MySQL и MariaDB
-- comment: команду для других СУБД можно узнать у ChatGPT
ALTER TABLE user MODIFY COLUMN password CHAR(60);
```

Как «расхешировать» секретные данные

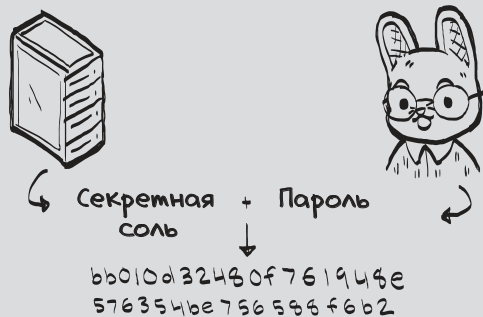
Хеширование осуществляется при помощи математической функции, на вход которой поступает строка переменной длины (к примеру, пароль), а на выходе образуется значение фиксированной длины — хеш. Такое преобразование необратимо, поэтому вы не найдете слово «*расхешировать*» в словаре! У хеш-функции множество применений, но те хеши, о которых мы говорим сейчас, специально предназначены для шифрования. Большинство хеш-функций (например, Sha512 и bcrypt) хорошо известно и стандартизировано.

Хранение паролей в виде обычного текста чревато тем, что их могут украсть из взломанной базы данных, после чего использовать в недобрых целях. По этой причине пароли принято хранить в виде хешей. Но даже это не дает полной защиты, поскольку злоумышленник может действовать так называемым методом грубой силы, или брутфорса (перебирать случайные пароли), или использовать «радужную таб-

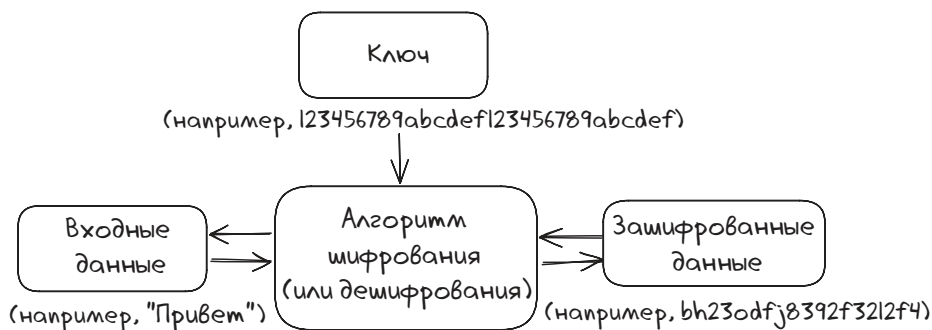
лицу» (rainbow table) — список известных паролей с заранее вычисленными хешами. Поэтому в большинстве приложений применяются модификаторы (так называемые соли): перед хешированием к паролю добавляется случайная строка, которая хранится в приложении. Не зная этой строки, злоумышленник не может подобрать пароль.

Вы спросите, как происходит аутентификация пользователей? Все очень просто:

1. Для входа в систему пользователь вводит пароль. Приложение принимает его и хеширует, используя тот же алгоритм, что и при сохранении в базе данных.
2. Приложение сравнивает два хеша: только что вычисленный и тот, что хранится в базе. Если они совпадают, считается, что пользователь аутентифицирован.

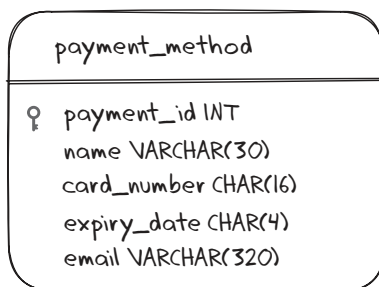


В отличие от одностороннего шифрования *симметричное шифрование* (symmetric encryption) обратимо, то есть можно расшифровать зашифрованную строку и получить исходную. Поэтому такой подход часто используют для защиты секретной информации, которую нужно использовать в первоначальном виде. К примеру, желательно запоминать номера кредитных карт, чтобы при следующем заказе клиентам не приходилось их набирать повторно. Однако хранить их в открытом виде опасно. Симметричное шифрование решает эту проблему: номер карты хранится в зашифрованном виде, а если покупатель захочет еще раз воспользоваться ею для оплаты — расшифровывается. При этом шифрование и дешифрование выполняются по одному и тому же алгоритму, с использованием одного и того же ключа — обычно строки из букв и цифр. Ключ следует безопасно хранить и правильно использовать: от этого зависит надежность шифрования.



Для симметричного шифрования чаще всего применяются три алгоритма: AES (Advanced Encryption Standard — продвинутый стандарт шифрования), 3DES (Triple Data Encryption Standard — стандарт тройного шифрования данных) и Blowfish. Каждый из них имеет несколько вариантов. Так, например, существуют алгоритмы AES-128, AES-192 и AES-256, которые отличаются длиной ключа (128, 192 и 256 бит соответственно). Последняя определяет степень защищенности и требования к вычислительным мощностям: чем длиннее ключ, тем более надежно шифрование, но тем больше времени необходимо на расчеты.

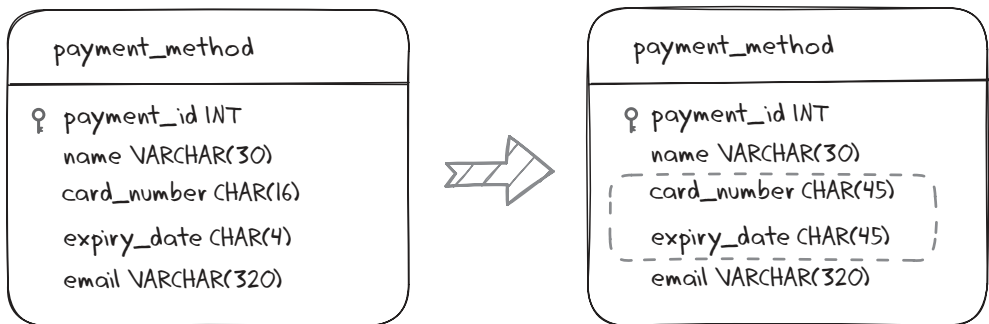
Понадобится ли симметричное шифрование в нашем примере? Да. В столбцах `card_number` и `expiry_date` таблицы `payment_method` хранятся секретные данные, которые нужно использовать повторно в первоначальном виде: номер и срок действия кредитной карты. Пока что мы храним их в виде обычного текста:



Шифрование потребует изменения проекта базы данных. Поговорим сначала о номере карты. Мы можем использовать алгоритм AES-256,

который работает с блоками по 16 байт. Поскольку номер карты состоит из 16 цифр, он уместится в одном таком блоке. Для повышения надежности в алгоритмах AES также используются *векторы инициализации* (initialization vectors) — двоичные значения длиной 16 байт, которые не являются секретными и поступают на вход алгоритма вместе с шифруемой строкой. При этом длина исходного значения увеличивается до 32 байт. Как и в случае с односторонним шифрованием, в результате получается двоичное значение, которое затем преобразуется в строку из ASCII-символов. И если использовать для этой цели алгоритм Base64, длина зашифрованной строки составит около 45 байт.

Теперь посмотрим на `expiry_date`. Для алгоритма AES-256 необходимы блоки по 16 байт, однако исходные значения из этого столбца короче. Поэтому перед шифрованием нам будет нужно дополнить их до 16 байт. Тогда с точки зрения алгоритма шифрования данные в столбцах `expiry_date` и `card_number` будут одной и той же длины (16 байт), а значит, и длины зашифрованных строк в обоих случаях будут одинаковы — около 45 байт. Поэтому нам нужно заменить типы данных этих столбцов на `CHAR(45)`:



Если проект базы данных уже реализован, замену типа можно произвести при помощи следующей команды:

```
-- comment: команда для MySQL и MariaDB
-- comment: команду для других СУБД можно узнать у ChatGPT
ALTER TABLE payment_method
  MODIFY COLUMN card_number CHAR(45),
  MODIFY COLUMN expiry_date CHAR(45);
```

Симметричное шифрование — не панацея

Симметричное шифрование кажется более удобным в работе, чем одностороннее, поскольку мы сохраняем возможность расшифровать исходные данные. Так почему же мы используем оба типа шифрования, а не один и не применяем симметричное шифрование при работе с паролями? Как мы увидим ниже, все дело в надежности. Для всех секретных данных, которые не требуют периодической дешифровки и применения в исходном виде, необходимо использовать одностороннее шифрование.

Одностороннее шифрование обычно надежнее симметричного, поскольку не требует применения ключа. Симметричное шифрование надежно лишь при условии, что ключ хранится в безопасном месте. Если каким-либо образом он попадет в руки злоумышленников, все зашифрованные им секретные данные окажутся под угрозой, поскольку могут быть расшифрованы.

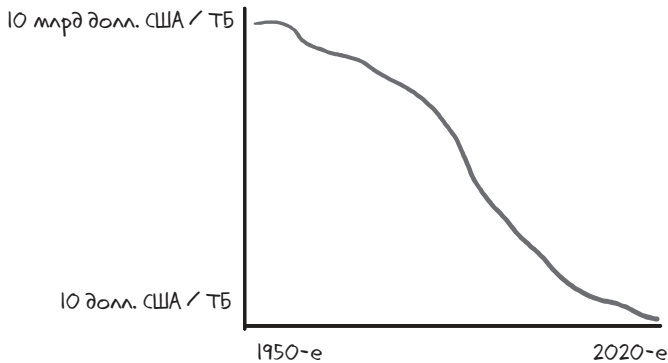
Поэтому вопрос о хранении ключей для симметричного шифрования критически важен. Нельзя, например, хранить их в той же базе данных, что и зашифрованные сведения. В рабочих приложениях для этого можно использовать один из следующих способов:

- переменные среды в файле на сервере приложений;
- специальные программы — *системы управления ключами*;
- специальное физическое устройство — *аппаратный модуль безопасности*.

Стоимость хранения

Спроектированная, пересмотренная, реализованная и протестированная база данных поступает на продакшен, где на ее работу тратятся деньги. Поэтому нужно заранее подумать не только о производительности запросов на чтение и запись, но и об эффективности хранения.

Раньше хранение данных стоило дорого: в семидесятых годах прошлого века стоимость 1 мегабайта в хранилище могла доходить до 250 долларов. Сегодня за доллар можно получить 1 терабайт.



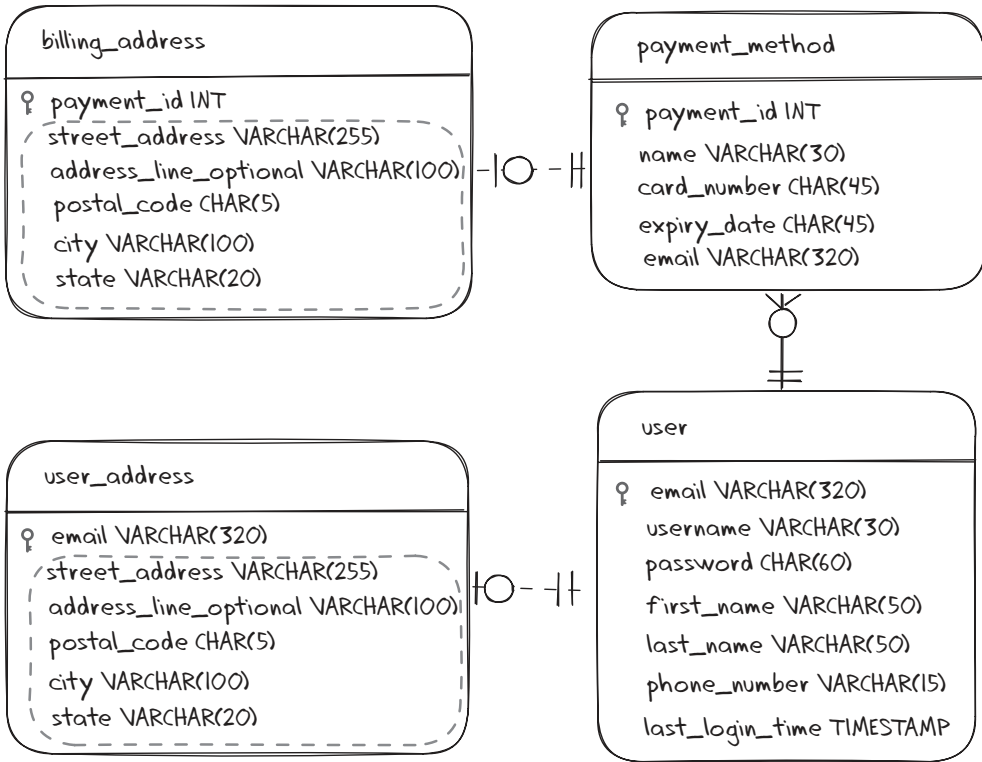
Но несмотря на эту тенденцию, улучшение проекта базы данных с точки зрения уменьшения ее объема хранилища остается важной задачей, особенно если речь идет о приложении с миллионами пользователей или возможным значительным расширением в будущем.

Что же предпринять? Уменьшить избыточность данных. В ходе нормализации мы уже значительно продвинулись в этом направлении, избавившись от аномалий, которые в том числе ведут к избыточности. Однако помимо нормализации есть и другие способы снизить избыточность. Поговорим о них в этом разделе.

Избыточные таблицы

Проект базы данных «Фантастической лавки» нормализован. Однако его можно еще улучшить, чтобы уменьшить избыточность данных.

Анализируя проект, зададим себе простой вопрос: нет ли у нас избыточных (или почти избыточных) таблиц? Такие таблицы есть, для хранения адресов мы используем две таблицы, и они почти одинаковы:



За исключением первичных ключей, состав столбцов в таблицах `billing_address` и `user_address` одинаков. Такой дизайн вполне допустим, поскольку адрес для доставки счетов может не совпадать с фактическим адресом покупателя. Впрочем, с высокой вероятностью, найдутся клиенты, желающие получать счета по месту жительства, и было бы хорошо консолидировать эти данные.

Как это сделать? Сначала необходимо проверить и обновить исходные требования. Чтобы объединить две таблицы, можно ввести еще одну сущность, которая будет хранить один адрес, если он одинаков

для `user` и `payment_method`, или же два адреса, если они не совпадают. Поэтому мы можем сделать такое уточнение в требованиях:

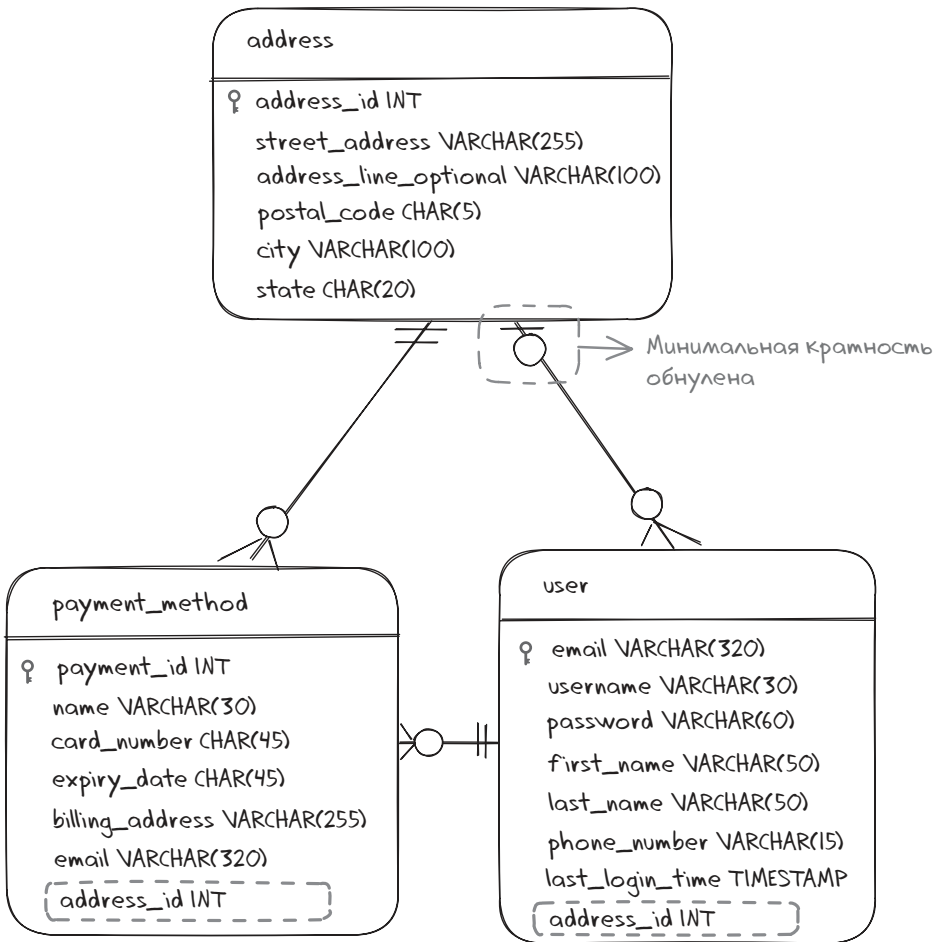
- у пользователя есть один адрес;
- у способа оплаты есть один адрес;
- адрес может быть связан с нулем, одним или несколькими пользователями либо с нулем, одним или несколькими способами оплаты.

Несложно перенести эти требования на ER-диаграмму. Однако каким будет первичный ключ единой сущности (например, `address`)? Как мы обсуждали в главе 4, необходимо подобрать реальное понятие, которое позволит отличить два адреса друг от друга. К примеру, сочетание из всех столбцов таблицы: `street_address`, `address_line_optional`, `postal_code`, `city` и `state`. Однако применение составных первичных ключей обычно считается плохим решением по двум причинам:

- составной ключ значительно снижает производительность;
- сущность `address` сопряжена с `user` и `payment_method` связью «один ко многим». И если использовать составной первичный ключ, придется дублировать все столбцы из `address` в `user` и `payment_method`.

Поэтому мы прибегнем к альтернативному решению: используем суррогатный ключ (например, `address_id`) и ограничение `UNIQUE`. Как было сказано в главе 4, суррогатный ключ чаще всего представляет собой числовое значение с автоматическим увеличением: простой и гибкий подход, позволяющий получить внешние ключи из одного столбца. Ограничение же мы наложим на сочетание всех столбцов таблицы `address`, за исключением суррогатного ключа (`address_id`). Это гарантирует отсутствие в ней избыточных данных.

Продумав все свойства измененного проекта, отобразим их на ER-диаграмме и перейдем к реализации:



Стоит отметить обнуление минимальной кратности связи между таблицами user и address. Действительно, клиенты «Фантастической лавки» могут не указывать адрес при регистрации, и если не обнулить минимальную кратность, новую запись в таблицу user нельзя будет внести без соответствующей строки в таблице address.

Если проект базы данных уже реализован, то, чтобы внести в него изменения, потребуется выполнить несколько действий:

1. Удалить две старые таблицы: billing_address и user_address.
2. Создать новую таблицу address и ввести ограничение UNIQUE на сочетание из всех ее столбцов, за исключением первичного ключа.

3. Обновить таблицы `payment_method` и `user`, добавив в них новый столбец `address_id` и сделав его внешним ключом в обеих таблицах.

Создадим таблицу `address`:

```
-- comment: команда для MySQL и MariaDB
-- comment: команду для других СУБД можно найти в репозитории
CREATE TABLE IF NOT EXISTS address (
  address_id INT AUTO_INCREMENT PRIMARY KEY,
  street_address VARCHAR(255) NOT NULL,
  address_line_optional VARCHAR(100),
  city VARCHAR(100) NOT NULL,
  state VARCHAR(20) NOT NULL,
  postal_code CHAR(5) NOT NULL,
  CONSTRAINT unique_address_constraint
  UNIQUE (street_address, address_line_optional, postal_code, city,
state)
);
```

Обновим таблицу `payment_method`:

```
-- comment: команды для MySQL и MariaDB
-- comment: команды для других СУБД можно узнать у ChatGPT
ALTER TABLE payment_method
  ADD COLUMN address_id INT NOT NULL;
ALTER TABLE payment_method
  ADD CONSTRAINT fk_address_payment_method
  FOREIGN KEY (address_id)
  REFERENCES address(address_id);
```

В таблице `user` столбец `address_id` не будет иметь ограничения `NOT NULL`, поскольку мы обнулили минимальную кратность связи между `user` и `address`. Более того, у новых записей в этом столбце по умолчанию должно быть значение `NULL`:

```
-- comment: команды для MySQL и MariaDB
-- comment: команды для других СУБД можно узнать у ChatGPT
ALTER TABLE user
  ADD COLUMN address_id INT NULL;
ALTER TABLE user
  ADD CONSTRAINT fk_address_user
  FOREIGN KEY (address_id)
  REFERENCES address(address_id);
```

Если внешний ключ может быть равен `NULL`, это говорит о том, что связь между сущностями не обязательна, то есть не каждая запись в дочерней таблице (`user`) должна соответствовать записи в родительской (`address`). Однако при этом необходимо понимать, что при чтении данных из двух таблиц потребуется дополнительная осторожность.

Категориальные данные

Вывав и устранив избыточные таблицы, перейдем к следующему шагу: поиску столбцов с избыточными данными. Взгляните на проект нашей базы данных и ответьте на вопрос: нет ли в нем таких столбцов? Такой столбец есть.

Действительно, в таблице `address`, которую мы только что добавили, имеется столбец `state`, в котором очень много повторяющихся значений. А как иначе? Товар в «Фантастической лавке» приобретают миллионы жителей США (в главе 4 мы говорили о том, что жители других стран не обслуживаются), а там всего лишь пятьдесят штатов, если не считать столичный округ Колумбия.

address

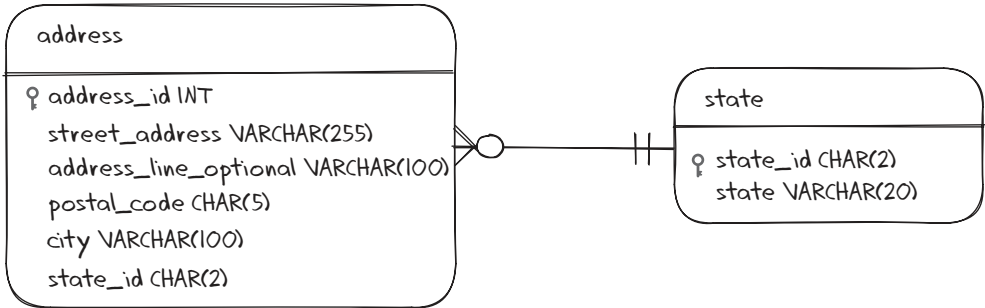
```
♀ address_id INT
street_address VARCHAR(255)
address_line_optional VARCHAR(100)
postal_code CHAR(5)
city VARCHAR(100)
state VARCHAR(20)
```

Как решить эту проблему? Выделить столбец `state` в таблицу с таким же названием, в которой будут перечислены все штаты и округ Колумбия. При этом между таблицами `state` и `address` образуется связь «один ко многим».

А что послужит первичным ключом для таблицы state? У каждого штата и, разумеется, у округа Колумбия есть уникальная аббревиатура, которая состоит из двух букв. Идеальный первичный ключ:

Алабама: AL
Аляска: AK
Калифорния: CA
Колорадо: CO
Округ Колумбия: DC

Теперь диаграмма таблицы address с учетом новой таблицы state будет выглядеть так:



Как мы говорили в главе 5, чтобы иметь возможность вводить данные, необходимо обнулить минимальную кратность связи в направлении таблицы address.

Индексация

Итак, мы полностью исключили избыточность данных из проекта. Настало время подумать, как сделать их выборку более эффективной, и в этой связи поговорить об *индексации* (indexing) — критически важном способе ускорить чтение, сортировку и поиск данных. Для этого на этапе реализации необходимо проиндексировать некоторые столбцы. В этом же разделе мы будем говорить о том, в каких сценариях это нужно и как это сделать.

Причины и способы индексации

Чтобы оптимизировать производительность запросов, в базах данных используют *индексы* нескольких типов, в том числе *стандартные* (standard), *кластеризованные* (clustered), *пространственные* (spatial) и *полнотекстовые* (full-text). Поговорим о стандартных и полнотекстовых индексах.

Стандартные индексы

Правильное использование индексов невозможно без понимания их важности и круга задач, которые они решают. Начнем с простой таблицы с фильмами из базы данных онлайн-кинотеатра.

Проект таблицы movie (фильм) выглядит так:

movie	
?	movie_id INT
	title VARCHAR(100)
	date TIMESTAMP
	rating DECIMAL(2, 1)
	photo VARCHAR(1000)



Поскольку наибольшей популярностью пользуются фильмы с высоким рейтингом (rating), приложению постоянно требуется сортировать их по этому показателю, то есть выполнять следующий запрос:

```
-- comment: команды для MySQL, MariaDB, PostgreSQL и SQLite
-- comment: команды для других СУБД можно узнать у ChatGPT
SELECT *
  FROM movie
 ORDER BY rating DESC
  LIMIT 30;
```

Он выполняется блок за блоком в таком порядке:

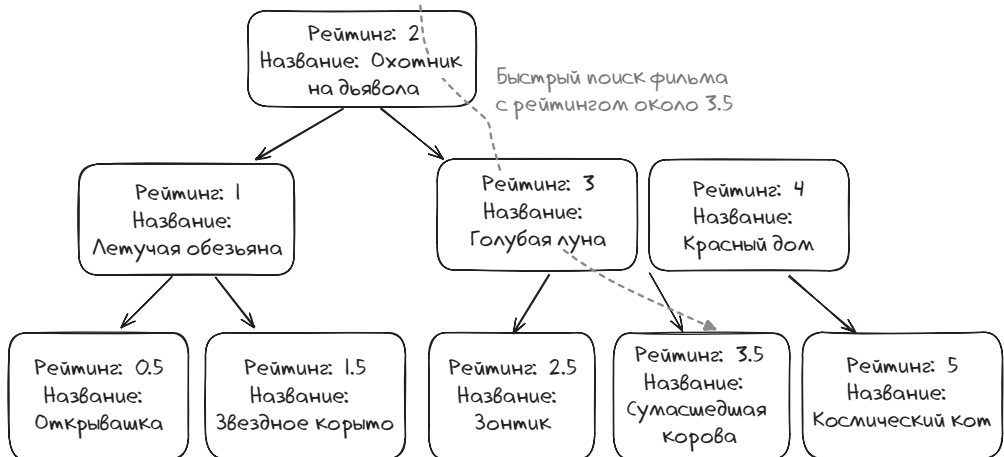
1. FROM. Команда FROM находит таблицу movie и считывает хранящиеся в ней данные.
2. SELECT. Команда SELECT выбирает все столбцы из всех строк.
3. ORDER BY. Команда ORDER BY сортирует все строки по рейтингу фильмов. Сортировка выполняется в порядке убывания.
4. LIMIT. Наконец, команда LIMIT выводит первые 30 строк из отсортированного списка.

Если бы не было индексов, СУБД пришлось бы при каждом таком запросе заново сортировать все фильмы по рейтингу. А если их, к при-

меру, миллион? Представьте себе, как это повлияет на производительность базы данных, насколько замедлится работа приложения. Чтобы решить эту проблему, необходимо проиндексировать столбец rating:

```
CREATE INDEX idx_rating ON movie (rating);
```

Какую функцию выполняет индекс? Он представляет собой дополнительную структуру данных, которая предназначена для быстрого доступа к строкам на основе значений из одного или нескольких столбцов. Обычно это *B-дерево* (B-tree) — самобалансирующаяся древовидная структура, благодаря которой скорость поиска, последовательного доступа, вставки и удаления ускоряется на порядки. При создании индекса СУБД сканирует всю таблицу и сохраняет значения из указанного столбца вместе со ссылками на соответствующие строки. В нашем случае индексируется столбец rating таблицы movie. В дальнейшем это ускорит сортировку и поиск значений из этого столбца:



Необходимо отметить, что все запросы, в которых имеется условие WHERE, фильтрующее фильмы по рейтингу, а значит выполняющее поиск, также оптимизируются при помощи этого индекса:

```
SELECT *
  FROM movie
 WHERE rating = 5;
```

Точно так же можно предположить, что пользователи могут сортировать и искать фильмы по дате выхода, а значит, нередко будут необходимы такие запросы:

```
-- comment: команды для MySQL, MariaDB, PostgreSQL и SQLite
-- comment: команды для других СУБД можно узнать у ChatGPT
SELECT *
  FROM movie
 ORDER BY date DESC
  LIMIT 30;
```

Поэтому имеет смысл индексировать и столбец `date`:

```
CREATE INDEX idx_rating ON movie (date);
```

Если же приложение позволяет сортировать фильмы одновременно и по рейтингу, и по дате, то есть выполнять запросы такого вида:

```
-- comment: команды для MySQL, MariaDB, PostgreSQL и SQLite
-- comment: команды для других СУБД можно узнать у ChatGPT
SELECT *
  FROM movie
 ORDER BY date DESC, rating DESC
  LIMIT 30;
```

потребуется индекс, объединяющий оба столбца:

```
CREATE INDEX idx_combo ON movie (rating, date);
```

Во всех этих случаях мы индексировали столбцы, отличные от первичного ключа. А что, если потребуется сортировать фильмы по `movie_id`? Об этом можно не беспокоиться: индексы для первичных ключей создаются автоматически.

Полнотекстовые индексы

Команда `CREATE INDEX` создает стандартные индексы, которые позволяют ускорить выполнение множества задач, в том числе сортировку и поиск по точному совпадению. Однако бывают случаи, когда стандартного индекса недостаточно. К примеру, он бесполезен, когда необходимо найти фильм по названию, но не по точному совпадению, а по ключевым словам или части названия и даже в том случае, когда пользователь набрал запрос с опечатками.

Для эффективного выполнения сложных задач, связанных с поиском по тексту, необходим *полнотекстовый индекс* (full-text index), который разделяет текст на слова или термины (токены) и сохраняет их в виде таблицы, пригодной для поиска. В отсутствие такого индекса поиск по ключевому слову выглядит так:

```
SELECT *
  FROM movie
 WHERE title LIKE '%шикарный%';
```

Как можно себе представить, даже если такой запрос выполним в принципе, поиск не будет эффективным, а при больших объемах данных потребует много времени. Полнотекстовый индекс значительно ускоряет эту работу. Чтобы его создать и использовать, нужны, например, такие запросы:

```
-- comment: команды для MySQL и MariaDB
-- comment: команды для других СУБД можно узнать у ChatGPT
-- comment: создание полнотекстового индекса
CREATE FULLTEXT INDEX ft_idx_title
  ON movie (title);
-- comment: поиск
SELECT *
  FROM movie
 WHERE MATCH(title)
  AGAINST('шикарный' IN NATURAL LANGUAGE MODE);
```

Несмотря на то что полнотекстовые индексы поддерживаются во многих СУБД, синтаксис команд для их создания и применения сильно варьируется. Сравните, к примеру, прошлый фрагмент кода, написанный для MySQL и MariaDB, со следующим, предназначенным для PostgreSQL. В обоих случаях индексируется столбец `title` таблицы `movie`:

```
-- comment: команды для PostgreSQL
-- comment: команды для других СУБД можно узнать у ChatGPT
-- comment: создание полнотекстового индекса
ALTER TABLE movie
  ADD COLUMN tsv_title;
UPDATE movie
  SET tsv_title = to_tsvector('english', title);
CREATE INDEX gin_idx_title
  ON movie USING gin(tsv_title);
```

Хотите разобраться, как работать с полнотекстовыми индексами в конкретной СУБД? Задайте вопрос ChatGPT.

Индексация на практическом примере

Чтобы понять, какие индексы нужно создать в базе данных «Фантастической лавки», необходимо подумать, какие запросы к ней будут выполняться приложением. Нам снова потребуется помощь его разработчиков и особенно бэкенд-инженеров. Теперь их задача сводится к следующему:

- Подробно рассказать, как приложение взаимодействует с базой данных, какие запросы будут выполняться.
- Поведать об узких местах с точки зрения производительности, в том числе при работе с пользователями.
- Убедиться, что изменения в логике приложения и новые функции соответствуют лучшим практикам проектирования баз данных.

Совместная работа с инженерами преследует определенные цели:

- Используя полученные сведения, понять, какие изменения нужно внести в структуру базы данных, какие столбцы проиндексировать, чтобы повысить производительность запросов.
- Убедиться, что изменения в проекте не снизят производительность приложения и не нарушат согласованность данных.
- Убедиться, что измененная база данных соответствует требованиям заказчика (если они не изменились).

Если же база данных уже находится в продакшене, то помимо сотрудничества с разработчиками можно прибегнуть к эмпирическим методам: определить потребности в индексах на основе метрик производительности и логов.

Итак, мы выяснили, какие запросы требуют оптимизации. Рассмотрим дальнейшие действия на следующих трех примерах.

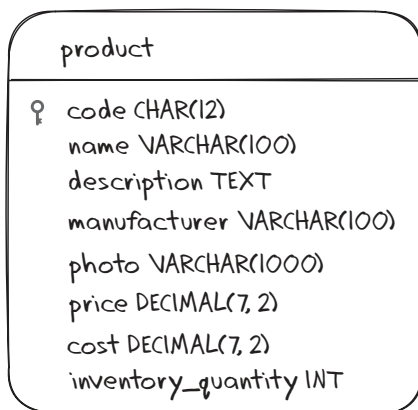
Таблица **product**

Сценарий работы с таблицей **product** можно сформулировать так:

Пользователи часто ищут товары по их названиям.

То есть предполагается выполнение следующего запроса:

```
-- comment: команды для MySQL и MariaDB
-- comment: команды для других СУБД можно узнать у ChatGPT
SELECT * FROM product WHERE MATCH(name)
AGAINST('какой-то текст' IN NATURAL LANGUAGE MODE);
```



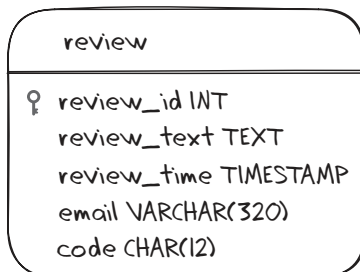
Для эффективного поиска по столбцу name необходимо создать для него полнотекстовый индекс:

```
-- comment: команды для MySQL и MariaDB
-- comment: команды для других СУБД можно узнать у ChatGPT
CREATE FULLTEXT INDEX ft_idx_name ON product(name);
```

Таблица review

А вот сценарий работы с таблицей review:

Пользователи часто читают отзывы на товары.



Чтобы вывести на экран отзывы о каком-то товаре, необходимо найти их в таблице review по артикулу, к примеру 23356. Поэтому часто будет выполняться запрос:

```
SELECT * FROM review WHERE code = '23356';
```

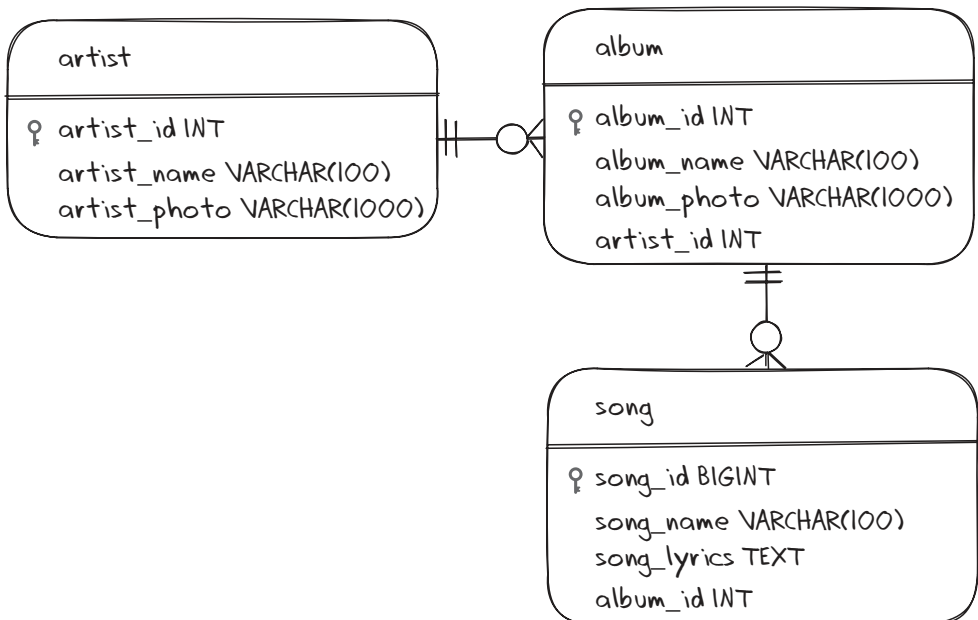

Процесс денормализации

Денормализация (denormalization) — последний этап оптимизации базы данных, когда в целях ускорения запросов в нее намеренно вносятся избыточность. Денормализация — явление нечастое. Однако когда какой-то запрос требует соединения нескольких таблиц и выполняется очень часто, она позволяет значительно повысить производительность базы данных.

Обычно денормализация проходит в два этапа:

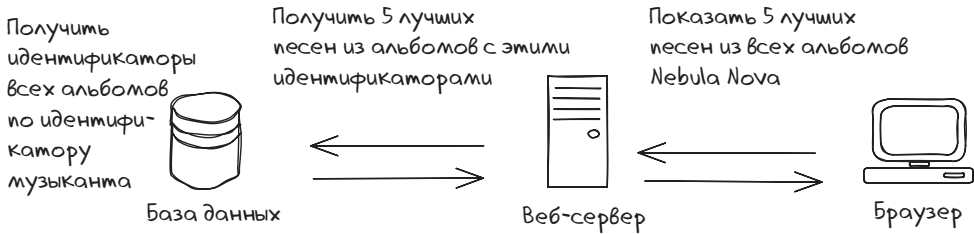
1. Выявление часто выполняемых запросов, которые требуют соединения нескольких таблиц.
2. Дублирование некоторых столбцов в другие таблицы, чтобы уменьшить количество таблиц, соединяемых при запросе.

Для лучшего понимания этих действий сначала рассмотрим упрощенный пример. Допустим, мы анализируем базу данных музыкального приложения, в которой есть информация о музыкантах (artist), альбомах (album) и песнях (song):



Анализируя связи между таблицами, можно сказать, что песни группируются по альбомам, а альбомы — по музыкантам. Все три таблицы полностью нормализованы. Однако приложению часто приходится выводить списки лучших песен того или иного музыканта:

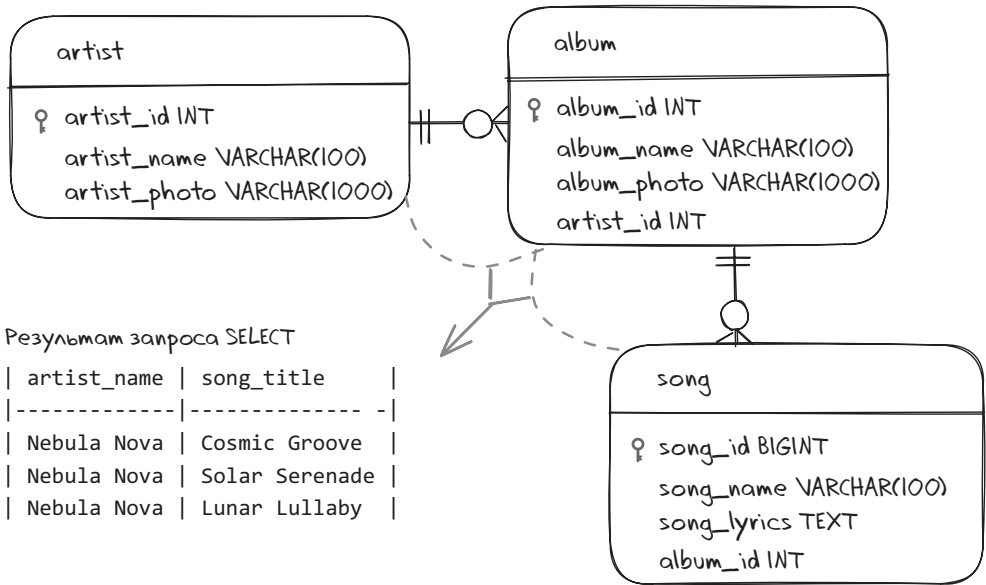
Получить идентификатор
музыканта Nebula Nova



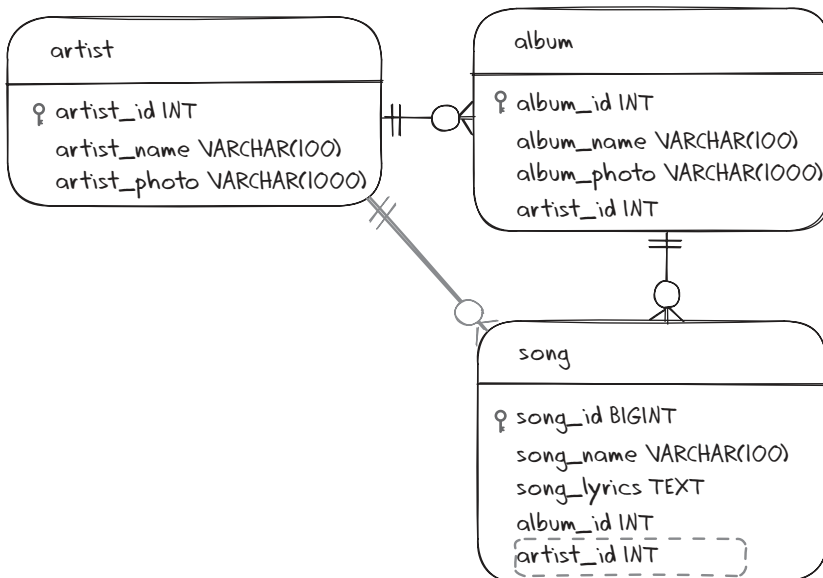
Для этого приложению нужно выполнить запрос с соединением всех трех таблиц:

```
-- comment: команды для MySQL, MariaDB и PostgreSQL
-- comment: команды для других СУБД можно узнать у ChatGPT
SELECT artist.artist_name, song.song_title
FROM artist
INNER JOIN album
    ON artist.artist_id = album.artist_id
INNER JOIN song
    ON album.album_id = song.album_id
WHERE artist.artist_name = 'Nebula Nova'
ORDER BY album.album_id, song.song_id
LIMIT 5;
```

Если учесть огромные объемы данных, можно сказать, что его выполнение потребует больше ресурсов и времени, а значит, и будет стоить дороже, чем ожидалось. К тому же поскольку такие запросы выполняются часто, основанные на них экраны и страницы приложения будут заметно тормозить.



Как решить эту проблему? Хорошо видно, что в результатах нет никакой информации из таблицы **album**. Однако она нужна для выполнения запроса, поскольку связывает **artist** и **song**. Прямая связь между этими таблицами могла бы заметно ускорить выборку данных. Попробуем создать эту связь и обновим ER-диаграмму:



Поскольку новая связь относится к типу «один ко многим», в таблицу song нужно добавить внешний ключ artist_id. Решит ли такое изменение проблему с запросами? Да. Теперь они будут более эффективными:

```
-- comment: команды для MySQL, MariaDB, PostgreSQL и SQLite
-- comment: команды для других СУБД можно узнать у ChatGPT
SELECT artist.artist_name, song.song_title
FROM artist
INNER JOIN song
    ON artist.artist_id = song.artist_id
WHERE artist.artist_name = 'Nebula Nova'
ORDER BY song.song_id
LIMIT 5;
```

С другой стороны, после обновления появилась другая проблема: транзитивная зависимость в таблице song:



Мы намеренно ввели избыточность в эту таблицу, ухудшили ее нормальную форму. Отсюда и слово «денормализация». Порой приходится взвешивать преимущества и недостатки и выбирать: стремиться ли к строгой нормализации проекта или закрыть глаза на небольшие нарушения в угоду эффективности и производительности.

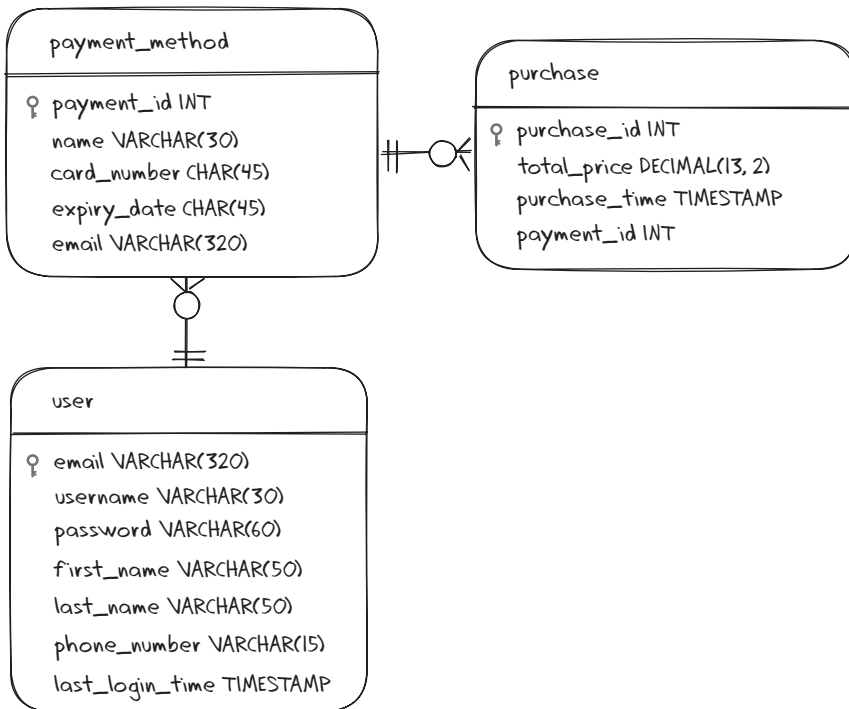
Важно понимать, что необдуманная денормализация может поставить под угрозу целостность данных, поскольку везде, где имеет место избыточность, придется обновлять несколько таблиц вместо одной. Поэтому следует тщательно оценить возможные последствия. К примеру, если она приведет к появлению множества новых операций INSERT, UPDATE и DELETE, итоговый прирост производительности может оказаться ничтожным.

Не менее важно, во избежание проблем в будущем, вести подробную документацию, в которой фиксировать принятые решения, их обоснования, учтенные факторы, плюсы и минусы. Наконец, нужно информировать обо всем этом разработчиков приложения, стейкхолдеров и других заинтересованных лиц, чтобы они были в курсе актуального состояния дел. В конце концов, денормализация — тема спорная

и противоречивая, и все должны быть согласны, что все сделано верно, а приложение будет работать эффективно и правильно.

Денормализация на практическом примере

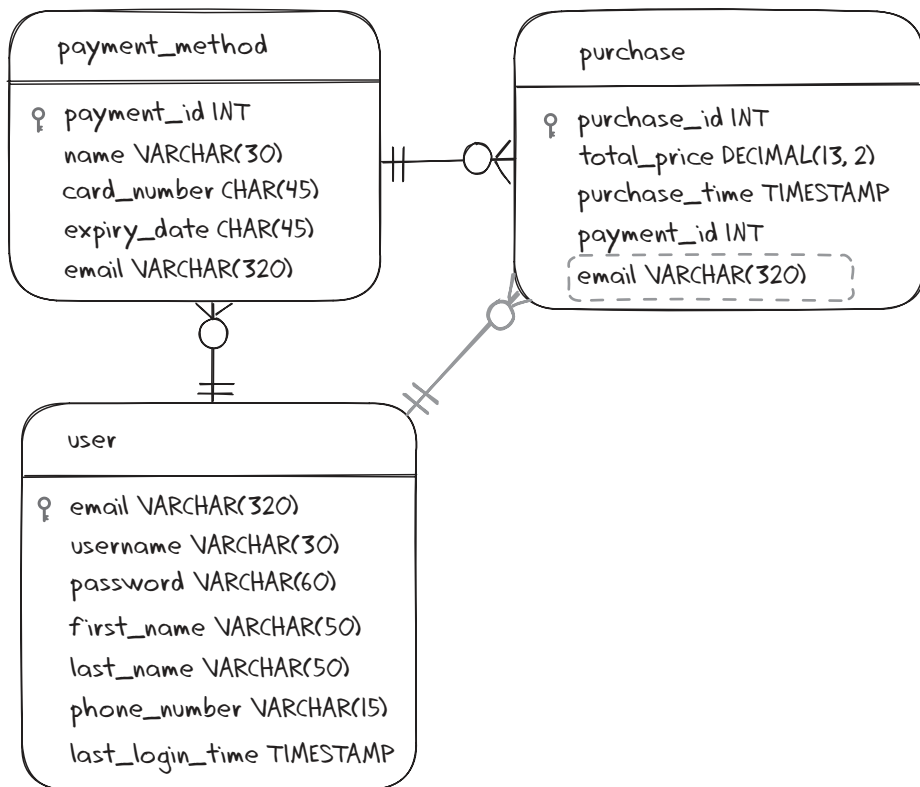
Посмотрим на базу данных «Фантастической лавки»: возможно, какие-то таблицы нуждаются в денормализации. Мы снова вернемся к тройке таблиц `user`, `payment_method` и `purchase`, которая на данный момент выглядит так:



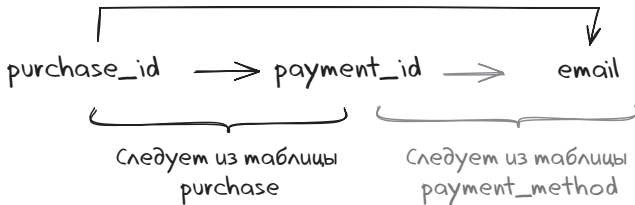
Клиенты «Фантастической лавки» внимательно относятся к проверке своих заказов и счетов, поэтому в приложении часто выполняется запрос:

```
SELECT * FROM purchase
  INNER JOIN payment_method
    ON purchase.payment_id = payment_method.payment_id
  INNER JOIN user
    ON purchase.email = user.email
 WHERE user.email = 'customerCat@humor.com';
```

Таблица `payment_method` не используется для вывода результатов, но должна присутствовать в запросе, поскольку связывает две другие таблицы — пользователей и их заказы. Если вы помните, в ходе нормализации в главе 6 мы разорвали прямую связь между `user` и `purchase`. Однако поскольку частое выполнение таких запросов замедляет работу приложения и повышает расходы на вычислительные мощности, придется подумать еще раз и, возможно, восстановить эту связь (заняться денормализацией):

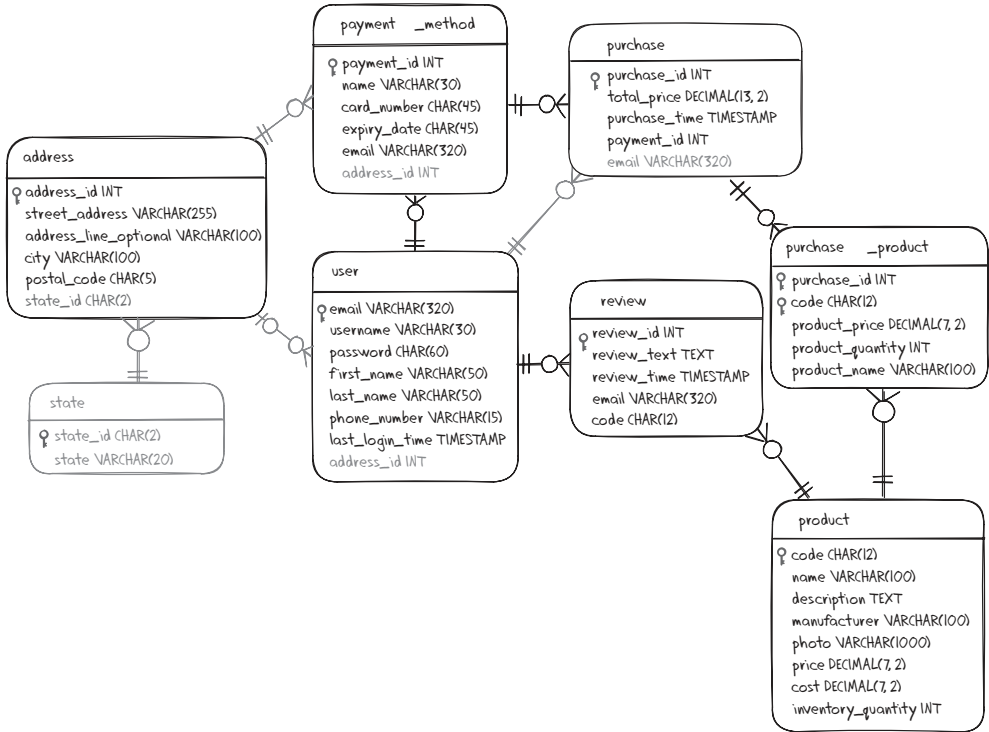


Естественно, при этом в таблице `purchase` появится избыточность, а требования нормальной формы Бойса — Кодда (BCNF; см. главу 6) будут нарушены. Действительно, при внимательном взгляде на эту таблицу мы вновь заметим в ней транзитивную зависимость между `email` и `purchase_id` через `payment_id`:



Необходимо взвесить плюсы и минусы возможного решения. Популярный онлайн-магазин — приложение, в котором часто приходится генерировать чеки, а значит, сопоставлять данные о заказах и пользователях. Естественно, что прямая связь между `user` и `purchase` облегчит запросы к этим таблицам, ведь без нее соединение будет происходить с участием третьей таблицы, а это дополнительные вычисления. Но не возникнут ли новые проблемы, к примеру аномалии добавления и удаления? Клиенты «Фантастической лавки» не могут менять адреса электронной почты, указанные при регистрации, а значит, данные в столбце `email` изменять не придется. Покупки без регистрации в приложении невозможны, а значит, к моменту создания новой строки в `purchase` мы будем знать, что записать в этот столбец. Делаем вывод: аномалии исключены, перед нами идеальная возможность для денормализации.

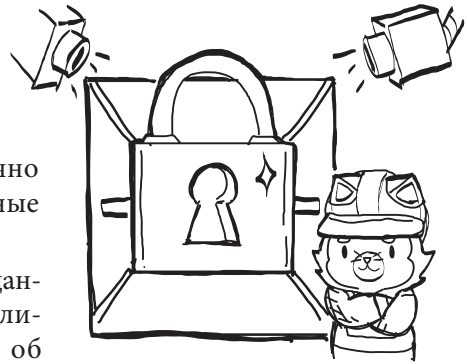
Подводя итоги сделанных изменений, построим окончательную ER-диаграмму базы данных «Фантастической лавки»:



Поздравляем всех, кто прошел этот путь от основ до готового проекта. Глава за главой постепенно накапливая знания, мы проследили сложный процесс трансформации разрозненных идей и требований в хорошо структурированную, эффективную и мощную базу данных. Ваше стремление к знаниям и желание учиться заслуживает всяческих похвал!

Итоги

- Ни в коем случае не храните секретные данные в виде обычного текста. Выберите и применяйте подходящий метод шифрования; убедитесь, что в столбцах достаточно места, чтобы хранить зашифрованные данные.
- Повысить эффективность хранения данных можно не только путем нормализации. К примеру, стоит подумать об устранении избыточных таблиц, а также столбцов с категориальными данными. Однако нужно действовать осмотрительно, чтобы при этом не пострадала производительность базы данных.
- Индексация — способ повысить производительность при выполнении запросов. Чтобы понять, какие столбцы индексировать, необходимо привлечь разработчиков приложения, а также проанализировать логи. В сценариях, когда предполагается лишь сортировка и поиск точных совпадений, достаточно стандартного индекса. Если же нужен поиск по ключевым словам или частичным совпадениям, не обойтись без полнотекстового индекса.
- Денормализация — достаточно редкий случай, когда в целях повышения производительности в базу данных искусственно вносится избыточность. При этом необходимо помнить о сопутствующих проблемах с добавлением, обновлением и удалением данных.



Часть 3

Искусственный интеллект в проектировании баз данных



Добро пожаловать в будущее баз данных! В этой части мы будем говорить о том, как усовершенствовать процесс проектирования при помощи генеративного ИИ, а также научимся эффективно взаимодействовать с ним, чтобы ускорить работу, но не утратить при этом контроль над происходящим.

В главе 8 мы снова начнем с нуля и спроектируем базу данных, но только теперь при поддержке ChatGPT. Поговорим о том, как встроить ИИ в процесс проектирования, чтобы повысить его эффективность, не жертвуя качеством результатов.

Приготовьтесь оттачивать навыки и углублять знания в ходе создания с нуля полностью функциональной базы данных.

8 | Проектирование баз данных в эпоху генеративного ИИ



В этой главе

- ✓ Знакомство с возможностями ИИ в проектировании баз данных
- ✓ Составление промптов для получения нужных результатов
- ✓ Полное проектирование базы данных для компании-производителя летательных аппаратов с помощью ИИ

Необходимые знания

Эта глава отличается от предыдущих. В силу вероятностной природы больших языковых моделей (large language models — LLM) и постоянного их развития вам вряд ли удастся точно воспроизвести примеры, которые мы рассмотрим. При работе над текстом авторы пользовались ChatGPT 4o, но даже он может выдать совсем другие ответы (верные или не очень) на промпты из этой главы, пусть даже вы повторите их слово в слово.

Проект рассматриваемой в этой главе базы данных (в полностью завершенном виде), реализованный при помощи часто применяемых на практике инструментов dbdiagram.io и MySQL Workbench, можно найти в репозитории GitHub (<https://github.com/Neo-Hao/grokking-relational>).

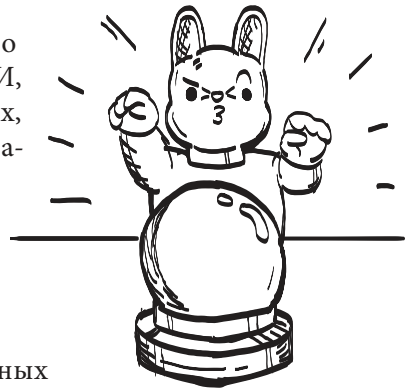
database-design). Чтобы загрузить этот проект в соответствующий инструмент, перейдите в папку `chapter_08` и следуйте инструкциям из файла `README.md`.

В той же папке можно найти полный текст запросов к ChatGPT, а также SQL-скрипты, соответствующие окончательному варианту проекта и подходящие для работы в нескольких распространенных СУБД, включая MySQL, MariaDB, PostgreSQL, SQLite, SQL Server и Oracle.

Введение

В этой главе мы поговорим о том, как можно использовать возможности генеративного ИИ, чтобы ускорить проектирование базы данных, и посмотрим, как изменяются стандартные этапы этого процесса, когда к работе подключается искусственный интеллект.

Из предыдущих семи глав мы получили основные знания о том, как проектировать надежные базы данных, рассмотрев большой практический пример — базу данных «Фантастической лавки». Настало время сменить декорации. В этой главе мы будем работать над базой данных для фирмы, выпускающей звездолеты. Мы снова пройдем весь путь от чистого листа до готового проекта, но в этот раз нам будет помогать искусственный интеллект. С его помощью мы сможем, во-первых, убедиться, что ИИ способен сильно облегчить нам жизнь, а во-вторых, понять, что его возможности имеют определенные границы.





Есть много LLM, которые могут помочь нам, и среди них — ChatGPT, Claude, Google Gemini и Llama. Для подготовки этой главы использовался ChatGPT (точнее, GPT-4o). Чтобы добиться самых точных ответов, необходимо строить промпты особым образом, учитывая то, как проходило обучение конкретной языковой модели.

Вот три железных правила, которые стоит соблюдать при общении с ChatGPT:

- *Промпты должны быть максимально конкретными, содержательными и подробными.* Не существует LLM, способной читать наши мысли и понимать, о чем мы действительно спрашиваем, и ChatGPT — не исключение. В ответ на туманный, нечетко сформулированный вопрос ИИ ответит по старому доброму принципу «что посеешь, то и пожнешь».
- *Инструкции следует отделять от контекста.* В этом случае большинство LLM дают более качественные ответы. ChatGPT ожидает, что мы поместим инструкции в начало промпта и отделим их от контекста при помощи знаков «###» или "''''''". Вот, например, как оформить запрос на краткое резюме текста:

CHATGPT GPT-4O

Кратко изложи следующий текст, оформив самые важные идеи в виде списка.

Текст: " " "

и далее какой-нибудь длинный текст
" " "

- *Желаемый формат результата необходимо описывать на примере.* Чтобы повысить качество ответа, имеет смысл предложить ChatGPT определенный формат, к примеру такой, в котором результат будет легче обработать программно. Продолжая пример с реферированием текста, можно задать такой запрос:

CHATGPT GPT-4O

Извлеки из следующего текста наиболее важные сущности: названия компаний, имена людей, конкретные и общие темы, которые соответствуют содержанию текста.

Формат:

Названия компаний: <список, разделенный запятыми>

Имена людей: - | | -

Конкретные темы: - | | -

Общие темы: - | | -

Текст: " " "

и далее какой-нибудь длинный текст
" " "

Подробные рекомендации по работе с ChatGPT можно найти по ссылке <https://mng.bz/QD41>. Однако по опыту авторов книги для проектирования баз данных наиболее важны и полезны именно эти три правила.

Последнее замечание, прежде чем мы приступим к работе. Не следует ожидать, что можно повторить приведенные в этой главе диалоги

с ChatGPT. По множеству причин, среди которых — случайность, предшествующий контекст, обновления и корректировки модели, на один и тот же запрос можно получить разные ответы. Старайтесь понять, как наиболее эффективно взаимодействовать с ChatGPT, как строить работу с ним, а в каких случаях стоит полагаться на собственный опыт и знания.

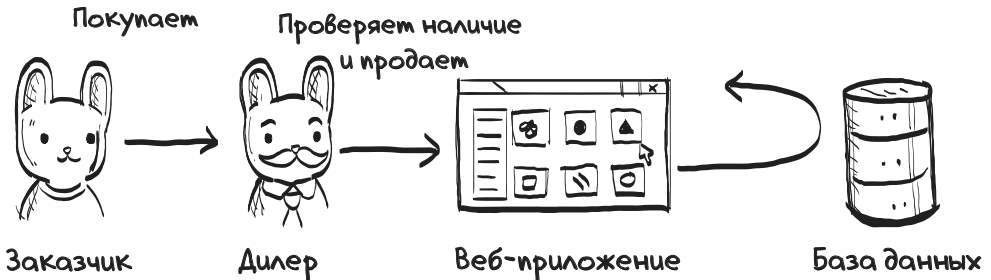
Анализ требований

Продолжая космическую тематику, спроектируем базу данных компании SHIPS R US («Звездолеты — это мы») — производителя звездолетов. Компания поставляет крупные партии товаров дилерам, а те продают их заказчикам в розницу. Подобным образом в наши дни работают Toyota и Ford. Уподобляясь им, руководство «Звездолетов» заказало веб-приложение и базу данных, в которой должны храниться сведения о моделях звездолетов, их производстве, дилерах и заказчиках. Наша задача — сделать проект этой базы.



Для краткости предположим, что мы уже воспользовались советами из главы 3, чтобы собрать и проанализировать требования к базе данных. В итоге мы получили следующий список:

- В базе данных должны храниться записи обо всех изготовленных звездолетах, их марках и моделях, дилерах, заказчиках, а также о связях между дилерами и заказчиками.
- Под одной маркой может выпускаться несколько моделей звездолетов. Звездолетов одной модели может быть несколько.
- Один или несколько звездолетов отгружаются дилеру, который обслуживает одного или нескольких заказчиков.
- Заказчик обслуживается одним или несколькими дилерами и может приобрести у него или у них один или несколько звездолетов.
- Звездолет может относиться только к одной модели, быть продан только одним дилером и только одному заказчику.



Эти требования в основном относятся к сущностям и связям между ними. Список требований к свойствам сущностей мы рассмотрим немного позже.

Сущности и свойства

В этом разделе мы определим список сущностей и свойств в соответствии с требованиями. ChatGPT поможет нам в этом.

Сущности

Просматривая расширенный список требований, мы можем выделить пять сущностей: марка, модель, звездолет, дилер и заказчик. Если это решение неочевидно для вас или же вы хотите проверить возможности ChatGPT, попробуйте обратиться к нему с таким промптом:

Я проектирую базу данных. Помоги мне выявить все сущности, исходя из следующих требований.

Требования: " " "

В базе данных должны храниться записи обо всех изготовленных звездолетах, их марках и моделях, дилерах, заказчиков, а также о связях между дилерами и заказчиками.

Под одной маркой может выпускаться несколько моделей звездолетов. Звездолетов одной модели может быть несколько.

Один или несколько звездолетов отгружаются дилеру, который обслуживает одного или нескольких заказчиков.

Заказчик обслуживается одним или несколькими дилерами и может приобрести у него или у них один или несколько звездолетов.

Звездолет может относиться только к одной модели, быть продан только одним дилером и только одному заказчику.

" " "

Как можно заметить, мы специально отделили инструкции от контекста, чтобы ChatGPT дал более точный ответ. Этот запрос, разумеется, настолько прост, что такое разделение вряд ли необходимо, однако лучше всегда придерживаться трех железных правил. Это поможет, когда у вас появятся действительно сложные вопросы.

В целом ответ ChatGPT будет похож на тот, что показан на рисунке ниже:

В базе данных должны
храниться записи обо всех звездолет
изготовленных звездолетах, марка
их марках и моделях, дилер модель
дилерах, заказчиков, дилер
а также о связях между заказчик
дилерами и заказчиками.

Свойства

Хорошо потрудившись, анализируя полученные у заказчиков сведения, мы составили следующий список требований к свойствам сущностей:

- *У марки есть название.* Количество марок ограничено и не превышает пяти.
- *У модели есть название и четырехзначный номер.* Номера моделей уникальны.
- *У звездолета есть 17-значный серийный номер (как VIN у автомобиля), рекомендуемая стоимость и год выпуска.* Серийные номера звездолетов уникальны. Единица измерения стоимости — доллар США.
- *У дилера есть название, адрес, номер телефона, электронная почта и URL сайта.* Дилер может располагаться в любой стране земного шара.
- *У заказчика есть имя, адрес, номер телефона и электронная почта.* Несколько заказчиков могут проживать по одному адресу. Заказчик может жить в любой стране земного шара.

Создание ER-диаграммы — важный этап работы, когда для каждой сущности нужно определить свойства, выбрать для них типы данных и установить первичный ключ. Это довольно утомительная задача, которая требует полного погружения в анализ собранных требований. Но ChatGPT может помочь, достаточно сформулировать подходящий промпт:

Я проектирую базу данных для производителя звездолетов. Я выявил пять сущностей: марка (brand), модель (model), звездолет (spaceship), дилер (dealer) и заказчик (customer). Помоги мне определить свойства этих сущностей, выбрать типы данных и первичные ключи. Дай ответ в удобном для меня формате.

Формат ответа:

Имя сущности: spaceship

Свойства:

Серийный номер: CHAR(17)

Рекомендуемая стоимость: NUMERIC(8, 2)

Первичный ключ: serial number

Требования: " " "

У марки есть название. Заказчик может жить в любой стране земного шара.

(Это фрагмент. Полный текст можно найти по ссылке <https://bit.ly/grddb>.)

" " "

Чтобы ответ ИИ соответствовал нашим ожиданиям, мы добавили в запрос описание формата вывода. Благодаря этому он будет не только более последовательным, но и удобным для дальнейшего взаимодействия с ИИ.

ChatGPT ответит примерно так:

CHATGPT GPT-4O

Имя сущности: Brand

Свойства:

Имя: VARCHAR(100)

Первичный ключ: Name

Имя сущности: Model

Свойства:

Номер модели: CHAR(4)

Имя модели: VARCHAR(100)

Первичный ключ: Model number

Имя сущности: Spaceship

Свойства:

Серийный номер: CHAR(17)

Рекомендуемая стоимость: NUMERIC(8, 2)

.....

Увидев этот текст, мы сразу же понимаем, что, формулируя запрос, упустили кое-что важное: именование сущностей и свойств. На данный момент они не соответствуют правилам, о которых мы говорили в главе 3. Чтобы решить эту проблему, нужно задать ИИ уточняющий запрос или же заново сформулировать исходный. В обоих случаях следует предложить ChatGPT пример формата. Вот, например, уточняющий запрос:

Переведи свой ответ в новый формат, но с теми же требованиями. Имена всех сущностей и свойств должны быть в змеином регистре.

Новый формат:

Имя сущности Spaceship: spaceship

Свойства:

Серийный номер: serial_number — CHAR(17)

Рекомендуемая стоимость: recommended_price — NUMERIC(8, 2)

Первичный ключ: serial_number

А вот что ИИ может ответить:

CHATGPT GPT-4O

Имя сущности Brand: brand

Свойства:

Имя: name — VARCHAR(100)

Первичный ключ: name

Имя сущности Model: model

Свойства:

Номер модели: model_number — CHAR(4)

Имя модели: model_name — VARCHAR(100)

Первичный ключ: model_number

Имя сущности Spaceship: spaceship

Свойства:

Серийный номер: serial_number — CHAR(17)

Рекомендуемая стоимость: recommended_price — NUMERIC(8, 2)

Год: year — YEAR

Первичный ключ: serial_number

Имя сущности Dealer: dealer

Свойства:

Имя: name — VARCHAR(100)

Адрес: address — VARCHAR(255)

Телефон: phone_number — CHAR(15)

Электронная почта: email_address — VARCHAR(100)

URL сайта: website_url — VARCHAR(255)

Первичный ключ: name

Имя сущности Customer: customer

Свойства:

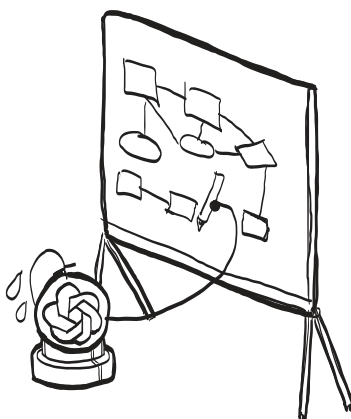
Имя: name — VARCHAR(100)

Адрес: address — VARCHAR(255)

Телефон: phone_number — CHAR(15)

Электронная почта: email_address — VARCHAR(100)

Первичный ключ: email_address



Как можно заметить, ChatGPT многое делает правильно: использует заданный формат, формирует имена в змеином регистре, в отсутствие требований делает разумные предположения о длине данных.

И все же с некоторыми решениями можно поспорить. Есть три хорошо заметные проблемы.

Во-первых, в качестве первичных ключей некоторых сущностей ИИ выбрал совсем не то, что нужно. Так вышло, поскольку в требованиях мало данных по этому вопросу, а значит, ИИ пришлось немного пофантазировать. Так, свойство `name` стало первичным ключом сущности `dealer`, а свойство `email_address` — сущности `customer`. Оба решения чреваты проблемами. Названия дилеров меняются редко, однако довольно легко найти двух дилеров с одинаковыми названиями. Даже в пределах одного штата, что уж тут говорить обо всем земном шаре. Поэтому первичным ключом должно быть какое-то другое свойство, но вот какое? Дальнейший анализ требований показывает:

- заказчик может изменить электронную почту и телефон;
- дилер может изменить электронную почту и телефон.

Иными словами, ни телефон, ни электронная почта не обладают стабильностью. Сделать любое из этих свойств первичным ключом — значит создать проблемы с их будущим обновлением. Стало быть, у сущностей `dealer` и `customer` на данный момент нет свойств, которые могут быть первичными ключами. Как мы уже знаем из главы 4, в таких случаях можно использовать суррогатный ключ с автоматическим увеличением. Такие ключи прекрасно справляются с идентификацией строк, но не могут предотвратить избыточность, ведь пользователь или бот может зарегистрироваться несколько раз, чтобы создать себе несколько учетных записей с одинаковыми данными. Но как мы читали в главе 6, здесь нам на помощь придут ограничения `UNIQUE`, которые мы наложим на свойства `email_address` и `phone_number`, чтобы их значения не повторялись.

Пересмотрев проект сущности `customer` с учетом этих соображений, получим:

Имя сущности Customer: `customer`
Свойства:
Идентификатор заказчика: `customer_id` — INT
Имя: `name` — VARCHAR(100)

Первичный ключ: `customer_id`
Примечание 1: Для свойства `email_address` необходимо ограничение UNIQUE. Для свойства `phone_number` необходимо ограничение UNIQUE.

Аналогичным образом обновим сущность `dealer`:

Имя сущности Dealer: `dealer`
Свойства:
Идентификатор дилера: `dealer_id` — INT
Имя: `name` — VARCHAR(100)

Первичный ключ: `dealer_id`

Вторая проблема — предлагая тип данных для номеров телефонов, ИИ не учел, что компания работает с дилерами и заказчиками из любых стран. Свойство `phone_number` есть у обеих сущностей: `customer` и `dealer`. Выбранный для него тип данных — `CHAR(15)` — не подойдет для всех номеров. Да, благодаря международной системе нумерации, все телефоны, включая код страны, состоят не более чем из 15 цифр, если мы запишем их без пробелов, дефисов и скобок. Однако в некоторых странах есть телефоны с буквами в номерах, и в таких случаях общая длина номера превысит 15 символов. Одно из возможных решений — сохранять код страны в отдельном от остальной части номера свойстве.

Пересмотрев проект сущности `customer` с учетом этих соображений, получим:

```
Имя сущности Customer: customer
Свойства:
Идентификатор заказчика: customer_id – INT
Имя: name – VARCHAR(100)
Телефон:
    country_code – CHAR(5)
    customer_phone_number – CHAR(15)
    . . . . .
```

Аналогичным образом обновим сущность `dealer`:

```
Имя сущности Dealer: dealer
Свойства:
Идентификатор дилера: dealer_id – INT
Имя: name – VARCHAR(100)
Телефон:
    country_code – CHAR(5)
    dealer_phone_number – CHAR(15)
    . . . . .
```

Отделение кода страны поможет стандартизировать номера телефонов, а также обеспечит единообразие их хранения. Но что еще более важно, типа `CHAR(15)` заведомо хватит для любых номеров без учета кода страны.

И наконец, третья проблема. Как и в прошлом случае, ChatGPT не учел возможное место жительства дилеров и заказчиков и предложил хранить адреса в виде строк типа `VARCHAR(255)`. Однако для некоторых очень длинных адресов этого может и не хватить. Что еще хуже, отсутствие четкого формата для ввода и хранения приведет к проблемам при парсинге, поиске и фильтрации адресов, не говоря уже о проверке данных и обработке ошибок. Нам нужен более надежный

и гибкий способ хранения адресов, а именно несколько свойств для их отдельных частей.

Поэтому мы изменим структуру сущности `customer`:

Имя сущности Customer: `customer`

Свойства:

.....

Адрес:

`street_address` – `VARCHAR(255)`

`suburb` – `VARCHAR(100)`

`city` – `VARCHAR(100)`

`state_province` – `VARCHAR(100)`

`postal_code` – `VARCHAR(20)`

`country` – `CHAR(2)`

.....

Аналогичным образом переделаем сущность `dealer`:

Имя сущности Dealer: `dealer`

Свойства:

.....

Адрес:

`street_address` – `VARCHAR(255)`

`suburb` – `VARCHAR(100)`

`city` – `VARCHAR(100)`

`state_province` – `VARCHAR(100)`

`postal_code` – `VARCHAR(20)`

`country` – `CHAR(2)`

.....

Теперь, когда вместо одного свойства стало шесть, можно заметить, что сущности `customer` и `dealer` во многом пересекаются. С подобной ситуацией мы уже сталкивались в главе 7, когда имели дело с адресами пользователей и адресами для отправки счетов. Здесь можно

применить то же решение: завести новую таблицу `address`, которая будет связана с сущностями `dealer` и `customer` и где будут собраны все адреса. Примерно так:

```
Имя сущности Address: address
Свойства:
Идентификатор адреса: address_id – INT
Улица и дом: street_address – VARCHAR(255)
Район: suburb – VARCHAR(100)
Город: city – VARCHAR(100)
Штат/провинция: state_province – VARCHAR(100)
Почтовый индекс: postal_code – VARCHAR(20)
Страна: country – CHAR(2)
Первичный ключ: address_id
```

Почтовые индексы применяются не везде. В некоторых странах их не используют, в других же используют, но по-разному. Единого международного формата для индексов не существует. Поэтому свойство `postal_code` (почтовый индекс) необходимо сделать необязательным: оно может быть равно `NULL`. То же касается и свойства `suburb` (район).

Теперь, когда у нас появилась сущность `address`, мы можем удалить все, что связано с адресами, из сущностей `customer` и `dealer`.

Важно сохранять все изменения в текстовом виде: проект еще не завершен, а ChatGPT — это прежде всего текстовый инструмент, и нам еще не раз придется отправить ему текущий вариант списка сущностей и свойств для доработок и реализации. Объединив внесенные нами изменения с ответами ChatGPT, получим следующее:

```
Имя сущности Brand: brand
Свойства:
Имя: name – VARCHAR(100)
Первичный ключ: name
```

Имя сущности Model: model

Свойства:

Номер модели: model_number – CHAR(4)

Название модели: model_name – VARCHAR(100)

Первичный ключ: model_number

Имя сущности Spaceship: spaceship

Свойства:

Серийный номер: serial_number – CHAR(17)

Рекомендуемая стоимость: recommended_price – NUMERIC(8, 2)

Год: year – YEAR

Первичный ключ: serial_number

Имя сущности Dealer: dealer

Свойства:

Идентификатор дилера: dealer_id – INT

Имя: name – VARCHAR(100)

Электронная почта: email_address – VARCHAR(100)

URL сайта: website_url – VARCHAR(255)

Телефон:

country_code – CHAR(5)

dealer_phone_number – CHAR(15)

Первичный ключ: dealer_id

Примечание 1: для свойства email_address необходимо ограничение UNIQUE. Для сочетания свойств (country_code, phone_number) необходимо ограничение UNIQUE.

Имя сущности Customer: customer

Свойства:

Идентификатор заказчика: customer_id – INT

Имя: name – VARCHAR(100)

Электронная почта: email_address – VARCHAR(100)

Телефон:

country_code – CHAR(5)

customer_phone_number – CHAR(15)

Первичный ключ: customer_id

Примечание 1: для свойства email_address необходимо ограничение UNIQUE. Для сочетания свойств (country_code, phone_number) необходимо ограничение UNIQUE.

Имя сущности Address: address

Свойства:

Идентификатор адреса: address_id – INT

Улица и дом: street_address – VARCHAR(255)

Район: suburb – VARCHAR(100)

Город: city – VARCHAR(100)

Штат/провинция: state_province – VARCHAR(100)

Почтовый индекс: postal_code – VARCHAR(20)

Страна: country – CHAR(2)

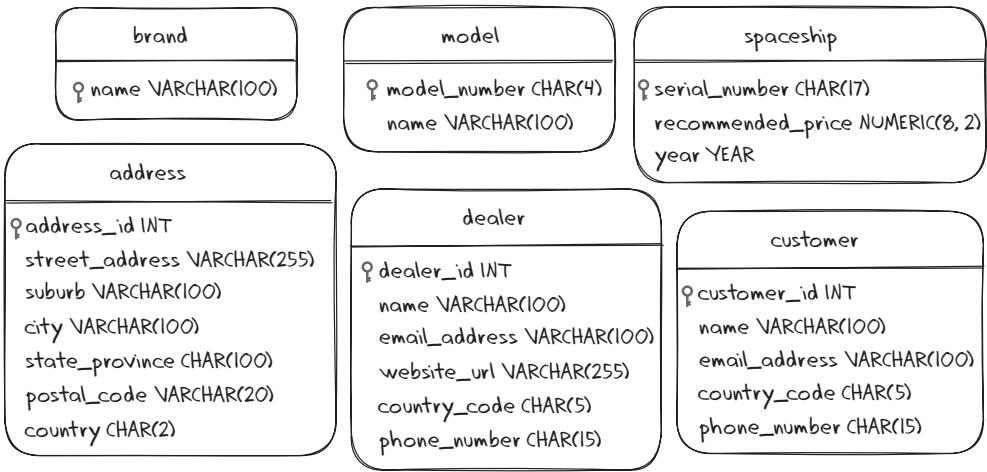
Первичный ключ: address_id

Примечание 1: свойства suburb и postal_code необязательные.

Демонстрация проекта

Итак, опираясь на требования к проекту и помощь ChatGPT, мы завершили работу над сущностями и свойствами. Результат представляет собой длинный форматированный текст, который необходим для общения с ИИ, но не подходит для демонстрации другим специалистам. Поэтому нам нужно нарисовать ER-диаграмму.

Когда список сущностей короток, их несложно изобразить в любом графическом редакторе. В нашем случае мы получим:



Однако в крупном проекте счет сущностей может идти на десятки, и в таких случаях можно снова прибегнуть к помощи ChatGPT. Он, разумеется, не нарисует нам диаграмму, зато сгенерировать SQL-скрипт ему вполне по плечу. Полученный код можно загрузить в любой инструмент, который позволяет построить схему базы данных по SQL-коду, к примеру dbdiagram.io (<https://dbdiagram.io>). Попробуем сделать это и в нашем случае:

Я проектирую базу данных. Помоги мне сгенерировать код MySQL на основе следующего списка сущностей и свойств. Считай, что все сущности независимы друг от друга.

Описание: ""

Имя сущности Brand: brand

Свойства:

Имя: name – VARCHAR(100)

Первичный ключ: name

.....

Страна: country – CHAR(2)

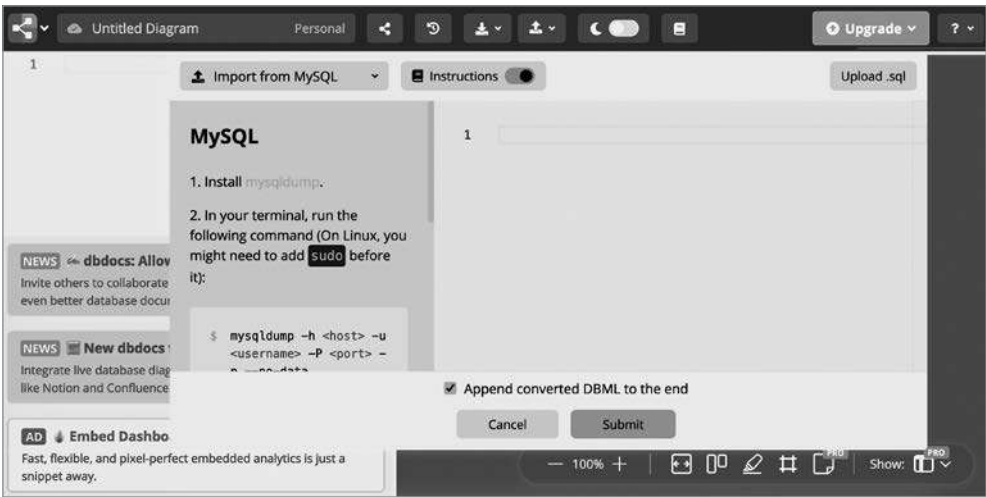
Первичный ключ: address_id

Примечание 1: свойства suburb и postal_code необязательные

(Это фрагмент. Полный текст можно найти по ссылке <https://bit.ly/grdb>.)

""

Получив код, загрузим его в dbdiagram.io, воспользовавшись функцией импорта:



Построенной таким образом схемой удобно делиться с другими. К тому же она гораздо нагляднее, чем текст:

brand	
name	VARCHAR(100)

model	
model_number	CHAR(4)
model_name	VARCHAR(100)

spaceship	
serial_number	CHAR(17)
recommended_price	NUMERIC(8,2)
year	YEAR

address	
address_id	INT
street_address	VARCHAR(255)
suburb	VARCHAR(100)
city	VARCHAR(100)
state_province	VARCHAR(100)
postal_code	VARCHAR(20)
country	CHAR(2)

dealer	
dealer_id	INT
name	VARCHAR(100)
email_address	VARCHAR(100)
website_url	VARCHAR(255)
dealer_phone_number	CHAR(15)
country_code	CHAR(5)

customer	
customer_id	INT
name	VARCHAR(100)
email_address	VARCHAR(100)
customer_phone_number	CHAR(15)
country_code	CHAR(5)

 dbdiagram.io

Вот так ChatGPT ускоряет нашу работу. Мы, разумеется, не можем использовать его ответы напрямую, но они служат отличной отправной точкой на долгом пути решения поставленных задач. В данном

случае мы попросили ИИ сгенерировать код, на основе которого сумели быстро нарисовать диаграмму, пригодную для демонстрации. Однако следует помнить о том, что, получив от ИИ какой-то ответ, необходимо проверить его и при необходимости устранить недочеты, используя собственный опыт и знание проектирования БД. Это — наша ответственность.

Связи между сущностями

Итак, у нас готов список сущностей, их свойства определены, типы данных выбраны. Пора переходить к связям между сущностями. Но для начала вспомним, какие требования предъявляются к проекту:

- В базе данных должны храниться записи обо всех изготовленных звездолетах, их марках и моделях, дилерах, заказчиках, а также о связях между дилерами и заказчиками.
- Под одной маркой может выпускаться несколько моделей звездолетов. Звездолетов одной модели может быть несколько.
- Один или несколько звездолетов отгружаются дилеру, который обслуживает одного или нескольких заказчиков.
- Заказчик обслуживается одним или несколькими дилерами и может приобрести у него или у них один или несколько звездолетов.
- Звездолет может относиться только к одной модели, быть продан только одним дилером и только одному заказчику.
- У каждого заказчика есть адрес. Адреса нескольких заказчиков могут совпадать. У каждого дилера есть только один адрес.

Последнее из требований появилось после решения о том, что адрес будет независимой сущностью. Оно необходимо, чтобы понять, как адрес связан с заказчиком и дилером.

Как и в прошлый раз, в начале работы имеет смысл попросить ChatGPT создать черновой вариант проекта. А чтобы ответ ИИ принес больше пользы, воспользуемся одним из железных правил (см. начало главы) и структурируем промпт надлежащим образом. К примеру, можно написать так:

Я проектирую базу данных для производителя звездолетов. Я выявил шесть сущностей: марка (Brand), модель (Model), звездолет (Spaceship), дилер (Dealer), заказчик (Customer) и адрес (Address). Помоги мне установить связи между ними с учетом заданных требований. Дай ответ в удобном для меня формате.

Формат: brand | model

Связь: один ко многим

Примечание: многие – со стороны model

Требования: ""

В базе данных должны храниться записи обо всех изготовленных звездолетах, их марках и моделях У каждого дилера есть только один адрес.

(Это фрагмент. Полный текст можно найти по ссылке <https://bit.ly/grdb>.)

""

ChatGPT ответит примерно так:

CHATGPT GPT-4O

brand | model

Связь: один ко многим

Примечание: многие – со стороны model

model | spaceship

Связь: один ко многим

Примечание: многие – со стороны spaceship

spaceship | dealer

Связь: один ко многим

Примечание: многие – со стороны spaceship

spaceship | customer

Связь: один ко многим

Примечание: заказчик может иметь несколько звездолетов. У звездолета может быть несколько заказчиков (например, совместная покупка).

dealer | address

Связь: один к одному

Примечание: у каждого дилера только один адрес

customer | address

Связь: один ко многим

Примечание: многие – со стороны customer

Как и в прошлый раз, ChatGPT многое делает правильно: большинство связей определены верно. Однако он ошибается в связи между customer и spaceship. Требования в отношении этих сущностей недвусмысленны:

Заказчик обслуживается одним или несколькими дилерами и может приобрести у него или у них один или несколько звездолетов.

Заказчик может приобрести один или несколько звездолетов

Звездолет может относиться только к одной модели, быть продан только одним дилером и только одному заказчику.

Звездолет может быть продан только одному заказчику

Возможно, ChatGPT просто не связал между собой два раздела, а потому неправильно понял их общий смысл. Может быть, именно в этот момент у ИИ случились галлюцинации. Как бы там ни было, если вы прочли эту книгу с первой страницы, вы уже знаете, как исправить эту простую ошибку:

spaceship | customer

Связь: один ко многим

Примечание: многие – со стороны spaceship

Кроме того, в ответе ИИ нет ничего о связи между customer и dealer. Однако в требованиях все четко:

- Дилер обслуживает одного или нескольких заказчиков.
- Заказчик обслуживается одним или несколькими дилерами.

Придется самим добавить недостающие сведения о связях:

customer | dealer

Связь: многие ко многим

Примечание: многие – с обеих сторон

Минимальная кратность и внешние ключи

ChatGPT ничего не сказал ни о минимальных кратностях связей, ни о внешних ключах для сущностей. Впрочем, мы ведь об этом его и не спрашивали: сначала нам нужно было понять, какие вообще есть связи, а также убедиться, что ChatGPT составил полный и правильный их список. Поэтому теперь мы можем задать ИИ новый вопрос:

Я проектирую базу данных для производителя звездолетов. Я выявил шесть сущностей: марка (brand), модель (model), звездолет (spaceship), дилер (dealer), заказчик (customer) и адрес (address). Помогите мне с учетом заданных требований и сведений о связях между сущностями оценить минимальную кратность этих связей, а также определить внешние ключи. Дай ответ в удобном для меня формате.

Формат:

brand | model

Связь: один ко многим

Примечание 1: многие – со стороны model

Примечание 2: внешний ключ у model

Примечание 3: минимальная кратность со стороны model обнулена

Требования: ""

В базе данных должны храниться записи обо всех изготовленных звездолетах, их марках и моделях У каждого дилера есть только один адрес.

(Это фрагмент. Полный текст можно найти по ссылке <https://bit.ly/grdb>.)

"""

Характеристики: """

brand | model

Связь: один ко многим

Примечание: многие – со стороны model

customer | address

Связь: один ко многим

Примечание: многие – со стороны customer

(Это фрагмент. Полный текст можно найти по ссылке <https://bit.ly/grdb>.)

Возможный ответ ChatGPT:

CHATGPT GPT-4O

brand | model

Связь: один ко многим

Примечание 1: многие – со стороны model

Примечание 2: внешний ключ у model

Примечание 3: минимальная кратность со стороны model обнулена

model | spaceship

Связь: один ко многим

Примечание 1: многие – со стороны spaceship

Примечание 2: внешний ключ у spaceship

Примечание 3: минимальная кратность со стороны spaceship обнулена

customer | dealer

Связь: многие ко многим

Примечание 1: необходима промежуточная таблица customer_dealer

Примечание 2: внешние ключи у customer_dealer

Примечание 3: минимальные кратности со стороны customer_dealer обнулены

spaceship | dealer

Связь: один ко многим

Примечание 1: многие – со стороны spaceship

Примечание 2: внешний ключ у spaceship

Примечание 3: минимальная кратность со стороны dealer обнулена

spaceship | customer

Связь: один ко многим

Примечание 1: многие – со стороны spaceship

Примечание 2: внешний ключ у spaceship

Примечание 3: минимальная кратность со стороны spaceship обнулена

dealer | address

Связь: один к одному

Примечание 1: у дилера один и только один адрес

Примечание 2: внешний ключ у dealer

Примечание 3: минимальная кратность со стороны address равна 1

customer | address

Связь: один ко многим

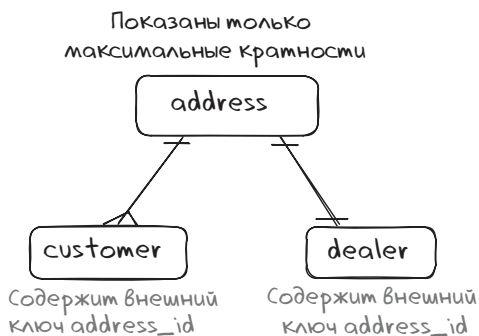
Примечание 1: многие – со стороны customer

Примечание 2: внешний ключ у customer

Примечание 3: минимальная кратность со стороны address обнулена

ChatGPT хорошо справился с размещением внешних ключей, особенно там, где связаны три таблицы. Мы уже обсуждали похожий случай:

customer, dealer и address связаны друг с другом так же, как user, payment_method и address из базы данных «Фантастической лавки». Изобразим эту связь на диаграмме:



Если же говорить о минимальных кратностях и в особенности об их обнулении, видно, что у ИИ нет четкого понимания этой темы. Пользуясь тем, что мы узнали из главы 5, исправим его ошибки самостоятельно. На всякий случай вспомним несколько правил обнуления минимальной кратности:

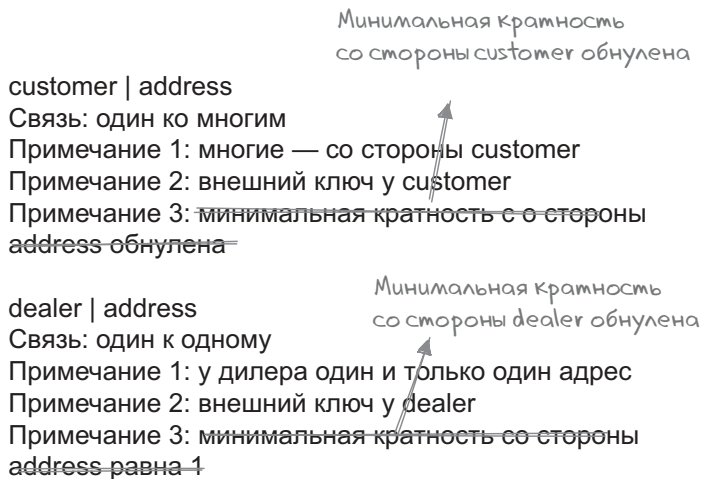
- При связи «один к одному»: поскольку обычно одна таблица создается раньше другой, необходимо обнулить минимальную кратность таблицы, записи которой создаются позже, чтобы можно было вносить записи в первую, пока соответствующие им записи в другой таблице еще отсутствуют.
- При связи «один ко многим» следует обнулить минимальную кратность со стороны «многие». В этом случае записи в таблице на стороне «один» смогут существовать без соответствующих записей в таблице на стороне «многие»: к примеру, покупатель может зарегистрироваться, не указывая адрес.
- При связи «многие ко многим» минимальную кратность часто обнуляют с обеих сторон промежуточной таблицы, что позволяет обеим сущностям существовать независимо друг от друга. Однако при наличии определенных требований в этом отношении мож-

но сохранить минимальную кратность с одной из сторон равной единице.

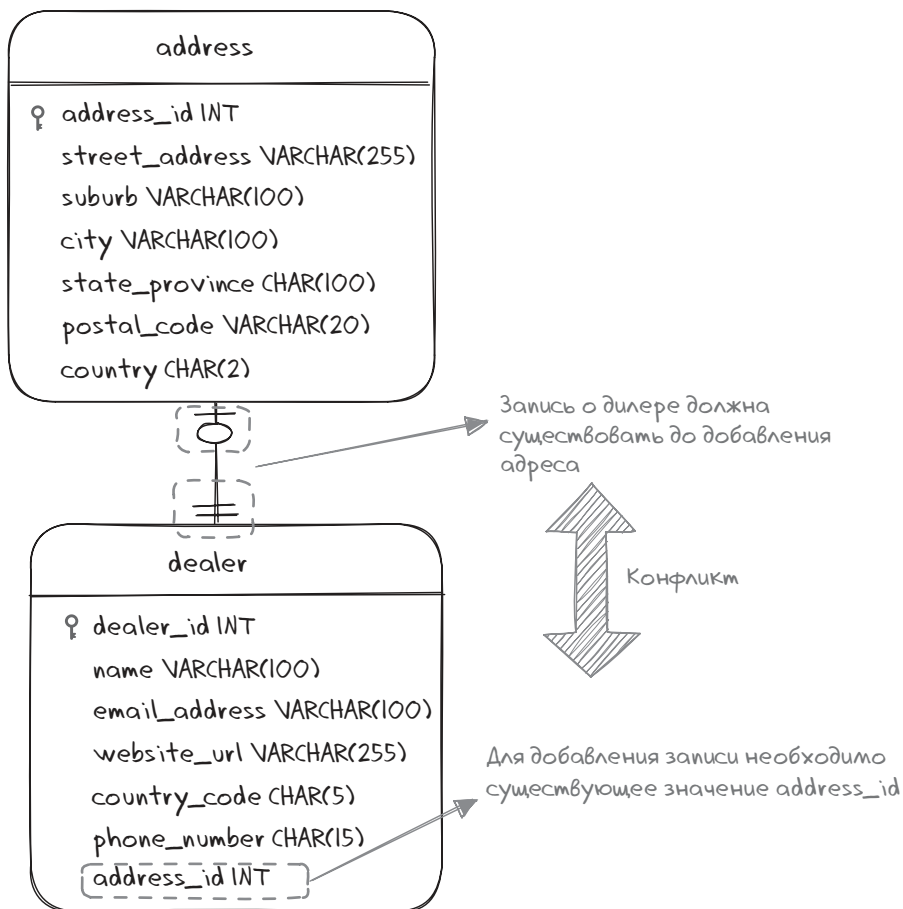
Начнем со связей между dealer и address, а также customer и address. Первоначально никаких требований в отношении этих связей у нас не было. Однако, поговорив с разработчиками приложения, мы получили новые сведения:

- Дилер не может завести учетную запись, не указав адрес. Веб-приложение SHIPS R US просто блокирует такие попытки. Другими словами, запись о дилере не может существовать без соответствующей записи об адресе.
- Компания SHIPS R US собирает данные о заказчиках для продвижения и отзыва товаров, однако сами заказчики в приложении не регистрируются. Вся информация о них, включая адреса, предоставляется дилерами. Другими словами, запись о заказчике не может существовать без соответствующей записи об адресе.

С учетом этих данных по приведенным выше правилам обнуления внесем в проект небольшие изменения:

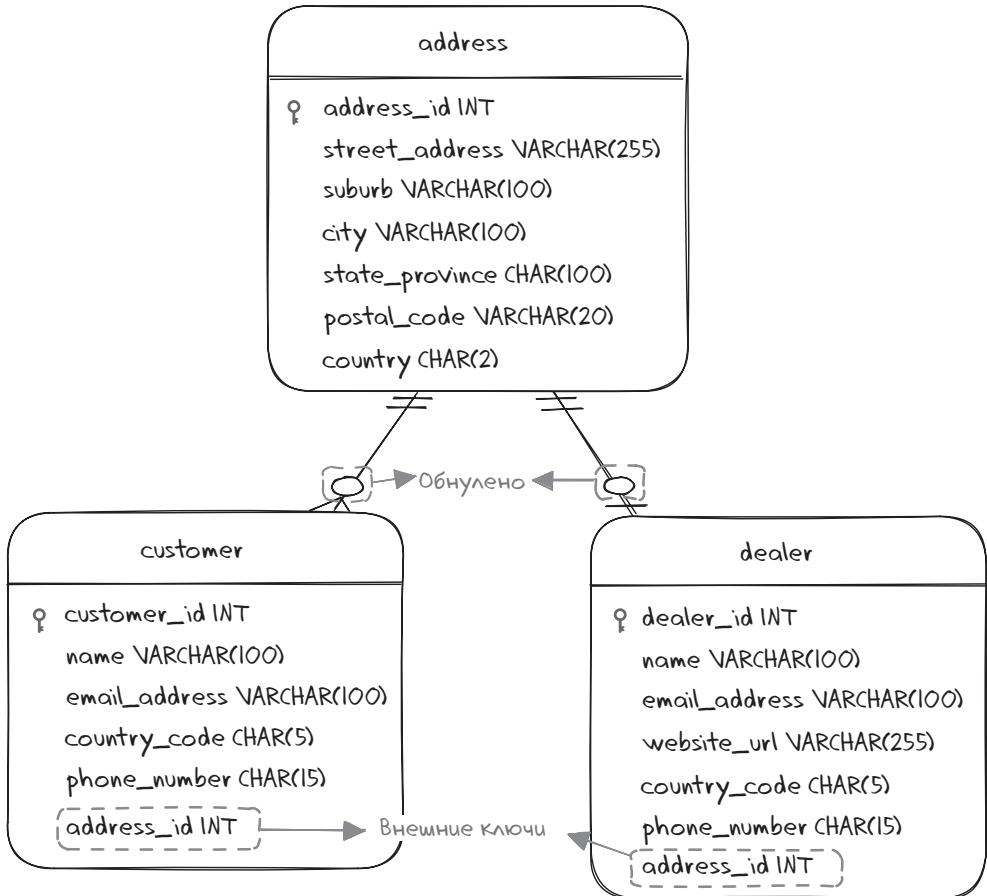


Коррекция связи между customer и address очевидна: поскольку она относится к типу «один ко многим», необходимо обнулить минимальную кратность со стороны «многие», иначе ввод данных будет невозможен. Со связью между dealer и address дело обстоит сложнее. Можно ли обнулить кратность со стороны address, а не dealer? Да, можно. Но это сильно усложнит проект и может нарушить согласованность данных:



Внешний ключ `address_id`, который связывает две таблицы, находится в дочерней таблице `dealer`, а значит, для добавления записи о новом дилере необходимо существующее значение `address_id`. Поэтому обнуление кратности со стороны `dealer` — более удобное решение.

Итоговые связи между таблицами `customer`, `dealer` и `address` можно изобразить так:



В обнулении кратности нуждается еще одна связь помимо уже рассмотренных выше: связь между таблицами `spaceship` и `customer`. О них нам известно следующее:

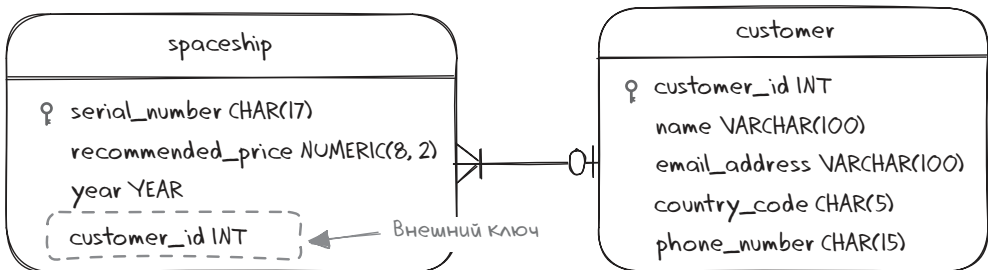
- Звездолет может относиться только к одной модели, быть продан только одним дилером и только одному заказчику.
- Заказчик может приобрести один или несколько звездолетов.

Иными словами, только что построенный звездолет заносится в базу данных, хотя еще и не приобретен заказчиком. Значит, нам нужно обнулить кратность связи со стороны customer, а не spaceship:

spaceship | customer
 Связь: один ко многим
 Примечание 1: многие — со стороны spaceship
 Примечание 2: внешний ключ у spaceship
 Примечание 3: ~~минимальная кратность со стороны spaceship обнуждена~~

Минимальная кратность со стороны customer обнулена

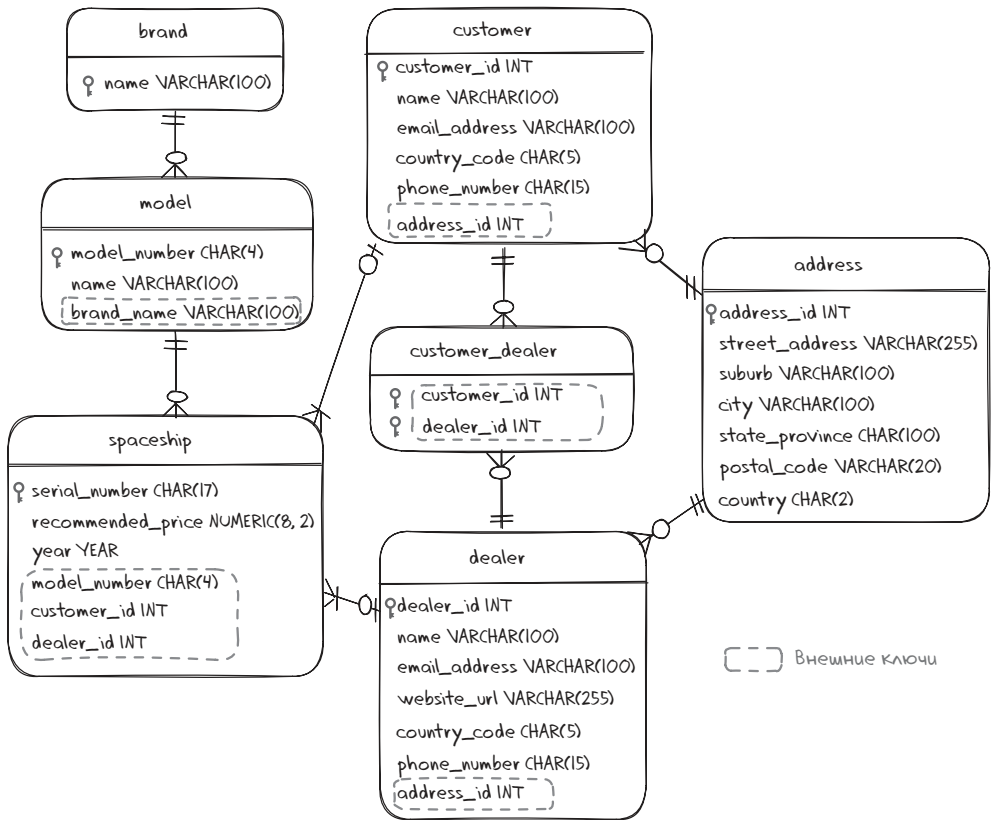
Внесем исправление в проект:



Как можно заметить, отвечающий за эту связь внешний ключ (`customer_id`) находится в таблице `spaceship`. Стало быть, чтобы добавить запись о звездолете, нам нужен существующий идентификатор заказчика. Поэтому чтобы обойти это ограничение и выполнить требования к базе данных, мы сделаем этот ключ необязательным. Иными словами, он сможет иметь значение NULL.

Демонстрация проекта

Как мы уже говорили в разделе «Сущности и связи», для обсуждения проекта с другими понадобится ER-диаграмма. Если изобразить ее вручную, получится примерно такой рисунок.



В более крупных проектах для этой цели мы можем прибегнуть к помощи ChatGPT и dbdigram.io. Сначала попросим ИИ подготовить код MySQL:

Я проектирую базу данных. Помоги мне сгенерировать код MySQL на основе следующего описания сущностей и связей между ними.

Описание: ""

начало списка сущностей

Имя сущности Brand: brand

Свойства:

Имя: name — VARCHAR(100)

Первичный ключ: name

.....

начало описания связей

brand | model

Связь: один ко многим

Примечание 1: многие – со стороны model

.....

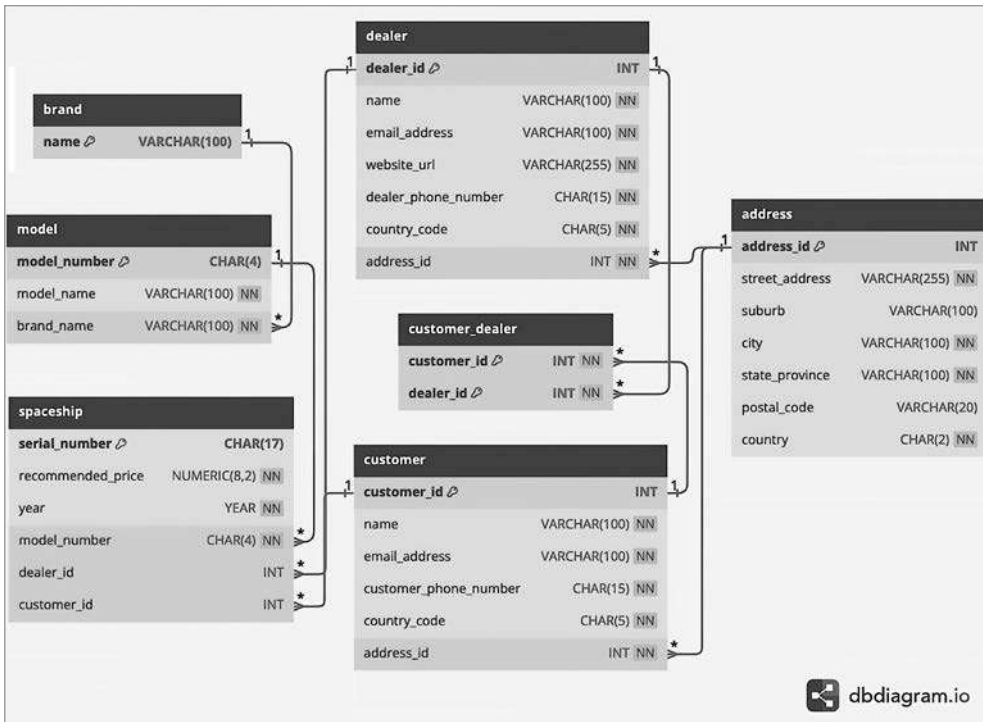
Страна: country – CHAR(2)

Первичный ключ: address_id

Примечание 1: свойства suburb и postal_code необязательные
(Это фрагмент. Полный текст можно найти по ссылке <https://bit.ly/grddb>.)

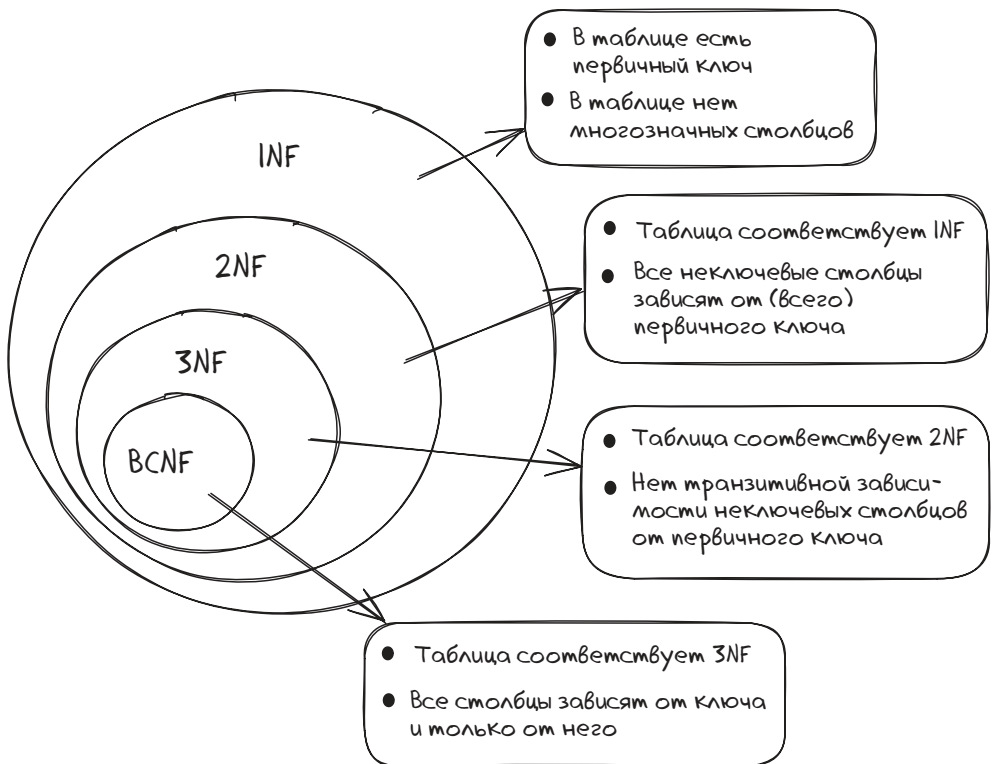
"" "

Получив код и загрузив его в dbdiagram.io, получим готовую ER-диаграмму:



Реализация

Итак, мы закончили работу со списком сущностей, их свойствами и взаимосвязями. Все готово к реализации базы данных. Но сначала нужно убедиться, что все таблицы нормализованы. Освежив в памяти требования к нормальным формам и просмотрев текущий вариант проекта, мы видим, что все таблицы уже находятся в нормальной форме Бойса — Кодда (BCNF). При желании эту работу можно поручить и ChatGPT («Нормализован ли этот проект базы данных?»). Однако нам все равно придется проверить его ответ.



Ограничения

Один из основных этапов реализации — введение ограничений на свойства. Ограничения (PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE и другие) мы обсуждали в главе 6. Чтобы понять, какие из них нужно ввести и для каких свойств, достаточно еще раз просмотреть требования к проекту. Однако и в этом вопросе мы можем рассчитывать на помощь ChatGPT. Для этого необходимо написать новый промпт, вставив в него и требования, и описание текущего варианта проекта:

Я проектирую базу данных. Помоги мне определить, какие свойства нуждаются в ограничениях, и выбрать типы этих ограничений на основе требований к проекту и списка сущностей, свойств и связей между ними. Дай ответ в удобном для меня формате.

Формат:

Сущность: brand

Свойства, требующие ограничений:

* model_number: PRIMARY KEY

* model_name: NOT NULL

* brand_name: FOREIGN KEY

Требования: ""

В базе данных должны храниться записи обо всех изготовленных звездолетах, их марках и моделях

(Это фрагмент. Полный текст можно найти по ссылке <https://bit.ly/grddb>.)

""

Сущности и свойства:

Имя сущности Brand: brand

Свойства:

Имя: name — VARCHAR(100)

(Это фрагмент. Полный текст можно найти по ссылке <https://bit.ly/grddb>.)

""

Связи между сущностями: ""

brand | model

Связь: один ко многим

Примечание 1: многие – со стороны model

(Это фрагмент. Полный текст можно найти по ссылке <https://bit.ly/grdb>.)

ChatGPT ответит примерно так:

CHATGPT GPT-4O

Сущность: brand

Свойства, требующие ограничений:

* name: PRIMARY KEY

Сущность: model

Свойства, требующие ограничений:

* model_number: PRIMARY KEY

* model_name: NOT NULL

* brand_name: FOREIGN KEY (ссылка на brand.name)

Сущность: spaceship

Свойства, требующие ограничений:

* serial_number: PRIMARY KEY

* recommended_price: CHECK (например, CHECK (recommended_price >= 0))

* year: CHECK (например, CHECK (year >= 1900))

* model_number: FOREIGN KEY (ссылка на model.model_number)

* dealer_id: FOREIGN KEY (ссылка на dealer.dealer_id)

* customer_id: FOREIGN KEY (ссылка на customer.customer_id)

Сущность: dealer

Свойства, требующие ограничений:

* dealer_id: PRIMARY KEY

```

* name: NOT NULL
* email_address: UNIQUE
* website_url: NOT NULL
* (country_code, dealer_phone_number): UNIQUE
* address_id: FOREIGN KEY (ссылка на address.address_id)

```

Сущность: customer

Свойства, требующие ограничений:

```

* customer_id: PRIMARY KEY
* name: NOT NULL
* email_address: UNIQUE
* (country_code, customer_phone_number): UNIQUE
* address_id: FOREIGN KEY (ссылка на address.address_id)

```

Сущность: customer_dealer

Свойства, требующие ограничений:

```

* (customer_id, dealer_id): PRIMARY KEY
* customer_id: FOREIGN KEY (ссылка на customer.customer_id)
* dealer_id: FOREIGN KEY (ссылка на dealer.dealer_id)

```

Сущность: address

Свойства, требующие ограничений:

```

* address_id: PRIMARY KEY
* country: CHECK (проверка правильности кода страны, например
CHECK (country IN ('US', 'CA', ...)))

```

Как видим, ChatGPT правильно установил большинство ограничений, но пропустил несколько свойств, которые нужно определить как NOT NULL. Кроме того, ИИ не учел сценарии, в которых свойство требует нескольких ограничений. К примеру, в отношении email_address в таблице dealer должны действовать как NOT NULL, так и UNIQUE. Исправим ответ ChatGPT и сохраним итоговый текст: он пригодится нам на следующем этапе.

Оптимизация

Цель оптимизации — сократить избыточность и повысить согласованность данных, а также эффективность проекта в целом. Лишь после этого можно приступать к написанию кода SQL. Можно задать ИИ несколько общих вопросов (к примеру, «Как мне оптимизировать проект базы данных?»). Но лучше задавать конкретные вопросы по поводу обнаруженных проблем: тогда ответы будут более полезны.



К примеру, можно заметить, что ChatGPT предложил ввести ограничение CHECK на столбец country таблицы address. Однако если учесть, что в мире есть около 200 стран, реализация такого ограничения будет сложна и запутанна. Спросим об этом ИИ:

Вопрос 1. Ввести ограничение CHECK на столбец country таблицы address затруднительно, поскольку в мире около 200 стран. Как можно решить проблему с проверкой правильности данных, не вводя это ограничение? Существуют ли общепринятые подходы для этого?

Аналогичный вопрос можно задать и о столбцах country_code из таблиц customer и dealer:

Вопрос 2. Существуют ли общепринятые подходы, позволяющие проверить правильность данных в столбцах country_code из таблиц customer и dealer?

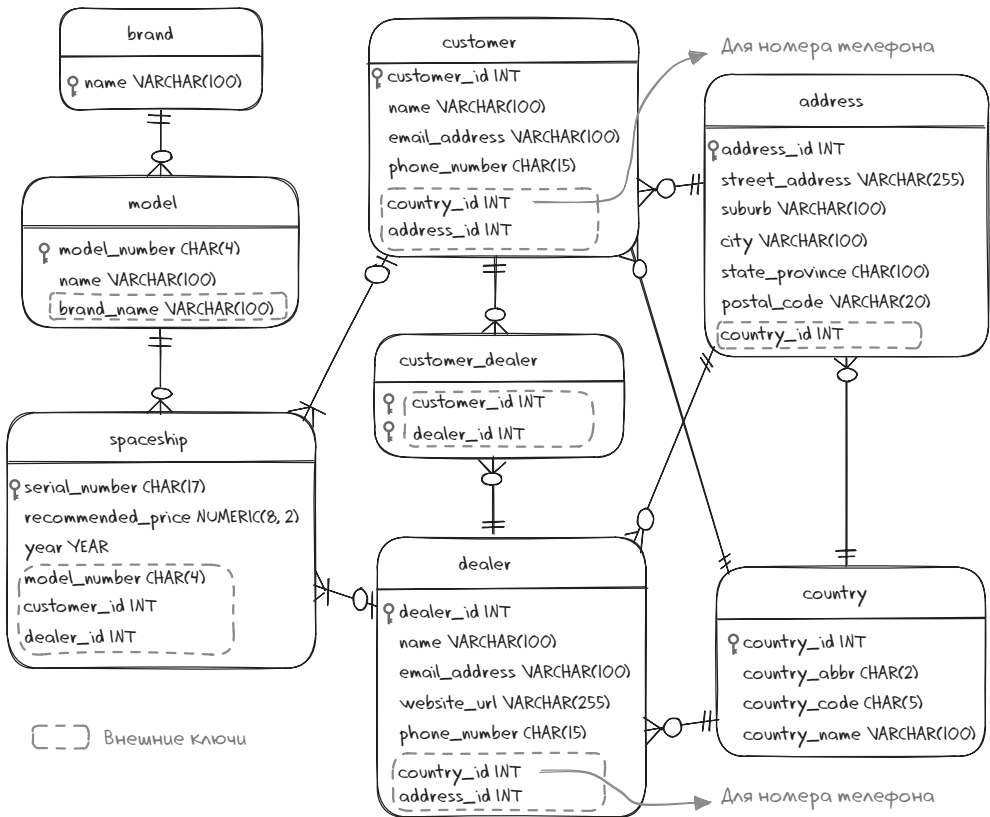
В ответ ChatGPT предложит довольно полезные решения:

CHATGPT GPT-4O

Ответ на вопрос 1. Вместо ограничения CHECK можно создать отдельную таблицу country, в которой будут храниться коды и названия стран...

Ответ на вопрос 2. Чтобы гарантировать правильность данных, можно использовать тот же подход: создать отдельную таблицу...

Объединяя эти два предложения, создадим новую таблицу, в которой будут храниться полные и сокращенные названия стран, а также их телефонные коды. Заранее заполнив ее, мы обеспечим правильность данных в таблицах customer, dealer и address. Обновленная ER-диаграмма выглядит так:



Внесем правки в текст с описанием сущностей, свойств, связей и ограничений. Теперь наш проект готов к конвертации в код SQL, которую мы также выполним при помощи ChatGPT. При этом не следует забывать, что качество результата зависит от того, насколько точно сформулирован промпт. К примеру, так мы получим код MySQL:

```
Я проектирую базу данных. Сгенерируй код MySQL по следующему описанию сущностей, свойств, связей и ограничений.
```

```
Сущности и свойства: ""
```

```
Имя сущности Brand: brand .....
```

```
(Это фрагмент. Полный текст можно найти по ссылке https://bit.ly/grdb.)
```

```
""
```

```
Связи: "" brand | model .....
```

```
(Это фрагмент. Полный текст можно найти по ссылке https://bit.ly/grdb.)
```

```
""
```

```
Ограничения: ""
```

```
Сущность: brand .....
```

```
(Это фрагмент. Полный текст можно найти по ссылке https://bit.ly/grdb.)
```

```
""
```

Если вы работаете с другой СУБД, измените промпт, и вы получите код на другом диалекте SQL. При наличии достаточно подробной информации на этом этапе ChatGPT едва ли допустит ошибки. Полный код можно найти в папке `chapter_08` в репозитории GitHub.

В нашем случае в требованиях к базе данных не сказано ничего об индексации. Если бы она требовалась, нам стоило бы еще раз обратиться к главе 7, чтобы определить, какие столбцы необходимо индексировать и как. Однако нам остается лишь подождать, пока в базе данных появятся реальные значения, и посмотреть, как она будет использоваться.

Отметим, что ChatGPT может помочь и в этом вопросе: создать для нас пробный набор данных, который мы можем использовать для тестирования и упреждающего выявления проблем. К примеру, сформируем такой промпт:

Я проектирую базу данных. Создай для меня пробный набор данных, который я мог бы использовать для тестирования. Вот код SQL для реализации базы данных.

Код SQL:

```
""
-- Создание таблицы brand
CREATE TABLE brand (name VARCHAR(100) PRIMARY KEY);
.....
-- Создание таблицы customer_dealer
CREATE TABLE customer_dealer (
    customer_id INT NOT NULL,
    dealer_id INT NOT NULL,
    CONSTRAINT pk_customer_dealer .....
    CONSTRAINT fk_customer_dealer_customer .....
    CONSTRAINT fk_customer_dealer_dealer .....
);

(Это фрагмент. Полный текст можно найти по ссылке https://bit.ly/grddb.)
""
```

На этом всё: при помощи ChatGPT наша база данных полностью готова. Отличная работа!

Итоги

- Чтобы эффективно использовать ChatGPT для проектирования баз данных, необходимо соблюдать три важных правила: составлять конкретные, содержательные и подробные промпты, отделять инструкции от контекста, а также приводить примеры нужного формата для ответа.
- Проектируя базу данных с помощью ChatGPT, следует действовать постепенно, проверять ответы ИИ и вносить в них необходимые исправления.
- Сохраняйте результаты работы в виде текста. Это облегчит дальнейшее взаимодействие с ChatGPT.
- Если необходимо показать результаты работы другим специалистам, преобразуйте текстовое описание в код SQL при помощи ChatGPT, а затем постройте ER-диаграмму, используя инструменты типа dbdiagram.io.

Цян Хао, Михаил Цикердекис

Грокаем проектирование реляционных баз данных

Перевел с английского А. Бойков

Научный редактор Е. Войнов

Руководитель дивизиона	<i>Ю. Сергиенко</i>
Ведущий редактор	<i>Е. Строганова</i>
Литературный редактор	<i>М. Трусковская</i>
Художественный редактор	<i>В. Мостипан</i>
Корректоры	<i>С. Беляева, Н. Викторова</i>
Верстка	<i>Л. Егорова</i>

Изготовлено в России. Изготовитель: ООО «Прогресс книга».

Место нахождения и фактический адрес: 194044, Россия, г. Санкт-Петербург,
Б. Сампсониевский пр., д. 29А, пом. 52. Тел.: +78127037373.

Дата изготовления: 04.2026. Наименование: книжная продукция. Срок годности: не ограничен.

Налоговая льгота — общероссийский классификатор продукции ОК 034-2014, 58.11.12 — Книги печатные профессиональные, технические и научные.

Импортер в Беларусь: ООО «ПИТЕР М», 220020, РБ, г. Минск, ул. Тимирязева, д. 121/3, к. 214,
тел./факс: 208 80 01.

Подписано в печать 12.02.26. Формат 70×100/16. Бумага офсетная. Усл. п. л. 21,930. Тираж 1000. Заказ 0000.



**ВОСПОЛЬЗУЙТЕСЬ
ВОЗМОЖНОСТЬЮ**

**ПРИБРЕСТИ КНИГИ
НА САЙТЕ ИЗДАТЕЛЬСТВА**

piter.com



со скидкой по промокоду

20%

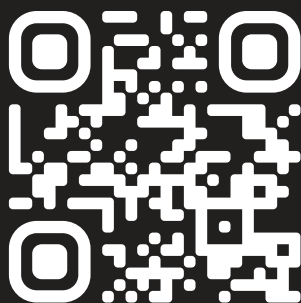
PITER



Read IT Club

Комьюнити рецензентов и переводчиков ИТ-литературы

Миссия участников клуба – обеспечить высокое качество профессиональной переводной литературы в России. «Книжные дебаггеры» проверяют корректность терминологии и подписей на схемах и иллюстрациях, чтобы сделать книги более понятными русскоязычному читателю. Стать участником Read IT Club может любой ИТ-специалист, готовый поделиться опытом с сообществом.



присоединиться к нам



Марчелло Ла Рокка

ГРОКАЕМ СТРУКТУРЫ ДАННЫХ

Каждый разработчик знает, насколько важны структуры данных. Без них не обходится ни один серьезный проект, будь то оптимизация запросов, работа с Big Data или просто написание чистого и эффективного кода. Не зря же на собеседованиях постоянно спрашивают про деревья, хеш-таблицы и сложность алгоритмов!

Вы только приступили к изучению структур данных? Хотите освежить знания, полученные в ходе обучения? В этой книге нет заумной математики, скучных доказательств и абстрактной теории. Вместо этого — понятные объяснения, рабочие примеры и реальные кейсы, с которыми ежедневно сталкиваются разработчики. Вы узнаете, как с помощью правильных структур данных ускорить поиск, эффективнее управлять очередями задач или, например, оптимизировать хранение данных.

Книга построена по принципу «от простого к сложному»: начинается с базовых структур, таких как массивы и связанные списки, и постепенно переходит к более сложным — стекам, очередям, деревьям, хеш-таблицам и графам. Каждая глава содержит практические примеры, упражнения и наглядные иллюстрации, которые помогают закрепить материал. Вся теория подкреплена примерами на Python — одним из главных языков современной разработки.

Если вы хотите не просто использовать структуры данных, а понимать их и применять осознанно — эта книга для вас.

КУПИТЬ